

Development of an Isothermal-Flow Digital Twin for the Compact Integral Effects Test

by

Theodore Kay Chen Ong

A thesis submitted in partial satisfaction of the

requirements for the degree of

Masters of Science

in

Nuclear Engineering

in the

Graduate Division

of the

University of California, Berkeley

Committee in charge:

Professor Per F. Peterson, Chair
Assistant Professor Raluca O. Scarlat
Assistant Professor GuanYu Su

Spring 2023

Development of an Isothermal-Flow Digital Twin for the Compact Integral Effects Test

Copyright 2023
by
Theodore Kay Chen Ong

Abstract

Development of an Isothermal-Flow Digital Twin for the Compact Integral Effects Test

by

Theodore Kay Chen Ong

Masters of Science in Nuclear Engineering

University of California, Berkeley

Professor Per F. Peterson, Chair

The objective of this study is to develop a isothermal flow digital twin for the Compact Integral Effects Test (CIET). CIET is a scaled Integral Effects Test (IET) of the Mark I Fluoride Salt Cooled High Temperature Reactor (FHR). Constructing a digital twin of CIET was motivated because it can be used for remote operation studies, fault detection studies and operator training studies. However, what made the digital twin essential was the advent of the COVID-19 pandemic, which made experiments on CIET nearly impossible to do. Therefore, constructing a digital twin was necessary to ensure that some studies requiring CIET could be performed on a desktop computer. However, access to software, especially open source software, for constructing CIET's digital twin was limited. Given the limited options for using existing software for CIET's digital twin, software libraries for the digital twin were written from the ground up using Rust. Thereafter, a digital twin was constructed using these written libraries and some existing open source Rust and Python libraries found online. This study presents the approach used to construct digital twin and its libraries. Additionally, it demonstrates that an isothermal digital twin can be run from desktop computers with satisfactory validation using experimental data from CIET's isothermal tests. This isothermal digital twin of CIET will be used as a baseline for future work in constructing CIET's digital twin with heat transfer capabilities. Should this endeavour prove successful, the developed digital twins can then be used for various applications such as operator training and controller development.

Contents

Contents	i
List of Figures	iii
List of Tables	v
1 Introduction	1
2 Literature Review	3
2.1 Motivation for Constructing a Digital Twin of the Compact Integral Effects Test (CIET)	3
2.2 Digital Twin: Origin and Definition	7
2.3 Utility of Digital Twins	14
2.4 Process of Constructing Digital Twins	22
3 Methodology	47
3.1 Overview	47
3.2 Justifying the use of Operator Splitting in Real Time Calculations for Fluid Mechanics and Heat Transfer	49
3.3 Solution Procedure for Parallel Branches of Fluid Flow	52
3.4 Choice of Coding Language for the Digital Twin	64
3.5 OPC-UA Server	69
3.6 OPC-UA Client	69
3.7 Code Validation	70
3.8 Approximations Made in Branch Model	76
3.9 Hardware used for Testing of Computation Speed	79
4 Results and Discussion	81
4.1 CTAH and Heater Branch Results	81
4.2 CTAH and DHX Branch Results	88
4.3 CTAH, DHX and Heater Branch Digital Twin Results	93
5 Conclusion	99

5.1	Future Work	100
	Bibliography	102
A	Hydrodynamic Parameters used for Nodalised Python and Rust Model	111
B	Source Code and Copyright Notices	114
B.1	Main Rust Interface Code	114
B.2	Friction Factor Code	127
B.3	Pipe Calculation Code	144
B.4	CIET Specific Code	155
B.5	Python Interface Code	220
B.6	OPC-UA Server and Calculation	237
C	Supplemental Logs and Discussion for Main Text	249
C.1	Literature Review Supplemental Discussion	249
C.2	Methods for Constructing Digital Twins	260
C.3	Methodology Supplemental Discussion	269
C.4	Residual Plot Discussion for DHX and CTAH Branch	288
C.5	Computation Speed in CTAH, Heater and DHX Branch Digital Twin	302
D	Useful References for CIET Schematics	303

List of Figures

2.1	Nodalised Model of CIET without Bypass Branch	5
2.2	Real-Time Data Flow Comparison for each Type of Digital Twin	12
2.3	Approximate Remote Operation Architecture for CIET	31
2.4	Dowtherm A vs Therminol VP-1 Prandtl Number Comparison	39
2.5	Dowtherm A vs Therminol VP-1 Prandtl Number Residual Comparison	40
3.1	Architectural Diagram of Isothermal Type I Digital Twin	48
3.2	Newton Raphson Method Algorithm for Tightly Coupled Solution	50
3.3	Approximate Nodalised Model of CIET without Bypass Branch and Flow Direction	55
3.4	Iterative Solution Procedure for Series of Fluid Components	58
3.5	Iterative Solution Procedure for Parallel Branches of Fluid Components	59
3.6	Iterative Solution Procedure for Series of Fluid Components with Nested Parallel Branches	61
3.7	Approximate Computation Time for Nested Iteration Loops	62
3.8	Summarised Chart of Some NBody Benchmark Simulation Times	66
3.9	Nodalised Model of CIET without Bypass Branch	77
3.10	Approximate Nodalised Model of CIET without Bypass Branch	78
4.1	Digital Twin System Curve Compared to Experimental System Curve from Manome- ter M-42 and M-43 Plotted in Absolute Pressure Units (CTAH and Heater Branch)	82
4.2	Residual Plot for CTAH and Heater Branch System Curves	85
4.3	Absolute Residual Plot for CTAH and Heater Branch System Curves with Error Bars	87
4.4	Digital Twin System Curve Compared to Experimental System Curve from Manome- ter M-42 and M-43 Plotted in Absolute Pressure Units (CTAH and DHX Branch)	89
4.5	Plot Data for Digital Twin Residual vs Experimental System Curve from Manome- ter M-42 and M-43 (CTAH and DHX Branches Only)	91
4.6	Absolute Residual Plot for CTAH and DHX Branch System Curves with Error Bars	92
4.7	Digital Twin Forward Flow vs Reverse Flow	95
A.1	Hydrodynamic Parameters used for the Python and Rust Model	112

C.1	MX10 Correlation and Experiemntal Data Plots	296
C.2	FM40 Correlation and Experiemntal Data Plots	296
C.3	CTAH and TCHX Correlation and Experiemntal Data Plots	298
C.4	MX10 Estimated Residual Plots	300
C.5	FM40 Estimated Residual Plots	300
C.6	CTAH and TCHX Estimated Residual Plots	301

List of Tables

2.1	Digital Twins and their Types	11
4.1	Digital Twin System Curve Compared to Experimental System Curve from Manometer M-42 and M-43 Plotted in Absolute Pressure Units (CTAH and Heater Branch)	83
4.2	Digital Twin System Curve Compared to Experimental System Curve from Manometer M-42 and M-43 Plotted in Absolute Pressure Units (CTAH and DHX Branch)	90
4.3	Reverse Flow Configuration Mass Flowrate vs Loop Pressure Drop Data	96
C.1	Prandtl Number Comparison for Therminol VP 1	263
C.2	Specific Heat Capacity Comparison ($J/(kg\ K)$) for Therminol VP 1	264
C.3	Thermal Conductivity Comparison ($W/(m\ K)$) for Therminol VP 1	265
C.4	Dynamic Viscosity Comparison ($Pa\ s$) for Therminol VP 1	267
C.5	Density Comparison (kg/m^3) for Therminol VP 1	268
C.6	Raw Data used to Plot Experimental Data Left MX-10, Right FM-40 using Graphreader, Significant Figures not Adjusted	297
C.7	Raw Data used to Plot Experimental Data Left: CTAH Downward Flow, Middle: CTAH Upward Flow, Right: TCHX Downward Flow using Graphreader, Significant Figures not Adjusted	299

Acknowledgments

I would first like to thank my supervisor Professor Per F. Peterson for his continuous support in this work and also in supporting me in various ways as an international student living in the Bay Area during the trials of the COVID-19 pandemic. This is in addition to helping me keep my work on track and ensuring that I stay within scope for research so that I don't bite off more than I can chew for my PhD journey. Also, I would like to thank Asst. Professor Raluca O. Scarlat for her valuable inputs on residual and error analysis as well as training me to present my work with precision. Additionally, thanks to Asst. Professor Su GuanYu for reading through long winded drafts of my work and making it much more readable than it was. Secondly, I thank the Singapore Nuclear Research and Safety Initiative (SNRSI) for funding my studies in the University of California, Berkeley. I also thank Clara, Omar, Ishak, Christopher, Dane, Shane and Oliver for helpful discussions and ideas in the laboratory. I thank James and Alan for constant encouragement throughout my PhD and thesis writing journey, where this master's thesis is a milestone.

Also, I thank my mum and dad for working hard in bringing me up in tough economic hardship and for constant support especially over my undergraduate and graduate studies. Also I'm thankful for support from my church and bridges family in California and Singapore, including but not limited to, the Lim family, the Gregory family, the Tronstad family, the Tyler family, the Wolfs, the Castueras and the Proceks. As well as Adrian, PL, Melvin, Shawn, among many many others. None of these could be done without you.

Finally, thanks be to my God and Lord and Saviour Jesus Christ who:

Was manifested in the flesh, vindicated by the Spirit, seen by angels, proclaimed among the nations, believed on in the world, taken up in glory.

אל תהי חכם בעיניך ירא אתי יהוה וסור מרע
רפאות תהי לשרך ושקוי לעצמותיך

Chapter 1

Introduction

Interest in Generation IV Nuclear Power Plants is growing because of their potential to decarbonise the world's energy supply. To develop such Gen IV Reactors which power such plants, initiatives and technologies are being developed which reduce the cost and increase development speed of nuclear power plants. One such technology that can speed up the development of these nuclear power plants is digital twin technology [McCausland, 2022]. A digital twin can be defined as a digital representation of a machine, device or asset which tracks the characteristics, properties, conditions and behaviours of the system by means of models, information and data [Sleiti, Kapat, and Vesely, 2022]¹.

Digital twins can bring many benefits to the end user. These benefits can be broadly categorised into benefits derived before construction of the physical asset and benefits derived after construction of the physical asset they are meant to represent. Before construction, digital twins can be used in product design and manufacturing, [M. Liu et al., 2021] and by extension should also have the potential for designing Generation IV Nuclear Reactors in the case that the desired product is the nuclear reactor. After construction, a digital twin can provide a means to monitor the health of a power plant and provide fault detection [Sleiti, Kapat, and Vesely, 2022], train operators and perform real time simulations [Pérez et al., 2020].

These benefits are highly desirable, and sometimes necessary for research and development to continue in the case of extreme circumstances such as the COVID-19 pandemic. During such circumstances, experimental work meant to further research and development of Gen IV nuclear power plants was severely impaired due to the COVID-19 lockdowns. One experimental apparatus used for Gen IV nuclear power plant development, known as the

¹To begin discussion about digital twin technology, we must first define what a digital twin (DT) is. This can, however, prove to be difficult to pinpoint because there are several different definitions within literature. Sometimes those definitions conflict, which adds to the confusion. To make sense out of the confusion, I have written an entire section discussing the definitions of digital twins. This section is quite verbose. To avoid overloading the reader with information in the introduction, I will first say that the digital twin I have develop and intend to showcase in this work more closely resembles a plant simulator. Only after the reader's interest has (hopefully) been piqued will I then invite the reader to come alongside me in venturing down the rabbit hole of defining digital twins in the literature review section.

Compact Integral Effects Test (CIET), was essentially non-operable during the COVID-19 pandemic lockdowns. To continue research work in such circumstances, it was desirable to have a digital twin acting as a real time simulation available to use as a surrogate for CIET when CIET was not available or too inconvenient to use. Therefore steps to develop a fully functional digital twin of CIET were taken. The first of these steps is described in this work. While CIET’s digital twin development was chiefly motivated by the pandemic, the value of building a digital twin for CIET extends far beyond the use case in the COVID-19 pandemic as well.

To understand the value of constructing a digital twin of CIET, we need to first understand why CIET was constructed in the first place. Secondly, we need to understand some of the practical difficulties in operating CIET. Thirdly, we need to understand how a digital twin can address some of these issues.

Once the “why” of constructing the digital twins is addressed, we explore the “what” of a digital twin. This means we discuss what digital twins are, attempt to classify them and discuss how different classes of digital twin benefit industry and especially nuclear technology development differently from a literature review. In this section, we also discuss what the requirements of a digital twin are for the use case of CIET. Thereafter, we discuss the “how” of a digital twin. Firstly, we perform a literature review of how digital twins should be constructed for various use cases. Secondly, we discuss how a digital twin should be designed for CIET for its use case from a conceptual standpoint. With that, we move on to the methods section outlining specifics of how the digital twin is constructed for this work. We then review how the constructed digital twin performs in the results section.

Chapter 2

Literature Review

2.1 Motivation for Constructing a Digital Twin of the Compact Integral Effects Test (CIET)

Purpose of CIET

To understand why CIET was constructed, we need to understand the motivation behind the construction of non-nuclear testbeds in general. The construction of non-nuclear testbeds such as CIET was meant to help and expedite reactor development. This is because engineers can simulate the thermal hydraulics of the system without the added complexity of a highly radioactive environment. This would enable studies to be performed without the need for governmental licenses to operate a research reactor or the need to consider criticality or radiation safety.

While nuclear reactor thermal hydraulics phenomena is easier to study without the added complexity of ongoing fission reactions, they can still prove challenging to study experimentally. In the case of designing Fluoride Salt Cooled High Temperature Reactors (FHRs), a Gen IV reactor, the coolant used in the primary loop is FLiBe (Li_2BeF_4) [N. Zweibaum et al., 2015]. The use of FLiBe salt in experimental work can be difficult because of these two reasons. Firstly, FLiBe must be heated in the range from 500 to 700 °C [R. Scarlat et al., 2014] in order to replicate conditions within a typical FHR. Secondly, FLiBe is toxic due to the beryllium in the salt [R. Scarlat et al., 2014]. Such complications can make even a non-nuclear thermal hydraulic testbed expensive and complicated for a molten salt reactor (MSR) such as the FHR.

To further expedite the development of FHR without using FLiBe in the early development stages, scaled experiments such as the Compact Integral Effects Test (CIET) have been used to simulate thermal hydraulics of the FHR without using FLiBe directly [N. Zweibaum et al., 2015]. Integral Effects Tests (IETs) such as CIET are important in early development stages of FHRs because it provides data for experimental validation of system simulations and system level thermal hydraulic code such as RELAP5 [N. Zweibaum et al., 2015; Nicolas

Zweibaum, 2015].

CIET in particular is designed to emulate the scaled thermal hydraulics behaviour of a Mark 1 pebble bed fluoride salt cooled high temperature reactor (Mk1 PB-FHR) under decay heat conditions (10% power) [Nicolas Zweibaum, 2015]. This is done by using Dowtherm A or Therminol VP-1 to match the Prandtl number (Pr), Reynolds number (Re) and Grashof number (Gr) at operating temperatures of FHR [N. Zweibaum et al., 2015]. This enables testing without the safety concerns of Beryllium within FLiBe and also testing at significantly lower temperatures (80 to 120 °C) as compared to FHR operating temperatures of 600 °C. While heat transfer effects such as radiation would not be accounted for in CIET [L. Liu et al., 2020], the resulting distortions are generally small except for a specific set of geometric configurations [Abou Dbai, R. O. Scarlat, and Trujillo, 2020]. These distortions can also be quantified and accounted for [Johnson, 2022] so that the advantages far outweigh the cost during the early stages of development. Leveraging such advantages, CIET can be used to validate FLiBe system simulation models much quicker than using a non nuclear molten salt thermal hydraulics system. The advantages provided by CIET are so valuable that several simulation models of CIET have been constructed in literature. For example, CIET has been used to validate RELAP5 [N. Zweibaum et al., 2015], Transform [Per F. Peterson and Greenwood, 2019] and System Analysis Module (SAM) [Zou, R. Hu, and Charpentier, 2019] simulations. The data from these models can then be used to inform development decisions for the FHR in the early stages.

To help the reader visualise what CIET looks like, a new nodalised diagram of CIET based on information from the SAM model [Zou, R. Hu, and Charpentier, 2019], process and instrumentation diagram (P&ID) [N. Zweibaum et al., 2015] and the early RELAP nodalised model is shown in figure 2.1:

Compact Integral Effects Test (CIET) Nodalised Model Diagram



Figure 2.1: Nodalised Model of CIET without Bypass Branch

Some Difficulties in using CIET

Despite these advantages discussed, even a scaled IET facility such as CIET can be mildly inconvenient to work with at times. While conducting of CIET experiments in the University of California, Berkeley, safety requirements mandate that two people operate CIET at any one time in case of a hot oil spill. This means that anyone who wants to use CIET must ensure that a colleague is also available to operate CIET. Consequently, one must ensure that schedules are planned in advance, and that experiments must be cancelled if either one of the persons encounters some form of contingency. Furthermore, CIET takes time to initialise valve alignments and to heat up to reach the desired initial state for the experiment. Moreover, as with any experiment, things do not always go as planned. One common issue is that CIET’s control system, the Advanced Reactor Control and Operations (ARCO) facility [Poresky et al., 2022], often had problematic startup processes due to software bugs within Labview and other systems. This often interfered with experimental progress and took much time to debug. Some of these bugs were challenging to trace because of data races, and were therefore difficult to replicate and eliminate. This would often slow down experimental progress.

Benefits of a Digital Twin for CIET

One option for speeding up experimental progress is with the use of a plant simulator for CIET. In essence, we are constructing a digital twin for CIET. Proper use of simulation tools such digital twins can effectively reduce the number of test runs required to obtain the same results, thus saving time. Nevertheless, running a reduced number of test runs may not even be possible under extreme circumstances such as the COVID-19 pandemic. Hence, developing a digital twin became a necessity.

It was conceived that perhaps by constructing a full digital twin which emulates the thermal hydraulics and transient behaviour of CIET in real time and on demand, research could be carried out on the digital twin instead of the physical asset provided the digital twin’s thermal hydraulics were validated. This would then enable research to continue during extreme circumstances such as the COVID-19 pandemic. Even though the COVID-19 pandemic’s effects have subsided at the time of this paper’s publication, the work of constructing the digital twin is still valuable. The more we work on this digital twin, the more we can explore its potential value in the post pandemic world.

If the user needed to perform experiments to obtain validation data from CIET, experimental progress could potentially be sped up by prototyping and rehearsing on experimental plans on the digital twin before moving onto the physical experiment. Thus, potential bugs and issues with physical operation of the system could be solved separately from bugs or other teething issues inherent in the experimental procedure, thus saving time and speeding up research progress. This workflow could be applied not just for obtaining data for validation studies, but for any study being performed on CIET. Additionally, a digital twin of CIET would still be able to perform most simulation studies that a system code such

as SAM or RELAP would be able to perform subject to the accuracy and fidelity of the physical models used within the digital twin.

Moreover, a digital twin of CIET could be used as a plant monitoring system for fault detection or online health monitoring studies, [Poresky, 2019]. It could also be used in remote operation studies and operator training [Poresky et al., 2022] and perhaps to prototype other ways digital twins could be integrated into future nuclear power plants. Some of these applications, such as online health monitoring, require the digital twin to be configured for two way real time data exchange with CIET. It is unlikely that one can simply adapt existing simulation software for this purpose, and therefore this provides additional motivation for developing a digital twin of CIET with source code freely available for viewing. There are perhaps many more benefits that can be reaped from writing a digital twin, but it would suffice to say that there is sufficient motivation for developing one with or without the pandemic.

2.2 Digital Twin: Origin and Definition

Now that we have established that developing a digital twin for CIET is indeed desirable, we now want to explore how digital twins originated and what they are exactly according to literature.

Origin

To understand the origin of digital twins, we should first understand what “twinning” is and some of the well known examples of twinning. For this, we can look at a typical definition of a digital twin. A digital twin can be defined as a digital representation of a machine, device or asset which tracks the characteristics, properties, conditions and behaviours of the system by means of models, information and data [Sleiti, Kapat, and Vesely, 2022]¹. Twinning in general therefore means to use a physical or digital method to track the characteristics, properties, conditions or behaviours of the asset one is trying to twin.

An early example of the idea of twinning came from NASA where two identical space vehicles were built during the Apollo space program. [M. Liu et al., 2021; Allen, 2021; Akbarian, Fitzgerald, and Kihl, 2020] The space vehicle on earth became known as a twin of the vehicle which was launched into space. This example is most frequently cited in literature [P. Li and H, 2022; Allen, 2021], but the idea of constructing exact replicas or prototypes of physical assets can be traced back to an even earlier time. In the development of Nautilus nuclear submarine, the S1W reactor was built in 1953 as a 1:1 copy of the submarine reactor [Marguet, 2022] as part of prototyping work. This reactor was placed in a portion of a submarine hull for more realism [Marguet, 2022]. While the term “twin” was not explicitly

¹We should again note that there are several competing definitions in literature. We will explore that in the definitions section of a digital twin

coined for the S1W reactor, it could also qualify as a onshore twin of the actual reactor in the Nautilus.

In both the naval and space examples, these prototypes or twins were extensively used for development of the actual physical assets in space or at sea. During space operations for the Apollo space program in particular, one vehicle would be on earth while the other in space. The vehicle on earth would be subjected to conditions thought to be found in space, as closely as possible, so it would mirror the flight conditions in space.

The physical twin would prove valuable during the Apollo 13 crisis. In that scenario, of April 1970, one of the crew’s oxygen tanks exploded and the life of the crew was at risk [Augustine, 2020]. The engineers on the ground needed a way to devise plans to bring the crew back safely before they were severely oxygen deprived. For example, when the crew was running out of oxygen, engineers on the ground were able to use the physical twin on earth to experiment on how to improvise carbon dioxide scrubbers using lithium hydroxide canisters and only other resources available in the twin [Leblanc, 1970]. Using this knowledge, they then instructed the crew on how to make such an adapter within the Apollo 13 ship in space. In this and several other instances, the physical twin helped the crew of the Apollo 13 survive [Augustine, 2020].

In non-emergency situations, for example prior to flight, the twin of the space vehicle on earth was used extensively for flight preparations [Boschert and Rosen, 2016]. Besides flight preparations, physical twins such as the iron bird were used to optimise and validate aircraft systems. In such a setup, new aircraft components are tested on the ground individually and to see how they interact with other in flight systems. The twin would not fly, but the iron bird can facilitate simulated flight on the ground to put these components to the test and discover incompatibilities between the components [Jones et al., 2020].

Compared to the NASA Apollo twins, the iron bird has in flight conditions simulated via computer. This went to show that there were aspects of the physical twin that could be simulated digitally. In the case of the Iron Bird, the engines, control systems, hydraulics and avionics would undergo simulated flight conditions. These flight conditions, for example airspeed and temperature would be simulated via computer. The utility of such testing comes when the designers need to iron out bugs for in flight software, and other issues in components interacting with each other. [Jones et al., 2020]

To take the concept of a physical twin to a fully digital twin, we take a look once again at NASA. The popularisation of the term “digital twins” can be attributed to NASA’s Grieves and Vickers in 2002 [Grieves and Vickers, 2017]. Of course, it must be noted that while popular use of the word “digital twin” can be attributed to Grieves and Vickers, the concept pre-dates the use of the word. For example, when using models for fault detection, the words “fault diagnostics” and “digital twin” can be used [Poresky, 2019]. Model-based fault detection, however, was established well before 2002 [Isermann and Balle, 1997].

This digital twin would be similar in function to the physical twin concept in NASA during the Apollo missions, in that a copy that mirrors the conditions of the process or product in real time would be available to the end user. So, in addition to simulating the conditions faced by the product or hardware such as in the iron bird, digital twins take this

one step further by simulating not only the conditions faced by the physical hardware, but the hardware itself.

Definitions in Literature

In the introduction, I dove straight into the benefits and motivations behind developing a digital twin without definitively exploring what a digital twin is. This was deliberate because there are many differing definitions of what constitutes a digital twin [Sleiti, Kapat, and Vesely, 2022; Kochunas and Huan, 2021; Van der Valk et al., 2020]. I wished not to bore the reader with an elaborate section exploring definitions of digital twins too early on until we had established why the reader should be interested in digital twins in the first place. I hope by now we have established why digital twins are interesting for engineering work. With that, we shall now discuss how there is disagreement within literature as to the proper definition of digital twins.

One definition necessitates that a digital twin cannot simply be a digital representation of a physical machine or entity, but must strictly receive and transmit data to the physical entity in real time [Kochunas and Huan, 2021]. The digital representation that does not interact with a physical asset was called a “digital model”, whereas a digital representation that only takes in information from the physical asset and does not give real time data to the physical asset can be called a “digital shadow” [Kochunas and Huan, 2021].

I chose not to adopt this naming convention because the term “digital model”, “digital shadow” and “digital twin” could sound very similar to the average reader. This could be very confusing for readers who are trying to figure out what a digital twin actually is. Moreover, defining that a digital twin needs two-way real-time information exchange with the physical asset would mean that certain uses of the word “digital twin” in literature would become obsolete. Also, it would mean that digital twins are no longer applicable for certain use cases such as Predictive Maintenance. While defining digital twins in such a narrow way has proved more popular in literature [Van der Valk et al., 2020], narrowing down the definition does not solve the inconsistent use of the word “digital twin” already existing in literature. Nevertheless, the effort to highlight the nuances of how digital twins interact with the physical entity is commendable because several definitions of the word digital twin appear in literature.

What could then be done is that instead of using the words model, shadow and twin to describe said nuances, we could perhaps say that a digital twin that does not interact with the said physical entity is a Type I digital twin. Other terms in literature that perhaps fit into the Type I digital twin category include “digital siblings” [Rasheed, San, and Kvamsdal, 2020] or “digital twin prototypes” [Enders and Hoßbach, 2019; Grieves and Vickers, 2017]. A digital twin that only receives data or what Kochunas describes as a digital shadow could be a Type II digital twin and a digital twin that interacts with the physical asset in a two way real time data exchange for model-based control could be called a Type III digital twin.

This would ensure that the word “digital twin” would encompass the many definitions in literature. However, the language of using a term such as Type I, Type II and Type III,

would highlight to users that there are technical nuances to pay attention to. This naming technique is used in classifying supernovae such as the Type Ia and Type II supernova in astrophysics, and nucleophilic substitution mechanisms S_N1 and S_N2 described in organic chemistry. In the case of astrophysics, this form of taxonomy suggests to the reader that no matter how a supernova is formed, a supernova still refers to an extremely large explosion of a star. Using this definition would mean that the word supernova would still encompass all the forms of star explosions. However, using the technical terms Type Ia, Type II and so on would alert the user that there are technical nuances to pay attention to ².

We can look at table 2.1 which summarises how Type I, Type II and Type III digital twins differ in terms of their capabilities and how they relate to some terms which already exist in literature.

²For the interested reader, I have written a section in the appendix (Page 249) which provides further justification as to why I chose this naming convention

Digital Twin Type	Real-Time Flow of Data	Similar Terms used in Literature	Applications
Type I	None	Digital Sibling [Rasheed, San, and Kvamsdal, 2020], Digital Thread [Boschert and Rosen, 2016] Digital Model [Kochunas and Huan, 2021], Digital Twin Prototype [Enders and Hoßbach, 2019; Grieves and Vickers, 2017]	Scenario and Risk Assessment [Rasheed, San, and Kvamsdal, 2020], Penetration Testing [Eckhart and Ekelhart, 2018]
Type II	From Physical Asset	Digital Shadow [Kochunas and Huan, 2021] Digital Angel [Van der Valk et al., 2020] Digital Twin Instance [Enders and Hoßbach, 2019; Grieves and Vickers, 2017]	Fault Detection [Jain et al., 2019] Predictive Maintenance
Type III	To and From Physical Asset	Digital Twin [Kochunas and Huan, 2021] Digital Twin Instance [Enders and Hoßbach, 2019; Grieves and Vickers, 2017]	Model-Based Control [Enders and Hoßbach, 2019; Semeraro et al., 2021]

Table 2.1: Digital Twins and their Types

Typing Digital Twins by Real Time Data Flows

The key factor that distinguishes Type I, Type II and Type III digital twins is the flow of real-time information. Figure 2.2 summarises how real-time data flows between the user

(operator), physical process or asset and the digital twin for each type of digital twin. This will help the reader distinguish between Type I, Type II and Type III digital twins. In Figure 2.2, the arrows denote the flow of real-time information whereas the ticks and crosses denote whether the flow of real-time information in each setup is necessary for the specific type of digital twin.

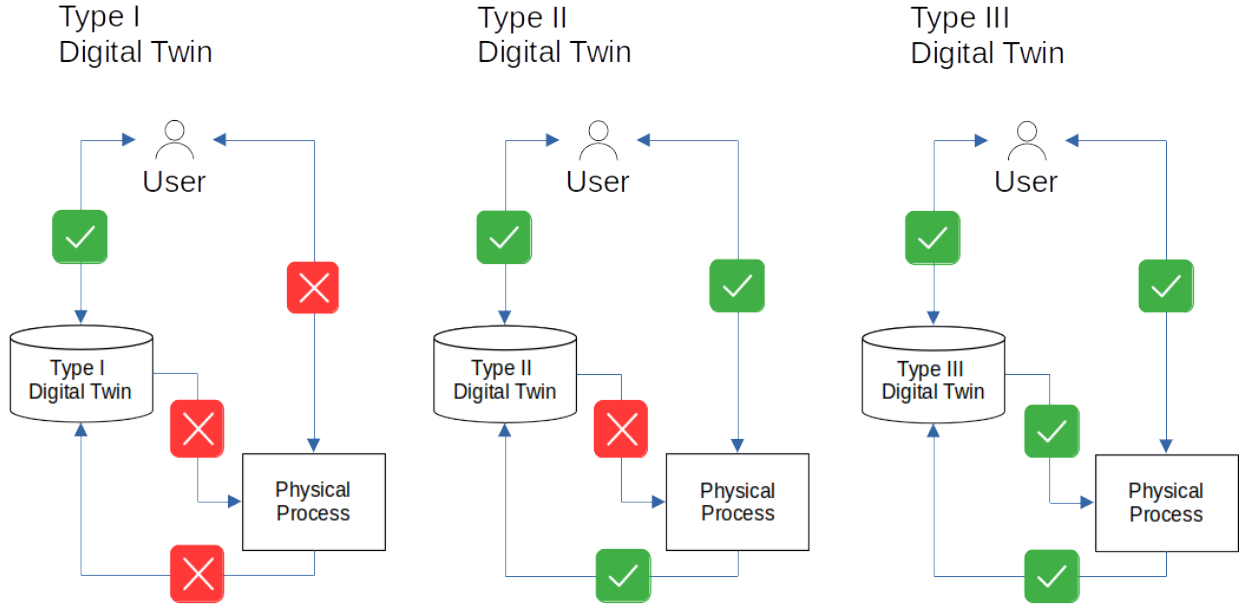


Figure 2.2: Real-Time Data Flow Comparison for each Type of Digital Twin

We wish to classify digital twins by how real-time data flows because real-time data flows determine the use cases each type of digital twin can offer to the end user. After all, why would one be interested in digital twins in the first place other than the fact that they provide the user with some specific utility? These different use cases would impose different real-time data flow requirements on the digital twin. These real-time data flow requirements would in turn have translate into real world time and resource cost implications for the end user. Therefore, it is better to classify digital twins by real time data flows rather than other metrics so that using this typing system, users can perhaps better gauge the cost and

benefits of implementing digital twin technology into their products. To illustrate this, let us examine the various Digital Twins ³ shown in Figure 2.2

The Type I digital twin only requires real-time information flow between the user and itself. This makes it ideal for scenarios where we do not want or need the plant operator to interact with the physical process such as plant simulators. Type I Digital Twins are possibly the cheapest to develop because it does not even require a physical asset to operate and also because there is no need for the Digital Twin to be kept in sync with the physical asset.

The Type II Digital Twin requires a two way real-time data flow between the user and itself, and also requires real-time information from the physical process to be fed into itself. However, the digital twin does not bypass the user and feed real-time data back into the physical process. Instead, the user reads data provided by the digital twin based on its models, and manually makes decisions based on the provided data. This makes Type II Digital Twins ideal for plant diagnostics and online health monitoring. Type II Digital Twins are relatively more extensive to develop as they are required to be synchronised⁴ with the physical asset. This means that time will be required to test the synchronisation capabilities of the Digital Twin when the physical asset is online. Using the physical asset to test and develop a Type II Digital Twin may mean that the physical asset will be less productive or non productive. This may translate into extra maintenance downtime for some plants if an end user wishes to incorporate a Type II Digital Twin into an existing plant.

Type III Digital Twins require two way real-time data flows between the user and itself, as well as the user and the physical process. This means it may receive information about a physical process which is telling the Digital Twin that there may be some kind of plant fault. The Digital Twin would then make a decision using this information to remedy the fault without user intervention. Type III Digital Twins may be used to help the end user control or even autonomously operate the plant or physical process, thus potentially lowering manpower requirements and costs. However, they may be the most costly to develop compared to Type I and Type II Digital Twins because extended time is not only required to test the synchronisation capabilities of the Type II Digital Twin, but also the control logic and autonomous operation logic. If the Digital Twin is being used to automate plant operations, then more extensive testing is required so that the Digital Twin does not run into a code bug and shutdown or destroy the plant. Thus, Type III Digital Twins could be the most costly to test and develop out of the three types discussed here.

Thus, we can see how typing digital twins by how data flows to and from the physical asset is useful to the end user in assessing the potential cost and benefit of using each type of digital twin.

³Benefits of Digital Twins will only be explored in the next literature review section, the discussion here only serves to illustrate why the typing convention is useful and how it can help the end user estimate the costs and benefits of developing and using digital twins of various types

⁴We will briefly explore synchronisation for digital twins when we explore how to build Type II and Type III digital twins

Concluding Remarks on the Typing System for Digital Twins

While I do not seek to exhaustively develop a full taxonomy on how digital twins are to be named in this thesis, I would rather the reader not be confused with more new technical terms than necessary and use the term “twin” in an intuitive way as far as possible. While defining nomenclature of what a digital twin is not the goal of this thesis, it is nevertheless not a trivial part. In fact, digital twin nomenclature is so important in the medical field that an entire article was written about the implications of digital twin nomenclature in the journal of medical ethics [Lupton, 2021]. While nomenclature need not be an ethical issue here, it is important to ensure that definitions are clear so as not to confuse the reader.

If one were to consider the terminology used by Kochunas [Kochunas and Huan, 2021], digital twins in this paper are equivalent to digital representation by definition in that review paper. Or in the naming convention I had suggested, Type I, Type II and Type III digital twins are all classified as digital twins.

2.3 Utility of Digital Twins

Now that we properly defined how the word “digital twin” is to be used in this thesis, we next discuss the benefits of digital twins according to their types their respective use cases. With this context, we shall re-examine the research requirements for developing a digital twin for CIET and thereby discuss the appropriate types which fulfil these research needs.

Benefits of Type I Digital Twins

Type I Digital Twins track the physical processes of a plant using digital models in a manner similar to plant simulators. These simulators are advantageous compared to using a physical asset because more physical resources need to be used to construct the physical asset or even a physical twin of this physical asset. For a physical twin, parts need to be designed, manufactured and then tested before data can be given to the designer. For a Type I Digital Twin however, we can skip the procurement of materials and obtain the benefits early on at lower cost.

Prototyping and What if Analysis

One such benefit for a Type I Digital Twin is its use as a prototyping tool. A Type I Digital Twin only needs a powerful enough computer and the right software to operate. Once a Type I Digital Twin is operational, we could use it to test potential product designs right after the model is constructed. This could be used for designing new plants or for retrofitting existing plants with new hardware or software. We could also use the Type I Digital Twin to predict how a plant or process might behave under an accident scenario. This is sometimes known as a what-if analysis because we are exploring the question “What happens if the process is subject to an adverse condition?”. What-if analysis is not limited to adverse conditions

however. The what-if analysis in weather forecasting seeks to predict local weather patterns using modelling tools and atmospheric sensor data. In the case of weather forecasting, we are asking “What if the status quo weather conditions persist?”. These predicted weather conditions are not necessarily adverse, but we still ask and answer a “what-if” question. In fact, prototyping often includes a series “what-if” analyses because it involves iterative design, which is by nature a “what-if” analysis. This is because we are asking “What if we designed the process in such and such a manner?”. Prototyping is therefore closely related to what-if analysis and therefore discussed together with what-if analysis here. However, it is still necessary to distinguish prototyping from what-if analysis because prototyping usually occurs in the design stage of the plant or process. In contrast, what-if analysis can occur at any stage of the plant’s life cycle. In any case, both of these are valuable use cases. The Type I Digital Twin allows us to perform prototyping, risk assessment and what-if analysis without interrupting safe operation of the physical asset [Rasheed, San, and Kvamsdal, 2020].

One example of Type I Digital Twins being used in this use case is in the aspect of cybersecurity [Eckhart and Ekelhart, 2018]. A Type I Digital Twin based on the physical asset but operating independent of the physical system can be subject to penetration testing without affecting the actual physical manufacturing process. Countermeasures developed using data from the penetration testing can be tested on the Type I Digital Twin before being put into production in the physical asset [Eckhart and Ekelhart, 2018].

It is evident that each cyber attack, simulated in penetration testing, would produce a specific behavioural pattern or fault within the physical asset. We could call such faults cyber-induced faults, and each behavioural pattern produced by these faults could be called fault signatures. If we wanted to use information generated from Type I Digital Twins in health monitoring, we could also compile data generated from penetration testing of a Type I Digital Twin into a cyber-induced fault signature library or database. In this regard, Type I Digital Twins could also help us detect cyber-induced faults for industrial systems including nuclear power plants. We can then see that Type I Digital Twins can support fault detection work in general by building up fault signature libraries for use in a Type II Digital Twin calibrated for use in fault detection or fault identification.

Another example of using Type I Digital Twins in what-if analysis could be in the area of simulated or virtual maintenance procedures [Zipper and Diedrich, 2019]. In such a use case, operators can practice maintenance procedures in the digital twin before proceeding to perform the maintenance on the physical system. Of course, operator training in general goes far beyond what-if analysis. It therefore deserves its own category which we will see in the next section. From the viewpoint of drafting maintenance procedures, it is useful to carry maintenance procedures out first in the digital twin and use information from early trial runs to refine these procedures before instructing operators to carry them out. In the former use case, this would fall under operator training. In the latter use case, however, we are essentially asking the question “What if we carried out a certain set of maintenance procedures when the plant is in some certain state?”. This would fall under what-if analysis as well. This benefit applies specifically to Type I Digital Twins, for example training simulators, since there need not be a physical entity for the Digital Twin to interact with

in real-time. This allows the component designs to be tested at a more preliminary design stage, and it could help iron out potential design flaws much earlier in the design process, possibly saving money and time so that fewer physical prototypes need to be built.

Operator Training

Type I Digital Twins have classically been used in industry as well for real time nuclear power plant simulation. For example GSE solutions as of September 2022, offers fully fledged real time plant simulator products for operator training [Kochunas and Huan, 2021; GSESolutions, 2016].

Other examples of Type I Digital Twins in industry [Kochunas and Huan, 2021] could include proprietary plant simulators by vendors such as Western Services Corporation and GSE Solution’s Generic Pressurised Water Reactor (PWR) product as well as the Compact Nuclear Simulator developed by the Korea Atomic Energy Research Institute (KAERI) [Park et al., 2022]. These have existed for decades and goes show that Type I Digital Twins are actively being used in the nuclear industry for both operator training and nuclear reactor development.

Artificial Intelligence Training and Type II and Type III Digital Twin Development

In some cases, we may even use artificial intelligence to perform nuclear reactor startup and shutdown. To provide data for training artificial intelligence, plant simulators such as the Compact Nuclear Simulator (CNS) by KAERI have been used to train deep neural networks [Park et al., 2022]. In essence, the Type I Digital Twin was used to train artificial intelligence. To train and test such neural networks, multiple instances of the CNS were run simultaneously to support multi-threaded reinforcement learning [Park et al., 2022]. This would only be possible with plant simulators or Type I Digital Twins since multiple copies of the plant need to be running simultaneously in order to train and test artificial intelligence for operating nuclear power plants.

Why Type I Digital Twins cannot be replaced by Type II or Type III Digital Twins

For prototyping, what-if analysis, operator training and refining maintenance procedures, the use of a digital twin here necessitates decoupling from the physical asset because we do not want to impact the performance of the physical asset during the early stages of devising and testing new or updated maintenance procedures. This means we can only use type I digital twins. Type II and Type III digital twins are not suitable for this use case. Nevertheless, Type II and Type III Digital Twins have their own specific use cases as well.

Benefits of Type II Digital Twins

Type II Digital Twins track their physical counterparts in real-time. This often makes them more costly to develop compared to their Type I counterparts. To justify the extra cost of developing a Type II Digital Twin, we shall discuss some benefits that come from them especially compared to a Type I Digital Twin.

Mitigation of Emergencies Where Physical Asset is Inaccessible

One justification for developing a Type II Digital Twin is that during operations Type II digital twins could potentially be used to mitigate emergency situations similar to how Apollo 13 used physical twins to help the crew on board make informed decisions to survive. [Augustine, 2020; Leblanc, 1970] In this case, the physical twin on the ground was the twin of the physical asset in space. As information was transmitted from the crew to ground control, ground control was able to devise solutions to help the crew survive and send it back to the crew in space. This is a two-way data exchange between the physical twin and the physical asset. However, only the data transfer from the physical asset to the physical twin was in real-time. If a digital twin was to be used in a similar manner, it would be a Type II digital twin. Such a benefit is impossible with a Type I digital twin.

In the case of Apollo 13, the physical asset was inaccessible to ground control. This inaccessibility could extend to cases beyond space exploration as well. In the case of emergency response, the rescue team would want as much information as possible about the emergency situation in order to make the best decisions. The quality of these decisions would directly impact how many lives are saved.

For nuclear reactors, the physical asset is difficult to access due to the highly radioactive environment amongst other factors. Therefore, the only way to monitor it is with the use of sensors. However, operators may find it difficult to make sense of sensor data in an emergency. Hence, using a digital twin paired with artificial intelligence to help the operator diagnose faults and make suitable decisions during stressful emergency situations could help reduce damage caused by operator error. This has motivated studies in developing Type II Digital Twin assisted control systems such as the Nearly Autonomous Control and Management System (NAMAC) [Lin et al., 2022]. In this system, the NAMAC makes recommendations to the operator for loss-of-flow scenarios in a validated Experimental Breeder Reactor II (EBR-II) simulator [Lin et al., 2022].

Online Plant Health Monitoring and Predictive Maintenance

While mitigation of emergencies is an attractive proposition, we would like to prevent emergencies or faults from happening if possible. Type II Digital Twins can help us to prevent potential faults with the use of online health monitoring [Kochunas and Huan, 2021]. In normal plant operation, we may install sensors around the plant to monitor its operation and to check if any threatening anomalies occur. However, we may not be able to install sensors everywhere in the physical asset as the sensors themselves may impede operation of

the physical system. For example, one may not wish to insert temperature sensors into the electrolytes or electrodes of a battery because this would impede battery operation or, at best, drive up production costs. Without sensors within the battery, one can only guess the temperature of a battery internal with the use of models. Therefore, one might decide to use models, such as Type II Digital Twins, in general to predict data beyond the reach of sensors. I will refer to this as “beyond-sensor-data” from now on. Beyond-sensor-data can be used to monitor the health of a process by detecting anomalies or faults within the system. Once we have beyond-sensor-data, we can then take informed action to mitigate those faults either in real-time or after some time delay.

If we understand our plant or process sufficiently well, we might even choose to use data from our Type II Digital Twin to perform predictive maintenance on our plant [Rasheed, San, and Kvamsdal, 2020; Fotland, Haskins, and Rølvåg, 2020; Kochunas and Huan, 2021]. Predictive maintenance can be defined as a condition driven maintenance program [Mobley, 2002]. We can best understand predictive maintenance in contrast to preventative maintenance. With preventative maintenance, maintenance on a physical system is usually done periodically based on accumulated hours and intensity of use. For example, a car may need an oil change when it accumulates a certain amount of mileage. Preventative maintenance is a more conventional approach to maintaining a physical system. However, the frequency of preventative maintenance can often be higher than what is actually required, and performing unnecessary maintenance can accelerate the ageing of components. While this may be the case, periodic preventative maintenance is done so that the physical asset can be operated in a safe manner. However, if we want to reduce the frequency of maintenance in a safe manner, we would need to understand how the physical asset undergoes wear and tear using some simulation model. Here is where predictive maintenance comes in. In predictive maintenance, we could use some model to estimate the wear and tear of a physical asset. In other words, we are estimating the health or condition of a plant or physical asset. When the estimated wear and tear of a physical asset exceeds some threshold, we stop the operation of the physical asset and perform maintenance. The overall effect of using predictive maintenance in comparison to preventative maintenance is a higher degree of operation time and increased component life. We can use Type II Digital Twins to model and track a physical process for online health monitoring purposes. Doing so will help furnish us with information for a safe and effective predictive maintenance schedule.

In literature, one example of online health monitoring using digital models is real time model based thermal fault diagnosis in battery systems [Dey, Biron, et al., 2016; Dey, Perez, and Moura, 2017]. In such a case, the goal is to detect thermal runaways early to prevent battery degradation or even battery fires [Dey, Biron, et al., 2016]. While these were not described as digital twins within the reference, it does show that real-time models in general can be used for online plant health monitoring and fault detection. Hence, these real-time models could just as well be replaced with Type II Digital Twins calibrated to perform the same function.

For predictive maintenance using a Type II Digital Twin ⁵, one example in literature is where we carry it out on cranes in offshore vessels [Fotland, Haskins, and Rølvåg, 2020]. Offshore cranes will experience stress and fatigue during lifting operations. To detect if the crane is experiencing unsafe levels of stress, we could in theory install sensors within the crane or in a physical twin of the crane. However, similar to the case of batteries, sensors are limited in their capabilities to measure stress along the entire structure of the crane. Moreover, having physical twins of the cranes would be quite costly to construct and operate. The alternative then was to construct a digital twin of the crane and predict the accumulated stresses in each part of the crane so that the operators can better predict if or when parts of the crane would fail. This digital twin can then be used as a basis for risk analysis when planning for operations [Fotland, Haskins, and Rølvåg, 2020].

Similar to the example of the cranes, a Type II Digital Twin can simulate conditions in places where it is difficult to install sensors due to the high pressure, high temperature or high radiation environment [Nguyen et al., 2022]. One example of this was in the use of high pressure feedwater systems where a physics based approach was used to simulate and monitor the thermal hydraulic aspects of such a system. Real-time plant data provided by a utility collaborator was then used to predict faults and performance degradations of components and sensors in nuclear power plants. The data derived from the digital twins was then compared against the data from the sensors. Some of the possible faults found through this method were fouling in feedwater heaters and sensor faults for total feedwater flowrate. [Nguyen et al., 2022] A second example is where machine learning was used to perform reduced order modelling of neutronics for the HPR1000 reactor core so that neutron flux fields can be reconstructed in real time for online health monitoring [Gong, Cheng, et al., 2022; Gong, Zhu, et al., 2023].

These examples go to show that Type II Digital Twins are valuable for use online plant health monitoring and predictive maintenance.

Benefits of Type III Digital Twins

Type III Digital Twins require two way real-time data flows between the physical asset and itself. Type III Digital Twins would receive sensor data from the physical asset and process that data into useful information for the operator to make decisions. This would mean that most benefits which apply to the Type II Digital Twin also apply to the Type III Digital Twin. This includes but is not necessarily limited to online plant health monitoring and predictive maintenance. Moreover, Type III Digital Twins can go even further by making the plant control decisions without operator intervention [Kochunas and Huan, 2021; Varé and Morilhat, 2020]. Automatically making decisions for plant control without operator intervention is the key feature that separates Type III Digital Twins from their Type II counterparts.

⁵In Fotland’s paper, the Digital Twin was defined as “a digital copy of a physical asset, collecting real-time data from the asset and deriving information not being measured directly in the hardware” [Fotland, Haskins, and Rølvåg, 2020]. Using our convention, this is a Type II Digital Twin

Fault Tolerant Control

One possible way that a Type III Digital Twin makes such automatic decisions is when it uses its own modelling data of a plant’s process variables in order to perform fault tolerant control. In the example of batteries, the goal might be to develop an algorithm which minimises the effect of thermal faults on battery longevity [Dey, Biron, et al., 2016]. Mitigating the effects of faults, such as thermal faults in batteries, via controllers is what fault tolerant control is about. Fault tolerant control is one way faults can be mitigated as plants are online ⁶.

This is also an area of research in the nuclear industry as researchers have built a digital twin of the Micro Agile Non-Nuclear Experimental Test Bed (MAGNET) [Idaho National Lab, 2022; Turner, 2020]. While the experiment is non-nuclear in nature, it is meant to support nuclear micro-reactor development [Turner, 2020]. This digital twin successfully predicted the future temperatures of a heat pipe when it was trending towards an unfavourable threshold temperature. This was an example of a thermal fault. Additionally, it was able to autonomously control the temperature of the heat pipe and reduce it to a favourable level [Idaho National Lab, 2022]. Since the digital twin replicated the behaviour of MAGNET and was able to control some of the temperatures within MAGNET autonomously, we can say this is a Type III Digital Twin. This shows that Type III Digital Twins are being researched for automated fault tolerant control within nuclear reactors.

Plant Automation

In addition to fault tolerant control, we can extend the automated control of an industrial plant or nuclear power plant to operations such as startup and shutdown with the use of Type III Digital Twins. Nuclear Power Plant automation using artificial intelligence has already been explored in literature [Lu et al., 2020]. For example, KAERI’s CNS has been used to train a deep neural network for automated control in the heat-up mode of a nuclear power plant [Park et al., 2022]. Such artificial intelligence based automation can certainly be enhanced by online plant health monitoring based on information from a Type II Digital Twin. In fact, if we were to pair a Type II Digital Twin with artificial intelligence (AI) capable of automatic plant control, we would then have an AI Powered Type III Digital Twin. While, AI powered Type III Digital Twins for fully autonomous plant operation in nuclear power plants have scarcely been explored, this idea has been explored for robotic arms [Groshev et al., 2021] and could potentially be explored for other industries such as nuclear power plants. Of course, AI powered Type III Digital Twins would have certain security concerns which are out of scope of this discussion. Nevertheless, the use of Type III Digital Twins for autonomous operation research could be a reason for developing Type III Digital Twins as well.

⁶Fault Tolerant Control is similar but not equal to Model Predictive Control [Jones et al., 2020]. Interested readers can read about some differences between Model Predictive Control and Digital Twins in the Appendix on Page 251

Applications and Requirements for a Digital Twin of CIET

Having known that developing Type I, Type II and Type III Digital Twins each have their advantages, we now want to revisit the motivation and use cases for building a digital twin for CIET. The requirements for the Digital Twin would then depend directly to some of the applications CIET was used for.

How Digital Twins can Benefit CIET

CIET has been used for fault detection studies, plant monitoring studies and has been intended for use in remote operation studies [Poresky, 2019; Poresky et al., 2022]. A Type II or III Digital Twin would be extremely helpful in facilitating and conducting such studies. Therefore, developing a Type II or III Digital Twin is desirable for CIET. However, a Type II or III Digital Twin would be quite difficult to construct from scratch. Therefore, we wish to start development on a Type II or III Digital Twin by first constructing a Type I Digital Twin, which is in essence a plant simulator. Constructing a Type I Digital Twin would not only enable studies for remote and autonomous operation, but also serve as a digital surrogate for when CIET is unavailable for experimental use eg. during the COVID-19 pandemic. This could fall under the scenario testing use case of digital twins i.e. what-if analysis. Additionally, a Type I Digital Twin could also help researchers speed up the iteration process when they are dealing with teething issues and bugs for their experiments. Once constructed, the Type I Digital Twin could then be used as a stepping stone to construct a Type II or Type III Digital Twin. Therefore, when constructing a Type I Digital Twin for CIET, we may want to keep some of the prerequisites for constructing Type II and Type III Digital Twins in mind.

In other words, we want to develop a Type I Digital Twin as a real-time simulator, experimental surrogate and a stepping stone for Type II and Type III Digital Twins in future.

Requirements for a Digital Twin for CIET

Since we wish to develop a Type I Digital Twin as a stepping stone for developing Type II and Type III Digital Twins, we need to consider the operational constraints and requirements of Type II or Type III Digital Twins. For example, a Type III Digital Twin of CIET for remote operation studies needs to have (1) its models regularly synchronised with CIET. It should also have (2) capabilities to connect with a server remotely similar to what was done in previous studies [Poresky et al., 2022]. Furthermore, (3) all this must happen in at near real-time speed. Therefore, when designing the Type I Digital Twin for CIET, I am designing it with these requirements in mind. To understand what requirements are needed, I want to first explore how a Type II or Type III Digital Twin is constructed, and based on this, make decisions on how the Type I Digital Twin is to be constructed. Since we focus solely on developing a Type I Digital Twin of CIET for the bulk of this thesis, I would also like to clarify the use of the generic term “digital twin”. From now on, the generic term

“digital twin” would normally mean a Type I Digital Twin or the superset of Type I, Type II and Type III Digital Twins based on context. Where ambiguities exist, the Type of the digital twin will be specified for the reader.

2.4 Process of Constructing Digital Twins

In this section, we wish to explore “how” as in how to construct a Digital Twin for CIET. This means we explore some of the requirements of a Digital Twin that we wish to build for CIET. After that, we explore some of the available software libraries we could use to construct the appropriate Digital Twin for CIET. Here, we are constructing a Type I Digital Twin keeping in mind that it should be easily modifiable in such a way that sections of its code could be re-used for the eventual Type II and Type III Digital Twin we may wish to construct in future.

The reader should also note that there is no consensus on a generic way to construct Digital Twins [Tao et al., 2018]. Each Digital Twin must be custom built to fit the system and the use cases presented. Therefore, special care and attention needs to be given for each Digital Twin constructed, including Digital Twins of CIET, whether they are Type I, Type II or Type III.

Requirements for Type II and Type III Digital Twins for CIET

Since the longer term end goal is a Type II or Type III Digital Twin, we want to begin with the end in mind and explore requirements for Type II and Type III Digital Twins briefly. This is so that we can design a Type I Digital Twin for this work in such a way that the parts of its code can be reused for Type II or Type III Digital Twins. As a full literature review into constructing Type II and Type III Digital Twins will be quite lengthy, we will only briefly discuss Type II and Type III Digital Twins here. Interested readers can look in the Appendix (Page 252) for a more detailed discussion.

As mentioned before, the requirements of a Type II or Type III Digital Twin used for fault detection, autonomous operation and remote operation studies are (1) synchronisation capabilities with CIET, the physical asset, (2) capability for remote operation via a server-client interface and (3) real-time calculation capabilities. We shall explore each requirement briefly and assess their implications on how we construct the Type I Digital Twin for this work.

Synchronisation Capabilities

Type II and Type III Digital Twins need to be synchronised with the physical asset they aim to model digitally. As such, the Type II or Type III Digital Twin may require some means of receiving and sending data to the physical asset in real-time. Therefore, a Type I Digital Twin should be built with the potential to communicate information with the physical asset.

To start this process, we first need to find out what kind of data needs to be sent to and from the physical asset for Type II and Type III Digital Twin synchronisation. This in turn necessitates that we know the degree of synchronisation in turn depends on the use case of the Type II or Type III Digital Twin. Hence, we will need to discuss some specific use cases for Type II or Type III Digital Twin synchronisation.

On the Matter of State Synchronisation Terminology As we continue exploring specific use cases of digital twins ⁷, we note that synchronisation requirements, (both the degree and frequency) for the digital twin depend on greatly on its use case. Additionally, the degree to which digital twins can be synchronised to the state of their physical assets is limited on the instrumentation available to measure state parameters in the physical asset. Consideration of these requirements is important when designing digital twins for certain use cases. Therefore, we will want to discuss (1) the degree of state synchronisation, (2) the limitations of for state synchronisation and (3) the frequency required for state synchronisation.

Degree of State Synchronisation The term often used in literature to describe digital twin synchronisation is “state synchronisation”. In literature, this may mean a number of different things based on context. For example, if we have a fault detection study, state synchronisation might mean that we want the user inputs for the physical asset and the digital twin to be the same, but we want to measure the difference in process output between the physical asset and the digital twin to discover faults. In this case, the Digital Twin does not completely replicate the state of the plant, but only the inputs. In contrast, other digital twins may seek to replicate both user inputs and process outputs for the plants when they perform state synchronisation perhaps for process control purposes. Thus, we can see that state synchronisation may mean different things depending on the use case and context. Given the many nuances present within the subject of state synchronisation, we can see that using the term “state synchronisation” to describe all these nuances may not be sufficient. Therefore, I would like to make a distinction here for the sake of clarity by distinguishing between some terms I use here to describe the degree of synchronisation of digital twins. These terms are “state model synchronisation”, “state input synchronisation” and “state output synchronisation”. These terms are meant to help clarify what exactly we are synchronising.

State model synchronisation is somewhat similar to model validation steps used in simulation studies. To perform state model synchronisation, we still need to use the sensor data from the physical asset somehow to calibrate the digital twin. For example, we may use flowmeter data along with manometer data to calibrate the type I digital twin to use the correct pressure loss correlations. When building the RELAP5 model of CIET, flowmeter

⁷To avoid repetition of specifying Type II and Type III Digital Twins, I will use the generic term “digital twin”. In the context of synchronising digital twins, the term will mean both Type II and Type III Digital Twins. Type I Digital Twins are rarely because they do not need to be synchronised in real-time

data, heater input power data and thermocouple data from natural and forced circulation experiments were used to validate RELAP5 models of CIET [Nicolas Zweibaum, 2015]. Whereas data from the same sensors in frequency response tests in CIET were used to validate TRANSFORM models of CIET [Per F. Peterson and Greenwood, 2019]. In these cases, we use sensor data from past experiments to validate the simulation models built. Similarly, calibrating the models in a digital twin to match sensor data for time domain and frequency domain would be required to perform state model synchronisation. Of course, validation or state model synchronisation can only be as successful as the availability and accuracy of sensor data available.

State input synchronisation is best illustrated through an example of fault detection. For fault detection, we may periodically perform state model synchronisation for our digital twin to sensor data from our physical asset so that it reflects the healthy state of the plant as it ages. However, we may require that the input of the digital twins and the sensor measured input of the physical system stay in sync in order to allow ourselves to carry out real time fault detection. At the very least, it means that the digital twin inputs must be equal to the sensor measured inputs of the physical system in real-time. I deem this process a real-time state input synchronisation. This means that for real time fault detection, we need a Type II digital twin with periodic state model synchronisation and real-time state input synchronisation.

The term state output synchronisation means that the digital twin has its outputs⁸ synchronised with the measured sensor data from the physical asset. State synchronisation as used in literature could refer to a synchronisation of model, inputs, outputs or any combination of the above depending on context. If I use state synchronisation in this thesis, it would carry this context based meaning. Additionally, in literature, one may come across the term passive state replication [Akbarian, Fitzgerald, and Kihl, 2020]. This means that the stimulus experienced by the physical system would be replicated in the digital twin. This is closest to state input synchronisation which I use here. However, I use the latter term as it simply describes that the inputs of the digital twin are the same as the physical system. Of course, one could then go on to distinguish between user input and other stimuli experienced by a digital twin. For example, in the case of a heat exchanger, ambient temperature would be an input but not a user input. For such cases, I would not coin new terms but distinguish them as the discussion requires it.

However, if I intend to specify that model, input and outputs of the digital twin are all synchronised to the measured sensor data from the physical asset, I would deem this as full state synchronisation in this thesis.

“State Synchronisation” as used in this work can mean a combination of any of the above types of synchronisation.

Limitations of Synchronisation especially for Beyond Sensor Data The capability of a digital twin in performing state synchronisation is limited by the degree that

⁸There may be some ambiguity of what output may mean, discussion is in the appendix on page 252

sensor data is available. While, model outputs within our digital twin could predict process variables beyond the reach of sensors, in other words, beyond-sensor-data, we cannot measure beyond-sensor-data directly even though the physical asset does contain beyond sensor data. For example, actual neutron flux does exist throughout the core of a nuclear reactor, but we cannot measure neutron flux everywhere in the core for practical reasons. Therefore, beyond sensor data cannot be synchronised from the physical asset to the digital twin. The best we can do is to ensure that the models and sensor data used to infer beyond-sensor-data are as accurate as possible.

This means that we need to periodically perform state model synchronisation to calibrate our models. Also, we need to perform state input and state output synchronisation so that modelling outputs from the digital twin would match all sensor data from the physical asset. If this is done properly, we can assure ourselves that we have done our homework to predict beyond-sensor-data the best we can.

Synchronisation Frequency Additionally, I would also like to clarify terms used to describe the frequency of synchronisation, which is sometimes known in literature as the twinning rate [Jones et al., 2020]. The two terms I would use most frequently as qualitative descriptors of synchronisation frequency are the terms periodic and real-time. I would start off by saying that real-time may refer to different time scales depending on the timescales of the process. For example, a process involving nuclear fission might occur on timescales of nanoseconds or one shake (10 nanoseconds) [Lamarsh, Baratta, et al., 2001], but for heat transfer, processes, such as water heaters, the timescales can be several seconds depending on the thermal inertia of the system. For the latter case, a real time update of the digital twin might mean an update every one second. However a one second timescale for a fission process might be considered periodic. Therefore, I would refrain from attaching a hard timescale to those terms. However, in almost all digital twin setups, human interaction is required and the characteristic timescale for human interaction could be human reaction time. This timescale is on the order of 0.3s [Deng et al., 2019]. Therefore, the largest timescale for a digital twin to be considered real-time is about 0.3 seconds. Anything larger and the end user could perceive the updates to be slower than what is perceptible, in other words, a lag. One could then define periodic as anything slower than real-time.

In summary, where synchronisation frequency is discussed, the term real-time can be defined as a timescale less or equal to human reaction timescales (0.3s) and the term periodic can refer to a timescale slower than this, perhaps 0.4s and up. The basis of defining such timescales is whether the end user perceives a lag in updates. Therefore, these timescales are not hard limits, but rather suggestions based upon whether the end user perceives a time lag.

Conclusion for State Synchronisation Terminology While it is quite laborious to clarify definitions, the purpose of doing so is to provide the reader clarity on what the discussions mean. This is important because as we explore the different use cases of digital

twins, the degree and frequency of synchronisation in each use case will differ. Thus, we want to be as unambiguous as possible when discussing these matters so as to avoid confusing the reader.

Digital Twin Requirements for Fault Detection and Identification To quantify the extent to which the Type II Digital Twin of CIET needs to be synchronised in real-time, it is useful to consider specific use cases. The point of this example is to illustrate the fact that not all sensor data will be perfectly synchronised with the Digital Twin for all use cases. Here, we shall consider fault detection as a use case to investigate the extent of synchronisation required. Where possible, we shall discuss how these requirements impact the design of the Type I Digital Twin in this work.

Now, for fault detection and identification, we require that a digital twin have real-time state input synchronisation with the physical asset. When calibrated to predict the behaviour of the healthy system, the digital twin would serve to act as a healthy baseline for which to compare the physical asset against. If the physical asset deviates from this behaviour beyond a certain threshold, we could infer that a fault has occurred. The characteristic behaviour of a certain fault as the system deviates from a normal healthy plant is known as a fault signature. We then need to collate a library of fault signatures to have a fault signature library [Jain et al., 2019] to compare existing real-time faults against⁹. For more complex systems, structural analysis for fault detection and identification can be used [Poresky, 2019]. Another requirement is that Type II Digital Twins should have their models periodically synchronised with the physical asset so that the digital twin accurately reflects the healthy model of the plant. This is so that plant ageing does not trigger any kind of false alarm that may prompt operators to shut down the plant.

In summary, for Type II and Type III Digital Twins to be effective in fault detection and identification, we need to make sure (1) state input synchronisation happens in real-time during the startup of the plant and digital twin, (2) the digital twin's should perform periodic state model synchronisation to account for plant ageing. (3) fault signature libraries should be periodically synchronised.

State Input Synchronisation For state input synchronisation to happen, the initial conditions of the digital twin and physical asset could be synchronised during plant startup. This could include calibration of the Type II digital twin's models using data from the physical plant. Such data can be obtained by probing the physical asset while online using various methods, for example, frequency response testing [Poresky, 2019]. Additionally, the inputs into the digital twin should be synchronised with the inputs of the physical asset in real time. However, the outputs of the digital twin need not match that of the physical system. [Jain et al., 2019; Akbarian, Fitzgerald, and Kihl, 2020] It is precisely the difference in the outputs of the physical asset and digital twin that allows fault detection to occur.

⁹Interested Readers can refer to the appendix on page 253 to look at some typical fault detection setups in literature

For this use case, where we use the digital twin as a baseline for healthy plant or physical asset operation, a calibrated Type II or Type III digital twin will be required with real time synchronisation of user inputs and system inputs.

State Model Synchronisation in Accounting for Plant Degradation and Ageing

For fault detection to be effective, we would need to perform state model synchronisation on the Type II digital twin periodically to account for ageing [Zipper and Diedrich, 2019]. State model synchronisation is important to prevent us from getting false alarms from our fault detection algorithms. This is because when a plant ages, the physical models describing a plant may change. For example, heat exchangers may undergo slight fouling over time, but not to the extent that it is considered a fault that requires us to shut down the plant and initiate maintenance. This may cause abnormal behaviour compared to the healthy, new plant our digital twin is meant to represent. Such deviations or abnormalities may trigger fault alarms that we may not want to consider as faults. We may instead want to tolerate the slight fouling and consider this state of the plant a new normal. Thus, to keep ourselves from getting false alarms, we may want to update the physical or data driven model in the Type II digital twin to account for this extra fouling. This should be done for Type II Digital Twins in general so that an accurate baseline for a healthy plant is established which accounts for plant ageing and degradation tolerable within operating conditions.

To perform state model synchronisation, the Type II Digital Twin needs to be properly calibrated to replicate the behaviour of the physical system. This step is needed to eliminate any modelling errors present in the Type II Digital Twin. For example, a shell and tube heat exchanger would have an inherent thermal capacity within its piping systems and in the shell. We would then need to perform experiments to determine the thermal capacity of these heat structures. Using empirical data such as experimentally determined thermal capacity of the heat structures, we can then set the simulated heat capacity of the heat structures within the Type II Digital Twin to the correct values. This model calibration process is important to ensure that we do not have an outdated plant model that alerts us to faults due to modelling errors.

This implies that for the Type I Digital Twin we intend to construct, we should design it in such a way that the physical models that describe the components should be easily updated by the end user.

Periodic Synchronisation of Fault Signature Libraries Fault identification requires that we have a reasonably accurate library of fault signatures. These fault signature libraries should also be synchronised periodically to account for ageing as well. This means that to have a fault signature library, we need to perform simulation studies regularly. This falls under the what-if analysis stated earlier in 2.3, as such, Type I digital twins would be used since synchronisation with the physical asset in real-time is not necessary. In fact, real time synchronisation with the physical asset is not desirable because we do not want to expose the physical asset to faulted conditions on purpose. However, we do want to perform

periodic state model synchronisation for the Type I Digital Twin used to build fault signature libraries so that the fault signature libraries are accurate similar to how we perform state model synchronisation for the Type II Digital Twin.

This means that for the Type I Digital Twin we are to construct, we wish to design it with the capability to accurately develop fault signature libraries. The most basic requirement for accuracy is that we need to quantify the impact of uncertainty on a typical fault simulation. Therefore, we will discuss some of these uncertainty propagations in the results section in the context of common faults being explored for FHRs. We shall explore one of these faults in the result section.

Moreover, for the fault signature libraries to be valid, the simulated fault in the Type I digital twin must take place under similar operating conditions as the actual fault in the physical asset. Otherwise the signature may not be applicable. As such, this fault signature library must not only contain a comprehensive database of the possible faults that occur, but also contain information on how these faults affect the physical system or asset in its various operating states. Let's take for example, heat exchanger fouling as a possible fault. If we had the same fouling fault in the cold fluid, the decrease in heat transfer coefficient might differ depending on whether the flow regimes in the cold side of the heat exchanger were laminar or turbulent. Thus for one fouling fault, we might need at least two fault signatures in our database, one signature for laminar flow and one for turbulent flow. This shows that the fault signature database would likely need to be extensive and comprehensive in order to be effective. This would be easier to perform in Type I Digital Twin rather than the physical asset.

This means for the Type I Digital Twin for CIET, we may want to build it with capability to predict flow and heat transfer in various conditions even when experimental data is not available for validation. Therefore, a more deterministic modelling approach is favoured for the Type I Digital Twin as compared to a data driven approach such as artificial neural networks. This is because we want some physical basis for confidence in predicted results beyond experimental data.

Furthermore, we might have any combination of faults which may occur at once, thus producing different signatures. If we wanted to diagnose anything more than singular faults, then our database may have to grow to encompass many combinations of possible fault cases. One suitable tool for this use case presented in literature is structural analysis [Poresky, 2019]. Structural analysis provides us with a systematic framework for identifying and isolating faults in complex systems with several modelling equations and several variables in the system. Structural analysis methods are extensively discussed in literature for fault detection [Izadi-Zamanabadi, 2002] and fault tolerant control [Izadi-Zamanabadi and Staroswiecki, 2000]. This framework helps us to investigate if a sensor system is able to distinguish one fault from another, or if installation is possible, we can then use structural analysis to systematically determine where to install these sensors. Using structural analysis is advantageous because it enables us to visualise faults signatures across multiple sensors rather than just one sensor. Thus, we can use data from multiple sensors, plug it into a structural analysis framework and potentially diagnose and isolate a fault. Exploring the

structural analysis framework in detail is beyond the scope of this thesis. Interested readers can refer to the Appendix in page 269 where a more detailed discussion is given or refer to Poresky’s [Poresky, 2019], Izadi-Zamanabadi’s [Izadi-Zamanabadi and Staroswiecki, 2000; Izadi-Zamanabadi, 2002] or Frisk’s [Frisk, Krysander, and Jung, 2017] work for more detail about structural analysis and some software used for structural analysis. Performing fault detection in real-time using methods such as structural analysis requires us to obtain real-time simulated data from the Type II Digital Twin to compare with sensor data in real-time. We require the Type II Digital Twin used for this to replicate all possible sensor data in real-time so that real-time comparisons can be made. Hence, we should be able to receive sensor data from a remote device via some networking protocol and we also need to perform physical calculations that reproduce sensor data of a healthy version of CIET in real-time or faster than real-time if possible.

Summary of Synchronisation Requirements For use of CIET’s Type II or Type III Digital Twin in fault detection, state synchronisation only involves real-time state input synchronisation and periodic state model synchronisation. Other use cases such as fault tolerant control may have other synchronisation requirements such as the need for the Digital Twin to synchronise the physical asset’s state to the state of the Digital Twin (Type III in this case)¹⁰. This includes the ability of the Type III Digital Twin to send control signals to CIET. This implies that real-time sensor data needs to be communicated by CIET’s Programmable Logic Controller (PLC) and the user control panel to its Digital Twin in real-time. The user must then be able to read data from the Digital Twin in real-time. Furthermore, the Digital Twin needs to be highly customisable so that the user can adapt its code and programming for specific use cases. We also want the Digital Twin to accurately model the baseline healthy behaviour of CIET so that we can use this as a basis of comparison for fault detection. This means that the digital twin we construct for CIET should be (1) highly customisable, (2) be able to have two way communication with the user in real-time and (3) be accurate enough for use in simulating healthy plant behaviour in real-time. Moreover, the Type I Digital Twin we develop for use in developing fault signature libraries should be accurate enough for simulating transients. Therefore, it should also model plant behaviour accurately in real-time under faulted conditions. To build a Type I Digital Twin which acts as a stepping stone to construct a Type II or even Type III Digital twin with these capabilities, we want to apply these same three requirements to the Type I Digital Twin as well.

Remote Operation Capabilities

The second key capability we discuss for CIET is remote operations. This is because CIET was used to support remote operation studies [Poresky et al., 2022]. This research was motivated due to the “work from home” situation in the COVID-19 pandemic. To facilitate

¹⁰There are several other applications of digital twins as well. These applications along with their synchronisation requirements can be read by interested users in the appendix in page 256

this, ARCO-CIET, primarily built around a server-client architecture, was broken into several systems. The Programmable Logic Controller (PLC) of CIET would stay physically with CIET while the server using the Open Platform Communications Unified Architecture (OPC-UA) protocol would be hosted remotely [Poresky et al., 2022]. A rough diagram of this setup can be visualised in figure 2.3.

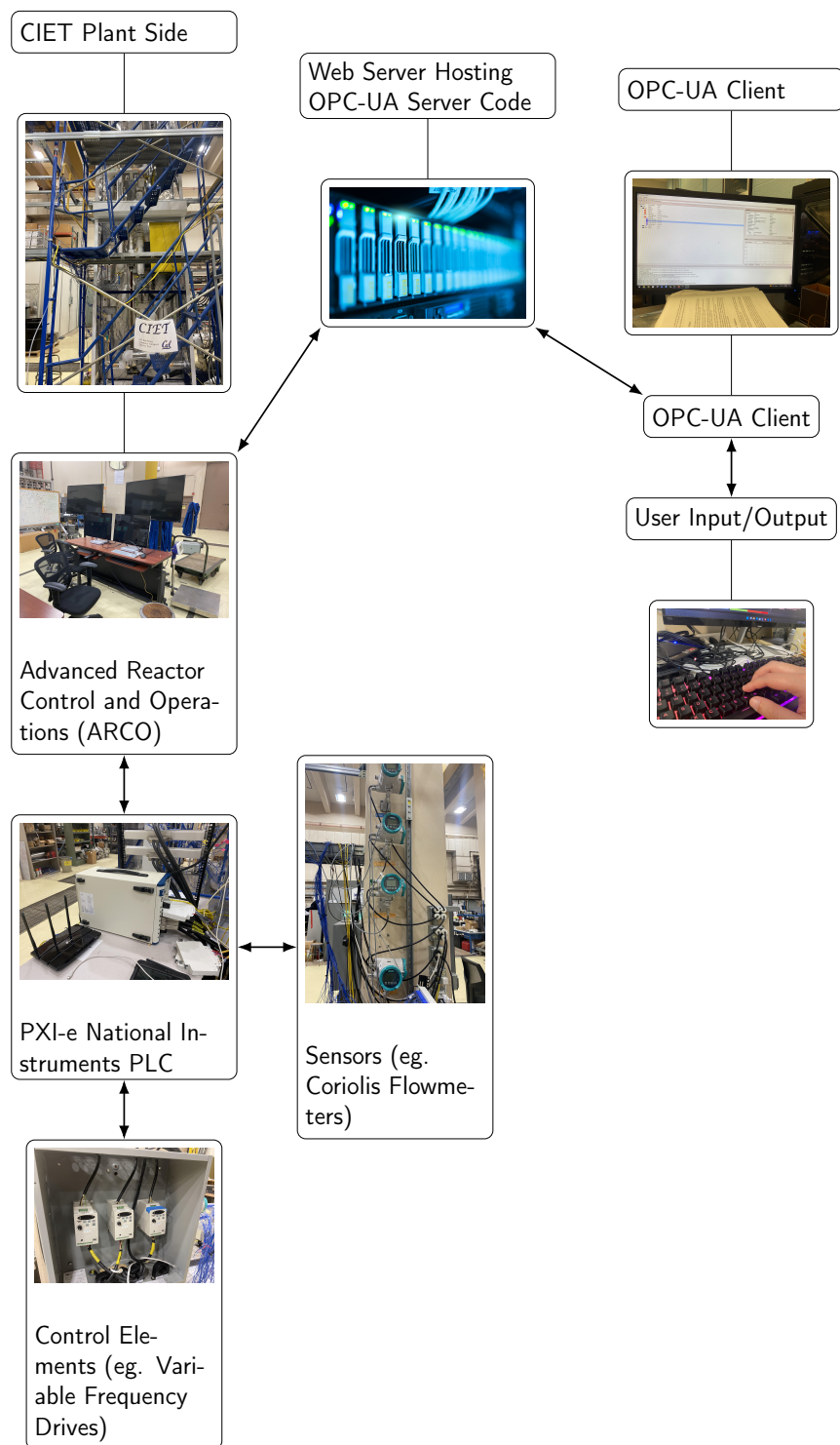


Figure 2.3: Approximate Remote Operation Architecture for CIET

The Type II or III Digital Twin should be able to interface with this OPC-UA server

to retain remote operation capability. Hence, at the very least, the Type I Digital Twin we are developing should also support an OPC-UA server-client architecture so that it can be easily upgraded to support remote operation capabilities. For the Type I Digital Twin, we aim to host the OPC-UA server on one computer and host the OPC-UA client on another computer. The client could be as simple as a graphical OPC-UA client application or as fully featured as a supervisory control and data acquisition (SCADA) interface. We wish to also have the server computer and client computer connect wirelessly over a local Wi-Fi network. This would give a demonstration in principle that the Type I Digital Twin can be operated over a wireless network and thus gives confidence that the Type I Digital Twin can be upgraded in future to support remote operations.

For remote operation, autonomous operation capabilities are also important as is the case for space reactors [Wood, Upadhyaya, and Floyd, 2017]. This may require a Type III Digital Twin with fault tolerant controllers or even artificial intelligence capabilities. Developing such capabilities is far outside the scope of this dissertation. However, some discussion for developing fault tolerant controls is available in the appendix on page 252 for interested readers.

Real-Time Calculation Capabilities

For any type of Digital Twin of CIET, whether Type I, Type II or Type III, we require calculations for the thermal hydraulics of the model to happen in real-time or near real-time such that the operator cannot distinguish it from operating CIET. As such, we need thermal hydraulics libraries which can be calculated at high speeds, preferably <100 ms per calculation for all heat transfer and mass balance equations. This requirement is imposed because ARCO would obtain data and send control signals in a loop occurring approximately every 95 ms. Therefore, we need to develop code to run at this speed or faster. We also want to exclude the use of supercomputing clusters because they cannot provide computing power on demand. Normally, users have to create “jobs” for the cluster and wait in line for their job to start because other users are using the supercomputing clusters at the same time. For a research involving digital twins, this is not practical since the digital twin should be able to perform state synchronisation (of whatever kind) with CIET in real-time and on demand. Therefore, we want any digital twin developed to run within this time limit on laptops or computers readily available to the end user.

Requirements for Developing a Type I Digital Twin

Based on the above requirements for Type II and Type III Digital Twins, we want to develop a Type I Digital Twin with the following requirements: (1) ability to replicate the thermal hydraulics behaviour of CIET in real-time without using computing clusters, (2) high degree of customisation for end user to change the thermal hydraulic models over time and add new capabilities, and (3) two way real-time data communication with the end user via OPC-UA server-client interface, preferably over two computers connected to a Wi-Fi network. I

also wish to add a fourth requirement that the Digital Twin to be built using open source software. We will elaborate on the rationale on some of these requirements before proceeding on to exploring software available to fulfil these requirements.

Rationale for Developing Open Source Digital Twins

While digital twins have many uses such as operator training and even training of students, [Al-Geddawy, 2020] the cost of procuring a digital twin can often make such advantages inaccessible. Furthermore, the paywall presented by most digital twin systems can make it more costly both in terms of time and money to try testing the results presented by a digital twin for the purposes of peer review and scientific research.

Of course, this does not prohibit the use of proprietary software for simulation studies in literature. Commercial software has been used extensively in literature for real-time simulation studies. One such example is the use of the software known as the Advanced Process Simulator (APROS) [Silvennoinen et al., 1989]. APROS contains well established thermal hydraulic libraries [Gallagher, 2021] which can be used for real-time or faster than real-time simulation [Puska et al., 1995]. Unfortunately, APROS is commercial software [Gallagher, 2021] and is not free. Thus, software such as APROS is not necessarily available to all members of the scientific community, but rather only to those who can afford it. Therefore, the ability to replicate and build upon the results of these studies is only available to those who can afford it.

I wish to avoid such issues in the course of research so that code used to build the digital twin can be readily peer reviewed and built upon. It is for this reason that developing open source digital twins is a priority.

Rationale for Requiring Real-Time Simulation Capability without Supercomputing Clusters

As mentioned earlier, we wish to develop digital twins capable of running in real-time on laptops so that digital twins can be run on demand without having to wait in line for jobs to be done in supercomputing clusters. The aspect of real time on demand availability is especially important for digital twins to achieve their objectives, and hence there is such a constraint on using readily available computing resources. This has the added benefit of saving cost because we do not have to purchase our own dedicated server ¹¹ to run the digital twin unless the cost is absolutely necessary.

Designing a Digital Twin this way allows us to have the benefits of digital twins as early in the design process as possible. This means that we can have a digital twin that is easily run in a manner that is low cost, on demand and in real time eg. from home laptops. Doing so makes Digital Twins accessible to as many students and researchers as possible.

¹¹If we wanted extra computing power, we could perhaps design a Digital Twin split the computing load amongst several readily available and affordable laptops.

For the purposes of constructing a digital twin prototype of CIET, I reiterate that computations for each timestep should be computed in under 100 milliseconds. This is because the control system, ARCO, for CIET would have taken in system data about every 95-100 milliseconds.

Rationale for Two-Way Real-Time Interaction with Users via OPC-UA

We intend that a user should be able to interact with the Type I digital twin of CIET in real-time as far as possible as if he or she were interacting with CIET. This is because a Type I Digital Twin is essentially a plant simulator. For this thesis, this means that we want the user to be able to control the pump in real-time and receive flowmeter readings in real-time. Additionally, we wish to have the Digital Twin interact with the user via OPC-UA server client interface so that it is easier to upgrade it for remote operation studies in future if desired.

Using OPC-UA also has the added benefit of demonstrating communication capabilities with CIET's PLC, a key requirement for Type II or Type III Digital Twins. In principle, if we can demonstrate capability of transmitting data between user and the Digital Twin via OPC-UA, then we should be confident that the Digital Twin can communicate with CIET's PLC as well via OPC-UA. This is because OPC-UA is platform independent protocol and an emerging industrial standard for industrial cyber-physical systems [Poresky et al., 2022]. If OPC-UA is a standard, then in theory, one should be able to interface with most hardware via an OPC-UA server client interface. This is advantageous because we want to design the digital twin to interact with PLCs regardless of platform, hardware or operating system. This is so that if the PLCs were updated or changed, we do not have compatibility issues.

On User Interaction with the Pump Now, when CIET is used to perform experiments physically, the user controls the pumps via a variable frequency drive (VFD). The pressure and flowrates of the loop can then be determined by finding the intersection between the pump curve and the system curve. This is known as the operating point. For the purposes of this thesis however, I am not intending to simulate the pump curve at all along with its various frequencies.

For simplicity of development, I would intend the user supply a loop pressure drop, and the digital twin of CIET will calculate the mass flowrate of the loop. This is similar to how one finds the mass flowrate of a system using a system curve.

By specifying the loop pressure drop, the user would essentially be able to control the mass flowrates in CIET's digital twin. However, when an operator operates CIET, the operator would be controlling a pump by specifying the frequency. I would like to replicate the "feel" of an operator changing settings on a pump to control the system flowrate. Therefore, I intend to name the loop pressure drop which the operator controls as the "pump pressure" variable in my digital twin code. While it does not reflect the most technically accurate use of the term, the use of such terminology in the control system may be slightly more intuitive to the end user.

Rationale for High Degree of Customisation

We are developing a Type I Digital Twin hopefully as a first step to developing Type II and/or Type III Digital Twins in future. Therefore, we want the code to be as customisable as possible so that we can add capabilities over time.

For example, we are developing a Type I digital twin with no real-time state synchronisation capabilities in this thesis, we want to keep in mind that the Type I digital twin may be upgraded in future to have real-time state synchronisation capabilities. This means that the Type I digital twin should be designed in such a way that it can interface with CIET sometime in future and receive real-time sensor readings from CIET. This is so that the codebase can be extended to include synchronisation methods such as Kalman Filters or PID controllers if so desired. Some literature review on synchronisation methods in the appendix on page C.2 for interested readers.

These requirements make it difficult to use existing system simulation models and codes which have been developed for CIET such as RELAP [N. Zweibaum et al., 2015], SAM [Zou, R. Hu, and Charpentier, 2019] and Transform [Per F. Peterson and Greenwood, 2019]. These simulations were not geared towards the requirements of a digital twin and the closed source nature of these codes makes them impractical to modify. For example, they were not able or designed to run calculations in real time, nor did they have inherent capability to interact with a supervisory control and data acquisition (SCADA) interface or with CIET's Programmable Logic Controller (PLC) or any computer via an OPC-UA server-client interface.

Therefore, we want to design the Type I digital twin in this work in a modular fashion where code can be easily customised without rewriting significant portions of the code.

With these requirements in mind, we shall go on exploring the available open source software that enables us to meet the requirements.

Survey of Available Resources for Constructing the Type I Digital Twin

Having established the motivation and requirements for our Type I Digital Twin, we now consider software options which could help us construct the Digital Twin. As mentioned earlier, open source software is preferred due to the source code being made available for a high degree of customisation and also relative ease for other members of the research community in reproducing the results presented here.

STOP WORK HERE Such a fine level of customisation could be best achieved using open source code with near bare metal performance. I use the term bare metal as used in high performance computing journals and conference proceedings such as Zhang's studies [Zhang et al., 2017]. Bare metal performance means that the performance of the code matches natively compiled machine code, for example code running at the kernel level; it is the fastest possible way of programming a piece of software given a set piece of computer hardware with

minimum or zero overhead. The alternative to writing open source performant ¹² code is to find libraries that cater specifically to the purpose outlined. To minimise the hours of unnecessary work done, we first want to utilise existing free and open source (FOSS) high level libraries as far as possible. Even better, if Digital Twin software already exists for other physical systems, we could perhaps adapt it for use in CIET. Therefore, we shall explore (1) firstly, existing open source digital twin software, (2) secondly, libraries we can use to construct CIET's Type I Digital Twin if the first option is not feasible

Available Open Source Software for Digital Twins

For a digital twin, we need to simulate the data and the physical system in real time. In literature, RoboDK has been used as an [Al-Geddawy, 2020] open source software to simulate robots. It interfaces well with Python, MATLAB and visual studio (C# code and visual basic) to provide the backend for the program.

It must be emphasised however, that digital twin software for robots is quite different from digital twin software for power plants as the underlying physical asset has a different design, purpose and behaviour.

Existing plant simulators or digital twins for nuclear power plants are, more often than not, proprietary software with source code not easily available for modification for uses in this project. Some examples include the GSE Solutions Nuclear Power Plant simulator used for NuScale [Kochunas and Huan, 2021; GSESolutions, 2016]. Options for Free and Open Source Software (FOSS) digital twin software for power plant simulation in general are often much more limited or not available to the best of my knowledge. Thus, it seems that writing a custom digital twin for CIET seems to be the most viable option presently.

Open Source Libraries for writing a Digital Twin Server

Writing a customised solution procedure that fits the real time constraints of digital twins is no easy venture. Therefore, it makes sense to leverage upon existing libraries used for calculation in order to speed up the development process.

One FOSS library available for power plant simulation is the Institute for the Design of Advanced Energy Systems (IDAES) [Lee et al., 2021] integrated platform. IDAES is well suited for computer aided process process engineering (CAPE) and can potentially serve to provide the libraries necessary for physical calculations at the backend of the digital twin.[Sleiti, Kapat, and Vesely, 2022]

IDAES has shown that for a hydrodalkylation process, 16.2 seconds were required for sequential decomposition and then 0.5s was required for solution on a i7-8750H CPU with 10.4GB of RAM [Lee et al., 2021]. This is a typical hardware specification for a home desktop computer in 2021.

This sequential decomposition is a time intensive step that is due to how the underlying pyomo libraries solve equations. These libraries are designed for optimisation and not real

¹²by performant I mean code performs fast enough for the required task

time simulation. As such they were not designed with time constraints in mind for real time simulation.

In this case, the Pyomo [Hart, Watson, and Woodruff, 2011] Network package is responsible for solving interconnected blocks or interconnected components and how they relate to each other. The sequential decomposition step leverages the modelling components to decompose a model and compute each unit in the model in a logically ordered sequence. [Hart, 2017]

In other words, when Pyomo is supplied with many interdependent equations, the sequential decomposition step is used to determine which equations are solved first. This would offload the burden from the user to figure out which equations must be solved in the physical model first, but if it were done in every time step, the time and computation cost would become unacceptable.

To ensure that this step was not needed for networks of components, the user would then need to understand Pyomo’s libraries at a low level alongside the IDAES library to use and calibrate the libraries to run a real-time simulation. Otherwise, it would seem that sequential decomposition would be required for every time step, and this is not acceptable for this scenario.

If one were to take away the problems caused by sequential decomposition, a 0.5s solution time is too long for CIET’s digital twin. While it can be argued that the hydrodalkylation process is much more complex than CIET and would therefore take more time, we would still need the process time to speed up by about 5 to 10 times for it to become viable in CIET’s digital twin. We may also find after constructing the model that the solution time may take too long. Should we desire to speed calculations up by increasing the degree of parallelisation in calculations, we would then require some low level code modification with Pyomo and the IDAES libraries. The degree of effort needed to be familiar with IDAES libraries so that low level modifications can be done is comparable to writing a library from scratch. This has discouraged me from using IDAES for CIET’s digital twin.

To the best of the author’s knowledge, FOSS libraries for power plant simulation that come close to the capabilities provided by IDAES but suitable for real time simulation are not currently available.

Justification for Writing Digital Twin Libraries Due to the degree of flexibility and speed required to construct a digital twin prototype for CIET, I may not be using the IDAES software libraries available but instead write my own libraries to solve the mass and energy balances for the digital twin. Furthermore, if I want to add libraries in future which enable state synchronisation capabilities in real-time, I would then require a very fine level of control within my code. This further justifies the writing of Digital Twin libraries from scratch.

Justification for not using Artificial Intelligence Artificial Intelligence (AI) has been used for digital twins in literature [Idaho National Lab, 2022; Gong, Cheng, et al., 2022]. This allows the most time intensive high fidelity models to be developed using machine

learning prior to the operation of digital twins so that digital twins can simulate the power plant or CIET processes in real time.

Nevertheless, for this thesis, my knowledge of AI is limited and it would be too time consuming to learn about AI and machine learning to the extent required for this thesis. I have therefore opted to avoid use and literature review of AI in general and reserve it for future work if higher fidelity models are required.

To fulfil the real time requirements, a nodalised approach similar to that used in RELAP5 or MELCOR for nuclear power plant transient simulations [Murgatroyd et al., 2017] can be used for a relatively low cost high speed simulation.

Open Source Methods for Writing Digital Twin Libraries

Given the stringent requirements for Digital Twins, we find that Open Source Digital Twin software is not available “as is” for use in CIET, nor are high level libraries for constructing CIET’s Digital Twins available so that we cannot simply copy code used for a pump or pipe in CIET. Since we might have to write our own libraries, it is important to explore some lower level libraries and fundamental methods available to write our Digital Twin Libraries.

Open Source Thermodynamic Libraries To set up a digital twin using a nodalised model, one would have to solve mass and energy balances for transient problems in real time. Such balances are routinely solved by CAPE software such as Aspen Plus and DWSIM. Of these, DWSIM is FOSS and written in visual basic in the .Net Framework [Tangsriwong et al., 2020]. In DWSIM, nodalised fluid mechanics and heat transfer solutions are often solved simultaneously without consideration for calculation time. DWSIM code was also first considered for use in contrast to IDAES as it had a graphical user interface (GUI) and due to prior user experience.

In closed loop simulation for DWSIM, a recycle stream function is often used so that mass balances and energy balances are iteratively solved until a steady state is reached. For the purposes of a digital twin, this calculation procedure would take too long. Nevertheless, DWSIM has useful thermodynamic libraries written in Visual Basic in the .NET ecosystem. It can interface well with C# code which is a compiled rather than an interpreted language. Such a language could be fast enough for a digital twin. Furthermore, we need our coding language to be fast enough so that real-time state synchronisation of the Digital Twin is possible in future. Therefore, in each timestep, we would need to calculate the outputs of the digital twin, and in future, perform synchronisation of the digital twin to the state of CIET. The degree of which depends on the use case of the digital twin. With that many tasks to perform in one timestep, it is obvious that the speed of the code is of utmost importance. Therefore, we would need to obtain a thermodynamic library in a language where execution speed is reasonably fast.

Of course, the end goal of using the thermodynamic libraries is to simulate our working fluid. Here, the working fluid used in CIET was mentioned in literature to be Dowtherm A [N. Zweibaum et al., 2015], but over time, we refilled CIET using Therminol-VP1 because it

was more affordable. However, the composition and thermophysical properties of Therminol VP-1 and Dowtherm A are remarkably similar.

To illustrate this, the temperature dependence of the Prandtl Number for both Dowtherm A and Therminol-VP1 is shown in figure 2.4. The residual plots in figure 2.5 are generated by subtracting the Therminol VP-1 Prandtl Number from the Dowtherm A Prandtl Number.

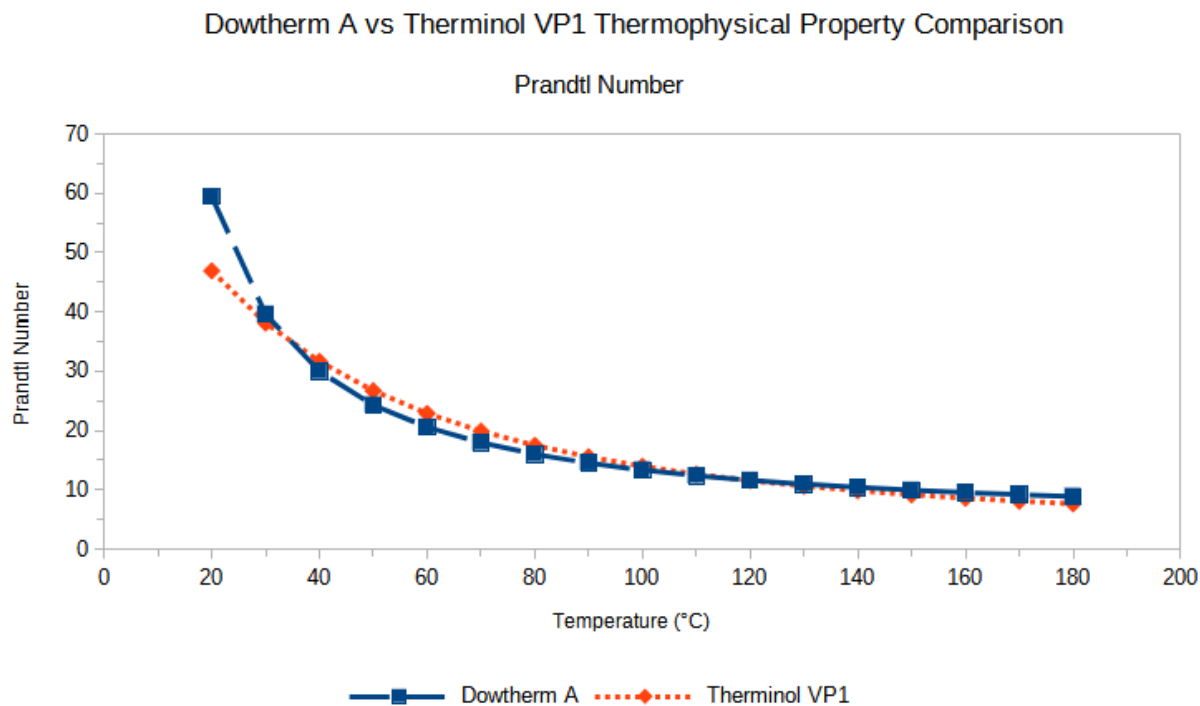


Figure 2.4: Dowtherm A vs Therminol VP-1 Prandtl Number Comparison

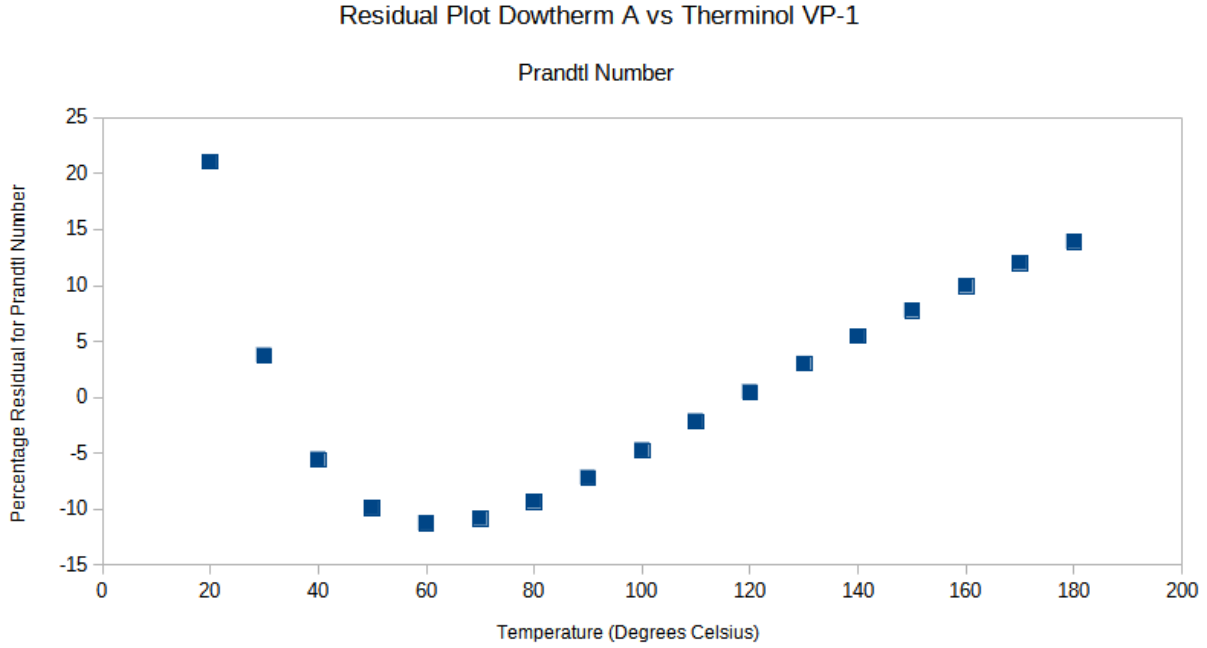


Figure 2.5: Dowtherm A vs Therminol VP-1 Prandtl Number Residual Comparison

Data for figure 2.4 was generated using Dowtherm A thermophysical property correlations [N. Zweibaum et al., 2015] and the Therminol VP-1 data from coolprop libraries [Bell et al., 2014]. Figure 2.5 was generated using the thermophysical property correlations in equations 2.1 to 2.4. No error bars are provided because I just want to provide a rough idea of the deviations present between Dowtherm A and Therminol VP-1. I'm not trying to prove that these deviations are within experimental or measurement error.

$$\rho(kg/m^3) = 1078 - 0.85(T^{\circ}C) \quad (2.1)$$

$$\mu(Pa \cdot s) = 0.130/(T^{\circ}C)^{1.072} \quad (2.2)$$

$$c_p(J/(kg \cdot K)) = 1518 + 2.82(T^{\circ}C) \quad (2.3)$$

$$k(W/(m \cdot K)) = 0.142 - 0.00016 \cdot (T^{\circ}C) \quad (2.4)$$

The Prandtl Number for both fluids matches well except at 20°C where the error is as large as 21%. This is largely due to discrepancies in viscosity at 20 °C¹³. At prototypical temperatures for CIET to operate however (about 80 to 110 °C [N. Zweibaum et al., 2015; De Wet and Per F Peterson, 2020]), the CoolProp Libraries have deviations for Prandtl Number by at most 10% using properties for Therminol VP-1 compared to Dowtherm A.

Whether these errors are important depends on the use case for these Prandtl Numbers. One example is shown for the reader's consideration. A common use for Prandtl Number is

¹³ Refer to Appendix on page 263.

for Nusselt Number calculation. For FHRs, Nusselt Numbers in packed beds are of interest. For packed beds in general, use of the Wakao correlation is common [Son et al., 2018; Huddar et al., 2015]. We can calculate a typical error percentage due to this Prandtl Number deviation and see if it lies in the range of correlation deviation from experimental data.

$$Nu = 2 + 1.1Pr^{1/3}Re^{0.6} \quad (2.5)$$

We can calculate the error propagation in the Wakao correlation in equation 2.5 using the following uncertainty propagation formula[BIPM et al., 2008]:

$$(\delta Nu)^2 = (\delta Pr)^2 \left(\frac{\partial Nu}{\partial Pr} \right)^2 + (\delta Re)^2 \left(\frac{\partial Nu}{\partial Re} \right)^2 \quad (2.6)$$

We can ignore the contributions due to Re and calculate the partial derivative as:

$$\frac{\partial Nu}{\partial Pr} = 0.3667Re^{0.6}Pr^{-2/3} \quad (2.7)$$

We can estimate the error δPr to be 21% of the value of the Prandtl number based on the residual plot in figure 2.5 so that we get:

$$(\delta Nu) = 0.21Pr * 0.3667Re^{0.6}Pr^{-2/3} \quad (2.8)$$

$$(\delta Nu) = 0.077Re^{0.6}Pr^{1/3} \quad (2.9)$$

The 95th percentile upper and lower bounds for experimental Nusselt Number deviation from the Wakao Correlation Nusselt Number can be expressed as [Son et al., 2018]:

$$Nu = 2 + 2.25Pr^{1/3}Re^{0.6} \quad (2.10)$$

$$Nu = 2 + 0.72Pr^{1/3}Re^{0.6} \quad (2.11)$$

Should we add and subtract the error in equation 2.9 from the Wakao correlation in equation 2.5 we would get:

$$Nu = 2 + 1.177Pr^{1/3}Re^{0.6} \quad (2.12)$$

$$Nu = 2 + 1.023Pr^{1/3}Re^{0.6} \quad (2.13)$$

Thus, we can see that the errors due to a difference in Prandtl Number deviation using Therminol VP-1 CoolProp Libraries as compared to the Dowtherm A correlation are well within the bounds of uncertainty encompassing 95% of experimental data even when we account for the 21% deviation at lower temperatures. Thus, we can conclude that at least for packed beds, use of the Prandtl Number correlations with CoolProp are quite acceptable if we are using the Wakao Correlation. If other Nusselt Number correlations are used, then similar error analysis should be performed for those applications.

Now, while DWSIM does not have thermodynamic libraries for Dowtherm A, it has libraries for Therminol-VP-1, which has a very similar composition to Dowtherm A. The underlying thermodynamic libraries for Therminol-VP-1 and other incompressible fluids such as air are found in the coolprop libraries. [Bell et al., 2014] Coolprop libraries have thermodynamic property data for several fluids besides Therminol-VP 1 and this may prove useful if we need to simulate heat exchanger behavior in the CTAH where air is the fluid which receives heat.

While having databases of thermodynamic data for various fluids is important, we must also consider speed. The coolprop libraries are written in C++, and therefore the library should be reasonably fast in terms of computation time.

Now, C++ libraries are not known for their thread safety and memory safety. Should this become an issue, we can still fall back on empirical correlations for Dowtherm A directly shown in equations 2.1. to 2.4[N. Zweibaum et al., 2015].

For an isothermal digital twin where we do not solve energy equations yet, these correlations would suffice. In future when we want to include enthalpy calculations, we may have to use temperature correlations for enthalpy in future.

Operator Split Methods We mentioned earlier that speed is an important requirement for creating a digital twin which must run in real time. Additionally, we want to have some computation time available for real-time synchronisation of the digital twin to CIET in future. In terms of solving equations, we know that the methods used in DWSIM may not be suitable for this purpose since DWSIM has to iteratively solve for mass and energy balances for the simulation.

The typical time it takes to solve these balances on a computer when a closed loop with a recycle stream is on the order of a few seconds to tens of seconds. One reason for this is that the mass and energy balances have to be solved simultaneously so that steady state is reached. To solve these equations simultaneously, several iterations are required to get the mass and energy balances to some sort of steady state.

The options for speed up in this case are not many since each iteration would require information from the last iteration in order to be calculated. Thus, parallelisation is not quite possible with this approach. As a result, the only way to speed this type of simulation up is to use a Central Processing Unit (CPU) with faster clock speeds. This is an undesirable situation.

To enable parallelisation, which can speed up calculations, we could solve the mass and energy balances separately. This could include the use of operator splitting methods to solve for flow rates [MacNamara and Strang, 2016] on one thread or computer, and in parallel, solve for heat transfer equations on a separate thread or computer.

This means that mass flow rates and temperature profiles for the digital twin can be calculated separately rather than simultaneously. Since that is the case, it is desirable to design a digital twin for isothermal flow in CIET as a first step. Such a solver can then be

used as the mass balance solver in the full digital twin. This operator splitting method has been demonstrated in Computational Fluid Dynamics (CFD).

One example is the use of Pressure Implicit with Splitting of Operators (PISO) algorithm solves for the values of velocity, pressure and potentially temperature for the next time step without tight coupling. This has been demonstrated for free convection flows [J. Oliveira, 2001] which have coupled heat transfer and fluid mechanics equations solved at each time step. This shows that operator splitting can be used for coupled heat transfer and fluid mechanics simulations.

While this method is used for solving partial differential equations, it should be possible for a PISO like operator splitting algorithm to help solve for thermal hydraulics equations within the digital twin.

In other words, we will attempt to tackle the fluid mechanics and heat transfer solutions separately.

A possible approach would be at a certain timestep:

- Calculate Flow Solution using Temperature data from last time step
- Calculate Heat Transfer Solution using Flow data from last time step

In future, heat transfer equations can be solved in parallel with the hydraulics equations. To speed this up however, we may require parallel computation capabilities. Code parallelisation would require a great degree of customisation however for the underlying code.

Furthermore, when real-time synchronisation becomes important, we may perform the synchronisation step in parallel with the calculation steps for monitoring purposes. As mentioned in the definition of state synchronisation terminology on page 23, we may need to tailor the degree and frequency of synchronisation to the use case of the digital twin.

We may, for example, choose to have a digital twin that exclusively models and monitors CIET's temperature where thermal sensors are not available. In such a case, we may forgo the solving of the flow solution altogether, and synchronise mass flowrate in the digital twin using CIET's mass flowrate data. Possible calculation steps during such a time step could be:

- Obtain Flow Data using Flowmeter Data from CIET
- Calculate Heat Transfer Solution using Flow Data

These calculations can be done sequentially or in parallel. We may even choose to solve conjugate for each heat structure in parallel threads as we collect flowmeter data in another thread.

For a digital twin meant for fault detection, we may still need to solve the flow solution and heat transfer solutions according to our physical model. As there are several use cases for developing digital twins in CIET, we can see that there is a real need to customise how each digital twin behaves at each timestep.

With this degree of customisation and constraint required for CIET's digital twin(s), and with most digital twin software having paywalls, it may be best to design some libraries from the ground up, especially in terms of calculation procedure. This is so that if we need the speed increase, we should have the freedom to optimise the code or, if needed, change the language.

Libraries for Solving Pipe Network Problems In CIET, even for just hydraulic flow, we have to solve the issue of obtaining mass flow rates for parallel flow in real time. A possible starting point to solve flow network problems is to take inspiration from existing solvers for electrical circuits such as Spice3f5 and their .NET implementations such as spiceSharp and NextGen Spice. [Zikmund, 2018]

Attempts were made in literature to draw inspiration from solving circuits in electrical networks using nodal analysis and applying them to fluid pipe networks. However, the algorithms used to solve Electrical networks iteratively such as the newton raphson method were known to cause numerical instabilities in pipe networks due to square root terms in the equations caused mostly by localised turbulent flow. [Hamam and Brameller, 1971]

The square root terms come from velocity and pressure losses being nonlinearly related.

$$\Delta P_{loss} = \frac{1}{2}\rho u^2 \left(f \frac{L}{D} + K \right) \quad (2.14)$$

Nevertheless, I still thought it worth the effort to try if this instability would affect this problem in particular. This was because SpiceSharp uses the Newton Raphson method to solve nonlinear equations. If they had such a solver that solved for problems with nonlinear resistors, perhaps it could be applied to pipes too. But alas, after many frustrating tries, I have confirmed that the convergence instabilities exist even for simple pipes in series.

In addition to convergence issues, we find that the analogy between electrical circuits and fluid circuits is not ideal. In the case of CIET, we have the additional problem of needing to account natural convection too, and this would further complicate the above equation since we have to take into account hydrostatic pressure. There isn't an analogue in electrical circuits for hydrostatic pressure, except that external electric fields be applied. Thus, using the electric circuit analogy could prove challenging.

Due to the convergence issue and the fact that I still had to consider hydrostatic pressure, SpiceSharp code and Spice methods were not pursued due to their complexity and the time needed to develop code.

Therefore, the problem had to be recast in such a way as to avoid using nodal analysis and perhaps use common numerical methods for single input single output equations. If the problem could be recast in such a manner, simple bisection methods [Perry and Green, 2015] or other numerical methods could be used. Several iterations of such methods are discussed in the methodology section.

Open Source OPC-UA Server Implementation Supposing that the numerical methods could solve the hydraulics equations within the stipulated timeframe, the next step is to

look for existing methods for the digital twin to communicate with the end user preferably via OPC-UA. Furthermore, we would also want to have the ability in future to read data from CIET’s temperature sensors and flowmeters so that the digital twin can be kept in sync with CIET. To satisfy both of these requirements, we should build CIET’s digital twin with networking capabilities.

We intend to do so via constructing the digital twin as a server and interact with it using a client similar to previous work on ARCO-CIET [Poresky et al., 2022]. Establishing an OPC-UA server-client architecture is helpful because when synchronisation becomes important, we may want to use a client to request CIET’s sensor data from CIET’s server connected to CIET’s PLC or PXI-e box. The client can then push data to the server where the digital twin resides so that the digital twin can be kept up to date with CIET. To construct this server client setup, it is convenient to use OPC-UA libraries already available for this use. This means looking for implementations of OPC-UA Servers for the digital twin to communicate with the end user via the OPC-UA client ¹⁴.

OPC-UA is an emerging industry standard machine to machine protocol for automation, process control and monitoring [Schwarz and Börcsök, 2013; Poresky et al., 2022]. It is not a trivial task to write an OPC-UA server from scratch so that we can interact with the digital twin in real time, perhaps from different computers. Fortunately, there are already OPC-UA implementations written by the OPC Foundation such as UA-.NETStandard written in C#, open62541 written in C++, node-opcua written in Typescript for Nodejs, and Python OPC-UA/AsyncUA written in Python. [Muhlbauer et al., 2021]. Additionally, an open source Rust OPC-UA library also exists. [Erba, Müller, and Tippenhauer, 2021] These OPC-UA Server implementations have been used to connect digital twins to physical assets in literature [Zidek et al., 2020] via their Programmable Logic Controllers (PLCs) and SCADA interfaces. This shows that using OPC-UA with digital twins is an established concept.

The source codes of these implementations were readily available on github as of the time of writing. We shall assess the suitability of each of these implementations for this thesis in the methods section.

Open Source Software for Digital Twin Client and Frontends In addition to an OPC-UA server, we also need a control front end interface for researchers or students to interact with the digital twin. This usually means we need an OPC-UA client. Moreover, it would be good to have a graphical user interface (GUI) in order for the end user to control the digital twin.

In literature, a typical front end being used is Supervisory Control and Data Acquisition (SCADA) [Cimino, Negri, and Fumagalli, 2019] software. Some of these include open source software such as SCADAbr and OpenSCADA [Kazala and Straczynski, 2019; Zhe Peng,

¹⁴Now of course, there are several protocols available for communication beside OPC-UA, and this includes but is not limited to, modbus [Kazala and Straczynski, 2019], MQTT and Data Distribution Service (DDS). Exploration of the optimal protocols to use was out of scope of this thesis; I just picked OPC-UA due to prior familiarity and convenience as prior work was done using OPC-UA [Poresky et al., 2022]

2012]. If a user does not want use SCADA and wants to build a custom interface, he or she could program one using underlying libraries. One example in literature is where, Python's OPC-UA libraries have been used to write code for both OPC-UA server and clients [Kazala and Straczynski, 2019]. Now of course, a large portion of SCADA software is proprietary, and a possible concern is that an open source OPC-UA server may not be able to connect with a proprietary OPC-UA client. Thankfully, this is not true; OPC-UA has been used in literature even when the software being used is not fully open sourced. [Cimino, Negri, and Fumagalli, 2019] The reason is that OPC-UA data exchange between server and client has been shown to be safe, reliable, manufacturer independent and platform independent.

Besides SCADA software, and writing our own GUI front end, we could also consider the minimalist OPC-UA client gui written in Python and PyQt5. [Oroulet, 2015] While it may not offer full SCADA capabilities, it is quick to set up and can allow the user to manipulate values and read values on an OPC-UA server within minutes. Its simplicity makes it easy to learn, and thus, the user won't be overloaded with data input from traditional SCADA software. It would also be lightweight in terms of computational resources compared to a full SCADA suite. Thus it would be ideal for debugging and server development and testing.

Other open source options for human machine interfaces with inbuilt OPC-UA clients would include Scilab (with the xcos package). [Kazala and Straczynski, 2019] Open Modelica is also another choice for such a use case. Its libraries are well suited for creating control systems. [Kazala et al., 2022]

However, it may not be as suited for controlling the digital twin as it mainly deals with server side code rather than client side code. I discuss which OPC-UA client is used in the methods section.

Chapter 3

Methodology

3.1 Overview

A Type I Digital Twin was constructed as shown in Figure 3.1 which simulates isothermal flow for CIET at around 20°C . The user controls the Digital Twin Server via a Python Graphical User Interface (GUI) client on a computer hosting the client software. This Python GUI client interacts with a Python OPC-UA server on a Linux Server Computer where Linux is installed on a laptop. The server and client are connected via a Wi-Fi router to demonstrate that communication and control is possible over a network of computers. The Python Server calls a Rust fluid mechanics library via the PyO3 interface set up through Maturin. The source code is provided in the Appendix on Page 114¹.

¹Do note that the source code shown here will likely take a different form from what is found in online repositories. As of the time of publishing this work, I have not yet published these works as an Open Source Rust Crate or Open Source repository just yet because these libraries are not yet fully complete in terms of features. However, I've chosen to place the tentative code used to produce the results in this work so that readers can use it as a reference to build their own libraries or if they wish to reproduce the results. These working libraries found here can be referenced and modified with citation.

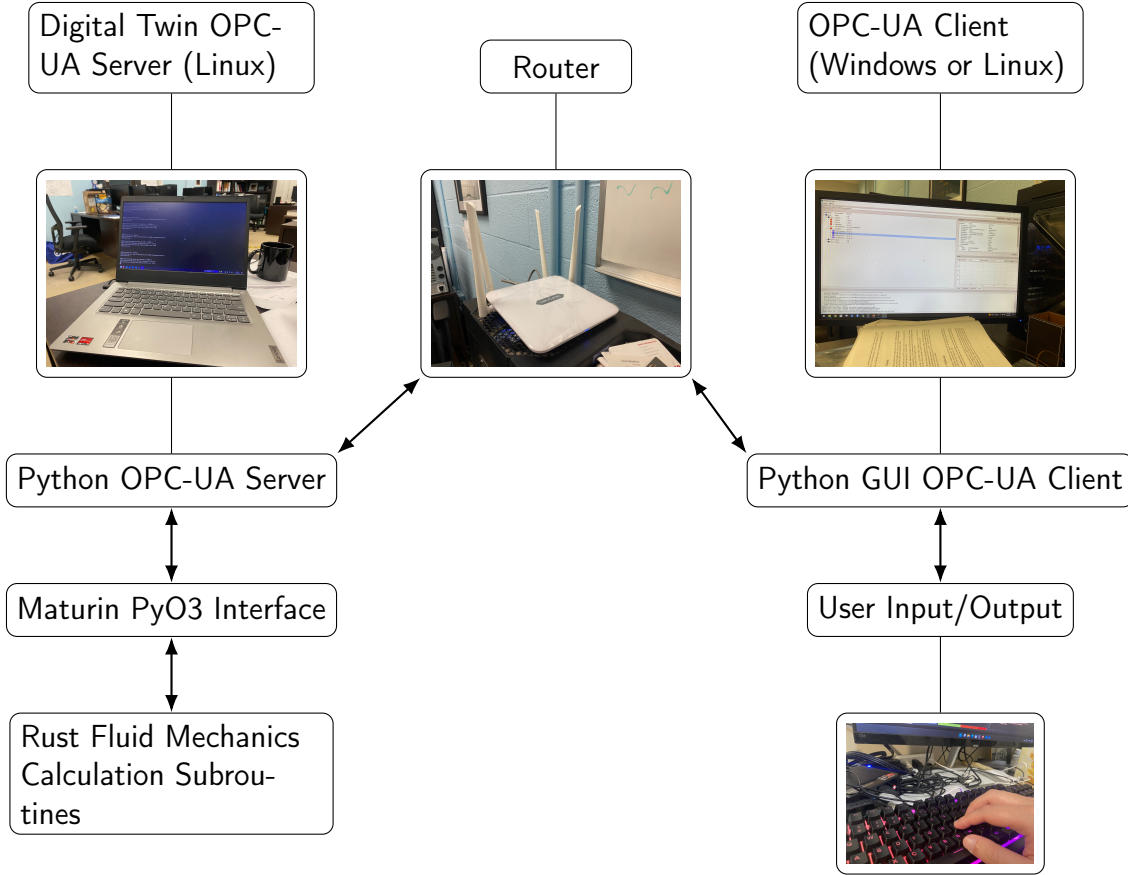


Figure 3.1: Architectural Diagram of Isothermal Type I Digital Twin

To create a digital twin of CIET, we first need to tackle the issue of solving fluid mechanics and heat transfer equations in real time. The second main issue is to construct an OPC-UA server-client interface.

For the digital twin's thermal hydraulics, I have decided to use operator splitting [MacNamara and Strang, 2016] which means that heat transfer equations and mass balances are solved separately and recombined to form the solution for the next timestep. Assuming operator splitting works, we could in theory develop separate solvers for heat transfer and fluid mechanics portions of the loop. As part of the effort to develop a solver which uses operator splitting, the fluid mechanics solvers are first developed. They are then used to construct an isothermal flow digital twin of CIET which can then be validated using the isothermal pressure drop flow tests in CIET.

This section begins by justifying why operator splitting is required in the context of digital twins. We then go on discussing methods used to solve parallel flow problems for the digital twin. Thirdly, we discuss the rationale for the choice of language used for constructing the solvers. Fourth, discussion methods for constructing the digital twin as an OPC-UA server and the choice of OPC-UA client are discussed. Fifth, we discuss methods used for

experimental code validation. Sixth, we discuss the caveats and approximations used in the digital twin. Seventh, we discuss the hardware used in the digital twin study because we wish to prove that we do not require supercomputing clusters to perform calculations for the digital twin constructed.

Lastly, I also wish to briefly document some of the failed methods I have tried in this section with lengthier elaboration in the appendix in page 269. This is in the spirit of sharing research failures [Young, 2019] in case users wish to better understand the process of constructing a digital twin for integral effects tests such as CIET.

3.2 Justifying the use of Operator Splitting in Real Time Calculations for Fluid Mechanics and Heat Transfer

We first wish to discuss why operator splitting is preferred and sometimes necessary for digital twins.

As the digital twin being developed seeks to emulate physical phenomena in CIET, we must first consider the nature of such phenomena. One key phenomena that CIET was originally purposed to study was natural convection in the FHR for decay heat removal [N. Zweibaum et al., 2015]. Natural convection normally includes a temperature differential which causes a density difference between to regions of the fluid and therefore a pressure difference. This pressure difference drives mass to flow around the system. However, the magnitude of mass flow inadvertently affects the rate of heat transfer and therefore the temperature difference between hot and cold regions of the fluid. We can see that while temperature drives mass to flow around the system, mass flowrates also affect temperature difference. This system can therefore be described as “tightly coupled”.

Since the system is tightly coupled, we might be inclined to form a system of tightly coupled fluid mechanics and heat transfer equations that can be solved simultaneously. The issue with this approach is that the systems of equations is quite nonlinear. For example, we know that we cannot expect the pressure and velocity correlations in fluid flow to be linear in all regimes. Therefore, we may end up solving a non linear system of equations with multiple variables. For such a system, we might conceive of an iterative approach such as the multivariate newton raphson method to solve it. A flowchart depicting such a procedure is in figure 3.2.

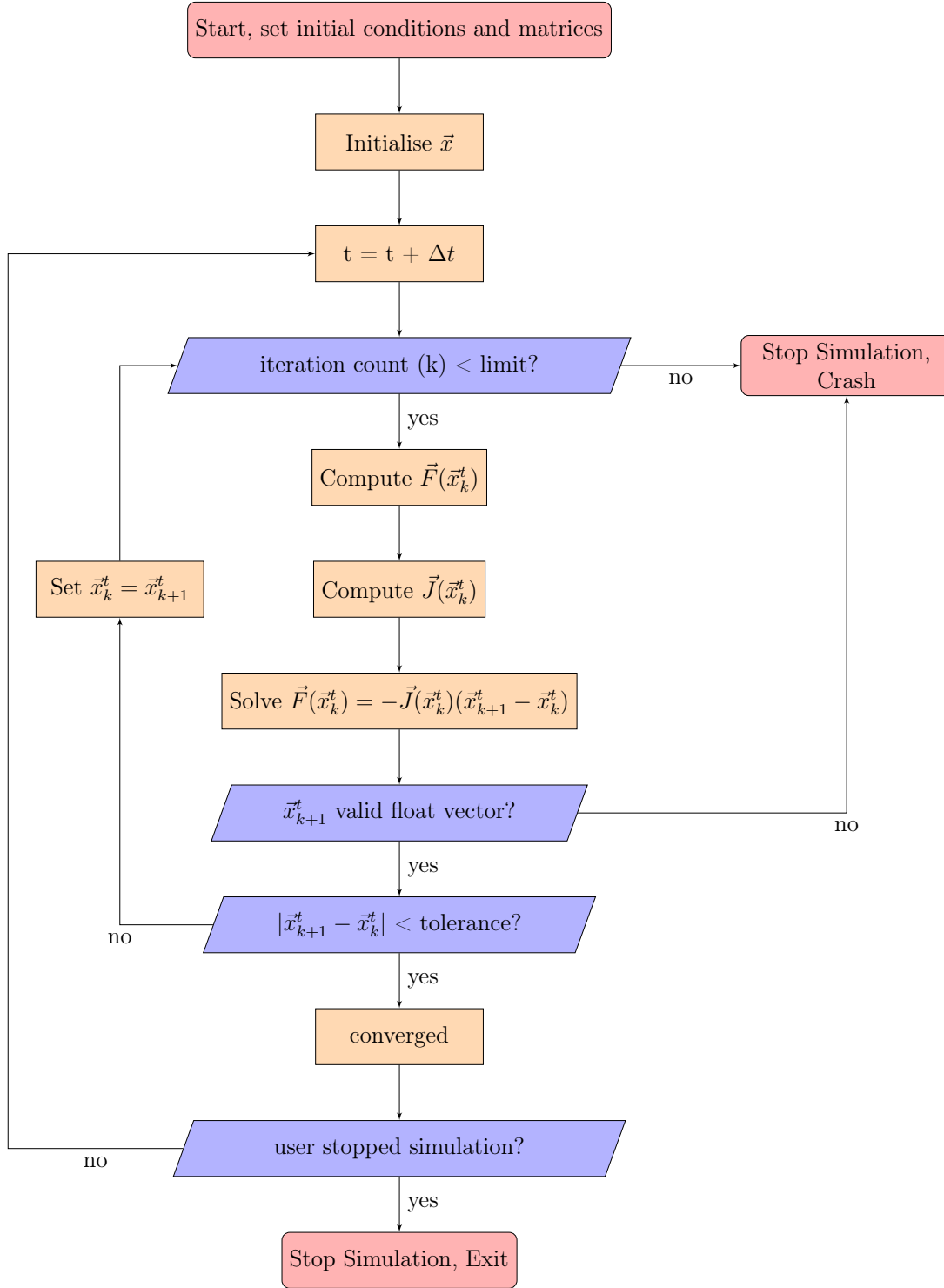


Figure 3.2: Newton Raphson Method Algorithm for Tightly Coupled Solution

In figure 3.2, $\vec{F}$ represents the nonlinear system of equations to be solved and $\vec{J}$ represents

the Jacobian. For interested readers, Appendix C.3 on page 269 contains some discussion of how one might obtain $\vec{F}$ and $\vec{J}$. No derivations are given, but a conceptual overview is presented.

The problem with methods such as the newton raphson method is, first of all, divergence [Kovacs, 2011]. Divergence can arise from the newton raphson method if we make a poor initial guess of all the relevant variables in our system of equations. This is shown as $\vec{x}_k^t$ in figure 3.2. The vector notation indicates that there are multiple variables stored within $\vec{x}$ which fully describe the physical state of the digital twin. k represents the number of iterations the solver has performed in the current timestep and t denotes the simulation time at which the calculations are being performed. If divergence occurs such that the iteration count exceeds a specified limit or if the solver somehow causes one of these variables to end up with a nonphysical value (eg. NaN or infinity), the simulation can crash.

The second issue with the newton raphson method is convergence time. Even if we do manage to find an initial point that leads to a stable solution, the total time taken for convergence may be more than what is tolerable for a digital twin. A digital twin requires a real-time calculation of system variables. For CIET in particular, sensor data is given to the user approximately every 95 ms. Therefore, an upper limit on the amount of time allowed to solve the system of equations at each timestep is about 100 ms. (I'm using 100 ms as a point of reference because it is easier to remember). The key calculation which might slow down the rates of convergence for the newton raphson method is matrix multiplication. Matrix multiplication is at best $\mathcal{O}(n^{2.37188})$ in literature [Duan, Wu, and Zhou, 2022]. This means that for an n by n matrix, the computational complexity and time increases by more than the square of the number of elements. Tightly coupled solution schemes require that both heat transfer and fluid mechanics equations be solved simultaneously. If we could solve these equations separately, we could reduce the computation time required. This comes at the expense of errors that occur due to the splitting [MacNamara and Strang, 2016]. Nevertheless, having a digital twin produce approximate results within the specified timeframe is better than having no results at all within the time limit. We only need ensure that the errors are within an acceptable range. This means that the sum of all errors should be within experimental uncertainty, or within percentage errors for system codes within literature. One typical estimate of percentage errors for a system code modelling CIET is within 3% which was the case for FHR Natural Circulation Analysis (FANCY) [N. Zweibaum et al., 2015].

Given these two reasons, we wish to avoid solving fluid mechanics and heat transfer equations simultaneously. Additionally, we also wish to avoid solving nonlinear systems of equations simultaneously as well because of the stability issues with the multivariate Newton Raphson method. Suffice to say that during code development, I have had enough frustration with multivariate Newton Raphson instability in general to want to avoid it altogether. A slightly more comprehensive account can be found in the appendix for interested readers in the section where I discuss the use of SpiceSharp Flow solvers.

3.3 Solution Procedure for Parallel Branches of Fluid Flow

Overview

In this section, I describe how the mass balance solver is developed. This is merely a first step as code from this solver can be further developed by adding energy balance and heat transfer equations in future iterations of CIET's digital twin.

We first start from a general discussion of what algorithms to choose for fluid mechanics solver and then focus specifically on parallel branch solvers.

Justifying the Choice of Algorithm

For this digital twin, we want to use nodalised solvers one-D pipe network solvers which are similar to those used in system level codes such as RELAP and SAM. These solvers have less emphasis on fidelity and more emphasis on speed compared to a partial differential equation solver. Thus, these are preferred for the digital twin outlined in this thesis.

To do this, first, we consider the mass balance equations. In CIET, we mainly deal with incompressible flow. In fact, Dowtherm A and air flows can be treated as incompressible as both are less than 33% the speed of sound in either medium. The only thing changing fluid density is temperature. This means that pressure and flow profiles equilibrate almost instantaneously and therefore we can treat fluid flow as steady state.

The incompressible assumption may be true for isothermal flow so that we can just assume density is constant. However, in a non isothermal case, the fluid density may change with temperature. If we were to write a solver assuming density is constant in the isothermal case and this assumption is wrong in the non isothermal case, we may have to rewrite the solver. This limits the usefulness of this work. However, the Boussinesq approximation can apply to Dowtherm A or Therminol VP1 here because they are liquids. The Boussinesq incompressible assumption is often used for natural convection boundary layer problems in literature [Bejan, 2013] and assumes that density is considered constant everywhere except where body force or buoyancy forces are concerned. This allows us to neglect any mass accumulation within each pipe. Thus, time dependent terms in the mass balance are zero. This supports us in the use of the steady state assumption.

This means that for a series of pipes, the mass flowrate through each component ($\dot{m}_i$) is the same throughout the series of pipes. This makes the mass balance trivial to solve.

$$\dot{m}_i = \text{constant} \quad (3.1)$$

For momentum balances, we can say that at steady state, the pressure loss across the series of pipes is the sum of the of the pressure losses through each pipe component. This can be written as follows:

$$\Delta P_{series} = \sum_i^n \Delta P_{loss\ i} \quad (3.2)$$

This equation of course supposes that the sum of the pressure losses is equivalent to the pressure supplied by a pump driving flow through the series of components (ΔP_{pump}) at the system's operating point. Note that P in our context here necessarily represents pressure in pascals.

Where the pressure loss across each component $\Delta P_{loss\ i}$ can be written as [Perry and Green, 2015]:

$$\Delta P_{loss\ i} = \frac{1}{2} \rho u^2 (f_{darcy} \frac{L}{D} + K) \quad (3.3)$$

Here ρ is the fluid density, u is fluid velocity, L is the component length, D is the hydraulic diameter (sometimes i might refer to it as D_H) f_{darcy} is the darcy friction factor, K is the form loss coefficient. This equation is known as the Darcy Weisbach equation

Of course, in CIET, we do not measure flow velocity directly, but rather mass flow rates from Coriolis flowmeters. It is therefore more convenient to rewrite the Darcy Weisbach in terms of mass flowrate:

$$\rho A_{xs} u = \dot{m} \quad (3.4)$$

ρ is fluid density, u is fluid superficial velocity and A_{xs} is cross sectional area. $\dot{m}$ is the mass flowrate.

$$\Delta P_{loss\ i} = \frac{1}{2} \frac{\dot{m}^2}{\rho A_{xs}^2} (f_{darcy} \frac{L}{D} + K) \quad (3.5)$$

The term in the brackets is known as the pressure loss coefficient. In my code however, I prefer to call it the “fldk” term or variable because naming the term “pressure loss coefficient” may lead users to question which pressure loss coefficient is being referred to. To prevent any ambiguity and increase code readability, I call it the “fldk” term in my code.

The pressure loss coefficient will be a function of the Reynolds number. As a reminder, we define the Reynolds number in terms of both fluid velocity and mass flowrate:

$$Re = \frac{\rho u D_H}{\mu} = \frac{\dot{m} D_H}{A_{xs} \mu} \quad (3.6)$$

Usually density ρ and dynamic viscosity μ are both a functions of temperature:

$$\rho = \rho(T); \mu = \mu(T) \quad (3.7)$$

Now eventually, to get the temperature of the fluid, we will have to solve heat transfer equations. Energy will be related to fluid temperature via the heat capacity and enthalpy:

$$\Delta H_{control\ volume} = \rho \int_{T_{initial}}^{T_{final}} c_p(T) dT \quad (3.8)$$

This heat capacity c_p is temperature dependent in addition to thermal conductivity, k , density ρ and viscosity μ . We will need such correlation data from a database or fluid package in order to solve these equations. This will give us a pathway to map heat transfer and enthalpy to temperature, and temperature to fluid properties. These fluid properties will then fluid affect the fluid flow equations.

Now we can see fluid flow equations are dependent in some form on temperature. And since heat transfer is involved, we must solve energy balance equations in order to get temperature profiles.

$$\frac{\partial H}{\partial t} = \sum_i H_{in} - \sum_i H_{out} + \Delta Q \quad (3.9)$$

In each pipe, the above energy balance equation will hold. Here ΔQ is heat transferred to the control volume of fluid either volumetrically or at the boundaries. H_{in} and H_{out} represent the flows of enthalpy flowing in and out of the control volume respectively due to advection.

Of course, as stated earlier, we use the operator splitting method so that solution procedures can be sped up. Hence, at each timestep, the temperature profile used for fluid mechanics calculations is assumed to be constant. We can then use a fixed temperature profile to calculate the fluid properties and buoyancy forces to be used in fluid mechanics calculations.

Practically speaking however, experimental data with known fixed temperature profiles and pressure loss data is difficult to find in literature outside the context of heat transfer experiments. For those, we might need to develop heat transfer solvers in order to try and reproduce that data. Outside heat transfer experiments, isothermal pressure drop experimental data is available [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]. Therefore, we can develop solvers for isothermal flow as a first step to developing a fluid mechanics solver with capability for a non uniform temperature profile across CIET.

Why Parallel Branch Solvers are Important in CIET

Even if we were to just solve fluid mechanics equations alone, we would also need to sort out the problem of flow in parallel series of pipes. This is because CIET is set up in such a way as to allow parallel flow. This is best seen in Figure 3.3, which is a graphical representation of the nodalised model for CIET used in the System Analysis Module (SAM) [Zou, R. Hu, and Charpentier, 2019]. I have excluded most of the instrumentation from figure 3.3 so that the figure will not look too overcrowded. Only manometers M-42 and M-43 and included because loop pressure drop experimental data used to validate CIET's Type I Digital Twin will use readings from these two manometers.

Compact Integral Effects Test (CIET) Nodalised Model Diagram



Figure 3.3: Approximate Nodalised Model of CIET without Bypass Branch and Flow Direction

If we refer to figure 3.3, we can see that Dowtherm A or Therminol VP 1 would flow from

the CTAH pump into one of two possible branches, (1) the heater branch, (2) the DRACS Heat Exchanger (DHX) branch. A third branch, known as the bypass branch, also physically exists in CIET and runs in parallel to the heater branch. However, it is not modelled in SAM [Zou, R. Hu, and Charpentier, 2019] or RELAP in earlier studies [N. Zweibaum et al., 2015]. Therefore it is excluded from figure 3.3.

The bypass branch can be valved off so that no flow can go through it depending on the experiment for CIET. Similarly, the DHX branch has a check valve [N. Zweibaum et al., 2015] which is configured as a check valve and adjustable needle valve in parallel [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]. This means that depending on valve alignment, upward flow through the DHX branch may be blocked in the upward direction.

Due to the presence of parallel flow paths in CIET, we must develop solvers that are capable of accounting for these situations. In such a case, the following equation may not suffice in fully describing flow in the entire loop:

$$\Delta P_{pump} = \sum_i^n \Delta P_{loss\ i} \quad (3.10)$$

As such, we would need to solve different equations which sufficiently describe parallel flow paths within CIET. Additionally, we must solve these equations within the time constraints stipulated for digital twins.

Choice of Algorithm for Solving Parallel Flow Paths

The chosen algorithm for solving flow through parallel paths in laminar, transition and turbulent regimes is one where we approximate CIET as a collection of parallel branches. In figure 3.3, we could consider the CTAH branch splitting into two separate flow paths in the heater branch and DHX branch at branch 17. This is shown as 17a and 17b in figure 3.3. Flow then rejoins at branch 5, shown as 5a and 5b in figure 3.3 and combines into the CTAH branch. The bypass branch was not modelled extensively in literature, but to fully model CIET, we may want to include the capability to include the bypass branch if new experimental data is available in future. If we wanted to do so, we would need to know exactly where the flow branching occurs. Existing model data in RELAP and SAM is not available in literature. However, the Piping and Instrumentation Diagram (P&ID) of CIET available in literature [N. Zweibaum et al., 2015] shows that the bypass branch does not branch off at exactly the same place as the heater branch. This branching occurs somewhere between 17b and flowmeter 40 (FM-40) and the flow regains somewhere between 5a and the CTAH.

A solver which replicates this flow setup may be extremely costly in time as we will later discuss. Hence, an approximation could be chosen if the parallel bypass branch can be considered to split off at branch 17 and rejoin at branch five at exactly where the heater and DHX branches diverge and rejoin back to the CTAH branch.

This solution was considered better compared to other alternatives I had during code development. For interested readers, a chronological account of the trial and error process is given in the appendix in page 269. I will however, present the tried and failed solutions in a logical and easy to read manner rather than a chronological manner.

How the Parallel Branch Solver Works

For parallel branch flow solutions in general, we consider two fundamental equations that describe flow across a parallel set of branches.

Firstly, if flow from the branch such as the CTAH branch splits into two parallel flow paths, conservation of mass can be represented by the following equation:

$$\dot{m}_{total} = \sum_i^n \dot{m}_i \quad (3.11)$$

What equation 3.11 is saying is that the total mass flowing from the CTAH branch is the sum of the two mass flowrates flowing through the DHX and heater branch. This can be extended to a general case where a larger pipe splits off into n parallel flow paths.

Secondly, the conservation of energy is represented in an analogue of Kirchoff's Voltage Law adopted for fluid components. Kirchoff's Voltage Law states that the potential difference (analogous to pressure change) around any closed loop is zero. Practically, what this means is that the pressure change across any branch is the same.

The pressure change here is the sum of contributions from pressure losses, hydrostatic pressure and any other pressure sources within the branch. The most common of which are pumps, which must have their respective pressures iteratively calculated using pump curves. For this discussion, we simplify this by assuming that the frequency of the pump's variable frequency drive (VFD) is adjusted to such a point where a fixed pressure increase is provided by the pump. These statements can be summarised in equation 3.12.

$$\Delta P_{change \ branch} = - \sum_i^n \Delta P_{loss \ i} + \sum_i^n \Delta P_{hydrostatic \ i} + \sum_i^n \Delta P_{source \ i} \quad (3.12)$$

For each branch of components in series, we can use equation 3.12 to iteratively guess a mass flowrate given a pressure change across the whole branch. This is illustrated in Figure 3.4.

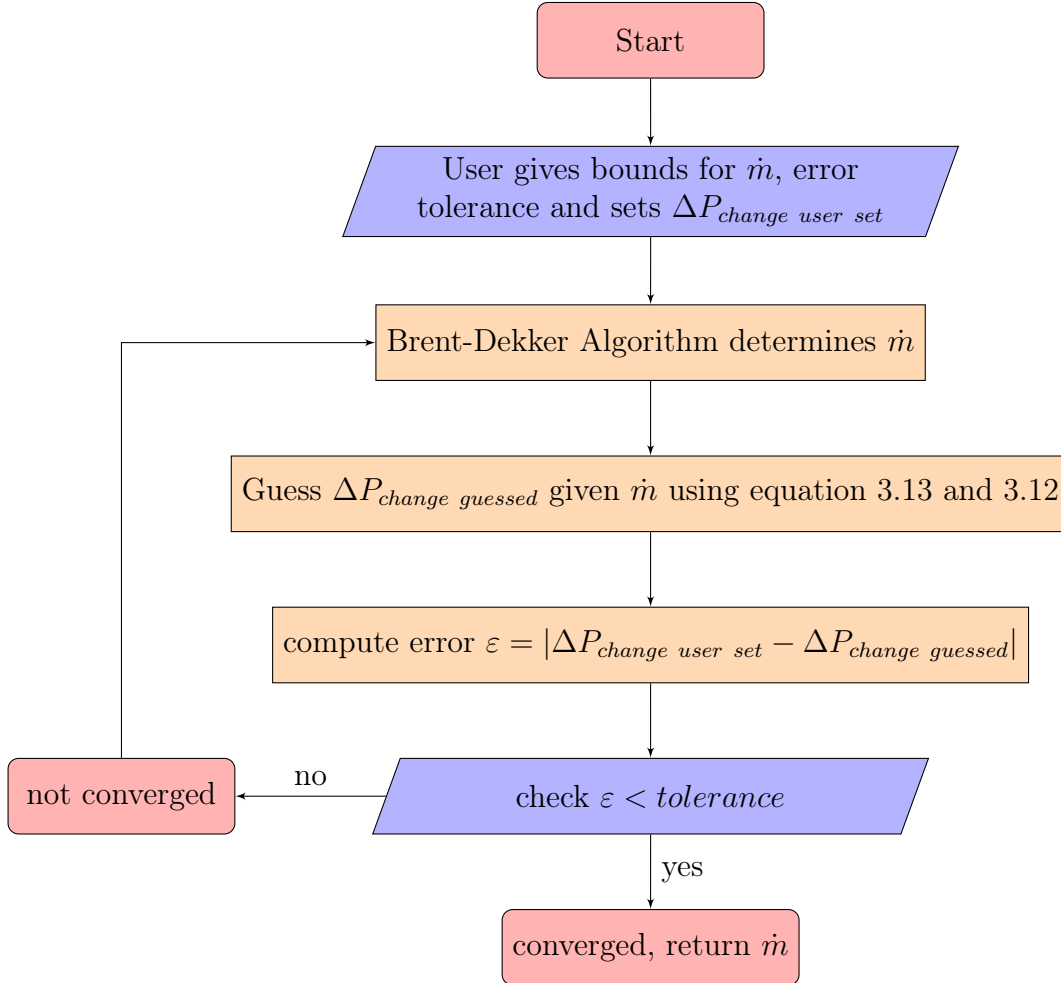


Figure 3.4: Iterative Solution Procedure for Series of Fluid Components

To calculate the pressure losses within each branch, we would then apply the Darcy Weisbach Equation to each component within the branch:

$$\Delta P_{loss\ i} = \frac{1}{2} \frac{\dot{m}_i^2}{\rho A_{xs}^2} (f_{darcy} \frac{L}{D} + K) \quad (3.13)$$

Using this procedure, we have found a way to iteratively find mass flowrate over each branch given a pressure change. We can use this procedure to solve for mass flowrates in parallel branches. As illustrated in Figure 3.5, the solution procedure would firstly involve specifying a total mass flowrate $\dot{m}_{total}$ through all branches. We would then guess the pressure change across each branch to obtain the mass flowrate through each branch $\dot{m}_i$. Once this is done, we sum $\dot{m}_i$ across all branches to see if it equals $\dot{m}_{total}$. If the error is too large, we iteratively guess the pressure change until the desired $\dot{m}_{total}$ is reached.

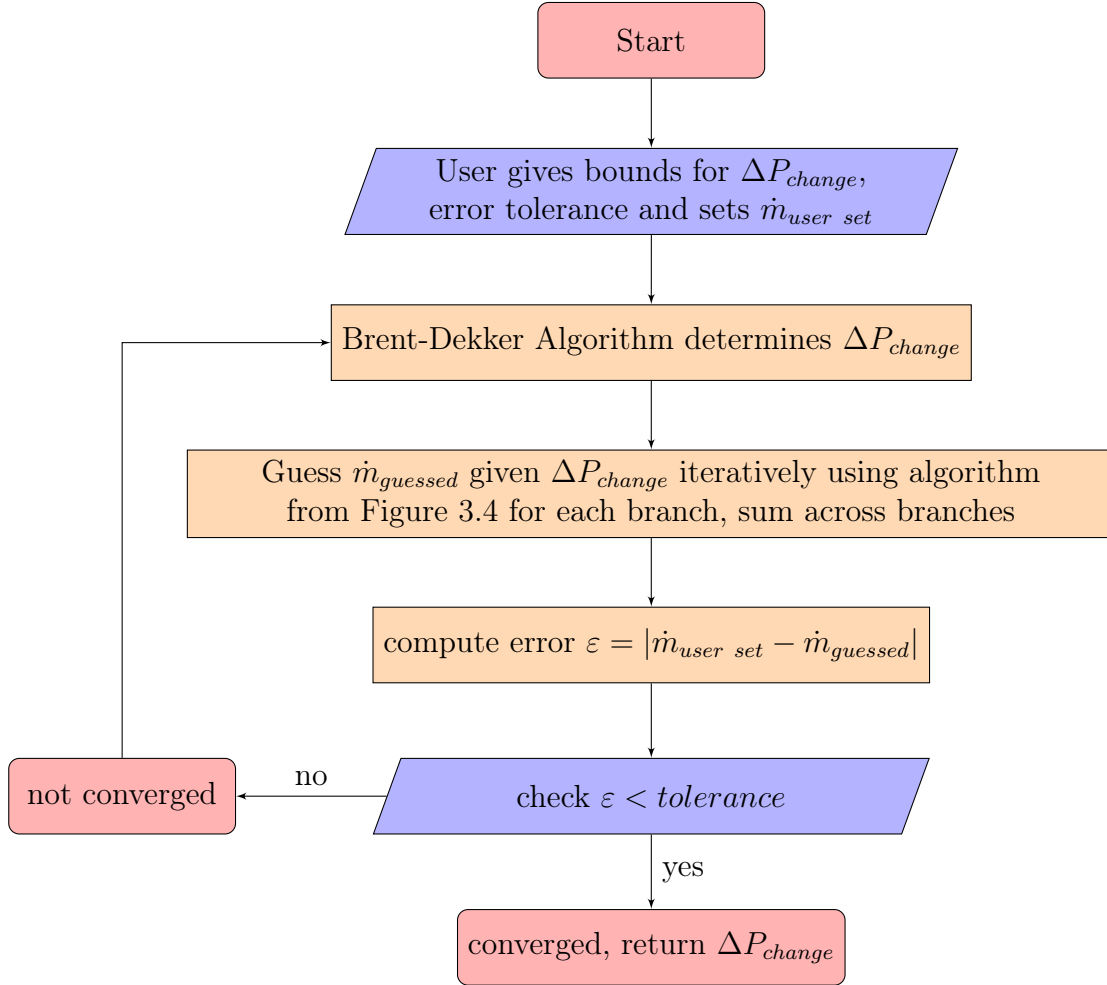


Figure 3.5: Iterative Solution Procedure for Parallel Branches of Fluid Components

Now, this is the general solution procedure. However, solving equation 3.12 using equation 3.13 is problematic because the pressure source $\Delta P_{source\ i}$ and is in general a function of $\dot{m}_i$ and pump frequency. To help solve this, we can shift the terms independent of $\dot{m}_i$ to the left hand side. The left hand side (LHS) would be constant while the right hand side is an expression in terms of $\dot{m}_i$. As fluid properties, pipe dimensions and pressure loss correlations are considered constant at every timestep, the only variable that is changed is $\dot{m}_i$. $\dot{m}_i$ is considered constant for all components as they are considered to be connected in series.

$$\Delta P_{change} - \sum_i^n \Delta P_{hydrostatic\ i} = - \sum_i^n \Delta P_{loss\ i} + \sum_i^n \Delta P_{source\ i} \quad (3.14)$$

We could then substitute equation 3.13 and the pump curve correlation into equation 3.14 and have a solver solve iteratively for $\dot{m}_i$. Iteration is a must because one cannot simply factorise $\dot{m}_i$ out. This is because the pressure loss coefficient $f_{darcy} \frac{L}{D} + K$ is a function of $\dot{m}_i$.

We can simplify our procedure by assuming that the ΔP_{source} is constant to simplify this our model similar to how the SAM model is simplified in literature [Zou, R. Hu, and Charpentier, 2019]. For this work, ΔP_{source} for the CTAH pump is a user specified input. However, in SAM, pump head rather than pump pressure ΔP_{source} is specified as a user input. This was done to speed up development as it was simpler to implement in code. With pump head, one would need to consider density in addition to pump head in order to calculate the required pressure source. Pump curve behaviour could perhaps be implemented in future iterations of the digital twin to give the end user a more realistic feel when operating this digital twin.

$$\Delta P_{change} - \sum_i^n \Delta P_{hydrostatic\ i} - \sum_i^n \Delta P_{source\ i} = - \sum_i^n \frac{1}{2} \frac{\dot{m}^2}{\rho A_{xs}^2} (f_{darcy} \frac{L}{D} + K) \quad (3.15)$$

The resulting equation 3.15 is to be iterated until the desired pressure change is reached. This means that the we have to solve iteratively for mass flowrate within the inner loop, and solve iteratively for pressure change in the outer loop. In such a case, we have a nested iterative procedure to solve for pressure change over a series of parallel branches given a fixed mass flowrate.

For CIET, we could then adopt this procedure and consider that the CTAH branch, DHX branch and Heater branch are three separate branches in a collection of parallel flow paths. The mass flowrate across this collection is set to 0 kg/s and we then execute the algorithm to iteratively find the pressure change across each of these branches. Once the pressure change across these branches is known, the mass flowrates across each branch can then be calculated using an iterative procedure.

For inner and outer iteration loops, I have opted to use the Brent Dekker method [Stage, 2013] in my code. This algorithm can be thought of as a hybrid iteration method which combines the speed of the secant method and the stability of the bisection method. I have set the iteration limits of pressure change to the hydrostatic pressure of any one of the branches ± 50000 Pa because during isothermal flow tests, the pressure supplied by the pump at the operating point does not usually exceed 25000 Pa. The bounds for mass flowrate in each branch is ± 1.00 kg/s because in isothermal flow tests, mass flowrates for the CTAH branch are at most 0.18 kg/s [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014; N. Zweibaum et al., 2015].

Failed Alternative Methods Attempted

In the course of trial and error, other methods were attempted for solving flow in parallel flow branches. One of these methods was to consider the DHX and Heater branches as parallel flow branches in a collection of fluid components in series. For this, a fixed loop pressure drop (or “pump pressure” as named in my source code) was supplied by the user. The solver would attempt to iteratively guess the mass flowrate through the series of components which could achieve the specified loop pressure drop. We would essentially solve for equation 3.12

iteratively setting ΔP_{change} and $\sum_i^n \Delta P_{hydrostatic\ i}$ to zero. However, calculating the pressure loss across the parallel flow branches would require a two more levels of nested iteration as shown in figure 3.6.

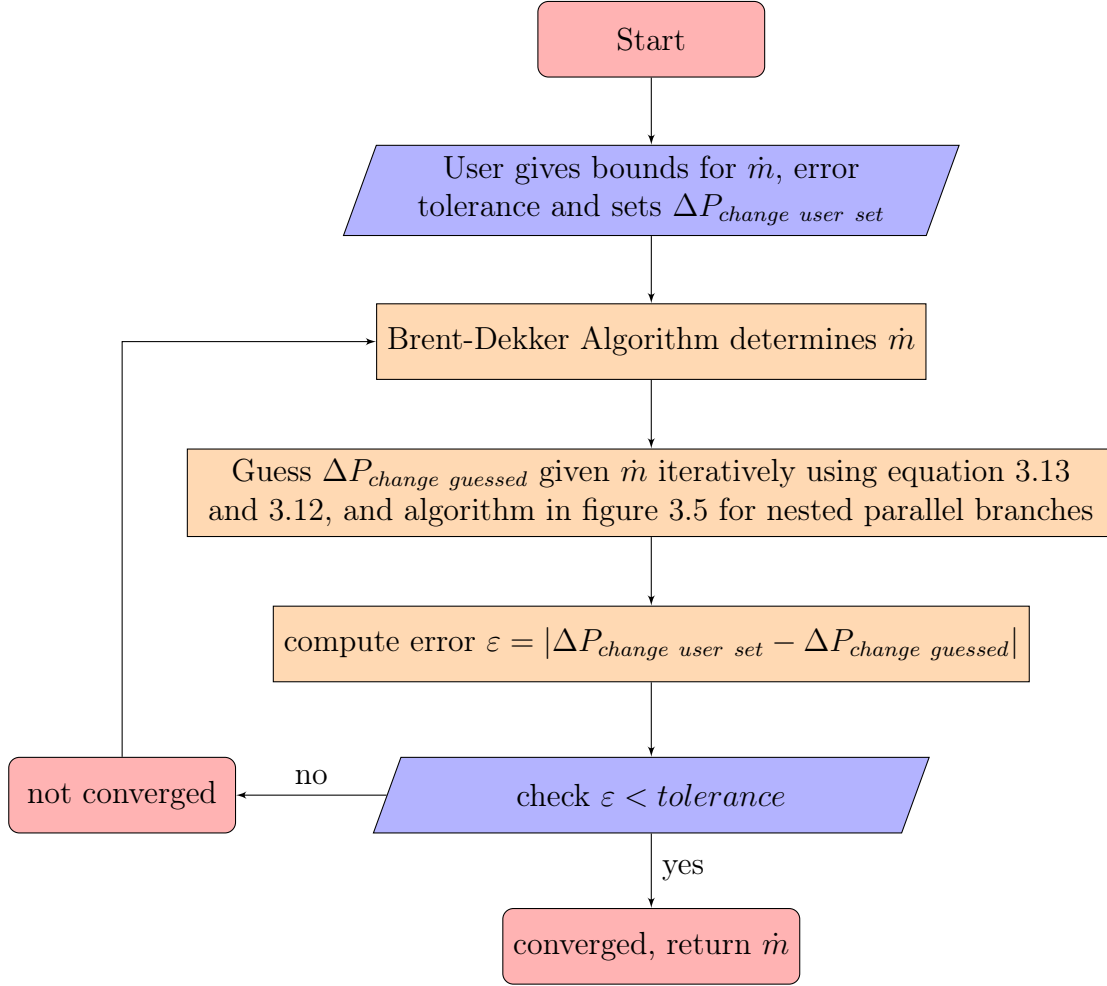


Figure 3.6: Iterative Solution Procedure for Series of Fluid Components with Nested Parallel Branches

Thus, calculations would have taken too long to be tolerable for use in digital twins as shown in figure 3.7.

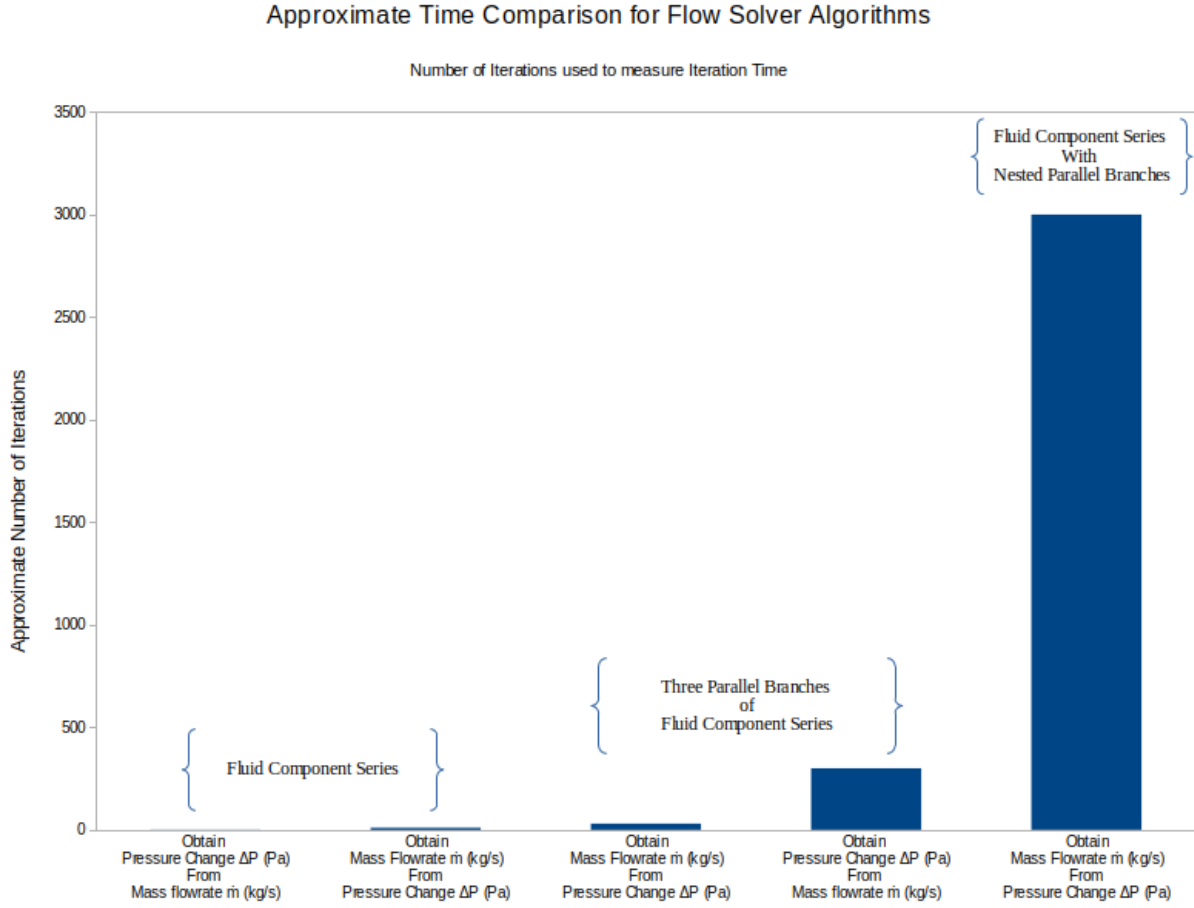


Figure 3.7: Approximate Computation Time for Nested Iteration Loops

Another method considered was to assume the pressure loss coefficient $f_{darcy} \frac{L}{D} + K$ effectively constant with respect to $\dot{m}_i$. This would accurately represent CIET if form losses of the system greatly exceed the darcy friction factor term. In such a case:

$$\Delta P_{change} = - \sum_i^n \frac{1}{2} \frac{\dot{m}^2}{A^2 \rho} (K) + \Delta P_{hydrostatic} + \Delta P_{source} \quad (3.16)$$

Now, if there is little density variation, we may proceed to factorise the other terms in the summation term out:

$$\Delta P_{change} - \Delta P_{hydrostatic} - \Delta P_{source} = - \frac{1}{2} \frac{\dot{m}^2}{\rho} \sum_i^n \left(\frac{K}{A^2} \right) \quad (3.17)$$

Rearranging and simplifying:

$$\Delta P_{hydrostatic} + \Delta P_{source} - \Delta P_{change} = \frac{1}{2} \frac{\dot{m}^2}{\rho} \sum_i^n \left(\frac{K}{A^2} \right) \quad (3.18)$$

$$\dot{m}^2 = 2\rho \frac{\Delta P_{hydrostatic} + \Delta P_{source} - \Delta P_{change}}{\sum_i^n \left(\frac{K}{A^2} \right)} \quad (3.19)$$

In such a case, mass flowrate in each series of pipes found without iteration provided we know the pressure change.

If the cross sectional area varies little across the series of pipes, we can find a simplified expression for mass flowrate over each branch:

$$\dot{m} = \sqrt{2\rho(\Delta P_{hydrostatic} + \Delta P_{source} - \Delta P_{change})} \frac{A}{\sqrt{\sum_i^n (K)}} \quad (3.20)$$

For parallel flow paths without pressure sources and equivalent hydrostatic pressure, and with nearly constant flow area, flow across parallel paths is proportional to $\frac{A}{\sqrt{K_{total}}}$. It is a useful case where we can save ourselves from iterating in order to obtain mass flowrate across each flow path. However, for CIET, flow is often in the laminar region especially when the loop operates in natural circulation mode [N. Zweibaum et al., 2015] or perhaps in loss of forced cooling (LOFC) scenarios where the pump ceases to work. If we wished to pick an algorithm which worked in all cases, then iteration would be the way to go.

A third method attempted but failed was to precalculate the system curve of the parallel branch collection and condense these branches into single equivalent component with an effective cross sectional area A_{xs} , density ρ , and form loss coefficient $f_{darcy} \frac{L}{D} + K$. This was an attempt to do away with the nested iteration procedure. However, derivations proved to be error prone and laborious. Problems also arose when non uniform temperature distributions across each branch were considered, thus leading to natural circulation within the parallel branches. These situations were too complicated to consider analytically and therefore the effort was abandoned.

A fourth method attempted was to attempt to adapt electrical circuit solvers such as SpiceSharp [Boulanger, Golebiowski, and Sodalom, 2017] for fluid flow. SpiceSharp is an object oriented circuit solver library which was designed for solving current and voltage for electrical circuits with several parallel flow paths for current. It was also designed to solve for systems with nonlinear components such as diodes and resistors which do not obey Ohm's law. This gave hope that the solvers within SpiceSharp could be adapted for fluid components which also behaved like resistors which do not obey Ohm's law. Unfortunately, the multivariate Newton Raphson solvers within SpiceSharp proved to be too unstable for any use in the context of this work. This further confirmed my preference in avoiding solution of coupled nonlinear equations as far as possible. For interested readers, a chronological account of the trial and error process is given in the appendix in page 269.

3.4 Choice of Coding Language for the Digital Twin

Justifying the Use of Rust

Now that we have discussed the algorithms used for parallel flow branches, we now discuss the choice of language for the digital twin and its libraries. The language of choice I used was Rust. Rust was chosen because it made memory safety and concurrency easier with its borrow checker and helpful compiler. It also tends to be less verbose and cognitively complex than C or C++ [Ardito et al., 2021]. Additionally it is a systems level code that can be used for high speed applications with speed comparable to C and C++. Furthermore, it contains useful libraries such as the Units of Measure (uom) library [Boutin, 2022] and various numerical solver libraries such as peroxide [Kim, 2022] which can help perform numerical calculations. The uom library forces unit safety which prevents accidental addition of two quantities of the wrong units. This has proven very helpful in reducing computational errors during code development of the digital twin.

For interested readers, a more verbose introduction into the thought process, code snippets and the source code are all available in the appendix.

Justifying the Use of Maturin, the Python-Rust Interface

However, solvers were not fully written in Rust because of the choice of OPC-UA Server used. The OPC-UA Server eventually chosen was the Python implementation [Oroulet et al., 2022]. Python OPC-UA servers have been used in digital twin creation and research [Zidek et al., 2020] and therefore this has set a precedent for using Python OPC-UA Servers.

As a result, a suitable interface between Rust and Python code was used. This was the Maturin PyO3 interface. This allows functions written in Rust to be called by python code. Thus, the speed of Rust binaries is made available when executing Python code.

Use of SciPy Libraries

Since Python Code is required, it is sometimes easier to use solvers existing in Python Libraries such as SciPy. SciPy has useful numerical algorithms and libraries written in C, C++ and Fortran [Virtanen et al., 2020]. This allows python to execute code with the speed of underlying C, C++ or Fortran binaries. In particular, the Brent Dekker Method in SciPy was used for this work.

Why other Languages were not Used

Fearless Concurrency in Rust Rust was used in preference to C and C# because of concurrency issues within the latter codes. In C#, data races occurred frequently during unit testing (via xUnit [James Newkirk, 2022]) likely as these unit tests were accessing shared libraries in a multithreaded manner. Thus, exceptions were thrown once every ten to twenty unit tests. For the non computer science reader, data races and race conditions usually occur

because two parallel processes try to access and change the same piece of data concurrently. The net result of this is that the same function can return different values depending on which process finishes first. These functions are in a sense “racing” against each other, and hence the name of “data race” and “race condition”. This is most commonly encountered during parallelisation. While parallel computing is not used in this work, it may be required in future if extra speed is needed. Thus, writing in a language that has built in safeguards against such data races is also a decision meant to enable less frustration with parallelism in future.

Rust has built in safeguards against such data races in that code will generally not compile if it tends to cause data races. Therefore, Rust was preferred over C#, C and C++, which lacks these comprehensive compile time checks. Interested readers may find more discussion about these issues in the Appendix C.3 on page 279.

Speed of Rust The second reason why I prefer Rust in place of Python and C# is due to its speed. To prove this point, we can look at some code benchmarks submitted online.

NBody Performance Benchmarks for Various Languages To compare the speeds of the codes Rust, Python, C#, C and C++, one could look into performance benchmarks for each of these codes in performing simulations. One simulation which has a similar computational intensity as the digital twin is the nbody simulation.

The Nbody simulation is one where there are a set number of entities or bodies in a region of space. Each entity exerts a gravitational force on all other entities. Newton’s Third law is used as the basis that each gravitational interaction has an action and reaction pair. And this would result in a densely populated matrix with which to solve.

The equations would be solved with some integrator to get the positions, velocities and accelerations of the next time step. In the code, we would encounter many for loops. This benchmark is similar enough to calculations performed in digital twins (except for the nested iterations), to help estimate the relative performance of each language when running digital twins. After searching an online benchmarking website online, some estimates of NBody calculation times were found [Hanabi1224, 2022a; Hanabi1224, 2022b; Hanabi1224, 2022c] and plotted in figure 3.8.

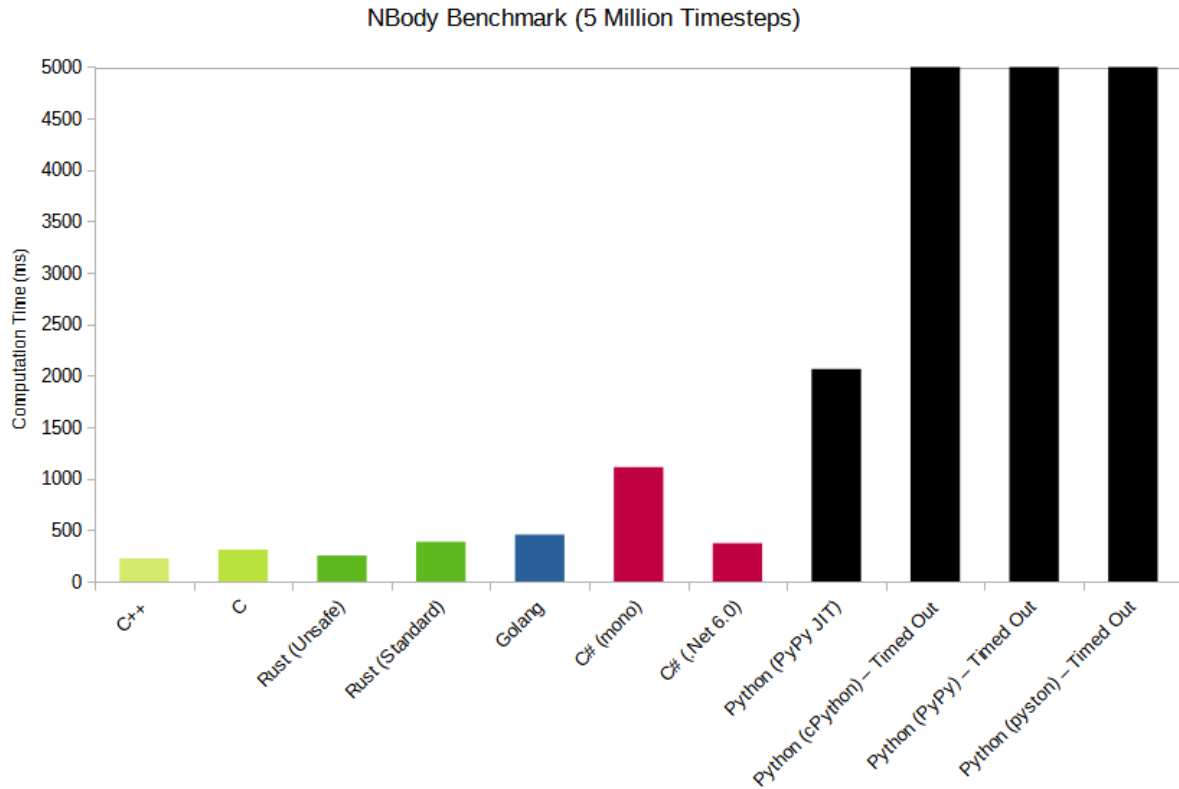


Figure 3.8: Summarised Chart of Some NBody Benchmark Simulation Times

The trials were carried out on the following hardware [Hanabi1224, 2022a] and were retrieved in Oct 2022:

```
CPU INFO:[x86_64][2 cores] Intel(R) Xeon(R)
Platinum 8370C CPU @ 2.80GHz (Model 106)
```

For this NBody particle benchmark, there were five bodies (Sun, Jupiter, Saturn, Uranus and Neptune) in the system (more bodies means a slower calculation). The calculations were carried out over 5 million time steps and the calculation times were tallied and reported.

We can see that from this benchmark, python was the worst performing of all the codes even using the PyPy Just in Time (JIT) compiler meant to speed up python code. Of these select codes that did not time out, PyPy's JIT compiler ran the slowest at 2065 ms. C# was the next slowest, and it ran at 373 ms using dotnet 6.0 and at 1113 ms using the mono implementation. Another popular new language, Golang had similar computation times to C# at 455 ms. The fastest codes were written in C++, C and Rust. C++ code ran the benchmark at 225 ms while C ran the benchmark at 308 ms. This is an important reference point because most scientific computation software requiring speed is written in C and C++. Two Rust code submissions were observed, a standard Rust binary which ran the benchmark

in 387 ms and an unsafe Rust binary which ran the benchmark in 252 ms. It should be noted that the Rust code in this dissertation is standard or “safe” Rust.

While C++ was indeed faster than Rust, the memory safety and relative ease of use of Rust made it the ideal low level, high speed, high memory efficiency and memory safe code to use for calculations both in serial and parallel mode.

We also can see that C# is faster than “safe” or standard Rust, the peak memory use for Rust was typically 0.6 to 0.7 MB, whereas for C#, we had peak memory usage of about 27.3 MB. In terms of memory usage, Rust is vastly superior to C# for this benchmark, because C# used about 39 times as much memory as Rust in terms of peak memory.

It should be noted however, that there are many factors when it comes to code performance besides language. For example, we should check if the code is optimised or not, and we should also ensure that an equal number of threads is used. The caveat with this quick search is that I did not inspect every code manually to see if the programs are optimised to a similar level. However, the point of looking into these benchmarks to get a rough estimate as to what speeds each language is capable of. And we can see evidence from this benchmark that Rust indeed has the capability to perform as well as C in terms of speed.

Libraries for Unit Safety The third reason I chose Rust over other languages such as Python is because of libraries written in Rust for unit safety. This means that the code should throw an error, preferably at compile time, when two values of dissimilar units are added or subtracted together. This is possible because of static typing in Rust as compared to Python. Having the code or compiler check for unit safety is important because it is common for engineers to make mistakes with units in their calculations especially when using software for calculations. Take for example the famous Mars Polar lander [Oberg, 1999], where the use of incorrect units caused US\$125 million (in 1999 dollars) of damage. As the benefits of Unit Safety seemed so attractive, I decided to pay less attention in searching for libraries written in Python such as Pyomo.

The reader should note that unit safety is not unique to Rust. C and C# also have libraries for unit safe calculations. One example of a C# package for unit safety is known as EngineeringUnits [MadsKirkFoged, 2021] which is based off a larger library known as UnitsNet. While the C# packages are more mature compared to Rust, (UnitsNet was copyrighted in 2013 while UOM was copyrighted in 2019 [Boutin, 2022]), the Rust libraries tend to favour compile time errors rather than runtime errors. This is advantageous for development work. Therefore I used the uom library (or crate as it is known in Rust), for development of the digital twin.

Notes about Rust Binary Compilation and Optimisation

While Rust was chosen because it is a high speed code, we would not necessarily want to optimise the Rust code for the sake of study. For example, I will compile Rust code in debug rather than release though release can help the code run faster.

The aim of this thesis is to understand how to construct a digital twin and what factors affect speed and accuracy. Since it is known already that compiling in release mode increases speed, we shall leave that aspect out of this study.

Implementation Issues with Unit Safety in a Two Language Digital Twin Library

Using two languages in a codebase can be quite cumbersome. One issue faced is where the Unit Safety library or “crate” within Rust is not quite compatible with a dynamically typed language such as Python. Therefore, within Rust code, unit safe types are used whenever possible. However, in the maturin PyO3 interface between Rust and Python, I can no longer use Unit Safe types conveniently. This is because of the extra complexity when introducing new classes or types across the Python, PyO3 and Rust interface. Instead I ensure that I always use primitive return types such as double floating points and annotate the input and output variables with the appropriate units. Within the Rust code however, I ensure that the uom crate is used.

The following excerpt of Rust code shows how this is performed:

```
let mut pressure_change_total =
    Pressure::new::(<pascal>(0.0));

// branch5 and pipe4
//
pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &branch_5,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_4,
        mass_flowrate,
        fluid_temp);
```

In the above code, I am calculating the pressure changes, according to the following equation:

$$\Delta P_{change} = - \sum_i^n \frac{1}{2} \frac{\dot{m}^2}{A^2 \rho} (f_{darcy}(Re) \frac{L}{D} + K) + \Delta P_{hydrostatic} + \Delta P_{source} \quad (3.21)$$

I start off by defining a variable of the type pressure in the pascal unit. The quantity is zero, so it is zero pascals. I then sequentially sum up the pressure changes from each

component until the pressure change for the entire branch is calculated. Once I calculate the pressure change as a unit safe object, I then convert it to a double floating point number (f64). This data type is compatible with Rust and Python. To add an additional layer of safety however, I try to include the unit type in the function name so it will be clear to the user what units to expect when calling the function even though it returns a f64 value type.

```
#[pyfunction]
fn get_heater_branch_isothermal_pressure_change_pascals_rust(
    mass_rate_kg_per_s: f64,
    temperature_degrees_c: f64) -> PyResult<f64> {
    return Ok(
        get_heater_branch_isothermal_pressure_change_pascals(
            mass_rate_kg_per_s,
            temperature_degrees_c
        ));
}
```

For example, in the above code, the return type is pascals.

3.5 OPC-UA Server

Now that we have settled the choice of language used to calculate the fluid equations within the code, I will now elaborate on why I chose Python's OPC-UA server [Oroulet et al., 2022]. Python was used because of the precedent set in literature where Python OPC-UA Servers were used for digital twin construction [Zidek et al., 2020]. Additionally Python OPC-UA Server code is much easier to implement compared to other languages as well.

Rust's implementation of OPC-UA [Locka99, 2022] was not used because it was slightly more complex than Python's implementation. C# OPC-UA Servers had an Open Source implementation as well [Regen, Barnstedt, and Moldovean, 2016]. However, they were not used as multiple third party libraries had to be used to [Armstrong, 2015; Manditereza, 2021] to even construct a simple server. This implementation proved to be too time consuming to learn (I spent at least three weeks on this before giving up), and therefore it was abandoned in favour of Python. For interested readers, a more verbose discussion of OPC-UA Servers selection and the trial and error process is presented in the appendix.

3.6 OPC-UA Client

While most of the work in writing the digital twin pertains to Server Side code, I also pay due attention to the OPC-UA Client. This is because the client gives the user an interface to interact with the digital twin. In this case, the user sets a loop pressure drop and obtains the mass flow rate within the loop just like in CIET.

While there are many choices for an OPC-UA Client, I decided to use the Python OPC-UA GUI client which is free and open source [Oroulet, 2015]. This client is minimalistic, lightweight, accessible and works out of the box. It is written in PyQt5 and also cross platform. A GUI makes saves time in terms of development work as I do not need to rewrite a OPC-UA client GUI code.

While Python also has a CLI OPC-UA Client, I opted not to use it to save time with debugging the code since there was already a useful GUI client online.

Other possible OPC-UA clients include proprietary SCADA, FOSS SCADA software, as well as Python implementations of OPC-UA within graphical programming languages within graphical programming languages eg. Labview. Nevertheless, in terms of out of the box experience for quick prototyping of OPC-UA Servers, for an initial test, the simple GUI OPC-UA Python client will suffice.

3.7 Code Validation

Is is also important to validate any simulation results to ensure that the digital twin code corroborates well with experimental data. Therefore, for this thesis, data from isothermal flow tests for CIET was used [N. Zweibaum et al., 2015; Nicolas Zweibaum, 2015; Bickel, Nicholas Zweibaum, and Per F Peterson, 2014].

These results published pertain to isothermal flow tests where the DHX and bypass branch were blocked off, and flow from the CTAH branch flowed into the heater branch and back [N. Zweibaum et al., 2015]. The second test used here was where the heater branch in figure 3.10 was blocked off and only the DHX branch and CTAH branch had their valves opened.

It should be noted that for a first iteration, the model would only reproduce mass flow rate data given a loop pressure drop. In CIET, the user would specify a pump frequency via the control system, ARCO, and would then read a relevant mass flow rate value provided by a Coriolis flowmeter. Here, I again state that the user would specify the loop pressure drop (which for the sake of mimicking the feel of controlling the pump, I call “pump pressure”) directly rather than a pump frequency, and then the mass flowrate value would be given. Emulation of pump curves and behaviour are out of the scope of this thesis, but it can be implemented in future versions of CIET’s digital twins. Data for pump curves is available in the Integrated Research Program (IRP) reports [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014].

Experimental Data used for Validation

Furthermore, while data exists for manometer heights at various places in the loop, only a limited set of data was used in code validation. Specifically M-42 and M-43, which are the manometers before and after the pump, which would measure the head increase due to the pump.

To obtain various data points from M-42 and M-43, software was used to visually obtain csv data from online graph reading software [Larsen, 2022] called graphreader. This software is used to convert visual graphical data into a csv file. However, there may be some random numerical errors introduced despite best efforts of converting graphical data into csv files. These errors largely depend on the resolution of the graph. When reading CIET's manometer data using graphreader, the estimated errors due to pixellation of the graph can result in as much as $\pm 0.5\text{cm}$ of Dowtherm A when using the graphreader software. This would introduce an uncertainty of approximately $\pm 50\text{ Pa}$.

A second issue is that we may not be able to plot residuals or compare experimental data directly to digital twin data read using graph reader. For example, a data point at 0.16 kg/s was perhaps registered at 0.161 kg/s for example. So if I wanted manometer data at 0.16 kg/s , I would then have to interpolate somehow.

To assist with this problem, a quadratic polynomial trendline function was inserted using LibreOffice Calc, and the resulting function put into Python code so that manometer data could be readily interpolated at any mass reading value.

The correlations of each polynomial are as follows:

$$h_{m42 \text{ heater } CTAH}(m) = -19.3710\dot{m}_{CTAH}^2 - 1.1014\dot{m}_{CTAH} + 1.0100 \quad (3.22)$$

$$h_{m43 \text{ heater } CTAH}(m) = 11.2059\dot{m}_{CTAH}^2 + 2.1099\dot{m}_{CTAH} + 1.0100 \quad (3.23)$$

$$h_{m42 \text{ dhx } CTAH}(m) = -19.5299\dot{m}_{CTAH}^2 - 1.1272\dot{m}_{CTAH} + 1.0090 \quad (3.24)$$

$$h_{m43 \text{ dhx } CTAH}(m) = 17.3617\dot{m}_{CTAH}^2 + 1.0204\dot{m}_{CTAH} + 1.0090 \quad (3.25)$$

One can take the residual ε by subtracting the manometer heights obtained using graphreader software from the manometer heights obtained using the correlations. The mean of the residuals $\mu(\varepsilon)$ and their standard deviation $\sigma(\varepsilon)$ is presented in the form $\varepsilon = \mu(\varepsilon) + \sigma(\varepsilon)$ below:

$$\varepsilon(h_{m42 \text{ heater } CTAH}(m)) = 0.00 \pm 0.003 \quad (3.26)$$

$$\varepsilon(h_{m43 \text{ heater } CTAH}(m)) = 0.00 \pm 0.003 \quad (3.27)$$

$$\varepsilon(h_{m42 \text{ dhx } CTAH}(m)) = 0.00 \pm 0.003 \quad (3.28)$$

$$\varepsilon(h_{m43 \text{ dhx } CTAH}(m)) = 0.00 \pm 0.003 \quad (3.29)$$

We can see that the standard deviations of the residuals $\sigma(\varepsilon)$ are about $\pm 3\text{mm}$. The pixellation errors within $2\sigma(\varepsilon)$. Therefore, any value of manometer height derived from these correlations would be statistically indistinguishable from the values obtained using graphreader software. Therefore, these correlations can be used with a standard deviation of $\pm 0.003\text{m}$ or $\pm 3\text{mm}^2$. This should be taken into account when using these correlations to interpolate manometer data.

²Do note that I did not correct this standard deviation using $\sqrt{n-1}$. Doing so would slightly increase the standard deviation but it would have been within rounding error. The sample size for the heater loop is 22 samples and for the dhx loop it was 16 samples.

Once that was done, the reference temperature of 20 degrees Celsius was used to obtain the Dowtherm A density, and the way to obtain the loop pressure drop was:

$$\Delta P_{experimental\ loss}(\dot{m}) = (h_{m43} - h_{m42})\rho g \quad (3.30)$$

Where h_{m42} and h_{m43} are the heights of the manometer readings at M-42 and M-43 respectively. ρ is the density of Dowtherm A at 1061 kgm^{-3} and g is the gravitational acceleration at 9.81ms^{-2} . It should be noted that the pressure loss term here is an explicit function of mass flow rate.

Since no iterations are needed, I saw little need to implement the code in Rust since it is not likely that this line of code would bottleneck the calculation speed.

The validation procedure in code would be firstly to obtain a mass flowrate value when the user specifies a loop pressure drop value. This mass flowrate value is then used to obtain the manometer heights from the correlations above and then used to plot a data point for the graph of experimental data.

Thereafter, both sets of data points are plotted on the system curve graphs in the results section.

Residuals and Error Bars

When validating the mass flowrate results produced by the Type I Digital Twin presented, residual plots are also provided with their corresponding error bars to determine if the simulated results are statistically different compared to the experimental data.

For these error bars, four main errors are considered. These are (1) manometer reading error of $\pm 1\text{mm}$, (2) manometer correlation error of $\sigma(\varepsilon) = \pm 3\text{mm}$, (3) Coriolis flowmeter errors of $\pm 2\%$ [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014] and (4) pressure loss coefficient $f + \frac{L}{D} + K$ error of $\pm 10\%$ [N. Zweibaum et al., 2015; Nicolas Zweibaum, 2015]. These errors are assumed to occur independently of each other.

While there are other potential sources of error such as manufacturing tolerances of the schedule 10 pipes used to construct CIET as well as pixellation error from using graphreader software, these are not considered because accounting for these four errors was enough to prove that the simulation results from the digital twin were statistically indistinguishable from the isothermal pressure drop experimental data. Any consideration of these other errors would only widen the error bars, and thus we would still end up at the same conclusion.

To prove this point, we need to consider if the residuals are statistically different from zero. The residuals are calculated as:

$$\delta(\Delta P_{loop\ pressure\ drop}) = \sum_i^n \frac{1}{2} \frac{\dot{m}_i^2}{\rho A_{xs\ i}^2} (f_{darcy\ i} \frac{L_i}{D_i} + K_i) - \Delta h_{correlation} \rho g \quad (3.31)$$

The first term pertains to the pressure drop value obtained from the digital twin, whereas the latter term refers to the loop pressure drop value obtained from the manometer reading correlations.

For independent variables, error propagation can be described as using a differential equation [Todreas, Kazimi, and Massoud, 2021; BIPM et al., 2008; Meyer, 2007]:

$$u_c^2(M) = \sum_{i=1}^N \left(\frac{\partial M}{\partial x_i} \right)^2 u^2(x_i) \quad (3.32)$$

The term $u_c(M)$ stands for combined uncertainty of a measurand M , and $u(x_i)$ stands for the uncertainty of a quantity x_i used in turn to calculate M . Here, our measurand M is the residual $\delta(\Delta P_{loop \text{ pressure drop}})$. In terms of partial derivatives,

On Partial Derivatives and Confidence Intervals

$$\begin{aligned} u_c^2(\delta(\Delta P_{loop \text{ pressure drop}})) &= \left(\frac{\partial \delta(\Delta P_{loop \text{ pressure drop}})}{\partial \dot{m}_i} \right)^2 u^2(\dot{m}_i) \\ &+ \left(\frac{\partial \delta(\Delta P_{loop \text{ pressure drop}})}{\partial \zeta_i} \right)^2 u^2(\zeta_i) + \left(\frac{\partial \delta(\Delta P_{loop \text{ pressure drop}})}{\partial \Delta h_{reading}} \right)^2 u^2(\Delta h_{reading}) \\ &+ \left(\frac{\partial \delta(\Delta P_{loop \text{ pressure drop}})}{\partial \Delta h_{correlation}} \right)^2 u^2(\Delta h_{correlation}) \end{aligned} \quad (3.33)$$

This means we will need to evaluate several partial derivatives in order to determine the error bars of the residual. Furthermore, we will need to quantify each uncertainty in order to evaluate the errors. This means assuming some distribution for each contribution uncertainty $u(x_i)$ and then summing them up. For example, the standard deviation of the correlation residual is calculated at about $\pm 3mm$. This would represent a 1σ variation. While for manometer reading error of $\pm 1mm$, we do not know what distribution it comes under. For such distributions, it may be prudent to assume a uniform distribution [BIPM et al., 2008] or at least that the measurement error will not fall outside $\pm 1mm$. If we assume a normal distribution, this could mean that we consider this $\pm 1mm$ as 3σ as we have a 99.7% chance that data will not fall outside 3σ for a Gaussian Distribution. It should be noted that error bars presented here are derived assuming that all readings have a Gaussian distribution for simplicity's sake³. We can use this same assumption that flowmeter error follows a normal distribution with 3σ at $\pm 2\%$ of the measured flowrate. In the same way, the pressure drop coefficient error ζ_i of $\pm 10\%$ is assumed at 3σ . If we were to use 3σ as our standard for uncertainty calculations, we would obtain:

$$(3\sigma_{combined})^2(M) = 9 \sum_{i=1}^N \left(\frac{\partial M}{\partial x_i} \right)^2 \sigma^2(x_i) \quad (3.34)$$

³Readers interested in a stricter error analysis can try Monte Carlo methods using their own assumed distributions for error propagation

To obtain our *error bar* = $2\sigma_{combined}$ for about 95% confidence interval, we can calculate:

$$error\ bar(Pa) = \frac{2}{3} \sqrt{\sum_{i=1}^N \left(\frac{\partial M}{\partial x_i} \right)^2 (3\sigma(x_i))^2} \quad (3.35)$$

Manometer Reading Error Variance

For manometer reading error, we consider first that we are calculating loop pressure drop by subtracting two manometer readings:

$$\Delta P_{loop\ pressure\ drop} = (h_1 - h_2)\rho g \quad (3.36)$$

Hence, we consider the 3σ variance to be:

$$\begin{aligned} & \left(\frac{\partial \delta(\Delta P_{loop\ pressure\ drop})}{\partial \Delta h_{reading}} \right) (3\sigma(\Delta h_{reading})) \\ &= (1061\text{kg}/\text{m}^3)(9.81\text{ms}^{-2})\sqrt{0.001^2\text{m}^2 + 0.001^2\text{m}^2} \approx 14.7\text{Pa} \end{aligned} \quad (3.37)$$

This assumes the density of dowtherm A or Therminol VP-1 to be $1061\text{ kg}/\text{m}^3$ and gravitational acceleration to be 9.81ms^{-2} . Uncertainty contributions of both of these quantities are neglected.

Manometer Correlation Error Variance

For manometer correlation error, we consider again that we need to subtract two loop pressure drop readings. σ is $\pm 3\text{mm}$ and therefore 3σ is $\pm 9\text{mm}$. Hence, we consider the 3σ variance to be:

$$\begin{aligned} & \left(\frac{\partial \delta(\Delta P_{loop\ pressure\ drop})}{\partial \Delta h_{correlation}} \right) (3\sigma(\Delta h_{correlation})) \\ &= (1061\text{kg}/\text{m}^3)(9.81\text{ms}^{-2})\sqrt{0.009^2\text{m}^2 + 0.009^2\text{m}^2} \approx 132\text{Pa} \end{aligned} \quad (3.38)$$

Do note that accounting for this error (or even the pixellation error) greatly widens the error bar for loop pressure drop as compared to using manometer readings obtained directly from CIET. It is likely that for flow less than $0.06\text{ kg}/\text{s}$ in the CTAH and Heater branch, as well as CTAH and DHX branch isothermal pressure drop tests, the deviations from experimental data exceed manometer reading error alone. A separate error and residual analysis will have to be performed using such data. This was not attempted in this work as data read directly from CIET in the form of tables was lacking.

Evaluating Coriolis Flowmeter Flowrate Variance

The first term $\left(\frac{\partial\delta(\Delta P_{loop\ pressure\ drop})}{\partial\dot{m}_i}\right)^2 u^2(\dot{m}_i)$ describes the error due to uncertainty in mass flowrate measurement. We see that the manometer measurement term becomes zero.

$$\left(\frac{\partial\delta(\Delta P_{loop\ pressure\ drop})}{\partial\dot{m}_i}\right)^2 u^2(\dot{m}_i) = u^2(\dot{m}_i) \left(\frac{\partial}{\partial\dot{m}_i} \sum_i^n \frac{1}{2} \frac{\dot{m}_i^2}{\rho A_{xs\ i}^2} (f_{darcy\ i} \frac{L_i}{D_i} + K_i)\right)^2 \quad (3.39)$$

The partial derivative here is evaluated numerically via finite difference method. This is done by perturbing the branch pressures over all parallel branches by a small amount (10 Pa) and then obtaining two sets of CTAH branch mass flowrate $\dot{m}_i$ and loop pressure drop $\sum_i^n \frac{1}{2} \frac{\dot{m}_i^2}{\rho A_{xs\ i}^2} (f_{darcy\ i} \frac{L_i}{D_i} + K_i)$. Once that is done, the partial derivative $\frac{\partial}{\partial\dot{m}_i} \sum_i^n \frac{1}{2} \frac{\dot{m}_i^2}{\rho A_{xs\ i}^2} (f_{darcy\ i} \frac{L_i}{D_i} + K_i)$ is estimated using a central difference approximation. This will allow us to obtain the 3σ for the Coriolis Flowmeter error of $\pm 2\%$.

Evaluating Pressure Loss Coefficient Flowrate Variance

Now let us calculate the variance due to pressure drop correlation ζ where $\zeta_i = f_i \frac{L_i}{D_i} + K_i$. We consider that $u(\zeta_i) = 0.1(\zeta_i)$ which is a $\pm 10\%$ error [Nicolas Zweibaum, 2015], so that:

$$\begin{aligned} \left(\frac{\partial\delta(\Delta P_{loop\ pressure\ drop})}{\partial\zeta_i}\right)^2 u^2(\zeta_i) &= \sum_i^N \left(\frac{1}{2} \frac{\dot{m}_i^2}{\rho A_{xs\ i}^2}\right)^2 u^2(\zeta_i) \\ &= 0.10^2 \sum_i^N \left(\frac{1}{2} \frac{\dot{m}_i^2}{\rho A_{xs\ i}^2} f_i \frac{L_i}{D_i} + K_i\right)^2 = 0.10^2 \sum_i^N (\Delta P_i)^2 \end{aligned} \quad (3.40)$$

This means that we can take the 3σ to be 10% of any pressure drop value obtained from pressure losses obtained through empirical correlations such as MX-10, FM-40 and the CTAH.

Notes on Error Bars for Single Flow Paths and Parallel Flow Paths

When presenting results, we may want to provide error percentages to give a reasonable estimate of uncertainty for these simulation results. For error treatment, we consider the model of the digital twin as if it were any other correlation. This means we will give some expression for the residual between the digital twin model and experimental data where experimental data is available.

Where experimental data is not available especially for a parallel flow loop situation, we will not provide a rigorous error bar assessment since the goal of this work is merely to validate the models within the digital twin and ensure that it runs within an acceptable timeframe.

However, we will still give a crude estimate for errors. These are (1) manometer reading error of $\pm 1mm$, (2) manometer correlation error of $\sigma(\varepsilon) = \pm 3mm$, (3) Coriolis flowmeter errors of $\pm 2\%$ [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014] and (4) pressure loss coefficient $f + \frac{L}{D} + K$ error of $\pm 10\%$ [N. Zweibaum et al., 2015; Nicolas Zweibaum, 2015] for all experimentally measured CTAH Branch components. This simplification is because measuring error in a parallel flow situation can get quite complicated. Here, we can no longer assume errors are independent of each other. A deviation of flowrate or diameter in one branch will inevitably impact mass flowrates in other branches as well. The approach in obtaining these deviations is not immediately obvious. Furthermore, obtaining loop pressure drop error in general becomes challenging to obtain analytically because where there are parallel branches, there exist multiple ways to calculate loop pressure drop. The question then becomes “which expressions do we use to calculate loop pressure drop?” Given these complications, we will only use a placeholder for error bars when presenting these results.

3.8 Approximations Made in Branch Model

For the parts of CIET where pipes branch, namely branches labelled 5 and 17 on the following diagram figure 3.9, Liberties were taken as to where the branch point occurred firstly because data for where the branch point occurred exactly was not readily available in the RELAP model [N. Zweibaum et al., 2015]. While it could be calculated, it was convenient to shift the branch point for expediency.

A second justification is that we can use the opportunity to test if the shifting the branch points by small amounts would significantly impact flow data. When discussing the parallel branch approximation, we note that the bypass branch does not branch off in CIET at the branch 5 and 17 just like the heater and DHX branches. However, we can shift the branch point to this area so that we can have an approximate model to solve using the parallel flow algorithm. If deviations in flowrate are not too large, it can still serve as an approximate Type I digital twin of CIET. If we are able to prove that the system curve of CIET can be reproduced to a satisfying degree despite shifting these branch points by small amount, then we can be more confident in the results given by the parallel branch approximation method.

How this happens can be shown in figures 2.1, and 3.10. In figure 3.9, branch 5 and 17 were split into two parts as it was meant to in the SAM model [Zou, R. Hu, and Charpentier, 2019] and Relap model [N. Zweibaum et al., 2015].

However, the model in this paper includes both 5a and 5b in figure 3.10 as branch 5 and 17a and 17b as a single branch 17. This was done primarily due to convenience as the earlier RELAP model [N. Zweibaum et al., 2015] only had dimensions for pipe 5 and 17 rather than the constituent parts 5a, 5b, 17a and 17b as shown in SAM [Zou, R. Hu, and Charpentier, 2019].

Compact Integral Effects Test (CIET) Nodalised Model Diagram

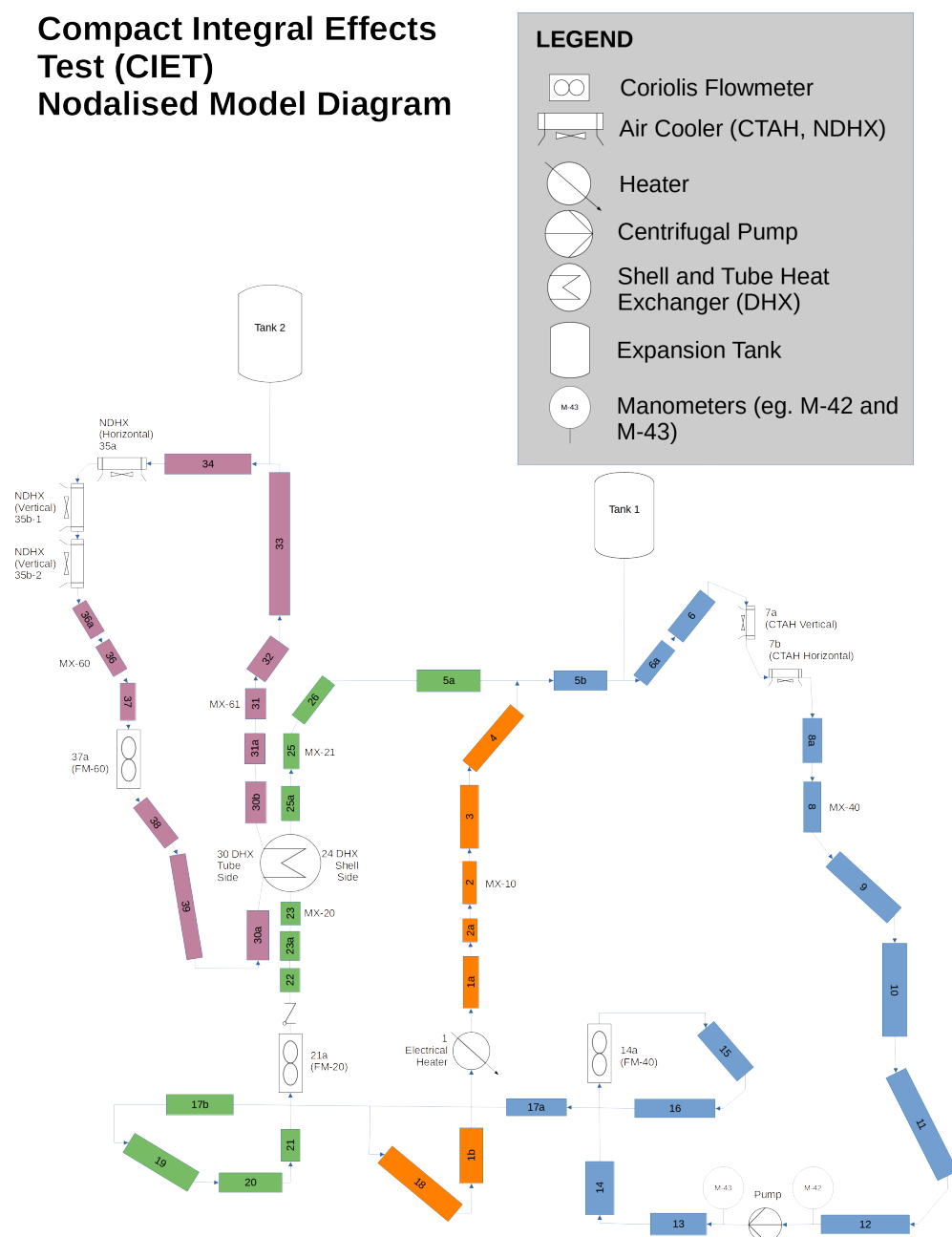


Figure 3.9: Nodalised Model of CIET without Bypass Branch

Compact Integral Effects Test (CIET) Nodalised Model Diagram



Figure 3.10: Approximate Nodalised Model of CIET without Bypass Branch

Figure 3.10 shows that I have included pipe 5 and 17 as one combined piping section rather than split them in two. This was the model used for this paper. Of course, the

DRACS loop was not simulated as this is an isothermal flow test.

The exact flow parameters for Figure 3.10 are found in the Appendix in part A.

Notes on the Heater Insert

Now the set of data used to validate CIET is based on the old heater insert [Nicolas Zweibaum, 2015]. The heater insert was subsequently replaced to improve the heat transfer characteristics of the heater. [Poresky, 2017]

Data from the new heater insert can be used for validation in future iterations, but not in this thesis.

3.9 Hardware used for Testing of Computation Speed

As mentioned earlier, we wanted the digital twin prototype to be readily available to users without immediate access to supercomputing clusters. As such, I performed all of these tests on typical laptops available for purchase by consumers. I decided to test the digital twin on two laptops: the first laptop is an ultrabook while the second laptop was a gaming notebook. The ultrabook had a lower clock speed but could serve as a conservative estimate for the hardware available to the end user since it is more affordable. The gaming laptop, on the other hand, was a less conservative estimate because it was more expensive to buy. Should the digital twin be able to run on the ultrabook even in a nonoptimised “debug mode” binary, we can be more confident that the full digital twin will not need supercomputing resources. However, if the ultrabook fails to run the nonoptimised binary, we can then test the digital twin’s nonoptimised binary with the gaming laptop. If it runs calculations for each timestep within 100ms, we can be more sure that the digital twin will not require supercomputing resources.

The specifications of the laptops are shown below:

Ideapad 3 with 8GB RAM, 1GB HDD and with the CPU:

AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx (8) @ 2.100GHz

Note that the AMD Ryzen tends to reserve about 2GB of RAM for graphics card memory on Arch Linux. The Ideapad was the lower budget end of the laptops I used.

For the gaming laptop, it was a custom built gaming laptop sold by AFTERSHOCK PC PTE. LTD. (from Singapore) with: 31GB RAM, 256 GB SSD and the CPU:

Intel i7-10875H (16) @ 5.100GHz

Both laptops were using Arch Linux as the Operating System at the time of testing. The reader should note that Windows is generally a more resource intensive Operating System as compared to Desktop Linux Distributions. Therefore, it is reasonable for the digital twin to potentially run faster on a Linux Operating System than a Windows Operating System. This

should be taken into account when considering the calculation time on lower end personal computers.

Chapter 4

Results and Discussion

Three digital twin servers written in Python and Rust were found to be able to replicate the system curves obtained using experimental data obtained from CIET. Except for the Ideapad, these servers were able to operate on laptops and perform calculations within the timeframe of 100 ms even when binaries were compiled without optimisation in Rust’s “Debug mode”.

The first server only includes the CTAH and Heater branches so that validation with CIET’s isothermal test data can be performed. The second server contains the CTAH and DHX branch for a similar purpose. Finally, the third server adds the CTAH, Heater and DHX branch in parallel. This third test setup is meant to investigate the amount of computational time and power required to run an isothermal digital twin of CIET.

4.1 CTAH and Heater Branch Results

Accuracy of Digital Twin Simulated Results

After setting up the digital twin server in Python and Rust, the Python script was run using nodemon. The OPC-UA GUI client was initialised on a different computer on the same private Wi-Fi network and connected to the digital twin using the endpoint. This was firstly to lighten the load of the OPC-UA Server and Client by splitting it on different computers. Secondly it was to show that the digital twin could be operated remotely over a wifi network.

System Curve Analysis

The digital twin was subject to user defined loop pressure drop values. Of course, it is more intuitive for the user to control a pump power or pressure rather than a loop pressure drop. Hence, loop pressure drop was named as “pump pressure” within the code. Subsequently, the flowrate and corresponding experimental pressure losses were recorded on a system curve like plot on Figure 4.1. Error bars of 2σ for mass flowrate deviation due to the $\pm 2\%$ Coriolis flowmeter reading uncertainty and manometer correlation standard deviation of $\pm 3mm$ are

added. The flowmeter deviation is assumed to be 3σ and the manometer correlation standard deviation is assumed to be 1σ as explained in the methods section. Therefore, the manometer experimental data correlation error bar here is about 90 Pa.

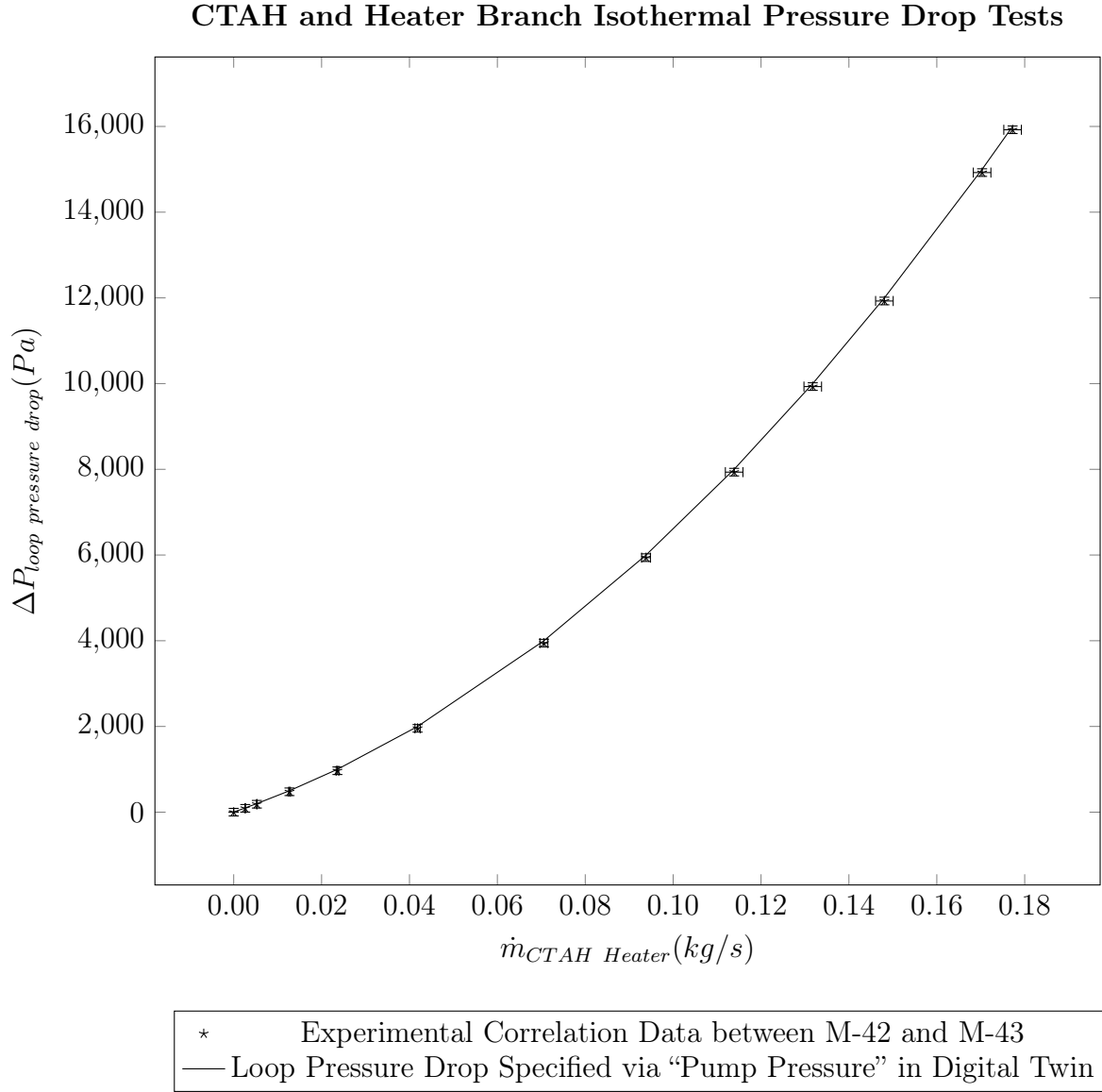


Figure 4.1: Digital Twin System Curve Compared to Experimental System Curve from Manometer M-42 and M-43 Plotted in Absolute Pressure Units (CTAH and Heater Branch)

Do note that the flow data is generated using a forward flow configuration as shown in figure 3.3 shown first in the methods section. Additionally, figure 3.3 in the methods section also shows the location of the manometers M-42 and M-43.

For the reader's convenience, a table of mass flowrates and loop pressure drop for the digital twin and the experimental setup is provided in table 4.1. The loop pressure drops are rounded to the nearest 10 Pa as because of measurement uncertainty in the manometers as well as the manometer correlation errors of about 90 Pa. Additionally, the flowmeter readings are given to three significant figures because the flowmeter measurement error is about $\pm 2\%$ [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014] ¹.

FM-40 Mass Flowrate (kg/s)	Digital Twin Loop Pressure Drop (Pa)	Experimental Data Correlation Pressure Drop (Pa)
0.177	16000	15920
0.170	15000	14920
0.148	12000	11930
0.132	10000	9930
0.114	8000	7930
0.0938	6000	5940
0.0706	4000	3950
0.0418	2000	1960
0.0236	1000	970
0.0127	500	480
0.00527	200	180
0.00263	100	90
0	0	0
-0.0418	-2000	outside data range
-0.132	-10000	outside data range

Table 4.1: Digital Twin System Curve Compared to Experimental System Curve from Manometer M-42 and M-43 Plotted in Absolute Pressure Units (CTAH and Heater Branch)

¹I am not entirely sure of the uncertainty at 0 kg/s flowrate, so I did not use three significant figures here, but just one

For figure 4.1, the loop pressure drop is plotted against mass flowrate in CIET for both digital twin and the experimental data set. From the system curve plots, it can be seen that the digital twin generated system curve closely conforms to the system curve generated using experimental data measured via the manometer and is essentially within the 2% flowmeter error and $\pm 90 \text{ Pa}$ experimental data correlation error obtained using graphreader software to extract the original manometer M-42 and M-43 readings in forward flow configuration. This meant that therminol or dowtherm A flows upward through the heater.

We also want the digital twin to emulate CIET's reverse flow capabilities. In fact, when deriving the empirical pressure drop correlations, data from both forward and reverse flow was used [Nicolas Zweibaum, 2015]. It was found that the magnitude of pressure drop did not deviate significantly in forward and reverse flow configurations [Nicolas Zweibaum, 2015]. Hence, for most components in CIET (except for the check valve in the DHX branch), the pressure drop should be similar in magnitude in both forward and reverse flow. This behaviour should be replicated in the digital twin. Therefore I wanted to check if the magnitude of the loop pressure drop was equal in forward and reverse flow configuration. This was done by sampling two data points for the digital twin at -2000 Pa and -10000 Pa of loop pressure drop. From table 4.1, we see that reverse flow configurations were working as expected. As shown in table 4.1 $\pm 2000 \text{ Pa}$, a mass flowrate of $\pm 0.0418 \text{ kg/s}$ was obtained and at $\pm 10,000 \text{ Pa}$ a mass flowrate of $\pm 0.132 \text{ kg/s}$ was obtained. Thus it showed that reverse flow setups work for the digital twin and corroborate well with experimental data for forward flow.

The reader may note that this curve is essentially the system curve, except that instead of plotting the system total head losses in meters, the system pressure drop or head loss was plotted using pascals. The conversion between head loss and pressure drop can be done using a Therminol VP-1 density of 1061 kg m^{-3} and gravitational acceleration of 9.81 m s^{-2} . This non traditional form of plotting the system curve is used because the user of the digital twin would specify a loop pressure drop (via a pump pressure setting) instead of a loop head loss. From the programming perspective, I personally think it is easier to program variables in terms of a loop pressure drop in terms of pressure units rather than head loss or length units. This would make it easier when the time comes for non-isothermal operations and the fluid density would vary due to temperature changes. Additionally, since the program outputs loop pressure values in units of pascal, a system curve in pascal units would be useful for users wishing to replicate the digital twin using my codebase. Users could use the graph to quickly check whether the digital twin was giving the right mass flowrate values at various loop pressure drop values without having to make unit conversions.

Overall the system curve plots seem to show good agreement between the system curve of the digital twin and experimental data. To ensure that residuals are truly within experimental error, we shall examine the residual plots.

Residual Plot Analysis

In the residual plots, we calculate the residuals ($\delta \Delta P_{\text{loop pressure drop}}$) by:

$$\delta\Delta P_{loop\ pressure\ drop} = \Delta P_{digital\ twin\ prediction} - \Delta P_{experimental\ data\ correlation} \quad (4.1)$$

In these plots, errors were $< 1\%$ for input loop pressure drop of 8000 Pa to 16,000 Pa (about 0.10 kg/s to 0.18 kg/s mass flowrate) , and were $< 5\%$ from 1000 Pa to 6000 Pa (about 0.02 kg/s to 0.1 kg/s) as shown in figure 4.2.

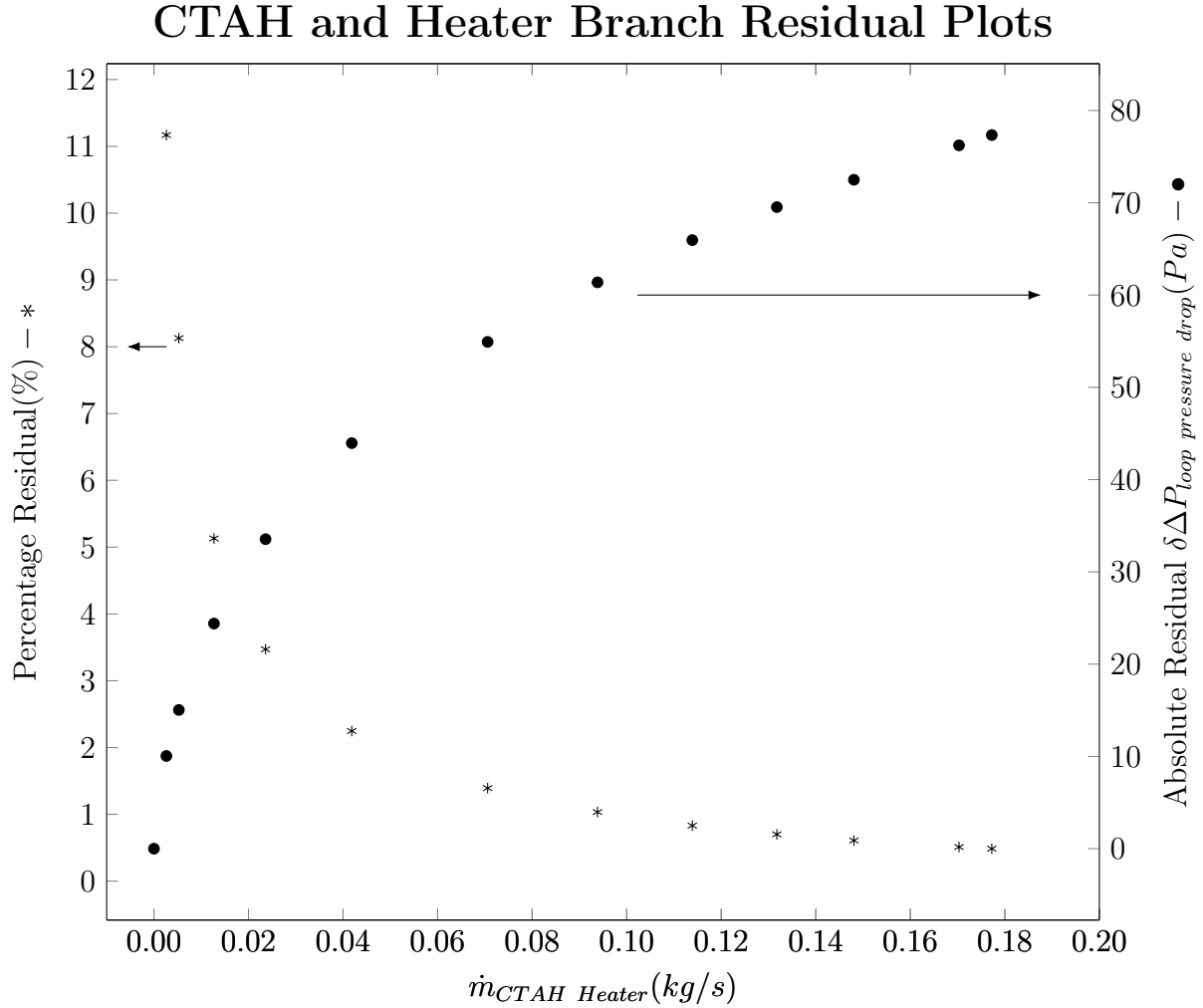


Figure 4.2: Residual Plot for CTAH and Heater Branch System Curves

From figure 4.2, we can see that the largest percentage error recorded was percentage error of 11% at 100 Pa of user specified loop pressure drop (< 0.01 kg/s mass flowrate) where the error was about 10 Pa and the manometer data predicted a loop pressure drop of about 90 Pa. In mass flowrates of 0.05 kg/s and above, the percentage error is generally below 2% while in mass flowrate regimes of 0.1 kg/s and above, percentage errors are within 1%. In the region of approximately 0.005 kg/s to 0.05 kg/s, the maximum percentage errors are about

9% while maximum absolute error of predicted loop pressure drop compared to experimental loop pressure drop is about 45 Pa. Note also that the residual plot consistently shows that the digital twin consistently overpredicts pressure loss in the CTAH and Heater branch given a fixed flowrate. This is likely because I combined branch 5a and branch 17b from figure 2.1 and put them into the CTAH branch as shown in figure 3.10. That would mean that the CTAH and Heater branch loop would have a slightly longer simulated pipe than what was in the experiment. As a result, the CTAH branch and Heater branch should experience slightly higher than normal pressure drop values given a fixed flowrate.

Having these errors may seem concerning, but it is worth noting that codes written in literature which model CIET have errors of about more than 2%. For example FHR Neutral Circulation Analysis (FANCY) correspond to experimental data with deviations of at most 3% [N. Zweibaum et al., 2015; Nicolas Zweibaum, 2015]. One can then conclude that a 2% error is quite acceptable. For the largest percentage error of about 11%, we note that the absolute error is about 10 Pa. This is still well within experimental error because the uncertainty of a manometer reading could be about $\pm 1mm$ or 10 Pa, if we obtain the final loop pressure drop reading by subtracting two manometers, then the experimental error is about 15 Pa. This would mean that the reading was well within experimental error due to uncertainties in reading manometers. Of course, the experimental correlation standard deviation is already about $\pm 3mm$, that constitutes a 2σ error of about 90 Pa. If we were to account for these errors, the pressure drop coefficient errors ζ and flowmeter reading errors of $\pm 2\%$, then we can see that the predictions are well within these errors. Figure 4.3 shows the residuals plotted with a 2σ error bar calculated using methods described in the methods section.

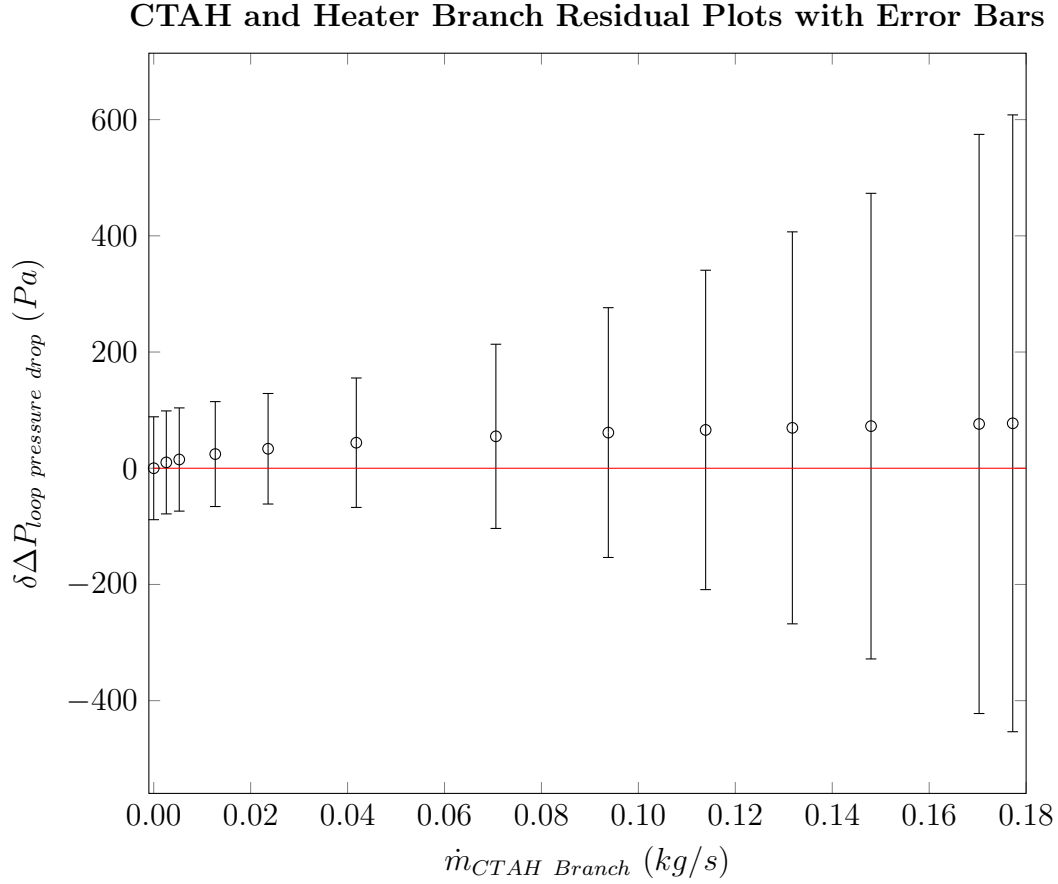


Figure 4.3: Absolute Residual Plot for CTAH and Heater Branch System Curves with Error Bars

To help the reader reproduce the error bars in figure 4.3, a polynomial regression was performed on these error bars and presented for the reader's reference:

$$\Delta P_{(error \text{ bar})} \text{ (Pa)} = 13879.6 \dot{m}_{CTAH \text{ Branch}}^2 \text{ (kg/s)} + 51.7 \dot{m}_{CTAH \text{ Branch}} \text{ (kg/s)} + 87.5$$

$$2\sigma = \pm 3 \text{ Pa}$$

We can see that the plotted CTAH and Heater Branch values are well within the 2σ error bar. Thus for all intents and purposes, the isothermal flow digital twin for CIET has replicated the results of the experimental data to within acceptable error bounds despite the approximations made with branch 5 and 17. This is quite a satisfying result.

Computation Speed

Another important aspect of digital twins is the requirement for real time computation. The data from Figure 4.1 was obtained by simulating only the heater branch and Coiled Tube

Air Heater (CTAH) branch since those were the only pieces of experimental data available. No further parallelisation or optimisation was used for either the Rust or Python code.

With the CPU:

AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx (8) @ 2.100GHz

The maximum simulation time obtained was 65 ms. This is the CPU of an IdeaPad 3 running Arch Linux on a Hard Disk Drive (HDD) with 6GB of available RAM.

I tried the same OPC-UA Server code on another computer with the CPU:

Intel i7-10875H (16) @ 5.100GHz

And the maximum simulation time per iteration was about 20.3 ms. This was running on an AFTERSHOCK PC PTE. LTD. (from Singapore) custom gaming laptop, with about 31 GB of available RAM. It also was run using Arch Linux. While the gaming computer was about three times faster than the former budget Ideapad, both laptops represent what is available to the end user on the market. With further optimisation and perhaps parallelisation, even the lower end laptop could perhaps solve transient heat transfer equations without breaking the 100 ms time limit. Nevertheless, this is pure speculation at this point. And should be proved in future work.

4.2 CTAH and DHX Branch Results

For the DHX Branch, isothermal tests were also done in a second digital twin by altering code to valve off the CTAH branch and pumping flow upwards towards the DHX branch [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]. Essentially, this setup is for the CTAH branch and DHX branch in series.

Accuracy of Digital Twin Simulated Results

System Curve Analysis

Once more, system curve plotted by the digital twin was compared against an experimental correlation based on experimental data. This time however, data from the Fluoride Salt Cooled High Temperature Reactor (FHR) Integrated Research Project (IRP) Final Report numbered as (UCBTH-14-009) [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014] was used. Error bars for 90 Pa for the experimental correlation uncertainty and $\pm 2\%$ Coriolis Flowmeter uncertainty are provided. These results were plotted in figure 4.4.

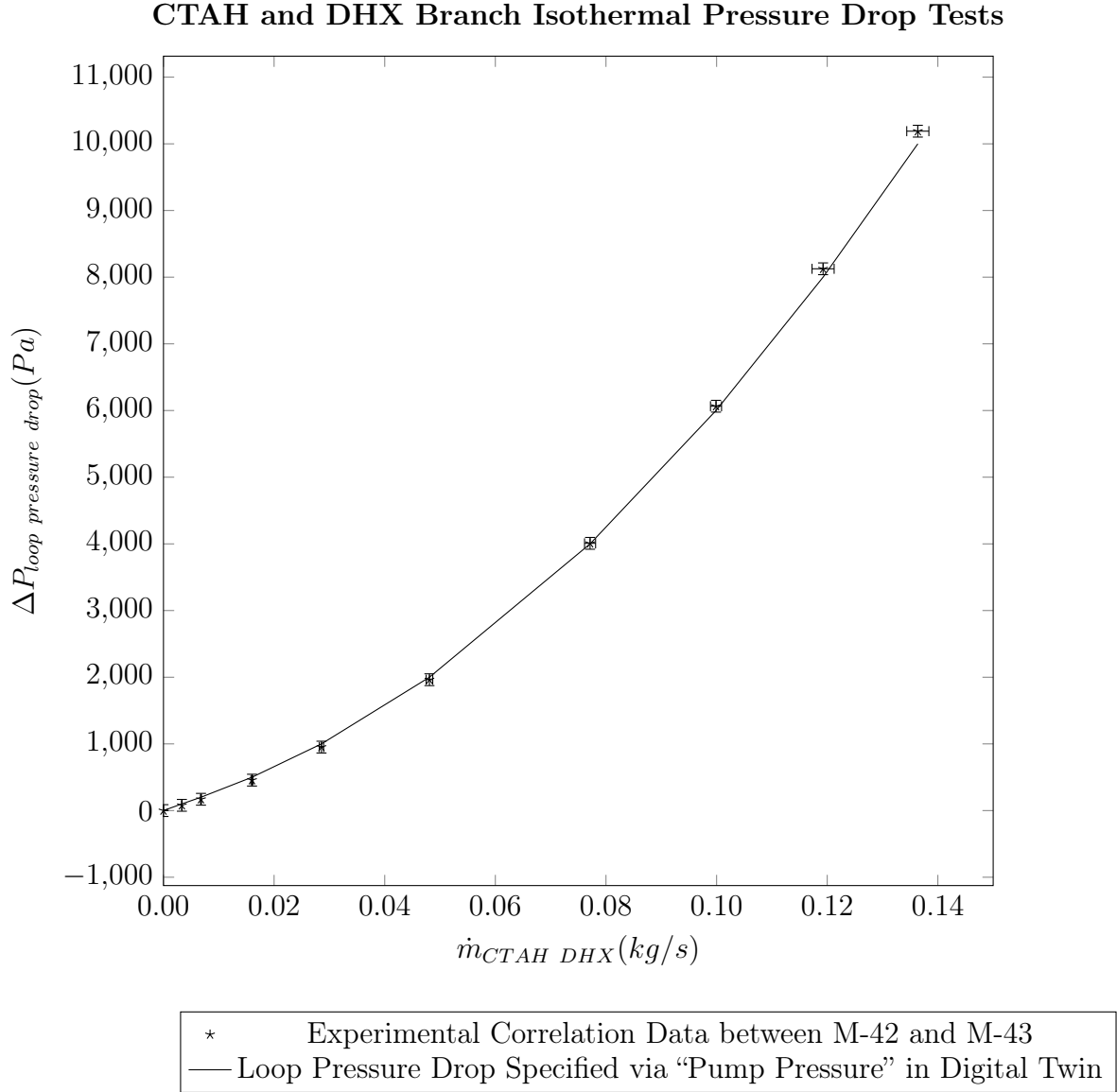


Figure 4.4: Digital Twin System Curve Compared to Experimental System Curve from Manometer M-42 and M-43 Plotted in Absolute Pressure Units (CTAH and DHX Branch)

The mass flowrate data from this dataset was obtained using flowmeter 20 (FM-20). However, the mass flowrate data generated from the digital twin was obtained using flowmeter 40 (FM-40). However, FM-20 and FM-40 are in series and there were no parallel branches or alternate flow paths present in this setup, therefore the mass flowrate through both of them is equal.

It can be seen from figure 4.4 that the shape of the system curve generated by the digital twin generally follows the shape of the system curve from the correlation obtained using experimental data. For the data point at 0.14 kg/s, we would need to extend the digital

twin simulated values beyond 0.14 kg/s in order to see it intersect the error bar. This is means that the deviations are still within experimental error and correlation error. We can examine these errors in more detail when we look at the residual plots.

Also, the reverse flow configuration works as expected for the digital twin because the magnitude of mass flowrate is the same regardless of forward or reverse flow configuration. Once again, data used for this plot is presented for the reader's reference in figure 4.2.

FM-40 Mass Flowrate (kg/s)	Digital Twin Loop Pressure Drop (Pa)	Experimental Data Correlation Pressure Drop (Pa)
0.136	10000	10190
0.119	8000	8130
0.0999	6000	6060
0.0771	4000	4010
0.0481	2000	1960
0.0286	1000	950
0.016	500	460
0.00678	200	170
0.00333	100	80
0	0	0
-0.0481	-2000	outside data range
-0.136	-10000	outside data range

Table 4.2: Digital Twin System Curve Compared to Experimental System Curve from Manometer M-42 and M-43 Plotted in Absolute Pressure Units (CTAH and DHX Branch)

Just as we did in the CTAH and Heater Branch results section, we need to examine the residuals plots to quantify if the errors present in our model are acceptable.

Residual Plot Analysis

The residual plot for table 4.2 is presented in figure 4.5.

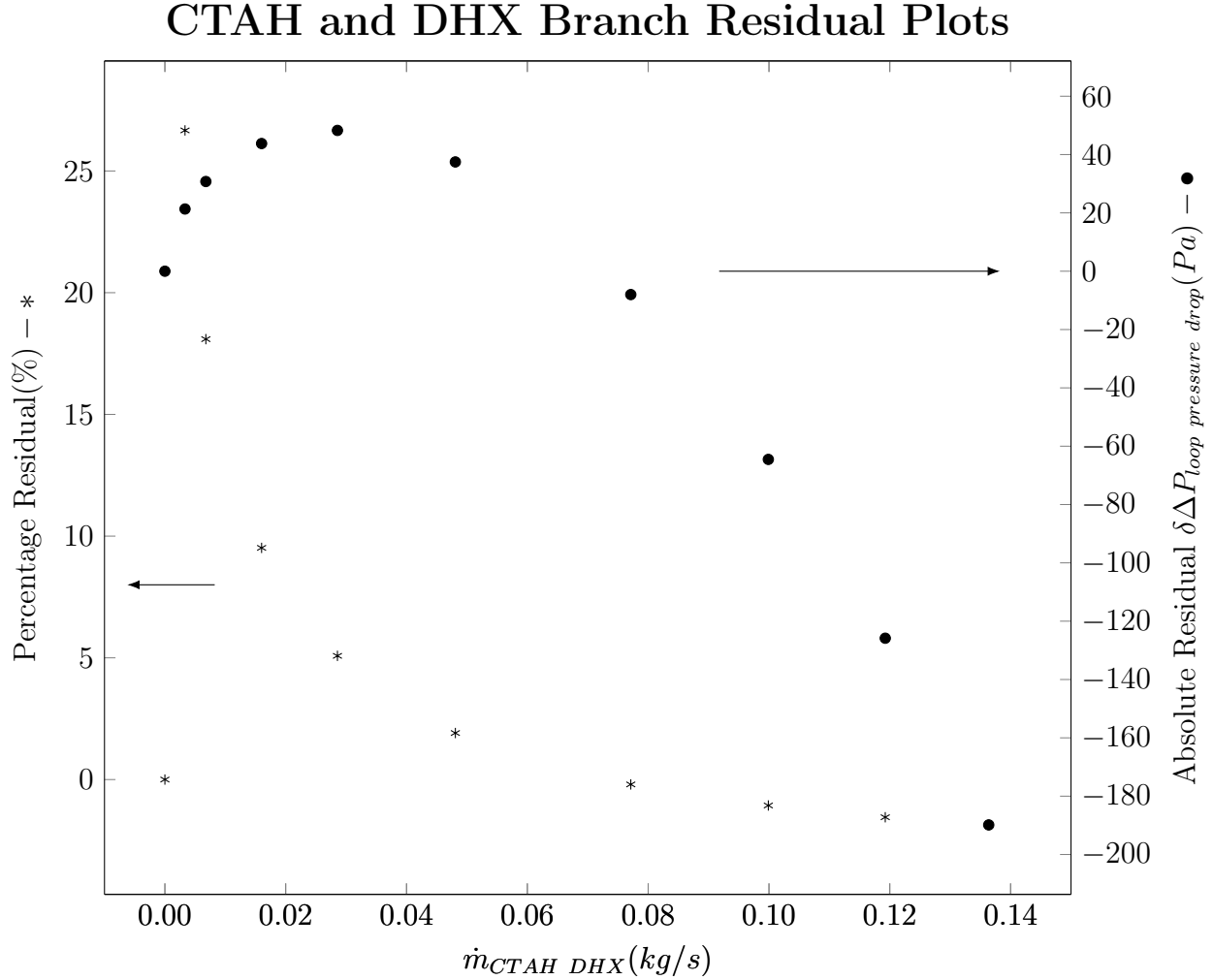


Figure 4.5: Plot Data for Digital Twin Residual vs Experimental System Curve from Manometer M-42 and M-43 (CTAH and DHX Branches Only)

Figure 4.5 shows that for flow from approximately 0.05 kg/s to 0.14 kg/s, the percentage errors are $<2\%$. Below approximately 0.05 kg/s of flowrate, the errors are about 5% to 25%. Furthermore, the digital twin often over predicts loop pressure drop value in this data range. These larger percentage errors occur at flowrates of 0.016 kg/s and less. In fact, systematic under prediction usually occurs at loop pressure drop below 2000 Pa or about 0.481 kg/s where the maximum error is about 50 Pa. This is about 3.5 times the reading uncertainty of the manometer data. While this may be cause for concern, it is well within the correlation error of the experimental manometer data. Therefore, an absolute error of 45 Pa should not be too much a cause for concern. To visualise this, we can once more consider the residuals in relation to the error bars contributed to by (1) manometer reading error, (2) manometer correlation error, (3) $\pm 2\%$ Flowmeter Reading Error, and (4) pressure drop coefficient error ζ . Figure 4.6 shows the residuals plotted with a 2σ error bar calculated using methods

described in the methods section.

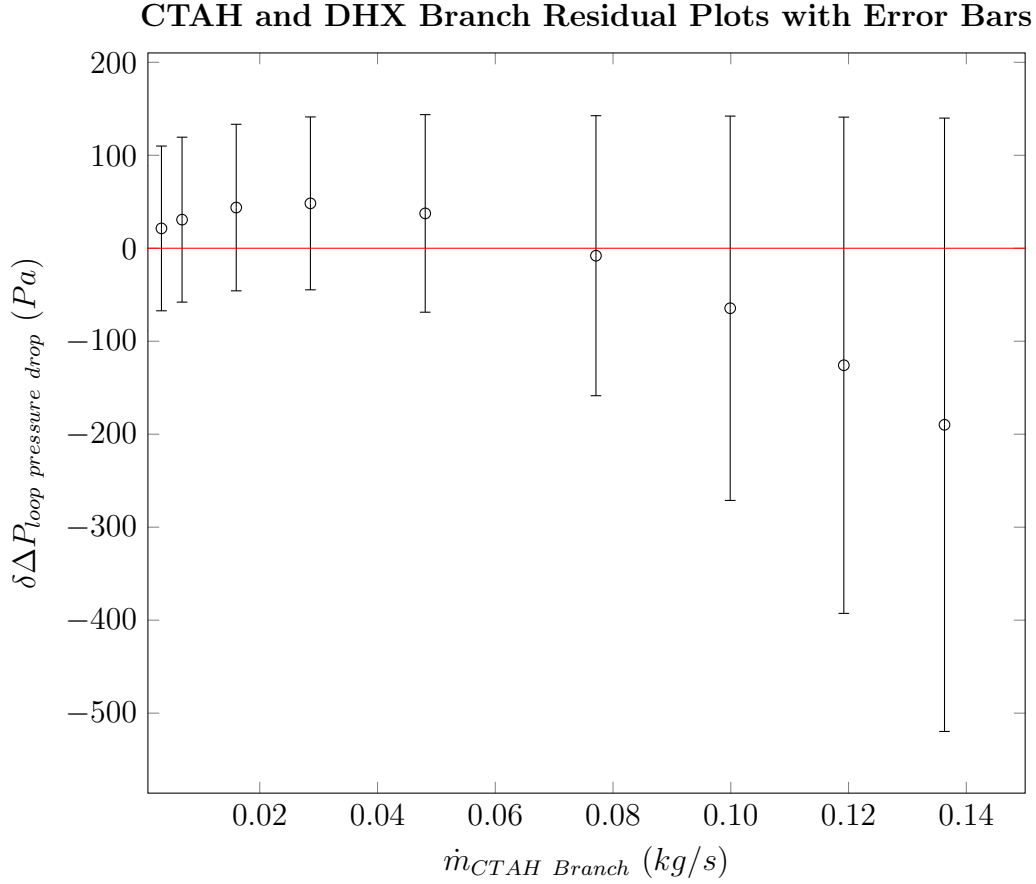


Figure 4.6: Absolute Residual Plot for CTAH and DHX Branch System Curves with Error Bars

We can see that once we take all these errors into account, the residuals are well within experimental error, measurement error and correlation error. To help the reader reproduce data, a polynomial regression was performed on these error bars and presented for the reader's reference:

$$\Delta P(\text{error bar}) = 16055.96 \dot{m}_{CTAH \text{ Branch}}^2 \text{ (kg/s)} - 437.88 \dot{m}_{CTAH \text{ Branch}} \text{ (kg/s)} + 90.51$$

$$2\sigma = \pm 3Pa$$

Furthermore, for flows in the flowrate $> 0.05 \text{ kg/s}$ an error of $< 2\%$ is quite acceptable compared to the errors of code found to emulate CIET in literature. This is about $< 3\%$ for FANCY [Nicolas Zweibaum, 2015]. For flows $< 0.05 \text{ kg/s}$, while these residuals relatively small compared to the error bars, we may want to evaluate their impact in future work

when validating flow through the DHX branch natural circulation regimes. This is the mass flowrate region where natural circulation data is available for the DHX branch (0.025 kg/s to 0.045 kg/s [N. Zweibaum et al., 2015; Zou, R. Hu, and Charpentier, 2019]).

We also note that unlike in the CTAH and Heater branch plots, the residuals are not consistently positive though they seem to follow a trend of a smooth curve. However, it is difficult to determine the exact cause of this deviation. Furthermore, it may not be worth discussing the cause of these errors at this time as they are within the range of correlation uncertainty at low flowrates (<0.03 kg/s). Therefore, I would say that the digital twin has reproduced the isothermal flowrate experimental results to within a reasonable accuracy.

Computational Speed

For the digital twin in the CTAH and DHX branch configuration, observed calculation times were in the range of 30 to 80 ms. This is similar to the computation speeds for CTAH and Heater branch configuration.

4.3 CTAH, DHX and Heater Branch Digital Twin Results

To ensure this study was more complete, I wanted to see the impact of adding the DHX branch in parallel to the heater branch. Specifically, I was looking for the impact on computation speed and time.

Computational Speed Test Setup

In the aspect of computation speed, we would expect that computation time would increase when extra computational load was added due to an extra branch of fluid components needing to be computed. Additionally, the DHX branch has a check valve [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014] which normally prevents flow in the upward direction through the DHX shell. Implementing this mathematically could slow down the solver because the system curves no longer become smooth and continuous functions. Therefore, tests were required to see how much time these calculations took. Note that when I mention forward flow I mean downwards flow through the CTAH, upwards flow through the Heater, and when I mention reverse flow I mean upwards flow through the CTAH, downwards flow through DHX and Heater Branches.

In both forward and reverse flow experiments, the following CPU was used:

AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx (8) @ 2.100GHz

Approximations made for Branch 5 and 17

At the risk of repetition, I reiterate that the digital twin here is constructed according to figure 3.10 where I took some liberties to simplify figure 3.9 and alter where the branches diverge out of convenience.

Implementing DHX Check Valve Behavior

In CIET, the DHX branch had a check valve which prevented forced convection through it, but allowed natural convection because it allowed flow in the downward direction but not the upward direction. Two ways were explored in this work:

One way to implement this was to introduce an extremely large form loss coefficient. This was done by artificially introducing the flowmeter in the DHX branch with the following loss coefficient in flow in the direction of forced convection:

$$\left(f \frac{L}{D} + K\right) = 1.0e10 + \frac{1.0e10}{Re} \quad (4.2)$$

The term dependent on Reynolds number was there to ensure K values were extremely high even in creeping flow regimes.

A second, more elegant, way was to return 0 kg/s of flow when the pressure gradient along the branch followed the direction of the pump.

Accuracy of Digital Twin Simulated Results

System Curve Analysis

While validation data isothermal flow in parallel across three branches is not provided, it is safe to say that one should expect the check valve behaviour to work such that when the isothermal digital twin operates in the normal forward flow (upwards through the heater, downwards in the CTAH branch), the results should be the same as the simulation containing only the heater branch and CTAH branch. In forward flow configuration, both check valve implementation techniques replicated the results for forward flow in the CTAH and Heater Branch. Thus, the system curves and residual plots were identical. I will not repeat the same analysis here for brevity.

In reverse flow configuration, we should expect there to be more flow through the CTAH branch because when extra parallel branches are added, flow resistance decreases. Both these behaviours are observed in figure 4.7. Crude error bars of 2σ are provided taking the (1) pressure loss coefficient (ζ) error to be a flat 10% of the loop pressure drop value based on ζ error estimates of 10% for CTAH, Static Mixer and Coriolis Flowmeter pressure loss coefficient correlations [Nicolas Zweibaum, 2015] (2) Manometer Experimental Correlation Error of 90 Pa, (3) Manometer Reading Error of 15 Pa, and (4) $\pm 2\%$ of Coriolis Flowmeter error for reverse flow across three branches. A rigorous approach was not taken because we wish to focus more on the computation speed potential of the algorithm in solving flow over

parallel flow branches rather than the accuracy and precision of the results. Accuracy only matters so far as to indicate that there is more mass flowrate in reverse flow configuration where the check valve is not active. For forward flow configuration, the error bar consists of (1) Manometer Experimental Correlation Error of 90 Pa and (2) $\pm 2\%$ of Coriolis Flowmeter error because we have already proven flow across the CTAH and Heater branches to be within this error.

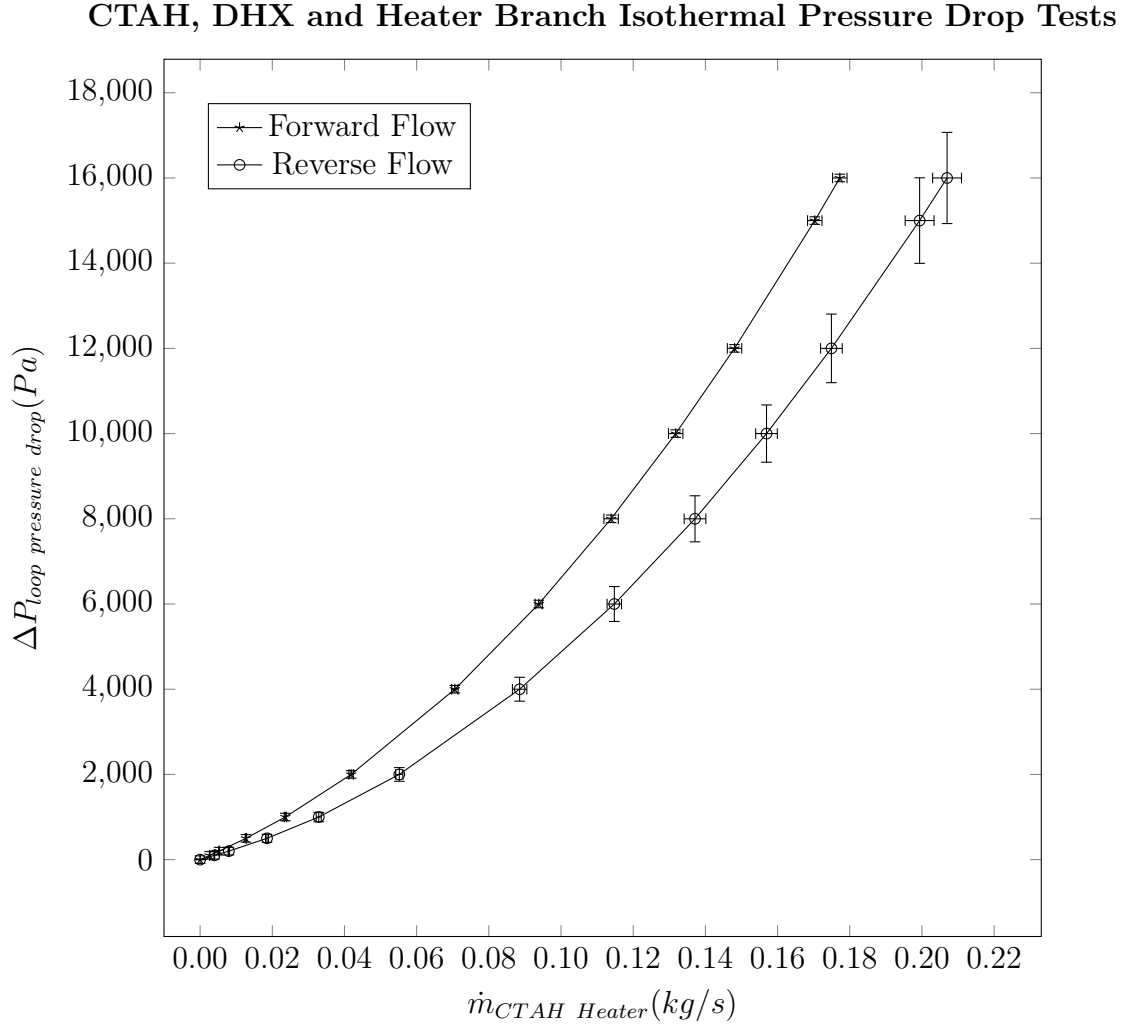


Figure 4.7: Digital Twin Forward Flow vs Reverse Flow

To the author's best knowledge, no experimental data was supplied for this case. Therefore, residual plots analysis is forgone in this work. The data for the flow in reverse configuration is provided in table 4.3 for the reader's reference. Mass Flowrate data is presented to three significant figures because of the $\pm 2\%$ flowmeter reading errors and loop pressure drop error is presented to two significant figures or the nearest 10 Pa because of the assumed crude error estimate of 10% ζ error and 90 Pa of manometer reading correlation error. We

assume that the correlation error of the manometer would be good enough for a crude error estimate to use here because the system was validated with a correlation of that error range.

FM-40 Mass Flowrate (kg/s)	Digital Twin Simulated Pressure Drop (Pa)
0.207	16000
0.199	15000
0.175	12000
0.157	10000
0.137	8000
0.115	6000
0.0885	4000
0.0552	2000
0.0329	1000
0.0185	500
0.00796	200
0.00396	100
0	0

Table 4.3: Reverse Flow Configuration Mass Flowrate vs Loop Pressure Drop Data

Computational Speed Tests

For computational speed tests, we wish to investigate the impact of implementing the DHX Branch with its check valve on the computation time taken. Two methods are to be investigated: (1) get ζ to be exceptionally high in reverse bias flow, (2) return 0 kg/s using an if statement.

For both these methods, the maximum time for computation exceeded 100ms given the Ryzen 5 CPU on the Ideapad. Peak computation times exceeded 120 ms for the high K check valve setting for both forward and reverse flow. By forward flow, I mean that fluid flows downwards in the CTAH branch. Whereas with the method of returning zero flowrate if pressure gradients favoured upward flow through the DHX, the peak computation times for reverse flow was 125 ms. However, time savings were indeed observed for forward flow with peak calculation times around 85 ms. These calculation times were mainly due to the

time it took for the solver to converge. A more detailed discussion on what could affect these convergence times is presented in the Appendix (Part C.C.5) for the reader's convenience.

Peak computation times of 125 ms were unacceptable because heat transfer should be implemented into CIET's digital twin in future work. This would ultimately increase computational loads and lead to longer computing times.

I was concerned if this computation time necessitated the use of computational clusters. Therefore I wanted to test this setup on a gaming laptop. If the calculation time could fall below 100 ms, it meant that consumer laptops had sufficient computational power to operate the digital twin so that supercomputing clusters were not necessary.

When the digital twin was run on a gaming laptop, peak computation times did not exceed 80 ms. This was good news because it showed that we did not need to resort to supercomputing clusters to operate the digital twin.

Moreover, these computation times are a conservative estimate because the Rust binaries were compiled in a non-optimised manner, otherwise known as "debug mode". If the binaries were compiled in an optimised manner, otherwise known as "release mode", then the performance gains could push computation times even lower. This however, is outside the scope of this work and is intended for future work.

Once more, the reader should note that that rigorous error analysis is not done on these values. The only purpose is to show that mass flowrate in reverse flow configuration is more than mass flowrate in forward flow configurations given a fixed loop pressure drop. This ensures that our check valve is working correctly. We are confident this is the case because the error values are significantly different at least beyond the error bounds $\pm 2\%$ of flowmeter reading.

Potential Use in Prediction of Reverse Flow Configuration Mass Flowrates

The lack of experimental data gives an opportunity for the digital twin to make predictions where experimental data was absent. This is indeed one of the potential uses of Type I digital twins described in the literature review section. In this case, reverse flow data could not be found in literature to the best of the author's knowledge. However, the digital twin has provided a reasonable prediction of what the total flowrate through the CTAH branch should be in reverse flow configuration. The digital twin is based on generic empirical correlations for pipes and fluid components such as the CTAH. These correlations are true independent whether the components are connected in series or parallel. Since these branches have been experimentally validated individually, it gives us a reasonable degree of confidence in the predictions made. The predicted mass flowrate and loop pressure drop data is given in table 4.3.

The exact degree of uncertainty is another topic altogether and should be explored when using this Digital Twin to predict flow in parallel flow paths. Without estimating the uncertainty of these predictions, the flow data provided here is of limited use.

Calculating this uncertainty is somewhat trickier than normal because solving for flow in parallel paths involves solving simultaneous nonlinear equations. Whereas the usual uncer-

tainty correlations almost always assume an explicit or at least implicit singular expression. Furthermore, assumptions must be made about errors and uncertainty propagation when predicted flow regimes extend beyond the experimental region of operation. For example, if flow goes beyond 0.18 kg/s as tested in the experiments, we cannot be absolutely sure about the flowmeter error. We could reasonably assume the manometer error is the same assuming the manometer still functions under the simulated flowrates. For values slightly above 0.18 kg/s we can perhaps assume that the manometer error is quite similar to those found below 0.18 kg/s. But far beyond that, we could simulate a pump speed of 1 kg/s or 10 kg/s. We may not know how the system really responds then unless we conduct experiments, if equipment starts breaking or the manometer stops functioning. In literature, there are methods for estimating errors in extrapolation [Cook and Stefanski, 1994]. If we manage to sort that out, we then need methodology for uncertainty quantification in simultaneous systems of nonlinear algebraic equations. This is also explored in literature [Lopez, Miguel, and De Cursi, 2013; Tang, Y. Li, and Wang, 2001; Vincent, T. Hu, et al., 2000].

This aspect is not explored here as I am deeming it out of scope of this work. The goal here is to demonstrate a Type I Digital Twin which is able to reproduce experimental data within the timeframe. Nevertheless, it is worth exploring in future work where extrapolation or predicting plant behaviour outside experimental data becomes important.

Chapter 5

Conclusion

This work shows that a prototype isothermal flow digital twin of CIET was constructed successfully using Rust and Python. This digital twin was validated using isothermal flow experimental data available in literature [N. Zweibaum et al., 2015; Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]. The data used was isothermal flow data from flow in the CTAH and Heater branches, as well as flow data in CTAH and DHX branches. The deviation from experimental data for both cases is about $\pm 50 Pa$ or $\pm 2\%$ whichever is greater. This was well within flowmeter error and manometer correlation error. The Type I Digital Twin could also estimate mass flowrates through the three branches, CTAH for reverse flow through the DHX and Heater Branches. For this setup, rigorous error estimation of the predicted data was not performed due to the complexity of error estimation through parallel branches. Instead, crude error bounds were given based on 10% ζ , manometer correlation error, and $\pm 2\%$ flowmeter error. More detailed error analysis will be required in future if the goal is to simulate flow through parallel flow paths. This work also shows that the parallel branch approximation approach for CIET is workable because even when branch points were shifted slightly for Branch 5 and 17 of CIET, the loop pressure drop data was still able to be replicated within experimental and correlation uncertainty. This allowed for simulation times to be reduced to an acceptable timeframe.

This digital twin could also be interacted with in real time via an OPC-UA server and client interface, and it was able to run within 100 ms for both the lower end Ideapad 3 and the gaming laptop if the only two out of three branches were included in calculations. Whereas when the DHX branch was added with check valve behaviour, only the gaming laptop managed to provide accurate results within 100 ms. This shows that even with unoptimised Rust binaries, the digital twin could be run without the need for high performance computer clusters.

It is also important that the libraries built here were constructed using open source software. The source code is available in the Appendix B for readers who are interested in replicating the results.

5.1 Future Work

To complete the digital twin, one must move beyond isothermal flow into heat transfer. Similar to the SAM model [Zou, R. Hu, and Charpentier, 2019], natural convection flows should be validated and at a later time, transients and frequency response test results from CIET should be used to validate the digital twin.

With each step however, the computational burden would increase with each timestep. It is therefore desirable to introduce parallelisation or optimised compilation in future work to ensure that at each timestep, all the required calculations can be completed within 100 ms or less.

Should we successfully complete this venture, the next steps could be adapting the Type I digital twin into a Type II or Type III digital twin with real-time state synchronisation capabilities. This could be a study to use the digital twin as a tool for fault detection or fault tolerant control as described in the literature review. We could even develop a prototype of a Type III digital twin for use in predictive maintenance.

One other use of the Type I digital twin could be in a simulated neutronics feedback study. Such a study requires development of a heater controller which simulates coupled reactor physics and thermal hydraulics behaviours. The Type I Digital Twin would then serve as a safe testing and development environment for such a controller. Should the controller be properly developed, I may want to investigate the impact of a loss of heat sink (LOHS) incident on the pipe temperatures when the reactor is running at full power. Unfortunately, CIET is that it is scaled to about 2% of the Mark I FHR at prototypical conditions [N. Zweibaum et al., 2015]. This means its 10 kWth heater is scaled to about 500 kWth of power or 0.5 MWth of heater power. A typical demonstration reactor such as the Karios Power Hermes Reactor would be around 35 MWth. To have CIET scale to that power, it would need to be about 70 times more power. In other words, we would need to have a 700 kWth heater at least or 0.7 MWth. This is not practical for CIET due to the level of power draw required and potential safety concerns of installing a heater that powerful. Even if those issues were solved, the cost of running such a setup would likely prohibit any extended operation of such an apparatus.

Thankfully, with type I digital twin of CIET, we may be able to simply increase heater power by changing one line of code. I may also be able to artificially increase pump power to reach an appropriate mass flowrate, and then perform a LOHS scenario study at this power level. In this case however, we must then delve into predicting the uncertainty of the extrapolated flow data.

As illustrated here, once the digital twin is constructed, there are many possibilities for research and scenario assessment. Also, once the libraries and source code for the Type I Digital Twin are completed, I intend to release them with an open source license so that research and development in digital twins can be accelerated rather than be bottlenecked by a paywall. If this is done well, this Digital Twin codebase could be one of the first Type I Digital Twins and perhaps the first Type I Digital Twin (and system level program) of CIET to be released under an open source license. This is intended for making Digital Twin

research more accessible.

Bibliography

- McCausland, Tammy (Jan. 2022). “Digital Twins”. In: *Research-Technology Management* 65.1, pp. 69–71. ISSN: 0895-6308. DOI: 10.1080/08956308.2022.1999637. (Visited on 10/04/2022).
- Sleiti, Ahmad K, Jayanta S Kapat, and Ladislav Vesely (2022). “Digital twin in energy industry: Proposed robust digital twin for power plant and other complex capital-intensive large engineering systems”. In: *Energy Reports* 8, pp. 3704–3726. ISSN: 2352-4847.
- Liu, Mengnan et al. (2021). “Review of digital twin about concepts, technologies, and industrial applications”. In: *Journal of Manufacturing Systems* 58, pp. 346–361.
- Pérez, Luis et al. (2020). “Digital twin and virtual reality based methodology for multi-robot manufacturing cell commissioning”. In: *Applied sciences* 10.10, p. 3633.
- Zweibaum, N. et al. (2015). “Design, fabrication and startup testing in the compact integral effects test facility in support of fluoride-saltcooled, high-temperature reactor technology”. In: *International Topical Meeting on Nuclear Reactor Thermal Hydraulics (NURETH-16), Chicago, IL, August*.
- Scarlat, Raluca et al. (Nov. 2014). “Design and licensing strategies for the fluoride-salt-cooled, high-temperature reactor (FHR) technology”. In: *Progress in Nuclear Energy* 77, pp. 406–420. DOI: 10.1016/j.pnucene.2014.07.002.
- Zweibaum, Nicolas (2015). *Experimental Validation of Passive Safety System Models: Application to Design and Optimization of Fluoride-Salt-Cooled, High-Temperature Reactors*. University of California, Berkeley. ISBN: 1339216086.
- Liu, Limin et al. (Mar. 2020). “Scaling and distortion analysis using a simple natural circulation loop for FHR development”. en. In: *Applied Thermal Engineering* 168, p. 114849. ISSN: 1359-4311. DOI: 10.1016/j.applthermaleng.2019.114849. URL: <https://www.sciencedirect.com/science/article/pii/S1359431119348677> (visited on 10/05/2022).
- Abou Dbai, Mohamed, Raluca O Scarlat, and Mario F Trujillo (2020). “Radiative heat transfer in FLiBe molten salt participating medium in a vertical heated tube under forced and mixed convection laminar flows”. In: *Nuclear Engineering and Design* 368, p. 110775.

- Johnson, Ishak (2022). “Quantifying Radiative Heat Transfer Scaling Distortions: System Code Development and a Scaling Methodology for Fluoride-Salt-Cooled High-Temperature Reactors”. PhD thesis. UC Berkeley.
- Peterson, Per F. and Michael Scott Greenwood (2019). *A frequency response approach to model validation for the compact integral effects test facility in transform*. Tech. rep. Oak Ridge National Lab.(ORNL), Oak Ridge, TN (United States).
- Zou, Ling, Rui Hu, and Anne Charpentier (2019). *SAM Code Validation using the Compact Integral Effects Test (CIET) Experimental Data*. Tech. rep. Argonne National Lab.(ANL), Argonne, IL (United States).
- Poresky, Christopher et al. (Sept. 2022). “Advanced Reactor Control and Operations (ARCO): A University Research Facility for Developing Optimized Digital Control Rooms”. In: *Nuclear Technology* 0.0, pp. 1–12. ISSN: 0029-5450. DOI: 10.1080/00295450.2022.2092366. (Visited on 10/05/2022).
- Poresky, Christopher (2019). “Model Network Methodology for Experimental Development of Industrial Monitoring Systems”. PhD thesis.
- Allen, B Danette (Nov. 2021). *Digital Twins and Living Models at NASA*. Webpage. NTRS Author Affiliations: Langley Research Center NTRS Meeting Information: Digital Twin Summit; 2021-11-03 to 2021-11-04; undefined NTRS Document ID: 20210023699 NTRS Research Center: Langley Research Center (LaRC). URL: <https://ntrs.nasa.gov/citations/20210023699> (visited on 10/06/2022).
- Akbarian, Fatemeh, Emma Fitzgerald, and Maria Kihl (2020). “Synchronization in digital twins for industrial control systems”. In: *arXiv preprint arXiv:2006.03447*.
- Li, Phd and H (2022). “Finding Roots of Equations”. In: *Numerical Methods Using Java*. Apress, pp. 207–228. DOI: 10.1007/978-1-4842-6797-4_3.
- Marguet, Serge (2022). “History of the Pressurized Water Reactor”. In: *The Technology of Pressurized Water Reactors*. Cham: Springer International Publishing, pp. 1–277. DOI: 10.1007/978-3-030-86638-9_1.
- Augustine, Peter (2020). “The industry use cases for the digital twin idea”. In: *Advances in Computers*. Vol. 117. 1. Elsevier, pp. 79–105.
- Leblanc, J. C. (July 1970). *Apollo 13 LiOH canister breakthrough test*. Tech. rep. CSD-A-1070. NTRS Author Affiliations: NASA Lyndon B. Johnson Space Center NTRS Document ID: 19720021432 NTRS Research Center: Legacy CDMS (CDMS). URL: <https://ntrs.nasa.gov/citations/19720021432> (visited on 10/06/2022).
- Boschert, Stefan and Roland Rosen (2016). “Digital twin—the simulation aspect”. In: *Mechatron. Futur*. Springer, pp. 59–74.
- Jones, David et al. (2020). “Characterising the Digital Twin: A systematic literature review”. In: *CIRP Journal of Manufacturing Science and Technology* 29, pp. 36–52.
- Grieves, Michael and John Vickers (2017). “Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems”. In: *Transdisciplinary perspectives on complex systems*. Springer, pp. 85–113.

- Isermann, Rolf and Peter Balle (1997). “Trends in the application of model-based fault detection and diagnosis of technical processes”. In: *Control engineering practice* 5.5, pp. 709–719.
- Kochunas, Brendan and Xun Huan (2021). “Digital twin concepts with uncertainty for nuclear power applications”. In: *Energies* 14.14, p. 4235.
- Van der Valk, Hendrik et al. (2020). “A Taxonomy of Digital Twins.” In: *AMCIS*.
- Rasheed, Adil, Omer San, and Trond Kvamsdal (2020). “Digital Twin: Values, Challenges and Enablers From a Modeling Perspective”. In: *IEEE Access* 8, pp. 21980–22012. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2020.2970143.
- Enders, Martin Robert and Nadja Hoßbach (2019). “Dimensions of digital twin applications-a literature review”. In.
- Eckhart, Matthias and Andreas Ekelhart (May 22, 2018). “Towards Security-Aware Virtual Environments for Digital Twins”. In: *Proceedings of the 4th ACM Workshop on Cyber-Physical System Security*. ACM, pp. 61–72. DOI: 10.1145/3198458.3198464.
- Jain, Palak et al. (2019). “A Digital Twin Approach for Fault Diagnosis in Distributed Photovoltaic Systems”. In: *IEEE Transactions on Power Electronics* 35.1, pp. 940–956. DOI: 10.1109/tpel.2019.2911594.
- Semeraro, Concetta et al. (2021). “Digital twin paradigm: A systematic literature review”. In: *Computers in Industry* 130, p. 103469.
- Lupton, Deborah (May 2021). “Language matters: the ‘digital twin’ metaphor in health and medicine”. eng. In: *Journal of Medical Ethics*, medethics–2021–107517. ISSN: 1473-4257. DOI: 10.1136/medethics-2021-107517.
- Zipper, Holger and Christian Diedrich (Sept. 2019). “Synchronization of Industrial Plant and Digital Twin”. In: *2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, pp. 1678–1681. DOI: 10.1109/etfa.2019.8868994.
- GSESolutions (2016). *Generic PWR: Control Room Simulator*. GSESolutions. URL: <https://www.gses.com/wp-content/uploads/GSE-GPWR-brochure.pdf>.
- Park, JaeKwan et al. (2022). “Control automation in the heat-up mode of a nuclear power plant using reinforcement learning”. In: *Progress in Nuclear Energy* 145, p. 104107.
- Lin, Linyu et al. (2022). “Digital-twin-based improvements to diagnosis, prognosis, strategy assessment, and discrepancy checking in a nearly autonomous management and control system”. In: *Annals of Nuclear Energy* 166, p. 108715.
- Fotland, Gaute, Cecilia Haskins, and Terje Rølvåg (2020). “Trade study to select best alternative for cable and pulley simulation for cranes on offshore vessels”. In: *Systems Engineering* 23.2, pp. 177–188.
- Mobley, R Keith (2002). *An introduction to predictive maintenance*. Elsevier.
- Dey, Satadru, Zoleikha Abdollahi Biron, et al. (2016). “Model-based real-time thermal fault diagnosis of Lithium-ion batteries”. In: *Control Engineering Practice* 56, pp. 37–48.
- Dey, Satadru, Hector Perez, and Scott Moura (2017). “Model-Based Battery Thermal Fault Diagnostics: Algorithms, Analysis, and Experiments”. In: *IEEE Transactions on Control Systems Technology* 27.2, pp. 576–587. DOI: 10.1109/tcst.2017.2776218.

- Nguyen, Tat Nghia et al. (2022). “A digital twin approach to system-level fault detection and diagnosis for improved equipment health monitoring”. In: *Annals of Nuclear Energy* 170, p. 109002.
- Gong, Helin, Sibio Cheng, et al. (2022). “Data-enabled physics-informed machine learning for reduced-order modeling digital twin: application to nuclear reactor physics”. In: *Nuclear Science and Engineering* 196.6, pp. 668–693.
- Gong, Helin, Tao Zhu, et al. (2023). “Parameter identification and state estimation for nuclear reactor operation digital twin”. In: *Annals of Nuclear Energy* 180, p. 109497.
- Varé, Christophe and Patrick Morilhat (2020). “Digital twins, a new step for long term operation of nuclear power plants”. In: *Engineering Assets and Public Infrastructures in the Age of Digitalization: Proceedings of the 13th World Congress on Engineering Asset Management*. Springer, pp. 96–103.
- Idaho National Lab (July 2022). *Idaho National Laboratory Demonstrates First Digital Twin of a Simulated Microreactor*. Office of Nuclear Energy. URL: <https://www.energy.gov/ne/articles/idaho-national-laboratory-demonstrates-first-digital-twin-simulated-microreactor>.
- Turner, Clayton G. (2020). “Microreactor AGile Non-Nuclear Experimental Testbed Aspen HYSYS Analysis”. PhD Thesis. University of Idaho.
- Lu, Chao et al. (2020). “Nuclear power plants with artificial intelligence in industry 4.0 era: Top-level design and current applications—A systemic review”. In: *IEEE Access* 8, pp. 194315–194332.
- Groshev, Milan et al. (2021). “Toward intelligent cyber-physical systems: Digital twin meets artificial intelligence”. In: *IEEE Communications Magazine* 59.8, pp. 14–20.
- Tao, Fei et al. (2018). “Digital twin in industry: State-of-the-art”. In: *IEEE Transactions on industrial informatics* 15.4, pp. 2405–2415.
- Lamarsh, John R, Anthony John Baratta, et al. (2001). *Introduction to nuclear engineering*. Vol. 3. Prentice hall Upper Saddle River, NJ.
- Deng, Chao et al. (2019). “Predicting drivers’ direction sign reading reaction time using an integrated cognitive architecture”. In: *IET Intelligent Transport Systems* 13.4, pp. 622–627.
- Izadi-Zamanabadi, Roozbeh (2002). “Structural analysis approach to fault diagnosis with application to fixed-wing aircraft motion”. In: *Proceedings of the 2002 American Control Conference (IEEE Cat. No. CH37301)*. Vol. 5. IEEE, pp. 3949–3954.
- Izadi-Zamanabadi, Roozbeh and Marcel Staroswiecki (2000). “A structural analysis method formulation for fault-tolerant control system design”. In: *Proceedings of the 39th IEEE Conference on Decision and Control (Cat. No. 00CH37187)*. Vol. 5. IEEE, pp. 4901–4902.
- Frisk, Erik, Mattias Krysander, and Daniel Jung (2017). “A toolbox for analysis and design of model based diagnosis systems for large scale models”. In: *IFAC-PapersOnLine* 50.1, pp. 3287–3293.

- Wood, Richard T, Belle R Upadhyaya, and Dan C Floyd (2017). “An autonomous control framework for advanced reactors”. In: *Nuclear Engineering and Technology* 49.5, pp. 896–904.
- Al-Geddawy, Tarek (2020). “A Digital Twin Creation Method for an Opensource Low-cost Changeable Learning Factory”. In: *Procedia Manufacturing* 51, pp. 1799–1805.
- Silvennoinen, Eero et al. (1989). *The APROS software for process simulation and model development*. VTT Technical Research Centre of Finland.
- Gallagher, Shane (2021). *Technology Readiness and Development of the Reheat Air-Brayton Combined Cycle Power Conversion System Using Molten Salt*. University of California, Berkeley.
- Puska, Eija Karita et al. (1995). “APROS simulation system for nuclear power plant analysis”. In.
- Zhang, Xiuxia et al. (Jan. 2017). “Understanding the GPU Microarchitecture to Achieve Bare-Metal Performance Tuning”. In: *ACM SIGPLAN Notices* 52.8, pp. 31–43. DOI: 10.1145/3155284.3018755.
- Lee, Andrew et al. (2021). “The IDAES process modeling framework and model library—Flexibility for process simulation and optimization”. en. In: *Journal of Advanced Manufacturing and Processing* 3.3, e10095. ISSN: 2637-403X. DOI: 10.1002/amp.2.10095. (Visited on 10/04/2022).
- Hart, William E., Jean-Paul Watson, and David L. Woodruff (2011). “Pyomo: modeling and solving mathematical programs in Python”. In: *Mathematical Programming Computation* 3.3, pp. 219–260.
- Hart, William E. (2017). *Pyomo Network Documentation Sequential Decomposition*. Sandia National Laboratories Webpage. URL: https://pyomo.readthedocs.io/en/stable/modeling_extensions/network.html?highlight=network#sequential-decomposition.
- Murgatroyd, Julian et al. (Nov. 2017). “Qualification of MELCOR and RELAP5 models for EU HCPB TBS accident analyses”. In: *Fusion Engineering and Design* 124, pp. 1251–1256. DOI: 10.1016/j.fusengdes.2017.02.083.
- Tangsriwong, Kwanchanok et al. (2020). “Modeling of chemical processes using commercial and open-source software: A comparison between Aspen Plus and DWSIM”. In: *IOP Conference Series: Earth and Environmental Science*. Vol. 463. 1. IOP Publishing, p. 012057.
- Bell, Ian H. et al. (2014). “Pure and pseudo-pure fluid thermophysical property evaluation and the open-source thermophysical property library CoolProp”. In: *Industrial & engineering chemistry research* 53.6, pp. 2498–2508.
- De Wet, Dane and Per F Peterson (2020). “A Frequency Domain Approach to Characterizing and Modeling Single Phased, Forced Circulation Advanced Nuclear Reactor Designs”. Dissertation. University of California, Berkeley.
- Son, Karen N et al. (2018). “Calibration and uncertainty analysis of a fixed-bed adsorption model for CO₂ separation”. In: *Adsorption* 24, pp. 781–802.

- Huddar, L et al. (2015). “Experimental Strategy for the Determination of Heat Transfer Coefficients in Pebble-Beds Cooled By Fluoride Salts”. In: *NURETH-16, Chicago, IL, August 30-September 4 2015*, pp. 1659–1675.
- BIPM, IFCC et al. (2008). “Evaluation of measurement data—guide to the expression of uncertainty in measurement, JCGM 100: 2008 GUM 1995 with minor corrections”. In: *Joint Committee for Guides in Metrology*.
- MacNamara, Shev and Gilbert Strang (2016). “Operator Splitting”. en. In: ed. by Roland Glowinski, Stanley J. Osher, and Wotao Yin. *Scientific Computation*. Cham: Springer International Publishing, pp. 95–114. ISBN: 9783319415895. DOI: 10.1007/978-3-319-41589-5_3. (Visited on 10/04/2022).
- J. Oliveira Paulo, Raad I. Issa (Dec. 2001). “An Improved Piso Algorithm for the Computation of Buoyancy-Driven Flows”. In: *Numerical Heat Transfer, Part B: Fundamentals* 40.6, pp. 473–493. ISSN: 1040-7790. DOI: 10.1080/104077901753306601. (Visited on 10/04/2022).
- Zikmund, Radek (2018). “NextGen SPICE-Electrical Circuit Simulation Library for. NET”. In.
- Hamam, Y. M. and A. Brameller (1971). “Hybrid method for the solution of piping networks”. In: *Proceedings of the institution of electrical engineers*. Vol. 118. 11. IET, pp. 1607–1612.
- Perry, Robert H and Don W Green (2015). “Perry’s chemical engineers’ handbook”. In: *Mc Graw*.
- Kazala, R. and P. Straczynski (Jan. 2019). “The Most Important Open Technologies for Design of Cost Efficient Automation Systems”. en. In: *IFAC-PapersOnLine*. 19th IFAC Conference on Technology, Culture and International Stability TECIS 2019 52.25, pp. 391–396. ISSN: 2405-8963. DOI: 10.1016/j.ifacol.2019.12.567. URL: <https://www.sciencedirect.com/science/article/pii/S2405896319324887> (visited on 10/04/2022).
- Schwarz, Michael H and Josef Börcsök (2013). “A survey on OPC and OPC-UA: About the standard, developments and investigations”. In: *2013 XXIV Int. Conf. Information, Commun. Autom. Technol.* IEEE, pp. 1–6. ISBN: 1479904317.
- Muhlbauer, Nikolas et al. (2021). “Open-Source OPC UA Security and Scalability”. In: *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. Vol. 2020. IEEE. DOI: 10.1109/etfa46521.2020.9212091.
- Erba, A, A Müller, and N Tippenhauer (2021). “Practical pitfalls for security in OPC UA”. In: *arXiv preprint arXiv:2104.06051*. eprint: arXiv:2104.06051.
- Zidek, Kamil et al. (2020). “Digital twin of experimental smart manufacturing assembly system for industry 4.0 concept”. In: *Sustainability* 12.9, p. 3658.
- Cimino, Chiara, Elisa Negri, and Luca Fumagalli (Dec. 2019). “Review of digital twin applications in manufacturing”. en. In: *Computers in Industry* 113, p. 103130. ISSN: 0166-3615. DOI: 10.1016/j.compind.2019.103130. URL: <https://www.sciencedirect.com/science/article/pii/S0166361519304385> (visited on 10/04/2022).

- Zhe Peng, Longhua Ma (Sept. 2012). *OPC ClientOLE for Process Control Toolbox for Scilab*. Zhejiang University. URL: https://atoms.scilab.org/toolboxes/opc_client/1.3.1.
- Oroulet (Sept. 2015). *opcua-client-gui*. Github. OPCUA Client Graphical User Interface (GUI) Github Repository. URL: <https://github.com/FreeOpcUa>.
- Kazała, Robert et al. (Jan. 2022). “An Enabling Open-Source Technology for Development and Prototyping of Production Systems by Applying Digital Twinning”. en. In: *Processes* 10.1, p. 21. ISSN: 2227-9717. DOI: 10.3390/pr10010021. URL: <https://www.mdpi.com/2227-9717/10/1/21> (visited on 10/04/2022).
- Young, Meredith (Dec. 2019). “The utility of failure: a taxonomy for research and scholarship”. In: *Perspectives on Medical Education* 8.6, pp. 365–371. ISSN: 2212-2761. DOI: 10.1007/s40037-019-00551-6. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6904373/> (visited on 01/29/2023).
- Kovacs, Zoltan (2011). “Understanding convergence and stability of the Newton-Raphson method”. In: *Interesting Mathematical Problems in Sciences and Everyday life*. Szeged.
- Duan, Ran, Hongxun Wu, and Renfei Zhou (2022). “Faster matrix multiplication via asymmetric hashing”. In: *arXiv preprint arXiv:2210.10173*.
- Bejan, Adrian (2013). *Convection heat transfer*. John Wiley & sons.
- Bickel, Jeffrey E, Nicholas Zweibaum, and Per F Peterson (2014). *Design, Fabrication and Startup Testing in the Compact Integral Effects Test (CIET 1.0) Facility in Support of Fluoride-Salt-Cooled, High-Temperature Reactor Technology*. University of California, Berkeley.
- Stage, S (2013). “Comments on an improvement to the Brent’s method”. In: *International Journal of Experimental Algorithms* 4.1, pp. 1–16.
- Boulanger, Sven, Marcin Golebiowski, and Sodalom (June 2017). *SpiceSharp*. Github. SpiceSharp Github Repository. URL: <https://github.com/SpiceSharp/SpiceSharp>.
- Ardito, Luca et al. (Feb. 2021). “Evaluation of Rust code verbosity, understandability and complexity”. In: *PeerJ Computer Science* 7, e406. ISSN: 2376-5992. DOI: 10.7717/peerj-cs.406. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7959618/> (visited on 10/04/2022).
- Boutin, Mike (June 2022). *Units of measurement (uom) v0.33.0*. Rust Crate io. URL: <https://crates.io/crates/uom>.
- Kim, Tae-Geun (Sept. 2022). *Peroxide*. Github. original-date: 2018-04-20T18:38:06Z. URL: <https://github.com/Axect/Peroxide> (visited on 10/04/2022).
- Oroulet et al. (Oct. 2022). *opcua-asyncio*. Github. original-date: 2018-08-02T07:45:42Z. URL: <https://github.com/FreeOpcUa/opcua-asyncio> (visited on 10/04/2022).
- Virtanen, Pauli et al. (Feb. 3, 2020). “SciPy 1.0: fundamental algorithms for scientific computing in Python”. In: *Nature Methods* 17.3, pp. 261–272. DOI: 10.1038/s41592-019-0686-2.
- James Newkirk Brad Wilson, Claire Novotny (2022). *Running Tests in Parallel*. .Net Foundation. Xunit Documentation on showing that tests are run in parallel by default since xUnit.Net Version 2. URL: <https://xunit.net/docs/running-tests-in-parallel>.

- Hanabi1224 (Oct. 2022a). *Python VS C Sharp benchmarks*. Webpage. URL: <https://programming-language-benchmarks.vercel.app/python-vs-csharp>.
- (Oct. 2022b). *Rust VS C Sharp benchmarks*. Webpage. URL: <https://programming-language-benchmarks.vercel.app/Rust-vs-csharp>.
- (Oct. 2022c). *C VS C++ benchmarks*. Webpage. URL: <https://programming-language-benchmarks.vercel.app/c-vs-cpp>.
- Oberg, J. (Dec. 1999). “Why the Mars probe went off course [accident investigation]”. In: *IEEE Spectrum* 36.12, pp. 34–39. ISSN: 1939-9340. DOI: 10.1109/6.809121.
- MadsKirkFoged (May 2021). *EngineeringUnits 1.1.580*. en. Nuget. URL: <https://www.nuget.org/packages/EngineeringUnits/> (visited on 10/04/2022).
- Locka99 (Sept. 2022). *OPC UA server and client API implementation for Rust*. Github. original-date: 2017-01-07T09:48:09Z. URL: <https://github.com/locka99/opcua> (visited on 10/04/2022).
- Regen, Martin, Erich Barnstedt, and Alin Moldovean (Feb. 2016). *OPCFoundation.NetStandard.Opc.Ua 1.4.370.12 Github*. Github and Nuget. URL: <https://github.com/OPCFoundation/UA-.NETStandard/>.
- Armstrong, Randy (Dec. 2015). *UA-ModelCompiler Github Repository by OPC Foundation*. Github. URL: <https://github.com/OPCFoundation/UA-ModelCompiler/>.
- Manditereza, Kudzai (Jan. 2021). *OPC UA .NET Tutorial - Creating Information Model and OPC UA Server Using NET [4 of 11]*. YouTube. URL: <https://www.youtube.com/watch?v=gxA7SDNLHgc> (visited on 10/04/2022).
- Larsen, Kristian Pontoppidan (Oct. 2022). *graphreader.com - Online tool for reading graph image values and save as CSV JSON*. Webpage. URL: <http://www.graphreader.com/>.
- Todreas, Neil E, Mujid S Kazimi, and Mahmoud Massoud (2021). *Nuclear systems Volume II: Elements of thermal hydraulic design*. CRC Press.
- Meyer, Veronika R (2007). “Measurement uncertainty”. In: *Journal of Chromatography A* 1158.1-2, pp. 15–24.
- Poresky, Christopher (2017). *Frequency response testing in the ciet facility*. Berkeley, CA.
- Cook, John R and Leonard A Stefanski (1994). “Simulation-extrapolation estimation in parametric measurement error models”. In: *Journal of the American Statistical association* 89.428, pp. 1314–1328.
- Lopez, RH, LF Fadel Miguel, and JE Souza De Cursi (2013). “Uncertainty quantification for algebraic systems of equations”. In: *Computers & Structures* 128, pp. 189–202.
- Tang, Shouzheng, Yong Li, and Yonghe Wang (2001). “Simultaneous equations, error-in-variable models, and model integration in systems ecology”. In: *Ecological Modelling* 142.3, pp. 285–294.
- Vincent, HT, T Hu, et al. (2000). “Generalized multivariate error propagation for zero memory nonlinear systems”. In: *Proceedings of 8th ASCE Specialty Conference on Probabilistic Mechanics and Structural Reliability*. Citeseer.
- Chen, Yubao (Oct. 2017). “Integrated and Intelligent Manufacturing: Perspectives and Enablers”. In: *Engineering* 3.5, pp. 588–595. DOI: 10.1016/j.eng.2017.04.009.

- Kholopov, VA et al. (2019). “Digital twins in manufacturing”. In: *Russian Engineering Research* 39.12, pp. 1014–1020.
- Garcia, Carlos E, David M Prett, and Manfred Morari (1989). “Model predictive control: Theory and practice—A survey”. In: *Automatica* 25.3, pp. 335–348.
- Majdzik, Pawel et al. (2022). “Integrated fault-tolerant control of assembly and automated guided vehicle-based transportation layers”. In: *International Journal of Computer Integrated Manufacturing*, pp. 1–18.
- Saraeian, Shideh and Babak Shirazi (2022). “Digital twin-based fault tolerance approach for Cyber–Physical Production System”. In: *ISA transactions*.
- He, Rui et al. (2019). “Data-driven digital twin technology for optimized control in process systems”. In: *ISA transactions* 95, pp. 221–234.
- Erber, T, SA Guralnick, and SC Michels (1993). “Hysteresis and fatigue”. In: *Annals of Physics* 224.2, pp. 157–192.
- Song, Houde, Meiqi Song, and Xiaojing Liu (2022). “Online autonomous calibration of digital twins using machine learning with application to nuclear power plants”. In: *Applied Energy* 326, p. 119995.
- Klabnik, Steve and Carol Nichols (June 2022). *References and Borrowing*. Github. Rust Borrow Checker Documentation assuming Rust 1.62 is used. URL: <https://doc.Rust-lang.org/book/ch04-02-references-and-borrowing.html>.
- Lostmsu, Vmuriart, and Filmor (May 2006). *Python .Net Github Repository*. Github and Nuget. Python .Net Github Repo. URL: <https://github.com/pythonnet/pythonnet#pythonnet---pythonnet>.
- Traeger (Oct. 2022). *OPC UA Client and Server in C Sharp VB.NET quick and easy*. Company Webpage. URL: <https://opcua.traeger.de/en/#prices>.

Appendix A

Hydrodynamic Parameters used for Nodalised Python and Rust Model

For the isothermal digital twin prototype, I used mainly data from the early RELAP model [N. Zweibaum et al., 2015]. I have reproduced the table here in figure A.1 and have also added Process and Instrumentation Diagram (P&ID) labels in case one is referring to the CIET P&ID for easy reference.

Component Description	P&ID Label For Mixers And Flowmeters	Number on Diagram	Component Length (m)	Vertical Angle (°)	Hydraulic Diameter (m)	Flow area (m²)	Form Loss K (dimensionless)
Heater Branch (Bottom to Top)							
Pipe		18	0.1778	-40.0052	2.79E-02	6.11E-04	5.15
Heater bottom head		1b	0.19685	90	6.60E-03	3.64E-04	3.95
Heater		1	1.6383	90	6.60E-03	3.64E-04	0
Heater top head		1a	0.0889	90	6.60E-03	3.64E-04	3.75
Static mixer pipe		2a	0.149425	90	2.79E-02	6.11E-04	1.8
Static mixer	MX-10	2	0.33	90	2.79E-02	6.11E-04	*
Pipe		3	1.2827	90	2.79E-02	6.11E-04	3.15
Pipe		4	0.2413	49.743387	2.79E-02	6.11E-04	2.4
Branch		5	0.7493	0	2.79E-02	6.11E-04	0
CTAH Branch (Top to Bottom)							
Static mixer pipe		6a	0.1526	51.526384	2.79E-02	6.11E-04	5.05
Static mixer	MX-41	6	0.33	51.526384	2.79E-02	6.11E-04	*
CTAH (Vertical)		7a	0.3302	-90	1.19E-02	1.33E-03	3.9
CTAH (Horizontal)		7b	1.2342	0	1.19E-02	1.33E-03	***
Static mixer pipe		8a	0.22245	-90	2.79E-02	6.11E-04	3.75
Static mixer	MX-40	8	0.33	-90	2.79E-02	6.11E-04	*
Pipe		9	0.7112	-42.73211	2.79E-02	6.11E-04	0.8
Pipe		10	2.4511	-90	2.79E-02	6.11E-04	0.45
Pipe		11	0.4826	-63.47465	2.79E-02	6.11E-04	2.4
Pipe		12	0.333375	0	2.79E-02	6.11E-04	21.65
Pipe		13	1.273175	0	2.79E-02	6.11E-04	12.95
Pipe		14	0.6687	90	2.79E-02	6.11E-04	2.4
Flowmeter	FM-40	14a	0.36	90	2.79E-02	6.11E-04	**
Pipe		15	0.3556	-49.36983	2.79E-02	6.11E-04	0.8
Pipe		16	0.644525	-90	2.79E-02	6.11E-04	1.9
Branch		17	0.473075	0	2.79E-02	6.11E-04	0
DHX Branch (Bottom to Top)							
Pipe		19	0.219075	-31.44898	2.79E-02	6.11E-04	7.5
Pipe		20	0.33655	0	2.79E-02	6.11E-04	0
Pipe		21	0.487725	90	2.79E-02	6.11E-04	4.4
Flowmeter	FM-20	21a	0.36	90	2.79E-02	6.11E-04	**
Pipe		22	0.69215	90	2.79E-02	6.11E-04	9.95
Static mixer pipe		23a	0.0891	90	2.79E-02	6.11E-04	1.35
Static mixer	MX-20	23	0.33	90	2.79E-02	6.11E-04	*
DHX shell side		24	1.18745	90	5.65E-03	9.43E-04	23.9
Static mixer pipe		25a	0.22245	90	2.79E-02	6.11E-04	1.35
Static mixer	MX-21	25	0.33	90	2.79E-02	6.11E-04	*
Pipe		26	0.2159	52.571994	2.79E-02	6.11E-04	1.75

Figure A.1: Hydrodynamic Parameters used for the Python and Rust Model

*** For CTAH fLDK correlation, see below

** For Flowmeter fLDK correlation, see below

* For Static Mixer fLDK correlation, see below

For the Coiled Tube Air Heater (CTAH), the Flowmeters and Static Mixers, we treat them as a custom component with a custom fLDK value. The fLDK term is found below:

$$\Delta P_{loss\ i} = \frac{1}{2} \frac{\dot{m}^2}{\rho A_{xs}^2} (f_{darcy} \frac{L}{D} + K)$$

$$fLDK = (f_{darcy} \frac{L}{D} + K)$$

Note that the darcy friction factor is not actually calculated from Churchill's Correlation [Perry and Green, 2015] in code when fLDK is calculated. Rather, empirical correlations [N. Zweibaum et al., 2015]. So for clarity, the empirical fLDK terms here already include the friction factor of the pipe.

For the CTAH, the following correlation is used [N. Zweibaum et al., 2015]:

$$fLDK = (f_{darcy} \frac{L}{D} + K) = 18 + \frac{93,000}{Re^{1.35}} \quad (A.1)$$

For all Coriolis Flowmeters, the following correlation is used [N. Zweibaum et al., 2015]:

$$fLDK = (f_{darcy} \frac{L}{D} + K) = 400 + \frac{52,000}{Re} \quad (A.2)$$

For all static mixers, the following correlation is used [N. Zweibaum et al., 2015]:

$$fLDK = (f_{darcy} \frac{L}{D} + K) = 21 + \frac{4000}{Re} \quad (A.3)$$

In all equations Re is:

$$Re = \frac{\dot{m} D_H}{A_{XS} \mu} \quad (A.4)$$

Appendix B

Source Code and Copyright Notices

Here is the source code used for this thesis.

B.1 Main Rust Interface Code

Here is the main Rust interface code used.

In this code, I made use of maturin PyO3 interface, as well as the units of measure (uom) crate.

Copyright Notices

As I used several packages within the code, I am obligated to ensure that the licenses are given whenever I present code which is based on these libraries.

PyO3 NOTICE

PyO3 has the following copyright notice:

Copyright (c) 2017-present PyO3 Project and Contributors. <https://github.com/PyO3>
Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file
except in compliance with the License. You may obtain a copy of the License at
<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the
License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
CONDITIONS OF ANY KIND, either express or implied. See the License for the specific
language governing permissions and limitations under the License.

Apache License Version 2.0, January 2004 <http://www.apache.org/licenses/>
TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as
defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50) outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

(a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License. You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions.

Notwithstanding the above, nothing herein shall supersede or modify the terms of any

separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

Maturin NOTICE

Maturin has the following copyright notice:

Apache License Version 2.0, January 2004 <http://www.apache.org/licenses/>
TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50) outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have

made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

- (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and
 - (b) You must cause any modified files to carry prominent notices stating that You changed the files; and
 - (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
 - (d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.
- You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for

reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

uom NOTICE

Units of Measure (uom) create is licensed under the MIT license or Apache License.

The MIT License (MIT)

Copyright (c) 2019 Mike Boutin

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Main Interface Code

```
// Note: //! indicates crate level documentation
//
//! A Library which calculates pressure losses to and from fluid
//! flowrates.
//!
//! For pipe flow, the library makes use of Churchill's Friction
//! Factor Correlation in order to calculate darcy or fanning
//! friction factor for laminar, turbulent and transitional flow.
#![warn(missing_docs)]
extern crate uom;
mod churchill_friction_factor;
mod custom_fldk;
mod dimensionalisation;
pub mod fluid_component_calculation;
pub mod therminol_component;

use uom::si::f64::*;
```

```

/// This function calculates darcy friction factor
#[allow(non_snake_case)]
pub fn darcy(ReynoldsNumber: f64, roughnessRatio: f64) -> f64 {
    return churchill_friction_factor::
        darcy(ReynoldsNumber, roughnessRatio);
}

#[allow(non_snake_case)]
pub fn moody(ReynoldsNumber: f64, roughnessRatio: f64) -> f64 {
    return churchill_friction_factor::
        moody(ReynoldsNumber, roughnessRatio);
}

#[allow(non_snake_case)]
pub fn fldk(ReynoldsNumber: f64,
            roughnessRatio: f64,
            lengthToDiameterRatio: f64,
            K: f64) -> f64{
    return churchill_friction_factor::
        fLDK(ReynoldsNumber,
            roughnessRatio,
            lengthToDiameterRatio,
            K);
}

#[allow(non_snake_case)]
pub fn get_bejan_d(ReynoldsNumber: f64,
                  roughnessRatio: f64,
                  lengthToDiameterRatio: f64,
                  K: f64) -> f64 {
    return churchill_friction_factor::
        getBe(ReynoldsNumber, roughnessRatio,
            lengthToDiameterRatio, K);
}

#[allow(non_snake_case)]
pub fn get_reynolds_number(Be_D: f64,
                           roughnessRatio: f64,
                           lengthToDiameter: f64,
                           formLossK: f64) -> f64 {
    return churchill_friction_factor::

```



```

    }

}

pub struct CalcReynolds {}
#[allow(non_snake_case)]
impl CalcReynolds {

    #[allow(non_snake_case)]
    pub fn from_mass_rate(fluidMassFlowrate: MassRate,
                        crossSectionalArea: Area,
                        hydraulic_diameter: Length,
                        fluidViscosity: DynamicViscosity) -> f64 {

        return dimensionalisation::CalcReynolds::from_mass_rate(
            fluidMassFlowrate,
            crossSectionalArea,
            hydraulic_diameter,
            fluidViscosity);
    }

    pub fn from_velocity(fluidDensity: MassDensity,
                        velocity: Velocity,
                        hydraulic_diameter: Length,
                        fluidViscosity: DynamicViscosity) -> f64 {

        return dimensionalisation::CalcReynolds::from_velocity(
            fluidDensity,
            velocity,
            hydraulic_diameter,
            fluidViscosity);
    }

}

#[allow(non_snake_case)]
pub fn to_mass_rate(crossSectionalArea: Area,
                    Re: f64,
                    hydraulicDiameter: Length,
                    fluidViscosity: DynamicViscosity) -> MassRate {

```

```

        return dimensionalisation::CalcReynolds::to_mass_rate(
            crossSectionalArea,
            Re,
            hydraulicDiameter,
            fluidViscosity);
    }
}

pub struct CalcBejan {}
impl CalcBejan {

    #[allow(non_snake_case)]
    pub fn from_pressure(fluidPressure: Pressure,
        hydraulicDiameter: Length,
        fluidDensity: MassDensity,
        fluidViscosity: DynamicViscosity) -> f64 {

        return dimensionalisation::CalcBejan::from_pressure(
            fluidPressure,
            hydraulicDiameter,
            fluidDensity,
            fluidViscosity);
    }

    #[allow(non_snake_case)]
    pub fn to_pressure(Be_D: f64,
        hydraulicDiameter: Length,
        fluidDensity: MassDensity,
        fluidViscosity: DynamicViscosity) -> Pressure {
        return dimensionalisation::CalcBejan::to_pressure(
            Be_D,
            hydraulicDiameter,
            fluidDensity,
            fluidViscosity);
    }
}
}

```

B.2 Friction Factor Code

We start with the nondimensionalised correlations in order To get nondimensionalised pressure loss from Re and vice versa.

The solver libraries used here come from the peroxide crate which is licensed under apache license.

Peroxide NOTICE

As I used several packages within the code, I am obligated to ensure that the licenses are given whenever I present code which is based on these libraries.

PyO3 NOTICE

PyO3 has the following copyright notice:

Copyright (c) 2017-present PyO3 Project and Contributors. <https://github.com/PyO3>
Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Apache License Version 2.0, January 2004 <http://www.apache.org/licenses/>
TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50) outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

(a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License. You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. **Limitation of Liability.** In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. **Accepting Warranty or Additional Liability.** While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

Maturin NOTICE

Maturin has the following copyright notice:

Apache License Version 2.0, January 2004 <http://www.apache.org/licenses/>

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50) outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution." "Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

- (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and
- (b) You must cause any modified files to carry prominent notices stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License. You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the

Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

uom NOTICE

Units of Measure (uom) create is licensed under the MIT license or Apache License.
The MIT License (MIT)

Copyright (c) 2019 Mike Boutin

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES

OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Churchill Friction Factor

```
extern crate peroxide;
use peroxide::prelude::*;

// here are the functions used for friction factor, rather messy but
// for fast prototyping and sandboxing don't really care too much
//

// first, to allow non snake case names..
#[allow(non_snake_case)]
fn B(Re: f64) -> f64 {
    let numerator = 37530.0_f64.powf(16.0);
    let denominator = Re.powf(16.0);
    return numerator/denominator;
}

#[allow(non_snake_case)]
fn A(Re: f64, roughnessRatio: f64) -> f64 {
    let seven_over_Re = 7.0/Re;
    let reynolds_term = seven_over_Re.powf(0.9);

    let roughness_term = 0.27 * roughnessRatio;

    let log_fraction = 1.0/(reynolds_term + roughness_term);
    // we will need natural logarithms:
    let inner_bracket_term = 2.457*log_fraction.ln();

    let A = inner_bracket_term.powf(16.0);

    return A;
}
```

```

}

#[allow(non_snake_case)]
fn churchillInnerTerm(Re: f64, roughnessRatio: f64) -> f64 {

    let eight_over_Re = 8.0/Re;
    let laminarTerm = eight_over_Re.powf(12.0);

    let Aterm = A(Re,roughnessRatio);
    let Bterm = B(Re);

    let APlusBInverse = 1.0/(Aterm+Bterm);
    let turbulentTerm = APlusBInverse.powf(3.0/2.0);

    return laminarTerm + turbulentTerm;
}

// this particular implementation uses the churchill correlation
#[allow(non_snake_case)]
fn fanning(ReynoldsNumber: f64, roughnessRatio: f64) -> f64{

    if ReynoldsNumber == 0.0 {
        panic!("Re = 0.0");
    }

    if ReynoldsNumber < 0.0 {
        panic!("Re<0.0");
    }

    if roughnessRatio < 0.0 {
        panic!("roughnessRatio<0.0");
    }

    let innerTerm = churchillInnerTerm(ReynoldsNumber, roughnessRatio);
    let powerTerm = innerTerm.powf(1.0/12.0);
    let fanningFrictionFactor = 2.0 * powerTerm;
    return fanningFrictionFactor;
}

#[allow(non_snake_case)]
pub fn darcy(ReynoldsNumber: f64, roughnessRatio: f64) -> f64 {
    return 4.0*fanning(ReynoldsNumber, roughnessRatio);
}

```

```

}

#[allow(non_snake_case)]
pub fn moody(ReynoldsNumber: f64, roughnessRatio: f64) -> f64 {
    return 4.0*fanning(ReynoldsNumber, roughnessRatio);
}

#[allow(non_snake_case)]
pub fn fLDK(ReynoldsNumber: f64,
            roughnessRatio: f64,
            lengthToDiameterRatio: f64,
            K: f64) -> f64{
    if ReynoldsNumber == 0.0 {
        panic!("Re = 0");
    }

    if ReynoldsNumber < 0.0 {
        panic!("Re < 0");
    }

    if roughnessRatio < 0.0 {
        panic!("roughnessRatio<0.0");
    }

    if lengthToDiameterRatio <= 0.0 {
        panic!("lengthToDiameterRatio<=0.0");
    }

    if K < 0.0 {
        panic!("For m loss coefficient K < 0.0");
    }

    let f = darcy(ReynoldsNumber, roughnessRatio);
    let fLDK = f*lengthToDiameterRatio + K;

    return fLDK;
}

#[allow(non_snake_case)]
pub fn getBe(mut ReynoldsNumber: f64,

```

```

        roughnessRatio: f64,
        lengthToDiameterRatio: f64,
        K: f64) -> f64{

    if ReynoldsNumber == 0.0 {
        return 0.0;
    }

    let mut isNegative = false;

    if ReynoldsNumber < 0.0 {
        isNegative = true;
        ReynoldsNumber = ReynoldsNumber * -1.0;
    }

    if roughnessRatio < 0.0 {
        panic!("roughnessRatio<0.0");
    }

    if lengthToDiameterRatio <= 0.0 {
        panic!("lengthToDiameterRatio<=0.0");
    }

    if K < 0.0 {
        panic!("Form loss coefficient K < 0.0");
    }

    let f = darcy(ReynoldsNumber, roughnessRatio);

    let fLDK = f*lengthToDiameterRatio + K;

    let mut Be = 0.5*fLDK*ReynoldsNumber.powf(2.0);

    if isNegative {
        Be = Be * -1.0;
        return Be;
    }

    return Be;
}

#[allow(non_snake_case)]

```

```

pub fn getRe(mut Be_D: f64,
             roughnessRatio: f64,
             lengthToDiameter: f64,
             formLossK: f64) -> f64 {

    if lengthToDiameter <= 0.0 {
        panic!("lengthToDiameterRatio<=0.0");
    }

    if roughnessRatio < 0.0 {
        panic!("roughnessRatio<0.0");
    }

    if formLossK < 0.0 {
        panic!("formLossK<0.0");
    }

    // this part deals with negative Be_L values
    // invalid Be_L values
    let mut isNegative = false;
    if Be_D < 0.0 {
        Be_D = Be_D * -1.0;
        isNegative = true;
    }

    let maxRe = 1.0e12;

    // i calculate the Be_D corresponding to
    // Re = 1e12
    let maxBe_D = getBe(maxRe, roughnessRatio,
                       lengthToDiameter, formLossK);

    if Be_D >= maxBe_D {
        panic!("Be too large");
    }

    // the above checks for all the relevant exceptions
    // including formLossK < 0
    //
    // now we are ready to do root finding
    //
    // the underlying equation is
    // Be = 0.5*fLDK*Re^2

```

```

let pressureDropRoot = |Re: AD| -> AD {
  // i'm solving for
  //  $Be - 0.5 * fLDK * Re^2 = 0$ 
  // the fLDK term can be calculated using
  // getBe
  //
  // now i don't really need the interpolation
  // term in here because when  $Re = 0$ ,
  //  $Be = 0$  in the getBe code.
  // so really, no need for fancy interpolation.
  //
  // Now in peroxide, the type taken in and out
  // is not a f64 double
  // but rather AD which stands for automatic
  // differentiation
  // https://docs.rs/peroxide/latest/peroxide/structure/ad/index.html

  let reynoldsDouble = Re.x();
  let fLDKterm = getBe(reynoldsDouble, roughnessRatio,
                      lengthToDiameter,
                      formLossK);

  return ADO(Be_D - fLDKterm);
};

let ReynoldsNumberResult = bisection(pressureDropRoot,
                                     (0.0, maxRe),
                                     100,
                                     1e-8);

// the unwrap turns the result into f64
let mut ReynoldsNumber = ReynoldsNumberResult.unwrap();

if isNegative
{
  ReynoldsNumber = ReynoldsNumber * -1.0;
}

```



```

        return ReynoldsNumber;
    }

    return ReynoldsNumber;
}

```

fldk code

```

extern crate peroxide;
use peroxide::prelude::*;

// here are the functions used for components with
// custom friction factor and K, rather messy but

// this first function allows for custom fldk,
// ie both friction factor and form loss k are user defined
// https://stackoverflow.com/questions/36390665/how-do-you-pass-a-rust-
//  function-as-a-parameter
#[allow(non_snake_case)]
pub fn custom_fLDK(customDarcy: &dyn Fn(f64, f64) -> f64,
    ReynoldsNumber: f64,
    roughnessRatio: f64,
    lengthToDiameterRatio: f64,
    customK: &dyn Fn(f64) -> f64) -> f64{

    if roughnessRatio < 0.0 {
        panic!("roughnessRatio<0.0");
    }

    if lengthToDiameterRatio <= 0.0 {
        panic!("lengthToDiameterRatio<=0.0");
    }

    let K = customK(ReynoldsNumber);

    let f = customDarcy(ReynoldsNumber, roughnessRatio);
    let fLDK = f*lengthToDiameterRatio + K;

```

```

    return fLDK;
}

// this is a special case of the fLDK component,
// where we just specify a custom K but friction factor is based
// on darcy friction factor
#[allow(non_snake_case)]
pub fn custom_Kpipe(ReynoldsNumber: f64,
                    roughnessRatio: f64,
                    lengthToDiameterRatio: f64,
                    customK: &dyn Fn(f64) -> f64) -> f64{

    let darcyFn = crate::churchill_friction_factor::darcy;

    let fLDK = custom_fLDK(&darcyFn,
                          ReynoldsNumber,
                          roughnessRatio,
                          lengthToDiameterRatio,
                          customK);

    return fLDK;
}

#[allow(non_snake_case)]
pub fn custom_Kpipe_Be_D(ReynoldsNumber: f64,
                        roughnessRatio: f64,
                        lengthToDiameterRatio: f64,
                        customK: &dyn Fn(f64) -> f64) -> f64{

    if ReynoldsNumber == 0.0 {
        return 0.0;
    }

    let fLDK = custom_Kpipe(ReynoldsNumber,
                          roughnessRatio,
                          lengthToDiameterRatio,
                          customK);

    let Be_D = 0.5*fLDK*ReynoldsNumber.powf(2.0);

```

```

    return Be_D;
}

#[allow(non_snake_case)]
pub fn custom_fLDK_Be_D(customDarcy: &dyn Fn(f64, f64) -> f64,
                        ReynoldsNumber: f64,
                        roughnessRatio: f64,
                        lengthToDiameterRatio: f64,
                        customK: &dyn Fn(f64) -> f64) -> f64{

    if ReynoldsNumber == 0.0 {
        return 0.0;
    }

    let fLDK = custom_fLDK(customDarcy,
                           ReynoldsNumber,
                           roughnessRatio,
                           lengthToDiameterRatio,
                           customK);

    let Be_D = 0.5*fLDK*ReynoldsNumber.powf(2.0);

    return Be_D;
}

// this code allos us to get Reynold's number from a Bejan
// number for a custom pipe.
// i make no assumptions about the symmetry of flow
// ie. i don't make assumptions about whether
// the pipe exhibits the same pressure loss
// in forwards and backwards flow,
// that is up to the user to decide when
// customDarcy and customK is put in
#[allow(non_snake_case)]
pub fn getRe(customDarcy: &dyn Fn(f64, f64) -> f64,
             Be_D: f64,
             roughnessRatio: f64,
             lengthToDiameter: f64,
             customK: &dyn Fn(f64) -> f64) -> f64 {

```

```

if lengthToDiameter <= 0.0 {
    panic!("lengthToDiameterRatio<=0.0");
}

if roughnessRatio < 0.0 {
    panic!("roughnessRatio<0.0");
}

// this part deals with negative Be_L values
// invalid Be_L values

let maxRe = 1.0e12;

// i calculate the Be_D corresponding to
// Re = 1e12
let maxBe_D = custom_fLDK_Be_D(
    customDarcy,
    maxRe,
    roughnessRatio,
    lengthToDiameter,
    customK);

if Be_D >= maxBe_D {
    panic!("Be too large");
}

// the above checks for all the relevant exceptions
// including formLossK < 0
//
// now we are ready to do root finding
//
// the underlying equation is
//  $Be = 0.5 * fLDK * Re^2$ 

let pressureDropRoot = |Re: AD| -> AD {
    // i'm solving for
    //  $Be - 0.5 * fLDK * Re^2 = 0$ 
    // the fLDK term can be calculated using
    // getBe
    //

```

```

// now i don't really need the interpolation
// term in here because when Re = 0,
// Be = 0 in the getBe code.
// so really, no need for fancy interpolation.
//
// Now in peroxide, the type taken in and out
// is not a f64 double
// but rather AD which stands for automatic
// differentiation
// https://docs.rs/peroxide/latest/peroxide/structure/ad/index.html

let reynoldsDouble = Re.x();
let fLDKterm = custom_fLDK_Be_D(
    customDarcy,
    reynoldsDouble,
    roughnessRatio,
    lengthToDiameter,
    customK);

return ADO(Be_D - fLDKterm);

};

let ReynoldsNumberResult = bisection(pressureDropRoot,
                                     (0.0,maxRe),
                                     100,
                                     1e-8);

// the unwrap turns the result into f64
let ReynoldsNumber = ReynoldsNumberResult.unwrap();

return ReynoldsNumber;
}

```

B.3 Pipe Calculation Code

Dimensionalisation

```

extern crate peroxide;
extern crate uom;

use uom::si::f64::*;
use uom::typenum::P2;

// uom stands for unit of measure.
//
// this set of functions here is simply to convert to and from
// dimensionless numbers to SI units, and back

// first and most important function here,
// this function converts a quantity into float given the correct dimensions
// if it's not dimensionless, it will throw a compile time error
pub fn convert_dimensionless_number_to_float(dimensionless_number: Ratio) ->
    ↪ f64 {
    return dimensionless_number.value.into();
}

pub struct CalcReynolds {}
impl CalcReynolds {
    #[allow(non_snake_case)]
    pub fn from_velocity(fluidDensity: MassDensity,
                        velocity: Velocity,
                        hydraulic_diameter: Length,
                        fluidViscosity: DynamicViscosity) -> f64 {

        if fluidViscosity.value <= 0.0 {
            panic!("fluid Viscosity <= 0.0, nonphysical");
        }

        if hydraulic_diameter.value <= 0.0 {
            panic!("hydraulic Diameter <= 0.0, nonphysical");
        }

        if fluidDensity.value <= 0.0 {
            panic!("fluidDensity <= 0.0, nonphysical");
        }

        let reynolds_number =
            fluidDensity *
            velocity *

```

```

        hydraulic_diameter /
        fluidViscosity;

    return convert_dimensionless_number_to_float(reynolds_number);
}

#[allow(non_snake_case)]
pub fn from_mass_rate(fluidMassFlowrate: MassRate,
                      crossSectionalArea: Area,
                      hydraulic_diameter: Length,
                      fluidViscosity: DynamicViscosity) -> f64 {

    if fluidViscosity.value <= 0.0 {
        panic!("fluid Viscosity <= 0.0, nonphysical");
    }

    if hydraulic_diameter.value <= 0.0 {
        panic!("hydraulic Diameter <= 0.0, nonphysical");
    }

    if crossSectionalArea.value <= 0.0 {
        panic!("pipe Area <= 0.0, nonphysical");
    }

    let reynolds_number = fluidMassFlowrate/
        crossSectionalArea*
        hydraulic_diameter/
        fluidViscosity;

    return convert_dimensionless_number_to_float(reynolds_number);
}

#[allow(non_snake_case)]
pub fn to_mass_rate(crossSectionalArea: Area,
                    Re: f64,
                    hydraulicDiameter: Length,
                    fluidViscosity: DynamicViscosity) -> MassRate {

```

```

    if fluidViscosity.value <= 0.0 {
        panic!("fluid Viscosity <= 0.0, nonphysical");
    }

    if hydraulicDiameter.value <= 0.0 {
        panic!("hydraulic Diameter <= 0.0, nonphysical");
    }

    if crossSectionalArea.value <= 0.0 {
        panic!("pipe Area <= 0.0, nonphysical");
    }

    let fluidMassFlowrate = fluidViscosity*
        crossSectionalArea/
        hydraulicDiameter*
        Re;

    return fluidMassFlowrate;
}
}

pub struct CalcBejan {}
impl CalcBejan {

    #[allow(non_snake_case)]
    pub fn from_pressure(fluidPressure: Pressure,
        hydraulicDiameter: Length,
        fluidDensity: MassDensity,
        fluidViscosity: DynamicViscosity) -> f64 {

        if fluidViscosity.value <= 0.0 {
            panic!("fluid Viscosity <= 0.0, nonphysical");
        }

        if hydraulicDiameter.value <= 0.0 {
            panic!("hydraulic Diameter <= 0.0, nonphysical");
        }

        if fluidDensity.value <= 0.0 {
            panic!("fluidDensity <= 0.0, nonphysical");
        }
    }
}

```



```

    }

    let Be = fluidPressure*
        fluidDensity *
        hydraulicDiameter.powi(P2::new())/
        fluidViscosity.powi(P2::new());

    return convert_dimensionless_number_to_float(Be);
}

#[allow(non_snake_case)]
pub fn to_pressure(Be_D: f64,
                    hydraulicDiameter: Length,
                    fluidDensity: MassDensity,
                    fluidViscosity: DynamicViscosity) -> Pressure {

    if fluidViscosity.value <= 0.0 {
        panic!("fluid Viscosity <= 0.0, nonphysical");
    }

    if hydraulicDiameter.value <= 0.0 {
        panic!("hydraulic Diameter <= 0.0, nonphysical");
    }

    if fluidDensity.value <= 0.0 {
        panic!("fluidDensity <= 0.0, nonphysical");
    }

    let fluidPressure = fluidViscosity.powi(P2::new())*
        Be_D/
        hydraulicDiameter.powi(P2::new())/
        fluidDensity;

    return fluidPressure;
}
}

```

Calculation for Standard Pipes

```
pub mod standard_pipe_calc;
pub mod custom_component_calc;

// using crate brings in the lib
// since i have to manually import files from above directories
use crate::churchill_friction_factor;
use crate::dimensionalisation;

use uom::si::f64::*;
pub struct CalcPressureLoss {}
impl CalcPressureLoss {
    // this calculates pressure loss in a pipe from mass flowrate
    #[allow(non_snake_case)]
    pub fn from_mass_rate(mut fluidMassFlowrate: MassRate,
                          crossSectionalArea: Area,
                          hydraulicDiameter: Length,
                          fluidViscosity: DynamicViscosity,
                          fluidDensity: MassDensity,
                          pipeLength: Length,
                          absolute_roughness: Length,
                          formLossK: f64) -> Pressure {
        // first let's calculate roughness ratio

        let roughnessRatioQuantity = absolute_roughness/hydraulicDiameter;

        let roughnessRatio =
            dimensionalisation::convert_dimensionless_number_to_float(
                roughnessRatioQuantity);

        // second i want to take care of reverse flow

        let mut reverseFlow = false;
        if fluidMassFlowrate.value < 0.0 {
            reverseFlow = true;
        }

        if reverseFlow {
            fluidMassFlowrate = fluidMassFlowrate * -1.0;
        }
    }
}
```

```

// and let's get the Re and L/D
let Re = dimensionalisation::CalcReynolds::from_mass_rate(
    fluidMassFlowrate,
    crossSectionalArea,
    hydraulicDiameter,
    fluidViscosity);

let lengthToDiameterRatio
    = dimensionalisation::convert_dimensionless_number_to_float(
        pipeLength/hydraulicDiameter);

// then let's obtain the pipe Bejan Number
// given the Re

let Be = churchill_friction_factor::getBe(
    Re,
    roughnessRatio,
    lengthToDiameterRatio,
    formLossK);

// once we get Be, we can get the pressure loss terms
//
let pressureLoss = dimensionalisation::CalcBejan::to_pressure(
    Be,
    hydraulicDiameter,
    fluidDensity,
    fluidViscosity);

// now before i exit, i want to make sure reverse flow is taken care
// of
if reverseFlow {
    return pressureLoss * -1.0;
}

return pressureLoss;
}

// this calculates mass flowrate in a pipe given pressure losses
#[allow(non_snake_case)]
pub fn to_mass_rate(pressureLoss: Pressure,
                    crossSectionalArea: Area,

```

```

        hydraulicDiameter: Length,
        fluidViscosity: DynamicViscosity,
        fluidDensity: MassDensity,
        pipeLength: Length,
        absolute_roughness: Length,
        formLossK: f64) -> MassRate {

// first let's get our relevant ratios:
let roughnessRatioQuantity = absolute_roughness/hydraulicDiameter;

let roughnessRatio =
    dimensionalisation::convert_dimensionless_number_to_float(
        roughnessRatioQuantity);

let lengthToDiameterRatio
    = dimensionalisation::convert_dimensionless_number_to_float(
        pipeLength/hydraulicDiameter);

// then get Bejan number:

let Be_D = dimensionalisation::CalcBejan::from_pressure(
    pressureLoss, hydraulicDiameter,
    fluidDensity, fluidViscosity);

// let's get Re
let Re_D = churchill_friction_factor::getRe(Be_D,
                                            roughnessRatio,
                                            lengthToDiameterRatio,
                                            formLossK);

// and finally return mass flowrate
//
let fluidMassFlowrate =
    dimensionalisation::CalcReynolds::to_mass_rate(crossSectionalArea
    ↪ ,
                                                    Re_D,
                                                    hydraulicDiameter,
                                                    fluidViscosity);

return fluidMassFlowrate;

```

```
}
}
```

Calculation for Custom Components

```
// using crate brings in the lib
// since i have to manually import files from above directories
use crate::custom_fldk;
use crate::dimensionalisation;

use uom::si::f64::*;
pub struct CalcPressureLoss {}
impl CalcPressureLoss {
    // this calculates pressure loss in a pipe from mass flowrate
    #[allow(non_snake_case)]
    pub fn from_mass_rate(fluidMassFlowrate: MassRate,
                          crossSectionalArea: Area,
                          hydraulicDiameter: Length,
                          fluidViscosity: DynamicViscosity,
                          fluidDensity: MassDensity,
                          pipeLength: Length,
                          absolute_roughness: Length,
                          customDarcy: &dyn Fn(f64, f64) -> f64,
                          customK: &dyn Fn(f64) -> f64) -> Pressure {

        // first we get our Reynolds number

        let ReQuantity = fluidMassFlowrate/
            crossSectionalArea*
            hydraulicDiameter/
            fluidViscosity;

        let Re = dimensionalisation::convert_dimensionless_number_to_float(
            ReQuantity);

        // second we get the darcy factor and custom K
        // note that reverse flow logic should be taken care of in
        // user supplied darcy factor and K, not here

        let roughnessRatioQuantity = absolute_roughness/hydraulicDiameter;
        let roughnessRatio =
```

```

        dimensionalisation::convert_dimensionless_number_to_float(
            roughnessRatioQuantity);

let lengthToDiameterQuantity =
    pipeLength/
    hydraulicDiameter;

let lengthToDiameter =
    dimensionalisation::convert_dimensionless_number_to_float(
        lengthToDiameterQuantity);

// now we have this, we can calculate bejan number

let Be_D = custom_fldk::custom_fLDK_Be_D(
    customDarcy,
    Re,
    roughnessRatio,
    lengthToDiameter,
    customK);

// once we get Be, we can get the pressure loss terms
//
let pressureLoss = dimensionalisation::CalcBejan::to_pressure(
    Be_D,
    hydraulicDiameter,
    fluidDensity,
    fluidViscosity);

return pressureLoss;
}

#[allow(non_snake_case)]
pub fn to_mass_rate(pressureLoss: Pressure,
                    crossSectionalArea: Area,
                    hydraulicDiameter: Length,
                    fluidViscosity: DynamicViscosity,
                    fluidDensity: MassDensity,
                    pipeLength: Length,

```

```

        absolute_roughness: Length,
        customDarcy: &dyn Fn(f64, f64) -> f64,
        customK: &dyn Fn(f64) -> f64 -> MassRate {

// first let's get our relevant ratios:
let roughnessRatioQuantity = absolute_roughness/hydraulicDiameter;

let roughnessRatio =
    dimensionalisation::convert_dimensionless_number_to_float(
        roughnessRatioQuantity);

let lengthToDiameterRatio
    = dimensionalisation::convert_dimensionless_number_to_float(
        pipeLength/hydraulicDiameter);

// then get Bejan number:
let Be_D = dimensionalisation::CalcBejan::from_pressure(
    pressureLoss, hydraulicDiameter,
    fluidDensity, fluidViscosity);

// let's get Re
let Re_D = custom_fldk::getRe(customDarcy,
                                Be_D,
                                roughnessRatio,
                                lengthToDiameterRatio,
                                customK);

// and finally return mass flowrate
//
let fluidMassFlowrate =
    dimensionalisation::CalcReynolds::to_mass_rate(crossSectionalArea
    ↪ ,
                                                    Re_D,
                                                    hydraulicDiameter,
                                                    fluidViscosity);

    return fluidMassFlowrate;
}
}

```

B.4 CIET Specific Code

CIET uses a mixture of therminol VP-1 and Dowtherm A oils for operation. The class of pipes used here is therminol pipes. But I ensure that Dowtherm A thermophysical properties are used in this code.

Therminol Pipe Generics

```
pub mod custom_therminol_component;
pub mod dowtherm_a_properties;
pub mod therminol_pipe;
pub mod factory;

// this allows for units
use uom::si::f64::*;

/// A generic base class or struct for therminol vp 1
/// or dowtherm A pipe like components
// i want to create an interface for a generic
// component, which could be a pipe or a custom component
pub struct PipeProperties {
    pub _name: String,
    pub hydraulic_diameter: Length,
    pub component_length: Length,
    pub absolute_roughness: Length,

    // also we should have incline angle in degrees
    // in case of dealing with elevation
    pub incline_angle: Angle,

    // for pipes, we specify a fixed form loss k term
    // not a custom one
    pub form_loss_k: f64,

    // internal pressure term, in case you want to have
    // a pump or something

    pub internal_pressure: Pressure,
}

/// A generic base class for therminol or dowtherm A
```



```

/// components with user defined fldk terms
pub struct CustomComponentProperties{
    pub _name: String,
    pub hydraulic_diameter: Length,
    pub xs_area: Area,
    pub component_length: Length,
    pub absolute_roughness: Length,

    // also we should have incline angle in degrees
    // in case of dealing with elevation
    pub incline_angle: Angle,

    // for custom fldk component, i have a custom
    // form loss term which is essentially a function
    // i put it as static because i want it to live through the
    // duration of the program
    pub custom_k: &'static dyn Fn(f64) -> f64,

    // the 'a thing i'm not really sure what it is
    // but the compiler says it is an explicit lifetime thingy
    // i'll just follow suit
    // it's supposed to be on stack memory (local variables)
    // rather than heap memory (global variables)

    pub custom_darcy: &'static dyn Fn(f64,f64) -> f64,

    // internal pressure term, in case you want to have
    // a pump or something

    pub internal_pressure: Pressure,
}

/// here is a trait (or interface for C# or java people),
/// however, traits only deal with methods, not properties.
/// note that traits are also types
/// A trait (or interface) for getting fluid properties from
/// temperature or enthalpy
pub trait FluidProperties {
    fn density(fluid_temp: ThermodynamicTemperature) -> MassDensity;

```

```

fn viscosity(fluid_temp: ThermodynamicTemperature) -> DynamicViscosity;

fn enthalpy(fluid_temp: ThermodynamicTemperature) -> AvailableEnergy;

fn specific_heat_capacity(
    fluid_temp: ThermodynamicTemperature) -> SpecificHeatCapacity;
fn thermal_conductivity(
    fluid_temp: ThermodynamicTemperature) -> ThermalConductivity;

fn get_temperature_from_enthalpy(
    fluid_enthalpy: AvailableEnergy) -> ThermodynamicTemperature;
}

/// A trait (or interface) for getting pressure change (not loss)
/// from mass flowrate and vice versa
/// it inherits from Fluid Properties as Reynolds number always needs
/// to be calculated
pub trait CalcPressureChange {
    fn from_mass_rate(&self, fluid_mass_flowrate: MassRate,
        fluid_temp: ThermodynamicTemperature) -> Pressure;

    fn to_mass_rate(&self, pressure_change: Pressure,
        fluid_temp: ThermodynamicTemperature) -> MassRate;
}

/// A trait (or interface) for getting pipe form losses and cross
/// sectional areas
/// i also force the implementation of a constructor
/// it inherits from CalcPressureChange
/// because it will need to calculate pressure changes
/// from the above properties,
/// also, constructor will need to be given
pub trait StandardPipeProperties : FluidProperties {
    fn new(name: String,
        hydraulic_diameter_meters: f64,
        component_length_meters: f64,
        absolute_roughness_millimeters: f64,
        incline_angle_degrees: f64,
        form_loss_k: f64) -> Self;

    fn get_cross_sectional_area(&self) -> Area;
    fn get_internal_pressure_term(&self) -> Pressure;

```

```

    fn set_internal_pressure_term(&mut self, pressure_pascals: f64);
    fn get_hydrostatic_pressure_change(
        &self, fluid_temp: ThermodynamicTemperature) -> Pressure;
}

pub trait StandardCustomComponentProperties : FluidProperties {
    fn new(name: String,
        hydraulic_diameter_meters: f64,
        cross_sectional_area_meters_sq: f64,
        component_length_meters: f64,
        absolute_roughness_millimeters: f64,
        incline_angle_degrees: f64,
        custom_darcy: &'static dyn Fn(f64,f64) -> f64,
        custom_k: &'static dyn Fn(f64) -> f64) -> Self;

    fn get_cross_sectional_area(&self) -> Area;
    fn get_internal_pressure_term(&self) -> Pressure;
    fn set_internal_pressure_term(&mut self, pressure_pascals: f64);
    fn get_hydrostatic_pressure_change(
        &self, fluid_temp: ThermodynamicTemperature) -> Pressure;
}

```

Dowtherm A Properties

```

use uom::si::f64::*;
use uom::si::thermodynamic_temperature::degree_celsius;
use uom::si::mass_density::kilogram_per_cubic_meter;
use uom::si::dynamic_viscosity::pascal_second;
use uom::si::thermal_conductivity::watt_per_meter_kelvin;
use uom::si::specific_heat_capacity::joule_per_kilogram_kelvin;
use uom::si::available_energy::joule_per_kilogram;

// this is for the root finding algorithms
extern crate peroxide;
use peroxide::prelude::*;

#[allow(non_snake_case)]
pub fn getDowthermADensity(
    fluidTemp: ThermodynamicTemperature) -> MassDensity {

```

```

// first we check if fluid temp is between 20-180C (range of validity)
// panic otherwise
rangeCheck(fluidTemp);

//then convert the fluidTemp object into a f64
// and plug it into the correlation
let densityValueKgPerM3 = 1078.0 - 0.85*fluidTemp
    .get::<degree_celsius>();

return MassDensity::new::<kilogram_per_cubic_meter>(densityValueKgPerM3)
    ↪ ;
}

#[allow(non_snake_case)]
pub fn getDowthermAViscosity(
    fluidTemp: ThermodynamicTemperature) -> DynamicViscosity{

    rangeCheck(fluidTemp);
    let temperatureDegreesCValue = fluidTemp.get::<degree_celsius>();
    let viscosityValuePascalSecond = 0.130/
        temperatureDegreesCValue.powf(1.072);

    return DynamicViscosity::new::<pascal_second>(viscosityValuePascalSecond
        ↪ );
}

#[allow(non_snake_case)]
pub fn getDowthermAConstantPressureSpecificHeatCapacity(
    fluidTemp: ThermodynamicTemperature) -> SpecificHeatCapacity{

    rangeCheck(fluidTemp);
    // note, specific entropy and heat capacity are the same unit...
    //
    let cp_value_joule_per_kg = 1518.0 + 2.82*fluidTemp.get::<degree_celsius>
        ↪ >();

    return SpecificHeatCapacity::new::<joule_per_kilogram_kelvin>(
        cp_value_joule_per_kg);
}

```

```

#[allow(non_snake_case)]
pub fn getDowthermAThermalConductivity(
    fluidTemp: ThermodynamicTemperature) -> ThermalConductivity {

    rangeCheck(fluidTemp);
    let thermalConductivityValue = 0.142 - 0.00016* fluidTemp
        .get::<degree_celsius>();

    return ThermalConductivity::new::<watt_per_meter_kelvin>(
        thermalConductivityValue);
}

// i also have an analytically integrated function for enthalpy of
// dowtherm A
// the thing is that with enthalpy
// we need a reference value
// i take mine to be 0 enthalpy at 20C
// integrating heat capacity with respect to T, we get
//
//  $cp = 1518 + 2.82 \cdot T$ 
//
//  $H = 1518 \cdot T + \frac{2.82}{2.0} \cdot T^2 + C$ 
// at  $T = 20C$ ,
//  $H = 30924 + C$ 
//  $H = 0$ 
//  $C = -30924$  (i used libre office to calculate this)
#[allow(non_snake_case)]
pub fn getDowtherMAEnthalpy(
    fluidTemp: ThermodynamicTemperature) -> AvailableEnergy{

    rangeCheck(fluidTemp);
    // note, specific entropy and heat capacity are the same unit...
    //
    //  $H = 1518 \cdot T + \frac{2.82}{2.0} \cdot T^2 - 30924$ 
    let tempCValue = fluidTemp.get::<degree_celsius>();
    let enthalpy_value_joule_per_kg
        = 1518.0 * tempCValue
        + 2.82/2.0 * tempCValue.powf(2.0) -
        30924.0;

    // the closest unit available is AvailableEnergy which is

```

```

    // joule per kg

    return AvailableEnergy::new::<joule_per_kilogram>(
        enthalpy_value_joule_per_kg);
}

// this functions enables us to get temperature from enthalpy using
// a root finding method
pub fn get_temperature_from_enthalpy(
    fluid_enthalpy: AvailableEnergy) -> ThermodynamicTemperature {

    // first let's convert enthalpy to a double (f64)
    let enthalpy_value_joule_per_kg =
        fluid_enthalpy.get::<joule_per_kilogram>();

    // second let's define a function
    // or actually a closure or anonymous function that
    // is aware of the variables declared
    // enthalpy value = 1518*T +2.82/2.0 T^2 - 30924
    // LHS is actual enthalpy value

    let enthalpy_root = |temp_degrees_c_value : AD| -> AD {
        let lhs_value = enthalpy_value_joule_per_kg;
        // convert AD type into double
        let temp_degrees_c_value_double = temp_degrees_c_value.x();

        let fluid_temperature =
            ThermodynamicTemperature::new::<degree_celsius>(
                temp_degrees_c_value_double);
        let rhs = getDowthermAEnthalpy(fluid_temperature);
        let rhs_value = rhs.get::<joule_per_kilogram>();

        return ADO(lhs_value-rhs_value);
    };

    // now solve using bisection

    let fluid_temperature_degrees_cresult
        = bisection(enthalpy_root,
            (20.0,180.0),
            100,
            1e-8);

```

```

let fluid_temperature_degrees_c = fluid_temperature_degrees_cresult.
    ↪ unwrap();

return ThermodynamicTemperature::
    new::<degree_celsius>(fluid_temperature_degrees_c);
}

// function checks if a fluid temperature falls in a range (20-180C)
// it is assumed that temperature here is in degrees C
// to avoid units, use the overload above.
#[allow(non_snake_case)]
pub fn rangeCheck(fluidTemp: ThermodynamicTemperature) -> bool{

    // first i convert the fluidTemp object into a degree
    // celsius
    let tempvalueCelsius =
        fluidTemp.get::<degree_celsius>();

    if tempvalueCelsius < 20.0 {
        let errorMsg = "Your fluid temperature \n";
        let errorMsg1 = "is too low :";
        let errorMsg3 = "C \n";
        let errorMsg4 = "\n the minimum is 20C";

        panic!("{}",
            errorMsg,
            errorMsg1,
            fluidTemp,
            errorMsg3,
            errorMsg4);
    }

    if tempvalueCelsius > 180.0 {
        let errorMsg = "Your fluid temperature \n";
        let errorMsg1 = "is too high :";
        let errorMsg3 = "C \n";
        let errorMsg4 = "\n the max is 180C";
    }
}

```

```

        panic!("{}",{:?}",{:?}},
            errorMsg,
            errorMsg1,
            fluidTemp,
            errorMsg3,
            errorMsg4);
    }

    return true;
}

```

Therminol Pipe Parent Class or Struct

All therminol pipes inherit from this base struct or class

```

use crate::fluid_component_calculation::standard_pipe_calc;
use crate::therminol_component::*;
use dowtherm_a_properties;

use uom::si::length::{meter, millimeter};
use uom::si::pressure::pascal;
use uom::si::angle::degree;
use uom::si::acceleration::meter_per_second_squared;
use uom::typenum::P2;
// now i will have dowtherm A pipe
// this is simliar in composition to therminol VP 1 so
// i class them as the same
pub struct DowthermAPipe {
    pub dowtherm_pipe_properties: PipeProperties,
}

impl FluidProperties for DowthermAPipe {
    fn density(fluid_temp: ThermodynamicTemperature) -> MassDensity {
        return dowtherm_a_properties::getDowthermADensity(fluid_temp);
    }

    fn viscosity(
        fluid_temp: ThermodynamicTemperature) -> DynamicViscosity{
        return dowtherm_a_properties::getDowthermAViscosity(fluid_temp);
    }
}

```



```

fn enthalpy(fluid_temp: ThermodynamicTemperature) -> AvailableEnergy{
    return dowtherm_a_properties::getDowthermAEnthalpy(fluid_temp);
}

fn specific_heat_capacity(
    fluid_temp: ThermodynamicTemperature) -> SpecificHeatCapacity{
    return dowtherm_a_properties::
        getDowthermAConstantPressureSpecificHeatCapacity(
            fluid_temp);
}

fn thermal_conductivity(
    fluid_temp: ThermodynamicTemperature) -> ThermalConductivity{
    return dowtherm_a_properties::
        getDowtherMAThermalConductivity(fluid_temp);
}

fn get_temperature_from_enthalpy(
    fluid_enthalpy: AvailableEnergy) -> ThermodynamicTemperature{
    return dowtherm_a_properties::
        get_temperature_from_enthalpy(fluid_enthalpy);
}
}

impl StandardPipeProperties for DowthermAPipe {
    // constructor
    fn new(name: String,
        hydraulic_diameter_meters: f64,
        component_length_meters: f64,
        absolute_roughness_millimeters: f64,
        incline_angle_degrees: f64,
        form_loss_k: f64) -> Self{

        let calculated_hydraulic_diameter = Length::new::<meter>(
            hydraulic_diameter_meters);
        let calculated_component_length = Length::new::<meter>(
            component_length_meters);
        let calculated_absolute_roughness = Length::new::<millimeter>(
            absolute_roughness_millimeters);
        let calculated_incline_angle = Angle::new::<degree>(

```

```

        incline_angle_degrees);
let calculated_internal_pressure = Pressure::new::<pascal>(
    0.0);

let custom_pipe_properties = PipeProperties {
    _name: name,
    hydraulic_diameter: calculated_hydraulic_diameter,
    component_length: calculated_component_length,
    absolute_roughness: calculated_absolute_roughness,
    incline_angle: calculated_incline_angle,
    form_loss_k: form_loss_k,
    internal_pressure: calculated_internal_pressure,
};

return Self { dowtherm_pipe_properties : custom_pipe_properties };
}

fn get_cross_sectional_area(&self) -> Area {
    let pipe_diameter = self.dowtherm_pipe_properties.hydraulic_diameter
    ↪ ;
    let pipe_xs_area =
        pipe_diameter.powi(P2::new())*
        std::f64::consts::PI/
        4.0;

    return pipe_xs_area;
}

fn get_internal_pressure_term(&self) -> Pressure {
    return self.dowtherm_pipe_properties.internal_pressure;
}

fn set_internal_pressure_term(&mut self, pressure_pascals: f64) {
    self.dowtherm_pipe_properties.internal_pressure =
        Pressure::new::<pascal>(pressure_pascals);
}

fn get_hydrostatic_pressure_change(
    &self, fluid_temp: ThermodynamicTemperature) -> Pressure {

```

```

    let pipe_length = self.dowtherm_pipe_properties.component_length;
    let incline_angle = self.dowtherm_pipe_properties.incline_angle;
    let fluid_density = DowthermAPipe::density(fluid_temp);

    let g: Acceleration =
        Acceleration::new::<meter_per_second_squared>(-9.81);
    let delta_h: Length = pipe_length*incline_angle.sin();

    let hydrostatic_pressure_increase: Pressure =
        fluid_density * g * delta_h;

    return hydrostatic_pressure_increase;
}
}

impl CalcPressureChange for DowthermAPipe {

    fn from_mass_rate(&self, fluid_mass_flowrate: MassRate,
                      fluid_temp: ThermodynamicTemperature) -> Pressure {

        // first let's get all the relevant properties...
        let hydraulic_diameter = self.dowtherm_pipe_properties.
            ↪ hydraulic_diameter;
        let pipe_length = self.dowtherm_pipe_properties.component_length;
        let absolute_roughness = self.dowtherm_pipe_properties.
            ↪ absolute_roughness;
        let xs_area = self.get_cross_sectional_area();

        let fluid_viscosity = DowthermAPipe::viscosity(fluid_temp);
        let fluid_density = DowthermAPipe::density(fluid_temp);

        let form_loss_k = self.dowtherm_pipe_properties.form_loss_k;

        // second let's get pressure loss from mass rate
        // by using this function or method, we assume that
        // the pipe behaves symmetrically in reverse flow
        let pressure_loss = standard_pipe_calc::CalcPressureLoss::
            from_mass_rate(
                fluid_mass_flowrate,
                xs_area,
                hydraulic_diameter,
                fluid_viscosity,

```

```

        fluid_density,
        pipe_length,
        absolute_roughness,
        form_loss_k);
// now to calculate pressure change
// we note this equation
//
// Pressure Change = - pressure loss + hydrostatic pressure +
// source pressure
//
//
// for hydrostatic pressure gain
// g is earth gravity at 9.81
// delta H is positive upwards

let hydrostatic_pressure_increase: Pressure =
    self.get_hydrostatic_pressure_change(
        fluid_temp);
// last but not least we need our source pressure

let source_pressure: Pressure = self.dowtherm_pipe_properties.
    internal_pressure;

// now we can calculate pressure change

let pressure_change =
    -pressure_loss +
    hydrostatic_pressure_increase +
    source_pressure;

return pressure_change;
}

fn to_mass_rate(&self, pressure_change: Pressure,
    fluid_temp: ThermodynamicTemperature) -> MassRate {
    // first let's get all the relevant properties...
    let hydraulic_diameter = self.dowtherm_pipe_properties.
        ↪ hydraulic_diameter;
    let pipe_length = self.dowtherm_pipe_properties.component_length;
    let absolute_roughness = self.dowtherm_pipe_properties.
        ↪ absolute_roughness;
    let xs_area = self.get_cross_sectional_area();

```

```

let fluid_viscosity = DowthermAPipe::viscosity(fluid_temp);
let fluid_density = DowthermAPipe::density(fluid_temp);

let form_loss_k = self.dowtherm_pipe_properties.form_loss_k;

// now we need to calculate a pressure loss term
// we use:
// Pressure Change = - pressure loss + hydrostatic pressure +
// source pressure
//
// so we just add pressure loss to both sides and subtract pressure
// change to both sides
// pressure loss = - pressure change + hydrostatic pressure +
// source pressure

// for hydrostatic pressure gain
// g is earth gravity at 9.81
// delta H is positive upwards
let hydrostatic_pressure_increase: Pressure =
    self.get_hydrostatic_pressure_change(
        fluid_temp);
// last but not least we need our source pressure

let source_pressure: Pressure = self.dowtherm_pipe_properties.
    internal_pressure;

// now calculate pressure loss
let pressure_loss =
    -pressure_change +
    hydrostatic_pressure_increase +
    source_pressure;

let mass_rate = standard_pipe_calc::CalcPressureLoss::
    to_mass_rate(
        pressure_loss,
        xs_area,
        hydraulic_diameter,
        fluid_viscosity,
        fluid_density,
        pipe_length,
        absolute_roughness,

```

```

        form_loss_k);

    return mass_rate;
}
}

```

Therminol Custom Component Parent Class or Struct

All therminol custom components with custom fdk factors inherit from this base struct or class.

```

use crate::fluid_component_calculation::custom_component_calc;
use crate::therminol_component::*;
use dowtherm_a_properties;

use uom::si::length::{meter, millimeter};
use uom::si::pressure::pascal;
use uom::si::angle::degree;
use uom::si::acceleration::meter_per_second_squared;
use uom::si::area::square_meter;

// the 'a' notation states that
// the dowthermACustomComponent only lives as long as
// the custom_component_properties object
pub struct DowthermACustomComponent {
    pub dowtherm_custom_component_properties: CustomComponentProperties,
}

impl FluidProperties for DowthermACustomComponent {
    fn density(fluid_temp: ThermodynamicTemperature) -> MassDensity {
        return dowtherm_a_properties::getDowthermADensity(fluid_temp);
    }

    fn viscosity(
        fluid_temp: ThermodynamicTemperature) -> DynamicViscosity{
        return dowtherm_a_properties::getDowthermAViscosity(fluid_temp);
    }

    fn enthalpy(fluid_temp: ThermodynamicTemperature) -> AvailableEnergy{
        return dowtherm_a_properties::getDowthermAEnthalpy(fluid_temp);
    }
}

```

```

fn specific_heat_capacity(
  fluid_temp: ThermodynamicTemperature) -> SpecificHeatCapacity{
  return dowtherm_a_properties::
    getDowthermAConstantPressureSpecificHeatCapacity(
      fluid_temp);
}

fn thermal_conductivity(
  fluid_temp: ThermodynamicTemperature) -> ThermalConductivity{
  return dowtherm_a_properties::
    getDowthermAThermalConductivity(fluid_temp);
}

fn get_temperature_from_enthalpy(
  fluid_enthalpy: AvailableEnergy) -> ThermodynamicTemperature{
  return dowtherm_a_properties::
    get_temperature_from_enthalpy(fluid_enthalpy);
}
}

impl StandardCustomComponentProperties for DowthermACustomComponent {
  // constructor
  fn new(name: String,
    hydraulic_diameter_meters: f64,
    cross_sectional_area_meters_sq: f64,
    component_length_meters: f64,
    absolute_roughness_millimeters: f64,
    incline_angle_degrees: f64,
    custom_darcy: &'static dyn Fn(f64,f64) -> f64,
    custom_k: &'static dyn Fn(f64) -> f64) -> Self {

    let input_hydraulic_diameter = Length::new::<meter>(
      hydraulic_diameter_meters);
    let input_xs_area = Area::new::<square_meter>(
      ↪ cross_sectional_area_meters_sq);
    let input_component_length = Length::new::<meter>(
      component_length_meters);
    let input_absolute_roughness = Length::new::<millimeter>(
      absolute_roughness_millimeters);
    let input_incline_angle = Angle::new::<degree>(

```

```

        incline_angle_degrees);
let input_internal_pressure = Pressure::new::<pascal>(
    0.0);

let custom_pipe_properties = CustomComponentProperties {
    _name: name,
    hydraulic_diameter: input_hydraulic_diameter,
    xs_area: input_xs_area,
    component_length: input_component_length,
    absolute_roughness: input_absolute_roughness,
    incline_angle: input_incline_angle,
    custom_darcy: custom_darcy,
    custom_k: custom_k,
    internal_pressure: input_internal_pressure,
};

return Self { dowtherm_custom_component_properties :
    ↪ custom_pipe_properties };
}

fn get_cross_sectional_area(&self) -> Area {

    return self.dowtherm_custom_component_properties.
        xs_area;
}

fn get_internal_pressure_term(&self) -> Pressure {
    return self.dowtherm_custom_component_properties.internal_pressure;
}

fn set_internal_pressure_term(&mut self, pressure_pascals: f64) {
    self.dowtherm_custom_component_properties.internal_pressure =
        Pressure::new::<pascal>(pressure_pascals);
}

fn get_hydrostatic_pressure_change(
    &self, fluid_temp: ThermodynamicTemperature) -> Pressure {

    let pipe_length = self.dowtherm_custom_component_properties.
        ↪ component_length;
    let incline_angle = self.dowtherm_custom_component_properties.

```



```

    ↪ incline_angle;
let fluid_density = DowthermACustomComponent::density(fluid_temp);

let g: Acceleration =
    Acceleration::new::<meter_per_second_squared>(-9.81);
let delta_h: Length = pipe_length*incline_angle.sin();

let hydrostatic_pressure_increase: Pressure =
    fluid_density * g * delta_h;

return hydrostatic_pressure_increase;
}
}

impl CalcPressureChange for DowthermACustomComponent {

    fn from_mass_rate(&self, fluid_mass_flowrate: MassRate,
                      fluid_temp: ThermodynamicTemperature) -> Pressure {

        // first let's get all the relevant properties...
let hydraulic_diameter = self.dowtherm_custom_component_properties.
    ↪ hydraulic_diameter;
let pipe_length = self.dowtherm_custom_component_properties.
    ↪ component_length;
let absolute_roughness = self.dowtherm_custom_component_properties.
    ↪ absolute_roughness;
let xs_area = self.get_cross_sectional_area();

let fluid_viscosity = DowthermACustomComponent::viscosity(fluid_temp
    ↪ );
let fluid_density = DowthermACustomComponent::density(fluid_temp);

let custom_k = self.dowtherm_custom_component_properties.custom_k;
let custom_darcy = self.dowtherm_custom_component_properties.
    ↪ custom_darcy;

// second let's get pressure loss from mass rate
// by using this function or method, we assume that
// the pipe behaves symmetrically in reverse flow
let pressure_loss = custom_component_calc::CalcPressureLoss::
    from_mass_rate(
        fluid_mass_flowrate,

```

```

        xs_area,
        hydraulic_diameter,
        fluid_viscosity,
        fluid_density,
        pipe_length,
        absolute_roughness,
        custom_darcy,
        custom_k);
// now to calculate pressure change
// we note this equation
//
// Pressure Change = - pressure loss + hydrostatic pressure +
// source pressure
//
//
// for hydrostatic pressure gain
// g is earth gravity at 9.81
// delta H is positive upwards
let hydrostatic_pressure_increase: Pressure =
    self.get_hydrostatic_pressure_change(
        fluid_temp);
// last but not least we need our source pressure

let source_pressure: Pressure = self.
    ↪ dowtherm_custom_component_properties.
        internal_pressure;

// now we can calculate pressure change

let pressure_change =
    -pressure_loss +
    hydrostatic_pressure_increase +
    source_pressure;

return pressure_change;
}

fn to_mass_rate(&self, pressure_change: Pressure,
    fluid_temp: ThermodynamicTemperature) -> MassRate {
// first let's get all the relevant properties...
let hydraulic_diameter = self.dowtherm_custom_component_properties.
    ↪ hydraulic_diameter;

```

```

let pipe_length = self.dowtherm_custom_component_properties.
    ↪ component_length;
let absolute_roughness = self.dowtherm_custom_component_properties.
    ↪ absolute_roughness;
let xs_area = self.get_cross_sectional_area();

let fluid_viscosity = DowthermACustomComponent::viscosity(fluid_temp
    ↪ );
let fluid_density = DowthermACustomComponent::density(fluid_temp);

let custom_k = self.dowtherm_custom_component_properties.custom_k;
let custom_darcy = self.dowtherm_custom_component_properties.
    ↪ custom_darcy;

// now we need to calculate a pressure loss term
// we use:
// Pressure Change = - pressure loss + hydrostatic pressure +
// source pressure
//
// so we just add pressure loss to both sides and subtract pressure
// change to both sides
// pressure loss = - pressure change + hydrostatic pressure +
// source pressure

// for hydrostatic pressure gain
// g is earth gravity at 9.81
// delta H is positive upwards
let hydrostatic_pressure_increase: Pressure =
    self.get_hydrostatic_pressure_change(
        fluid_temp);
// last but not least we need our source pressure

let source_pressure: Pressure = self.
    ↪ dowtherm_custom_component_properties.
        internal_pressure;

// now calculate pressure loss
let pressure_loss =
    -pressure_change +
    hydrostatic_pressure_increase +
    source_pressure;

```

```

        let mass_rate = custom_component_calc::CalcPressureLoss::
            to_mass_rate(
                pressure_loss,
                xs_area,
                hydraulic_diameter,
                fluid_viscosity,
                fluid_density,
                pipe_length,
                absolute_roughness,
                custom_darcy,
                custom_k);

        return mass_rate;
    }
}

```

Hydrodynamic Data for CIET Components

As the base classes or structs are already developed, I then put the pipe components into a factory class which helps instantiate all the thermol components.

```

extern crate uom;

// adding units from uom

// custom thermol component imports
use crate::thermol_component::custom_thermol_component::
DowthermACustomComponent;

use crate::thermol_component::
StandardCustomComponentProperties;

use crate::thermol_component::thermol_pipe::
DowthermAPipe;

use super::StandardPipeProperties;

/// CTAH branch items
/// I'm still in the midst of rearranging things though...

pub struct Pipe6a {
    // pipe 6a

```

```

    // otherwise known as the static mixer pipe 6a
}

impl Pipe6a {

    pub fn get() -> DowthermAPipe {
        let pipe_6a: DowthermAPipe
            = StandardPipeProperties::new(
                "static_mixer_pipe_6a".to_string(),
                2.79e-2, // component diameter in meters
                0.1526, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                51.526384, // angle in degrees
                5.05 // form loss K value
            );

        return pipe_6a;
    }
}

pub struct StaticMixer41 {
    // static mixer 41 (MX-41) on CIET diagram
    // in the pump and CTAH branch
    // just before CTAH (AKA IHX)
    // from top to bottom
    //
    // label 6 on diagram
}

impl StaticMixer41 {

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {

```

```

    let mut reverse_flow = false;

    // the user account for reverse flow scenarios...
    if reynolds_number < 0.0 {
        reverse_flow = true;
        reynolds_number = reynolds_number * -1.0;
    }

    let custom_k_value =
        21.0 + 4000.0/reynolds_number;

    if reverse_flow {
        return -custom_k_value;
    }

    return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let static_mixer_41: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "static_mixer_41_label_6".to_string(),
            2.79e-2, // component diameter in meters
            6.11e-4, //component area in sq meters
            0.33, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
                ↪ here,
                // but i need to fill in
            51.526384, //incline angle in degrees
            &StaticMixer41::custom_darcy,
            &StaticMixer41::custom_k);

    return static_mixer_41;
}
}

pub struct CTAHVertical {

    // coiled tube air heater,
    // uses pipe friction factors but has a constant K value

```

```

    // also pipe isn't circular
    // so we'll have to use custom fldk to help
    // label 7a
}

impl CTAHVertical {

    pub fn custom_darcy(mut reynolds_number: f64, roughness_ratio: f64) ->
        ↪ f64 {

        if roughness_ratio < 0.0 {
            panic!("roughness_ratio < 0.0");
        }

        use crate::churchill_friction_factor;
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }

        let darcy = churchill_friction_factor::darcy(reynolds_number,
                                                    roughness_ratio);

        if reverse_flow {
            return -darcy;
        }
        return darcy;
    }

    pub fn custom_k(reynolds_number: f64) -> f64 {

        let custom_k_value = 3.9;

        if reynolds_number < 0.0 {
            return -custom_k_value
        }

        return custom_k_value;
    }
}

```

```

}

pub fn get() -> DowthermACustomComponent {

    let ctah_vertical: DowthermACustomComponent
    = StandardCustomComponentProperties::new(
        "ctah_vertical_label_7a".to_string(),
        1.19e-2, // component diameter in meters
        1.33e-3, //component area in sq meters
        0.3302, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
            ↪ here,
            // but i need to fill in
        -90.0, //incline angle in degrees
        &CTAHVertical::custom_darcy,
        &CTAHVertical::custom_k);

    return ctah_vertical;
}
}

pub struct CTAHHorizontal {

    // coiled tube air heater
    // has fldk = 400 + 52,000/Re
    //
    // label is 7b
    // empirical data in page 48 on pdf viewer in Dr
    // Zweibaum thesis shows reverse flow has same
    // pressure drop characteristics as forward flow
}

impl CTAHHorizontal {

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {

```



```

    let mut reverse_flow = false;

    // the user account for reverse flow scenarios...
    if reynolds_number < 0.0 {
        reverse_flow = true;
        reynolds_number = reynolds_number * -1.0;
    }

    let custom_k_value =
        400.0 + 52000.0/reynolds_number;

    if reverse_flow {
        return -custom_k_value;
    }

    return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let ctah_horizontal: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "ctah_horizontal_label_7b".to_string(),
            1.19e-2, // component diameter in meters
            1.33e-3, //component area in sq meters
            1.2342, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
                ↪ here,
                // but i need to fill in
            0.0, //incline angle in degrees
            &CTAHHorizontal::custom_darcy,
            &CTAHHorizontal::custom_k);

    return ctah_horizontal;
}

pub struct Pipe8a {
    // pipe 8a
    // otherwise known as the static mixer pipe 8a

```

```

}

impl Pipe8a {

    pub fn get() -> DowthermAPipe {
        let pipe_8a: DowthermAPipe
            = StandardPipeProperties::new(
                "static_mixer_pipe_8a".to_string(),
                2.79e-2, // component diameter in meters
                0.22245, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                -90.0, // angle in degrees
                3.75 // form loss K value
            );

        return pipe_8a;
    }
}

pub struct StaticMixer40 {
    // static mixer 40 (MX-40) on CIET diagram
    // just after CTAH (AKA IHX)
    // from top to bottom
    // label 8 on diagram
    //
    // forced convection flow direction is same as top to bottom
    //
    // has a fldk of 21+4000/Re
}

impl StaticMixer40 {

    // let's import everything necessary:

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {
        return 0.0;
    }
}

```

```

pub fn custom_k(mut reynolds_number: f64) -> f64 {
    let mut reverse_flow = false;

    // the user account for reverse flow scenarios...
    if reynolds_number < 0.0 {
        reverse_flow = true;
        reynolds_number = reynolds_number * -1.0;
    }

    let custom_k_value =
        21.0 + 4000.0/reynolds_number;

    if reverse_flow {
        return -custom_k_value;
    }

    return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let static_mixer_40: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "static_mixer_40_label_8".to_string(),
            2.79e-2, // component diameter in meters
            6.11e-4, //component area in sq meters
            0.33, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
                ↪ here,
                // but i need to fill in
            -90.0, //incline angle in degrees
            &StaticMixer40::custom_darcy,
            &StaticMixer40::custom_k);

    return static_mixer_40;
}

pub struct Pipe9 {
    // pipe 9
}

```

```

impl Pipe9 {

  pub fn get() -> DowthermAPipe {
    let pipe_9: DowthermAPipe
      = StandardPipeProperties::new(
        "pipe_9".to_string(),
        2.79e-2, // component diameter in meters
        0.7112, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
          ↪ here,
          // but i need to fill in
          // in millimeters
        -42.73211, // angle in degrees
        0.8 // form loss K value
      );

    return pipe_9;
  }
}

pub struct Pipe10 {
  // pipe 10
}

impl Pipe10 {

  pub fn get() -> DowthermAPipe {
    let pipe_10: DowthermAPipe
      = StandardPipeProperties::new(
        "pipe_10".to_string(),
        2.79e-2, // component diameter in meters
        2.4511, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
          ↪ here,
          // but i need to fill in
          // in millimeters
        -90.0, // angle in degrees
        0.45 // form loss K value
      );
  }
}

```

```

        return pipe_10;
    }
}

pub struct Pipe11 {
    // pipe 11
}

impl Pipe11 {

    pub fn get() -> DowthermAPipe {
        let pipe_11: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_11".to_string(),
                2.79e-2, // component diameter in meters
                0.4826, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                -63.47465, // angle in degrees
                2.4 // form loss K value
            );

        return pipe_11;
    }
}

pub struct Pipe12 {
    // pipe 12
}

impl Pipe12 {

    pub fn get() -> DowthermAPipe {
        let pipe_12: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_12".to_string(),
                2.79e-2, // component diameter in meters
                0.333375, // component length in meters
            );
    }
}

```

```

        0.015, // estimated component wall roughness (doesn't matter
        ↪ here,
            // but i need to fill in
            // in millimeters
        0.0, // angle in degrees
        21.65 // form loss K value
    );

    return pipe_12;
}

pub struct CTAHPump {
    // ctah pump is an empty component
    // ie no friction factor losses
    // but it provides a source pressure
}

impl CTAHPump {

    // let's import everything necessary:

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
    ↪ {
        return 0.0;
    }

    pub fn custom_k(_reynolds_number: f64) -> f64 {
        return 0.0;
    }

    pub fn get(pressure_pascals: f64
        ) -> DowthermACustomComponent {

        let mut ctah_pump: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "ctah_pump".to_string(),
            2.79e-2, // component diameter in meters
            6.11e-4, // cross sectional area in meters sq
            0.36, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
            ↪ here,

```

```

        // but i need to fill in
        // in millimeters
        0.0, //incline angle in degrees
        &CTAHPump::custom_darcy,
        &CTAHPump::custom_k);

    ctah_pump.set_internal_pressure_term(pressure_pascals);

    return ctah_pump;
}
}

pub struct Pipe13 {
    // pipe 13 on the diagram in Nico Zweibaum nodalisation
    // probably some combination of V-42,
    // F-40 and F-41 on CIET diagram
}

impl Pipe13 {

    pub fn get() -> DowthermAPipe {
        let pipe_13: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_13".to_string(),
                2.79e-2, // component diameter in meters
                1.273175, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                0.0, // angle in degrees
                12.95 // form loss K value
            );

        return pipe_13;
    }
}

pub struct Pipe14 {
    // pipe 14 on the diagram in Nico Zweibaum nodalisation
    // probably some combination of V-42,

```

```

    // F-40 and F-41 on CIET diagram
    // it is inclined 90 degrees upwards in direction
    // of flow
    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is also 90 degrees
}

impl Pipe14 {

    pub fn get() -> DowthermAPipe {
        let pipe_14: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_14".to_string(),
                2.79e-2, // component diameter in meters
                0.6687, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                90.0, // angle in degrees
                2.4 // form loss K value
            );

        return pipe_14;
    }
}

pub struct Flowmeter40 {
    // ctah line flowmeter 40
    // label 14a on simulation diagram
    // fldk = 18.0+93000/Re
}

impl Flowmeter40 {

    // let's import everything necessary:

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {
        return 0.0;
    }
}

```



```

pub fn custom_k(mut reynolds_number: f64) -> f64 {
    let mut reverse_flow = false;

    // the user account for reverse flow scenarios...
    if reynolds_number < 0.0 {
        reverse_flow = true;
        reynolds_number = reynolds_number * -1.0;
    }

    let custom_k_value =
        18.0 + 93000.0/reynolds_number.powf(1.35);
    // coriolis flowmeter

    if reverse_flow {
        return -custom_k_value;
    }

    return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let flowmeter_40_14a: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "flowmeter_40_14a".to_string(),
            2.79e-2, // component diameter in meters
            6.11e-4, // cross sectional area in meters sq
            0.36, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
                ↪ here,
                // but i need to fill in
                // in millimeters
            90.0, //incline angle in degrees
            &Flowmeter40::custom_darcy,
            &Flowmeter40::custom_k);

    return flowmeter_40_14a;
}
}

```

```

pub struct Pipe15 {
    // pipe 15 on the diagram in Nico Zweibaum nodalisation
    // probably corresponds of F30 on CIET's P&ID
    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is also
    // -49.36983 degrees
}

impl Pipe15 {

    pub fn get() -> DowthermAPipe {
        let pipe_15: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_15".to_string(),
                2.79e-2, // component diameter in meters
                0.3556, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                -49.36983, // angle in degrees
                0.8 // form loss K value
            );

        return pipe_15;
    }
}

pub struct Pipe16 {
    // pipe 16 on the diagram in Nico Zweibaum nodalisation
    // probably corresponds of F30 on CIET's P&ID
    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is also
    // -49.36983 degrees
}

impl Pipe16 {

    pub fn get() -> DowthermAPipe {
        let pipe_16: DowthermAPipe

```

```

        = StandardPipeProperties::new(
            "pipe_16".to_string(),
            2.79e-2, // component diameter in meters
            0.644525, // component length in meters
            0.015, // estimated component wall roughness
                // (doesn't matter here,
                // but i need to fill in
                // millimeters
            -90.0, // angle in degrees
            1.9 // form loss K value
        );

    return pipe_16;
}
}

pub struct Branch17 {
    // pipe 17 on the diagram in Nico Zweibaum nodalisation
    // probably corresponds of F30 on CIET's P&ID
    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is 0 degrees
    //
}

impl Branch17 {

    pub fn get() -> DowthermAPipe {
        let pipe_17: DowthermAPipe
            = StandardPipeProperties::new(
                "branch_17".to_string(),
                2.79e-2, // component diameter in meters
                0.473075, // component length in meters
                0.015, // estimated component wall roughness
                    // (doesn't matter here,
                    // but i need to fill in
                    // millimeters
                0.0, // angle in degrees
                0.0 // form loss K value
            );

        return pipe_17;
    }
}

```

```

    }
}

/// Heater Branch (top to bottom)
// this is reverse order compared to table A-1 in
// the Zweibaum nodalised relap model

pub struct Branch5 {
    // pipe 5 on the diagram in Nico Zweibaum nodalisation
    // and from a top to bottom direction from pipe 5
    // to pipe 5, the incline angle is also
    // 0 degrees
    // i add 180 degrees so that it is
    // properly reversed in
    // inclination angle from top to bottom
}

impl Branch5 {

    pub fn get() -> DowthermAPipe {
        let pipe_5: DowthermAPipe
            = StandardPipeProperties::new(
                "branch_5".to_string(),
                2.79e-2, // component diameter in meters
                0.7493, // component length in meters
                0.015, // estimated component wall roughness
                    // (doesn't matter here,
                    // but i need to fill in
                    // millimeters
                0.0 + 180.0, // angle in degrees
                0.0 // form loss K value
            );

        return pipe_5;
    }
}

pub struct Pipe4 {
    // pipe 4 on the diagram in Nico Zweibaum nodalisation
    // probably corresponds of V11 and F12

```

```

    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is also
    // 49.743387 +180.0 degrees
}

impl Pipe4 {

    pub fn get() -> DowthermAPipe {
        let pipe_4: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_4".to_string(),
                2.79e-2, // component diameter in meters
                0.2413, // component length in meters
                0.015, // estimated component wall roughness
                    // (doesn't matter here,
                    // but i need to fill in
                    // millimeters
                49.743387 + 180.0, // angle in degrees
                2.4 // form loss K value
            );

        return pipe_4;
    }
}

pub struct Pipe3 {
    // pipe 3 on the diagram in Nico Zweibaum nodalisation
    // probably corresponds of V11 and F12
    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is also
    // 90.0 +180.0 degrees
}

impl Pipe3 {

    pub fn get() -> DowthermAPipe {
        let pipe_3: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_3".to_string(),
                2.79e-2, // component diameter in meters

```

```

        1.2827, // component length in meters
        0.015, // estimated component wall roughness
                // (doesn't matter here,
                // but i need to fill in
                // millimeters
        90.0 + 180.0, // angle in degrees
        3.15 // form loss K value
    );

    return pipe_3;
}
}

pub struct StaticMixer10 {
    // static mixer 10 (MX-10) on CIET diagram
    // just before the heater in the heater branch
    // from top to bottom
    // label 2 on diagram (fig A-1 on Nico Zweibaum thesis)
    // pg 125 on pdf viewer, pg 110 on printed page number on bottom right
    //
    // though in reality flow goes from bottom to
    // top in forced convection
    // so from a flow perspective it is before the
    // heater
    //
}

impl StaticMixer10 {

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
    ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }
    }
}

```

```

    let custom_k_value =
        21.0 + 4000.0/reynolds_number;

    if reverse_flow {
        return -custom_k_value;
    }

    return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let static_mixer_10: DowthermACustomComponent
    = StandardCustomComponentProperties::new(
        "static_mixer_41_label_2".to_string(),
        2.79e-2, // component diameter in meters
        6.11e-4, //component area in sq meters
        0.33, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
            ↪ here,
            // but i need to fill in
        90.0-180.0, //incline angle in degrees
        &StaticMixer10::custom_darcy,
        &StaticMixer10::custom_k);

    return static_mixer_10;
}
}

pub struct Pipe2a {
    // pipe 2a on the diagram in Nico Zweibaum nodalisation
    // probably corresponds of V11 and F12
    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is also
    // 90.0 +180.0 degrees
}

impl Pipe2a {

```

```

pub fn get() -> DowthermAPipe {
    let pipe_2a: DowthermAPipe
        = StandardPipeProperties::new(
            "pipe_2a_static_mixer".to_string(),
            2.79e-2, // component diameter in meters
            0.149425, // component length in meters
            0.015, // estimated component wall roughness
                // (doesn't matter here,
                // but i need to fill in
                // millimeters
            90.0 + 180.0, // angle in degrees
            1.8 // form loss K value
        );

    return pipe_2a;
}

pub struct HeaterTopHead1a {

    // heater top head
    // diagram label 1a
    //
    // inclined at 90 degrees bottom to top
    // or 90 degrees + 180 top to bottom orientation
}

impl HeaterTopHead1a {

    pub fn custom_darcy(mut reynolds_number: f64, roughness_ratio: f64) ->
        ↪ f64 {

        if roughness_ratio < 0.0 {
            panic!("roughness_ratio < 0.0");
        }

        use crate::churchill_friction_factor;
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {

```



```

        reverse_flow = true;
        reynolds_number = reynolds_number * -1.0;
    }

    let darcy = churchill_friction_factor::darcy(reynolds_number,
                                                roughness_ratio);

    if reverse_flow {
        return -darcy;
    }
    return darcy;
}

pub fn custom_k(reynolds_number: f64) -> f64 {

    let custom_k_value = 3.75;

    if reynolds_number < 0.0 {
        return -custom_k_value
    }

    return custom_k_value;

}

pub fn get() -> DowthermACustomComponent {

    let heater_top_head: DowthermACustomComponent
    = StandardCustomComponentProperties::new(
        "heater_top_head_label_1a".to_string(),
        6.60e-3, // component diameter in meters
        3.64e-4, //component area in sq meters
        0.0889, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
            ↪ here,
            // but i need to fill in
            90.0 + 180.0, //incline angle in degrees
            &HeaterTopHead1a::custom_darcy,
            &HeaterTopHead1a::custom_k);

    return heater_top_head;
}

```

```

}

pub struct CietHeaterVersion1 {

    // this is the first version of the ciet heater
    // without any insert within the heater
    // the heater behaves like a pipe
    //
    // inclined at 90 degrees bottom to top
    // or 90 degrees + 180 top to bottom orientation
}

impl CietHeaterVersion1 {

    pub fn custom_darcy(mut reynolds_number: f64, roughness_ratio: f64) ->
        ↪ f64 {

        if roughness_ratio < 0.0 {
            panic!("roughness_ratio < 0.0");
        }

        use crate::churchill_friction_factor;
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }

        let darcy = churchill_friction_factor::darcy(reynolds_number,
                                                    roughness_ratio);

        if reverse_flow {
            return -darcy;
        }
        return darcy;
    }

    pub fn custom_k(reynolds_number: f64) -> f64 {

```

```

    let custom_k_value = 0.0;

    if reynolds_number < 0.0 {
        return -custom_k_value
    }

    return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let heater_version_1_label_1: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "heater_version_1_label_1_label_1a".to_string(),
            6.60e-3, // component diameter in meters
            3.64e-4, //component area in sq meters
            1.6383, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
                ↪ here,
                // but i need to fill in
            90.0 + 180.0, //incline angle in degrees
            &CietHeaterVersion1::custom_darcy,
            &CietHeaterVersion1::custom_k);

    return heater_version_1_label_1;
}
}

pub struct HeaterBottomHead1b {

    // heater top head
    // diagram label 1b
    //
    // inclined at 90 degrees bottom to top
    // or 90 degrees + 180 top to bottom orientation
}

impl HeaterBottomHead1b {

    pub fn custom_darcy(mut reynolds_number: f64, roughness_ratio: f64) ->

```

```

→ f64 {

    if roughness_ratio < 0.0 {
        panic!("roughness_ratio < 0.0");
    }

    use crate::churchill_friction_factor;
    let mut reverse_flow = false;

    // the user account for reverse flow scenarios...
    if reynolds_number < 0.0 {
        reverse_flow = true;
        reynolds_number = reynolds_number * -1.0;
    }

    let darcy = churchill_friction_factor::darcy(reynolds_number,
                                                roughness_ratio);

    if reverse_flow {
        return -darcy;
    }
    return darcy;
}

pub fn custom_k(reynolds_number: f64) -> f64 {

    let custom_k_value = 3.95;

    if reynolds_number < 0.0 {
        return -custom_k_value
    }

    return custom_k_value;

}

pub fn get() -> DowthermACustomComponent {

    let heater_bottom_head: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "heater_bottom_head_label_1b".to_string(),
            6.60e-3, // component diameter in meters

```

```

        3.64e-4, //component area in sq meters
        0.19685, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
            ↪ here,
            // but i need to fill in
        90.0 + 180.0, //incline angle in degrees
        &HeaterBottomHead1b::custom_darcy,
        &HeaterBottomHead1b::custom_k);

    return heater_bottom_head;
}
}

pub struct Pipe18 {
    // pipe 18 on the diagram in Nico Zweibaum nodalisation
    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is also
    // -40.00520 +180.0 degrees
}

impl Pipe18 {

    pub fn get() -> DowthermAPipe {
        let pipe_18: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_18".to_string(),
                2.79e-2, // component diameter in meters
                0.1778, // component length in meters
                0.015, // estimated component wall roughness
                    // (doesn't matter here,
                    // but i need to fill in
                    // millimeters
                -40.00520 + 180.0, // angle in degrees
                5.15 // form loss K value
            );

        return pipe_18;
    }
}

/// DHX Branch from Top to Bottom

```

```

/// this is volume

pub struct Pipe26 {
    // pipe 26 on the diagram in Nico Zweibaum nodalisation
    //
    // and from a top to bottom direction from pipe 5
    // to pipe 17, the incline angle is also
    // -40.00520 +180.0 degrees
}

impl Pipe26 {

    pub fn get() -> DowthermAPipe {
        let pipe_26: DowthermAPipe
            = StandardPipeProperties::new(
                "pipe_26".to_string(),
                2.79e-2, // component diameter in meters
                0.2159, // component length in meters
                0.015, // estimated component wall roughness
                // (doesn't matter here,
                // but i need to fill in
                // millimeters
                52.571994 + 180.0, // angle in degrees
                1.75 // form loss K value
            );

        return pipe_26;
    }
}

pub struct StaticMixer21 {
    // static mixer 21 (MX-21) on CIET diagram
    // in the DRACS branch in primary loop
    // just before the DRACS heat exchanger
    // from top to bottom
    // label 25
    //
    // in reality flow goes from bottom to
    // top in natural convection
    // also in the DRACS
    // loop there are flow diodes to make
    // it such that flow going from bottom to top

```

```

    // encounters more resistance
    //
}
impl StaticMixer21 {

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
    ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }

        let custom_k_value =
            21.0 + 4000.0/reynolds_number;

        if reverse_flow {
            return -custom_k_value;
        }

        return custom_k_value;
    }

    pub fn get() -> DowthermACustomComponent {

        let static_mixer_21: DowthermACustomComponent
            = StandardCustomComponentProperties::new(
                "static_mixer_21_label_25".to_string(),
                2.79e-2, // component diameter in meters
                6.11e-4, //component area in sq meters
                0.33, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                // but i need to fill in

```

```

        90.0-180.0, //incline angle in degrees
        &StaticMixer21::custom_darcy,
        &StaticMixer21::custom_k);

    return static_mixer_21;
}
}

pub struct Pipe25a {
    // pipe 25a
    // otherwise known as the static mixer pipe 25a
}

impl Pipe25a {

    pub fn get() -> DowthermAPipe {
        let pipe_25a: DowthermAPipe
            = StandardPipeProperties::new(
                "static_mixer_pipe_25a".to_string(),
                2.79e-2, // component diameter in meters
                0.22245, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                90.0-180.0, // angle in degrees
                1.35 // form loss K value
            );

        return pipe_25a;
    }
}

pub struct DHXShellSideHeatExchanger {

    // this is the DRACS DHX shell side heat exchanger
    // in the DHX branch
    // inclined at 90 degrees bottom to top
    // or 90 degrees + 180 top to bottom orientation
    //

```



```

    // label 24
}

impl DHXShellSideHeatExchanger {

    pub fn custom_darcy(mut reynolds_number: f64, roughness_ratio: f64) ->
        ↪ f64 {

        if roughness_ratio < 0.0 {
            panic!("roughness_ratio < 0.0");
        }

        use crate::churchill_friction_factor;
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }

        let darcy = churchill_friction_factor::darcy(reynolds_number,
                                                    roughness_ratio);

        if reverse_flow {
            return -darcy;
        }
        return darcy;
    }

    pub fn custom_k(reynolds_number: f64) -> f64 {

        let custom_k_value = 23.9;

        if reynolds_number < 0.0 {
            return -custom_k_value;
        }

        return custom_k_value;
    }
}

```

```

pub fn get() -> DowthermACustomComponent {

    let dhx_shell_side: DowthermACustomComponent
    = StandardCustomComponentProperties::new(
        "dhx_shell_side_label_24".to_string(),
        5.65e-3, // component diameter in meters
        9.43e-4, //component area in sq meters
        1.18745, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
            ↪ here,
            // but i need to fill in
            90.0 + 180.0, //incline angle in degrees
            &DHXShellSideHeatExchanger::custom_darcy,
            &DHXShellSideHeatExchanger::custom_k);

    return dhx_shell_side;
}
}

pub struct StaticMixer20 {
    // static mixer 20 (MX-20) on CIET diagram
    // in the DRACS branch in primary loop
    // just after the DRACS heat exchanger
    // from top to bottom
    // label 23
    //
    // in reality flow goes from bottom to
    // top in natural convection
    // also in the DRACS
    // loop there are flow diodes to make
    // it such that flow going from bottom to top
    // encounters more resistance
    //
    // original angle is is 90 degrees
    // but i orientate from top to bottom
}

impl StaticMixer20 {

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {

```

```

    return 0.0;
}

pub fn custom_k(mut reynolds_number: f64) -> f64 {
    let mut reverse_flow = false;

    // the user account for reverse flow scenarios...
    if reynolds_number < 0.0 {
        reverse_flow = true;
        reynolds_number = reynolds_number * -1.0;
    }

    let custom_k_value =
        21.0 + 4000.0/reynolds_number;

    if reverse_flow {
        return -custom_k_value;
    }

    return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let static_mixer_20: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "static_mixer_20_label_23".to_string(),
            2.79e-2, // component diameter in meters
            6.11e-4, //component area in sq meters
            0.33, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
                ↪ here,
                // but i need to fill in
            90.0-180.0, //incline angle in degrees
            &StaticMixer20::custom_darcy,
            &StaticMixer20::custom_k);

    return static_mixer_20;
}
}

```

```

pub struct Pipe23a {
    // pipe 23a
    // otherwise known as the static mixer pipe 23a
}

impl Pipe23a {

    pub fn get() -> DowthermAPipe {
        let pipe_23a: DowthermAPipe
            = StandardPipeProperties::new(
                "static_mixer_pipe_23a".to_string(),
                2.79e-2, // component diameter in meters
                0.0891, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                90.0-180.0, // angle in degrees
                1.35 // form loss K value
            );

        return pipe_23a;
    }
}

pub struct Pipe22 {
    // pipe 22
    // otherwise known as the static mixer pipe 22
}

impl Pipe22 {

    pub fn get() -> DowthermAPipe {
        let pipe_22: DowthermAPipe
            = StandardPipeProperties::new(
                "static_mixer_pipe_22".to_string(),
                2.79e-2, // component diameter in meters
                0.69215, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in

```

```

        // in millimeters
        90.0-180.0, // angle in degrees
        9.95 // form loss K value
    );

    return pipe_22;
}

}

pub struct Flowmeter20 {
    // DHX flow flowmeter 20
    // natural convection heat exchanger in primary loop
    // diagram label is 21a
    // we use the convention of top of bypass branch to bottom
    // hence degree is -90
    //
    // However in DHX, i also expect there to be
    // a check valve which only allows flow from top to bottom
    //
    // That is the forward direction of flow for FM20,
    //
}

impl Flowmeter20 {

    // let's import everything necessary:

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
    ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }

        let custom_k_value =

```

```

        18.0 + 93000.0/reynolds_number.powf(1.35);
    // coriolis flowmeter

    if reverse_flow {
        return -custom_k_value;
    }

    return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let flowmeter_20: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "flowmeter_20_21a".to_string(),
            2.79e-2, // component diameter in meters
            6.11e-4, // cross sectional area in meters sq
            0.36, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
                ↪ here,
                // but i need to fill in
            90.0 - 180.0, //incline angle in degrees
            &Flowmeter20::custom_darcy,
            &Flowmeter20::custom_k);

    return flowmeter_20;
}
}

pub struct Flowmeter20WithHighKCheckValve {
    // DHX flow flowmeter 20
    // natural convection heat exchanger in primary loop
    // diagram label is 21a
    // we use the convention of top of bypass branch to bottom
    // hence degree is -90
    //
    // However in DHX, i also expect there to be
    // a check valve which only allows flow from top to bottom
    //
    // That is the forward direction of flow for FM20,
    //

```

```

}
impl Flowmeter20WithHighKCheckValve {

    // let's import everything necessary:

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }

        let custom_k_value =
            18.0 + 93000.0/reynolds_number.powf(1.35);
        // coriolis flowmeter

        if reverse_flow {
            return -1.0e10/reynolds_number - 1.0e10;
        }

        return custom_k_value;
    }

    pub fn get() -> DowthermACustomComponent {

        let flowmeter_20: DowthermACustomComponent
            = StandardCustomComponentProperties::new(
                "flowmeter_20_21a".to_string(),
                2.79e-2, // component diameter in meters
                6.11e-4, // cross sectional area in meters sq
                0.36, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                // but i need to fill in

```

```

        90.0 - 180.0, //incline angle in degrees
        &Flowmeter20::custom_darcy,
        &Flowmeter20::custom_k);

    return flowmeter_20;
}
}

pub struct Pipe21 {
    // pipe 21
}

impl Pipe21 {

    pub fn get() -> DowthermAPipe {
        let pipe_21: DowthermAPipe
            = StandardPipeProperties::new(
                "static_mixer_pipe_21".to_string(),
                2.79e-2, // component diameter in meters
                0.487725, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                90.0-180.0, // angle in degrees
                4.4 // form loss K value
            );

        return pipe_21;
    }
}

pub struct Pipe20 {
    // pipe 20
}

impl Pipe20 {

    pub fn get() -> DowthermAPipe {
        let pipe_20: DowthermAPipe
            = StandardPipeProperties::new(

```



```

        "static_mixer_pipe_20".to_string(),
        2.79e-2, // component diameter in meters
        0.33655, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
            ↪ here,
            // but i need to fill in
            // in millimeters
        0.0 - 180.0, // angle in degrees
        0.0 // form loss K value
    );

    return pipe_20;
}

pub struct Pipe19 {
    // pipe 19
}

impl Pipe19 {

    pub fn get() -> DowthermAPipe {
        let pipe_19: DowthermAPipe
            = StandardPipeProperties::new(
                "static_mixer_pipe_19".to_string(),
                2.79e-2, // component diameter in meters
                0.219075, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                -31.44898 - 180.0, // angle in degrees
                7.5 // form loss K value
            );

        return pipe_19;
    }
}

/// bypass line times in ciet,

```

```

/// tbd, still organising
pub struct Flowmeter30 {
    // bypass flow flowmeter FM30
    // not labelled on diagram
    // we use the convention of top of bypass branch to bottom
    // hence degree is a 180-90 degrees = -90 degrees
}
impl Flowmeter30 {

    // let's import everything necessary:

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }

        let custom_k_value =
            18.0 + 93000.0/reynolds_number.powf(1.35);
        // coriolis flowmeter

        if reverse_flow {
            return -custom_k_value;
        }

        return custom_k_value;
    }

    pub fn get() -> DowthermACustomComponent {

        let flowmeter_30: DowthermACustomComponent
            = StandardCustomComponentProperties::new(
                "flowmeter_30".to_string(),

```

```

        2.79e-2, // component diameter in meters
        6.11e-4, // cross sectional area in meters sq
        0.36, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
            ↪ here,
            // but i need to fill in
        90.0 -180.0, //incline angle in degrees
        &Flowmeter30::custom_darcy,
        &Flowmeter30::custom_k);

    return flowmeter_30;
}
}

pub struct Flowmeter60 {
    // DHX flow flowmeter 60
    // natural convection heat exchanger in DRACS loop
    // this is the secondary loop equivalent for
    // decay heat removal
    //
    // diagram label 37a on simulation model
    // we use the convention of top of bypass branch to bottom (Tank 2)
    // hence degree is -90
}

impl Flowmeter60 {

    // let's import everything necessary:

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }
    }
}

```

```

    let custom_k_value =
      18.0 + 93000.0/reynolds_number.powf(1.35);
    // coriolis flowmeter

    if reverse_flow {
      return -custom_k_value;
    }

    return custom_k_value;
  }

  pub fn get() -> DowthermACustomComponent {

    let flowmeter_60: DowthermACustomComponent
      = StandardCustomComponentProperties::new(
        "flowmeter_60_37a".to_string(),
        2.79e-2, // component diameter in meters
        6.11e-4, // cross sectional area in meters sq
        0.36, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
          ↪ here,
          // but i need to fill in
        -90.0, //incline angle in degrees
        &Flowmeter60::custom_darcy,
        &Flowmeter60::custom_k);

    return flowmeter_60;
  }
}

/// static mixers are here
///
///

```

```

pub struct StaticMixer60 {
    // static mixer 60 (MX-60) on CIET diagram
    // in the NDHX branch in secondary DRACS loop
    // just after the NDHX heat exchanger
    // from top to bottom
    // ie this is where hot fluid gets cooled by a fan
    // label 36
    //
    // in reality flow goes from top to
    // bottom in natural convection
    //
}

impl StaticMixer60 {

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
    ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {
        let mut reverse_flow = false;

        // the user account for reverse flow scenarios...
        if reynolds_number < 0.0 {
            reverse_flow = true;
            reynolds_number = reynolds_number * -1.0;
        }

        let custom_k_value =
            21.0 + 4000.0/reynolds_number;

        if reverse_flow {
            return -custom_k_value;
        }

        return custom_k_value;
    }

    pub fn get() -> DowthermACustomComponent {

```

```

    let static_mixer_60: DowthermACustomComponent
    = StandardCustomComponentProperties::new(
        "static_mixer_60_label_36".to_string(),
        2.79e-2, // component diameter in meters
        6.11e-4, //component area in sq meters
        0.33, // component length in meters
        0.015, // estimated component wall roughness (doesn't matter
            ↪ here,
            // but i need to fill in
        -58.99728, //incline angle in degrees
        &StaticMixer60::custom_darcy,
        &StaticMixer60::custom_k);

    return static_mixer_60;
}
}

pub struct StaticMixer61 {
    // static mixer 61 (MX-61) on CIET diagram
    // in the DHX branch in secondary DRACS loop
    // just before the DHX heat exchanger
    // from top to bottom
    // ie this is where cool fluid gets heated by the
    // primary loop heat exchanger
    // label 31
    //
    // in reality flow goes from bottom to
    // top in natural convection
    // so it is actually after the DHX from perspective of flow
    //
}

impl StaticMixer61 {

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
    ↪ {
        return 0.0;
    }

    pub fn custom_k(mut reynolds_number: f64) -> f64 {
        let mut reverse_flow = false;

```

```

// the user account for reverse flow scenarios...
if reynolds_number < 0.0 {
    reverse_flow = true;
    reynolds_number = reynolds_number * -1.0;
}

let custom_k_value =
    21.0 + 4000.0/reynolds_number;

if reverse_flow {
    return -custom_k_value;
}

return custom_k_value;
}

pub fn get() -> DowthermACustomComponent {

    let static_mixer_61: DowthermACustomComponent
        = StandardCustomComponentProperties::new(
            "static_mixer_61_label_31".to_string(),
            2.79e-2, // component diameter in meters
            6.11e-4, //component area in sq meters
            0.33, // component length in meters
            0.015, // estimated component wall roughness (doesn't matter
                ↪ here,
                // but i need to fill in
            90.0 - 180.0, //incline angle in degrees
            &StaticMixer61::custom_darcy,
            &StaticMixer61::custom_k);

    return static_mixer_61;
}
}

/// test components
// pump and pipe combination

```

```

pub struct PumpWithResistance {
    // this is a pump with resistance
    // i can set the internal pressure term to some value
    // and i ensure that the pressure change between
    // the two ends is zero
    // i should see some flowrate
}

impl PumpWithResistance {

    // let's import everything necessary:

    pub fn custom_darcy(_reynolds_number: f64, _roughness_ratio: f64) -> f64
        ↪ {
        return 64.0/_reynolds_number;
    }

    pub fn custom_k(_reynolds_number: f64) -> f64 {
        return 5.25;
    }

    pub fn get(pressure_pascals: f64
        ) -> DowthermACustomComponent {

        let mut pump_with_resistance: DowthermACustomComponent
            = StandardCustomComponentProperties::new(
                "pump_with_resistance".to_string(),
                2.79e-2, // component diameter in meters
                6.11e-4, // cross sectional area in meters sq
                0.36, // component length in meters
                0.015, // estimated component wall roughness (doesn't matter
                    ↪ here,
                    // but i need to fill in
                    // in millimeters
                0.0, //incline angle in degrees
                &PumpWithResistance::custom_darcy,
                &PumpWithResistance::custom_k);

        pump_with_resistance.set_internal_pressure_term(
            pressure_pascals);

        return pump_with_resistance;
    }
}

```



```
}

```

B.5 Python Interface Code

In order to interface with the OPC-UA server written in Python, Rust code must be called via the maturin interface.

```
use pyo3::prelude::*;

/// Formats the sum of two numbers as string.
#[pyfunction]
fn sum_as_string(a: usize, b: usize) -> PyResult<String> {
    Ok((a + b).to_string())
}

#[pyfunction]
fn add_four_rust(x: f64) -> PyResult<f64>{
    return Ok(x + 4.0);
}

#[pyfunction]
fn fldk_rust(reynolds_number: f64, roughness_ratio: f64,
             length_to_diameter: f64,
             k: f64) -> PyResult<f64> {
    return Ok(fLDK(reynolds_number, roughness_ratio, length_to_diameter, k))
    ↪ ;
}

#[pyfunction]
fn moody_rust(reynolds_number: f64,
              roughness_ratio: f64) -> PyResult<f64> {
    return Ok(moody(reynolds_number, roughness_ratio));
}

#[pyfunction]
fn get_ctah_branch_isothermal_pressure_change_pascals_rust(
    mass_rate_kg_per_s: f64,
    temperature_degrees_c: f64,
    pump_pressure_pascals: f64) -> PyResult<f64> {
    return Ok(
        get_ctah_branch_isothermal_pressure_change_pascals(

```

```

        mass_rate_kg_per_s,
        temperature_degrees_c,
        pump_pressure_pascals));
}

#[pyfunction]
fn get_heater_branch_isothermal_pressure_change_pascals_rust(
    mass_rate_kg_per_s: f64,
    temperature_degrees_c: f64) -> PyResult<f64> {
    return Ok(
        get_heater_branch_isothermal_pressure_change_pascals(
            mass_rate_kg_per_s,
            temperature_degrees_c
        ));
}

#[pyfunction]
fn get_dhx_branch_isothermal_pressure_change_pascals_rust(
    mass_rate_kg_per_s: f64,
    temperature_degrees_c: f64) -> PyResult<f64> {
    return Ok(
        get_dhx_branch_isothermal_pressure_change_pascals(
            mass_rate_kg_per_s,
            temperature_degrees_c
        ));
}

#[pyfunction]
fn get_heater_branch_isothermal_hydrostatic_pressure_pascals_rust(
    temperature_degrees_c: f64) -> PyResult<f64> {

    return Ok(
        get_heater_branch_isothermal_hydrostatic_pressure_pascals(
            temperature_degrees_c
        ));
}

/// A Python module implemented in Rust.
#[pymodule]
fn rust_functions_in_python(_py: Python, m: &PyModule) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(sum_as_string, m)?)?;
    m.add_function(wrap_pyfunction!(add_four_rust, m)?)?;
}

```

```

m.add_function(wrap_pyfunction!(fldk_rust, m)?)?;
m.add_function(wrap_pyfunction!(moody_rust, m)?)?;
m.add_function(wrap_pyfunction!(
    get_ctah_branch_isothermal_pressure_change_pascals_rust,
    m)?)?;
m.add_function(wrap_pyfunction!(
    get_heater_branch_isothermal_pressure_change_pascals_rust,
    m)?)?;
m.add_function(wrap_pyfunction!(
    get_heater_branch_isothermal_hydrostatic_pressure_pascals_rust,
    m)?)?;
m.add_function(wrap_pyfunction!(
    get_dhx_branch_isothermal_pressure_change_pascals_rust,
    m)?)?;
Ok(())
}

// here are the functions used for friction factor, rather messy but
// for fast prototyping and sandboxing don't really care too much
//
//

/// code for ciet isothermal digital twin
///

fn get_dhx_branch_isothermal_pressure_change_pascals(
    mass_rate_kg_per_s: f64,
    temperature_degrees_c: f64) -> f64 {
    use fluid_mechanics_rust::therminol_component::factory;
    use uom::si::mass_rate::kilogram_per_second;
    use uom::si::thermodynamic_temperature::degree_celsius;
    use uom::si::pressure::pascal;
    use fluid_mechanics_rust::therminol_component::CalcPressureChange;

    use uom::si::f64::*;
    // fluid temp and pressure
    let fluid_temp = ThermodynamicTemperature::new::<
        degree_celsius>(temperature_degrees_c);
    let mass_flowrate = MassRate::new::<
        kilogram_per_second>(mass_rate_kg_per_s);

```

```

// pipe 26, static mixer pipe 25a and static mixer 25
let pipe_26 = factory::Pipe26::get();
let mx21_25 = factory::StaticMixer21::get();
let pipe_25a = factory::Pipe25a::get();

// DHX heat exchanger
let dhx_shell_side_24 = factory::DHXShellSideHeatExchanger::get();

// static mixer pipe 23a, static mixer 23, and pipe 22
let mx20_23 = factory::StaticMixer20::get();
let pipe_23a = factory::Pipe23a::get();
let pipe_22 = factory::Pipe22::get();

// flowmeter 21a (FM-20)
let flowmeter_20_21a = factory::Flowmeter20::get();

// pipe 21 to 19
let pipe_21 = factory::Pipe21::get();
let pipe_20 = factory::Pipe20::get();
let pipe_19 = factory::Pipe19::get();

let mut pressure_change_total =
    Pressure::new::<pascal>(0.0);

// pipe 26, static mixer pipe 25a and static mixer 25
pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_26,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &mx21_25,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(

```

```

        &pipe_25a,
        mass_flowrate,
        fluid_temp);

// DHX heat exchanger
//
pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &dhx_shell_side_24,
        mass_flowrate,
        fluid_temp);

// static mixer pipe 23a, static mixer 23, and pipe 22
pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &mx20_23,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_23a,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_22,
        mass_flowrate,
        fluid_temp);

// flowmeter 21a (FM-20)

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &flowmeter_20_21a,
        mass_flowrate,
        fluid_temp);

// pipe 21 to 19

pressure_change_total = pressure_change_total +

```

```

        CalcPressureChange::from_mass_rate(
            &pipe_21,
            mass_flowrate,
            fluid_temp);

    pressure_change_total = pressure_change_total +
        CalcPressureChange::from_mass_rate(
            &pipe_20,
            mass_flowrate,
            fluid_temp);
    pressure_change_total = pressure_change_total +
        CalcPressureChange::from_mass_rate(
            &pipe_19,
            mass_flowrate,
            fluid_temp);

    // convert to f64 and return
    return pressure_change_total.get::<pascal>();
}

fn get_heater_branch_isothermal_pressure_change_pascals(
    mass_rate_kg_per_s: f64,
    temperature_degrees_c: f64) -> f64 {
    //import necessary things...
    use fluid_mechanics_rust::therminol_component::factory;
    use uom::si::mass_rate::kilogram_per_second;
    use uom::si::thermodynamic_temperature::degree_celsius;
    use uom::si::pressure::pascal;
    use fluid_mechanics_rust::therminol_component::CalcPressureChange;

    use uom::si::f64::*;
    // fluid temp and pressure
    let fluid_temp = ThermodynamicTemperature::new::<
        degree_celsius>(temperature_degrees_c);
    let mass_flowrate = MassRate::new::<
        kilogram_per_second>(mass_rate_kg_per_s);

    // let's get and pipe 4
    //
    let pipe_4 = factory::Pipe4::get();

```

```

// lets get pipe 3 and static mixer 2 and pipe 2a
let pipe_3 = factory::Pipe3::get();
let mx10_2 = factory::StaticMixer10::get();
let pipe_2a = factory::Pipe2a::get();

// let's get the heater components 1a, 1 and 1b
let heater_top_head_1a =
    factory::HeaterTopHead1a::get();
let heater_version_1_1 =
    factory::CietHeaterVersion1::get();
let heater_bottom_head_label_1b =
    factory::HeaterBottomHead1b::get();

// now we'll get pipe 18

let pipe_18 = factory::Pipe18::get();

// now that we've gotten our items, we can
// then sum up the pressure change contributions
// given

let mut pressure_change_total =
    Pressure::new::(<pascal>(0.0));

// pipe4
//

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_4,
        mass_flowrate,
        fluid_temp);

// pipe 3 and static mixer 2 and pipe 2a
//
pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_3,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +

```

```

        CalcPressureChange::from_mass_rate(
            &mx10_2,
            mass_flowrate,
            fluid_temp);

    pressure_change_total = pressure_change_total +
        CalcPressureChange::from_mass_rate(
            &pipe_2a,
            mass_flowrate,
            fluid_temp);

    // heater
    //
    pressure_change_total = pressure_change_total +
        CalcPressureChange::from_mass_rate(
            &heater_top_head_1a,
            mass_flowrate,
            fluid_temp);

    pressure_change_total = pressure_change_total +
        CalcPressureChange::from_mass_rate(
            &heater_version_1_1,
            mass_flowrate,
            fluid_temp);

    pressure_change_total = pressure_change_total +
        CalcPressureChange::from_mass_rate(
            &heater_bottom_head_label_1b,
            mass_flowrate,
            fluid_temp);

    //pipe 18
    //
    pressure_change_total = pressure_change_total +
        CalcPressureChange::from_mass_rate(
            &pipe_18,
            mass_flowrate,
            fluid_temp);

    // convert the object to f64 and return
    return pressure_change_total.get::<pascal>();
}

```



```

// get hydrostatic pressure change
// for heater branch
//
//

fn get_heater_branch_isothermal_hydrostatic_pressure_pascals(
    temperature_degrees_c: f64) -> f64 {
    //import necessary things...
    use fluid_mechanics_rust::therminol_component::factory;
    use uom::si::thermodynamic_temperature::degree_celsius;
    use uom::si::pressure::pascal;
    use fluid_mechanics_rust::therminol_component::
        StandardCustomComponentProperties;
    use fluid_mechanics_rust::therminol_component::
        StandardPipeProperties;

    use uom::si::f64::*;
    // fluid temp and pressure
    let fluid_temp = ThermodynamicTemperature::new::<
        degree_celsius>(temperature_degrees_c);

    // let's get pipe 4
    //
    let pipe_4 = factory::Pipe4::get();

    // lets get pipe 3 and static mixer 2 and pipe 2a
    let pipe_3 = factory::Pipe3::get();
    let mx10_2 = factory::StaticMixer10::get();
    let pipe_2a = factory::Pipe2a::get();

    // let's get the heater components 1a, 1 and 1b
    let heater_top_head_1a =
        factory::HeaterTopHead1a::get();
    let heater_version_1_1 =
        factory::CietHeaterVersion1::get();
    let heater_bottom_head_label_1b =
        factory::HeaterBottomHead1b::get();

    // now we'll get pipe 18

```

```

let pipe_18 = factory::Pipe18::get();

// now that we've gotten our items, we can
// then sum up the pressure change contributions
// given

let mut hydrostatic_pressure_change_total =
    Pressure::new::<pascal>(0.0);

// branch5 and pipe4

hydrostatic_pressure_change_total =
    hydrostatic_pressure_change_total +
    pipe_4.get_hydrostatic_pressure_change(
        fluid_temp);

// pipe 3 and static mixer 2 and pipe 2a
//
hydrostatic_pressure_change_total =
    hydrostatic_pressure_change_total +
    pipe_3.get_hydrostatic_pressure_change(
        fluid_temp);

hydrostatic_pressure_change_total =
    hydrostatic_pressure_change_total +
    mx10_2.get_hydrostatic_pressure_change(
        fluid_temp);

hydrostatic_pressure_change_total =
    hydrostatic_pressure_change_total +
    pipe_2a.get_hydrostatic_pressure_change(
        fluid_temp);

// heater
//
hydrostatic_pressure_change_total =
    hydrostatic_pressure_change_total +
    heater_top_head_1a.get_hydrostatic_pressure_change(
        fluid_temp);

hydrostatic_pressure_change_total =
    hydrostatic_pressure_change_total +

```

```

        heater_version_1_1.get_hydrostatic_pressure_change(
            fluid_temp);

hydrostatic_pressure_change_total =
    hydrostatic_pressure_change_total +
    heater_bottom_head_label_1b.get_hydrostatic_pressure_change(
        fluid_temp);

//pipe 18
//
hydrostatic_pressure_change_total =
    hydrostatic_pressure_change_total +
    pipe_18.get_hydrostatic_pressure_change(
        fluid_temp);

// convert the object to f64 and return
return hydrostatic_pressure_change_total.get::<pascal>();
}

// get ctah branch pressure

fn get_ctah_branch_isothermal_pressure_change_pascals(
    mass_rate_kg_per_s: f64,
    temperature_degrees_c: f64,
    pump_pressure_pascals: f64) -> f64 {

    //import necessary things...
    use fluid_mechanics_rust::therminol_component::factory;
    use uom::si::mass_rate::kilogram_per_second;
    use uom::si::thermodynamic_temperature::degree_celsius;
    use uom::si::pressure::pascal;
    use fluid_mechanics_rust::therminol_component::CalcPressureChange;

    use uom::si::f64::*;
    // fluid temp and pressure
    let fluid_temp = ThermodynamicTemperature::new::<
        degree_celsius>(temperature_degrees_c);
    let mass_flowrate = MassRate::new::<
        kilogram_per_second>(mass_rate_kg_per_s);

    // let's get branch 5 and pipe 6a and static mixer 41

```

```

// which is pipe 6 on diagram
//
let branch_5 = factory::Branch5::get();
let pipe_6a = factory::Pipe6a::get();
let static_mixer_41_6 = factory::StaticMixer41::get();

// let's get the component for ctah
let ctah_vertical_7a = factory::CTAHVertical::get();
let ctah_horizontal_7b = factory::CTAHHorizontal::get();

// let's get the static mixer and pipe 8 and 8a

let pipe_8a = factory::Pipe8a::get();
let static_mixer_40_8 = factory::StaticMixer40::get();

// now let's get pipe 9 to 12

let pipe_9 = factory::Pipe9::get();
let pipe_10 = factory::Pipe10::get();
let pipe_11 = factory::Pipe11::get();
let pipe_12 = factory::Pipe12::get();

// and now for the pump
let ctah_pump = factory::CTAHPump::get(
    pump_pressure_pascals);

// let's now get pipe 13 and 14
let pipe_13 = factory::Pipe13::get();
let pipe_14 = factory::Pipe14::get();

// let's get flowmeter 14a
let flowmeter_40_14a = factory::Flowmeter40::get();

// let's get pipe 15 and 16
let pipe_15 = factory::Pipe15::get();
let pipe_16 = factory::Pipe16::get();

// let's now get branch 17
let branch_17 = factory::Branch17::get();

// now that we've gotten our items, we can
// then sum up the pressure change contributions

```

```

// given

let mut pressure_change_total =
    Pressure::new::<pascal>(0.0);

// branch 5 pipe 6a, static mixer 6,
// ctah 7a, 7b and static mixer 8 and pipe 8a
pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &branch_5,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_6a,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &static_mixer_41_6,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &ctah_vertical_7a,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &ctah_horizontal_7b,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_8a,
        mass_flowrate,
        fluid_temp);

```

```

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &static_mixer_40_8,
        mass_flowrate,
        fluid_temp);

// pipe 9 tp 12

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_9,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_10,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_11,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_12,
        mass_flowrate,
        fluid_temp);

// ctah pump
pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &ctah_pump,
        mass_flowrate,
        fluid_temp);

// pipe 13 to 17

```

```

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_13,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_14,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &flowmeter_40_14a,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_15,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &pipe_16,
        mass_flowrate,
        fluid_temp);

pressure_change_total = pressure_change_total +
    CalcPressureChange::from_mass_rate(
        &branch_17,
        mass_flowrate,
        fluid_temp);

return pressure_change_total.get::<pascal>();
}

/// code for friction factor

// first, to allow non snake case names..

```

```

#[allow(non_snake_case)]
fn B(Re: f64) -> f64 {
    let numerator = 37530.0_f64.powf(16.0);
    let denominator = Re.powf(16.0);
    return numerator/denominator;
}

#[allow(non_snake_case)]
fn A(Re: f64, roughnessRatio: f64) -> f64 {
    let seven_over_Re = 7.0/Re;
    let reynolds_term = seven_over_Re.powf(0.9);

    let roughness_term = 0.27 * roughnessRatio;

    let log_fraction = 1.0/(reynolds_term + roughness_term);
    // we will need natural logarithms:
    let inner_bracket_term = 2.457*log_fraction.ln();

    let A = inner_bracket_term.powf(16.0);

    return A;
}

#[allow(non_snake_case)]
fn churchillInnerTerm(Re: f64, roughnessRatio: f64) -> f64 {

    let eight_over_Re = 8.0/Re;
    let laminarTerm = eight_over_Re.powf(12.0);

    let Aterm = A(Re,roughnessRatio);
    let Bterm = B(Re);

    let APlusBInverse = 1.0/(Aterm+Bterm);
    let turbulentTerm = APlusBInverse.powf(3.0/2.0);

    return laminarTerm + turbulentTerm;
}

// this particular implementation uses the churchill correlation
#[allow(non_snake_case)]

```



```

fn fanning(ReynoldsNumber: f64, roughnessRatio: f64) -> f64{

    if ReynoldsNumber == 0.0 {
        panic!("Re = 0.0");
    }

    if ReynoldsNumber < 0.0 {
        panic!("Re<0.0");
    }

    if roughnessRatio < 0.0 {
        panic!("roughnessRatio<0.0");
    }

    let innerTerm = churchillInnerTerm(ReynoldsNumber, roughnessRatio);
    let powerTerm = innerTerm.powf(1.0/12.0);
    let fanningFrictionFactor = 2.0 * powerTerm;
    return fanningFrictionFactor;
}

#[allow(non_snake_case)]
fn darcy(ReynoldsNumber: f64, roughnessRatio: f64) -> f64 {
    return 4.0*fanning(ReynoldsNumber, roughnessRatio);
}

#[allow(non_snake_case)]
fn moody(ReynoldsNumber: f64, roughnessRatio: f64) -> f64 {
    return 4.0*fanning(ReynoldsNumber, roughnessRatio);
}

#[allow(non_snake_case)]
fn fLDK(ReynoldsNumber: f64,
        roughnessRatio: f64,
        lengthToDiameterRatio: f64,
        K: f64) -> f64{
    if ReynoldsNumber == 0.0 {
        panic!("Re = 0");
    }

    if ReynoldsNumber < 0.0 {
        panic!("Re < 0");
    }

```

```

}

if roughnessRatio < 0.0 {
    panic!("roughnessRatio<0.0");
}

if lengthToDiameterRatio <= 0.0 {
    panic!("lengthToDiameterRatio<=0.0");
}

if K < 0.0 {
    panic!("For m loss coefficient K < 0.0");
}

let f = darcy(ReynoldsNumber, roughnessRatio);
let fLDK = f*lengthToDiameterRatio + K;

return fLDK;
}

```

B.6 OPC-UA Server and Calculation

Python asyncio OPC-UA NOTICE

The OPC-UA server is written in Python 3 and licensed using the GNU Lesser General Public License v3.0.

GNU LESSER GENERAL PUBLIC LICENSE Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <<http://fsf.org/>> Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

This version of the GNU Lesser General Public License incorporates the terms and conditions of version 3 of the GNU General Public License, supplemented by the additional permissions listed below.

0. Additional Definitions.

As used herein, "this License" refers to version 3 of the GNU Lesser General Public License, and the "GNU GPL" refers to version 3 of the GNU General Public License.

"The Library" refers to a covered work governed by this License, other than an Application or a Combined Work as defined below.

An "Application" is any work that makes use of an interface provided by the Library, but which is not otherwise based on the Library. Defining a subclass of a class defined by the Library is deemed a mode of using an interface provided by the Library.

A "Combined Work" is a work produced by combining or linking an Application with the Library. The particular version of the Library with which the Combined Work was made is also called the "Linked Version".

The "Minimal Corresponding Source" for a Combined Work means the Corresponding Source for the Combined Work, excluding any source code for portions of the Combined Work that, considered in isolation, are based on the Application, and not on the Linked Version.

The "Corresponding Application Code" for a Combined Work means the object code and/or source code for the Application, including any data and utility programs needed for reproducing the Combined Work from the Application, but excluding the System Libraries of the Combined Work.

1. Exception to Section 3 of the GNU GPL.

You may convey a covered work under sections 3 and 4 of this License without being bound by section 3 of the GNU GPL.

2. Conveying Modified Versions.

If you modify a copy of the Library, and, in your modifications, a facility refers to a function or data to be supplied by an Application that uses the facility (other than as an argument passed when the facility is invoked), then you may convey a copy of the modified version:

- a) under this License, provided that you make a good faith effort to ensure that, in the event an Application does not supply the function or data, the facility still operates, and performs whatever part of its purpose remains meaningful, or
- b) under the GNU GPL, with none of the additional permissions of this License applicable to that copy.

3. Object Code Incorporating Material from Library Header Files.

The object code form of an Application may incorporate material from a header file that is part of the Library. You may convey such object code under terms of your choice, provided that, if the incorporated material is not limited to numerical parameters, data structure layouts and accessors, or small macros, inline functions and templates (ten or fewer lines in length), you do both of the following:

- a) Give prominent notice with each copy of the object code that the Library is used in it and that the Library and its use are covered by this License.
- b) Accompany the object code with a copy of the GNU GPL and this license document.

4. Combined Works.

You may convey a Combined Work under terms of your choice that, taken together, effectively do not restrict modification of the portions of the Library contained in the Combined Work and reverse engineering for debugging such modifications, if you also do each of the following:

- a) Give prominent notice with each copy of the Combined Work that the Library is used in it and that the Library and its use are covered by this License.
- b) Accompany the Combined Work with a copy of the GNU GPL and this license document.

c) For a Combined Work that displays copyright notices during execution, include the copyright notice for the Library among these notices, as well as a reference directing the user to the copies of the GNU GPL and this license document.

d) Do one of the following:

0) Convey the Minimal Corresponding Source under the terms of this License, and the Corresponding Application Code in a form suitable for, and under terms that permit, the user to recombine or relink the Application with a modified version of the Linked Version to produce a modified Combined Work, in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.

1) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (a) uses at run time a copy of the Library already present on the user's computer system, and (b) will operate properly with a modified version of the Library that is interface-compatible with the Linked Version.

e) Provide Installation Information, but only if you would otherwise be required to provide such information under section 6 of the GNU GPL, and only to the extent that such information is necessary to install and execute a modified version of the Combined Work produced by recombining or relinking the Application with a modified version of the Linked Version. (If you use option 4d0, the Installation Information must accompany the Minimal Corresponding Source and Corresponding Application Code. If you use option 4d1, you must provide the Installation Information in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.)

5. Combined Libraries.

You may place library facilities that are a work based on the Library side by side in a single library together with other library facilities that are not Applications and are not covered by this License, and convey such a combined library under terms of your choice, if you do both of the following:

a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities, conveyed under the terms of this License.

b) Give prominent notice with the combined library that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

6. Revised Versions of the GNU Lesser General Public License.

The Free Software Foundation may publish revised and/or new versions of the GNU Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library as you received it specifies that a certain numbered version of the GNU Lesser General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that published version or of any later version published by the Free Software Foundation. If the Library as you received it does not specify a version number of the GNU Lesser General Public License, you may choose any version of the GNU Lesser General Public License ever published by the Free Software Foundation.

If the Library as you received it specifies that a proxy can decide whether future versions of the GNU Lesser General Public License shall apply, that proxy's public statement of acceptance of any version is permanent authorization for you to choose that version for the Library.

Server Code

The OPC-UA Server code used for tests with the DHX loop is as follows. It is not optimised, but rather is a prototype. As such, there is residual code from earlier attempts with sandboxing that I have not cleaned up yet. Nevertheless, the code is still able to provide satisfactory performance as shown in this thesis.

```
from asyncua import ua, Server
from asyncua.common.methods import uamethod

import logging
import asyncio
import sys
sys.path.insert(0, "..")

# importing rust functions in maturin
import rust_functions_in_python

# get ip address
import socket

# this is for iterative methods
import scipy
import numpy

# this is for timing
import time

# uamethods are methods meant for clients to invoke,
# not necessarily on the server side
@uamethod
def func(parent, value):
    return value * 2

# add two to whatever the value is
```

```

def add2(value):
    return value + 2.0

def m42_height_meters(mass_flowrate_kg_per_s):
    return -20.4218536540637* mass_flowrate_kg_per_s**2 - 0.874137365300828
    ↪ * mass_flowrate_kg_per_s + 1

def m43_height_meters(mass_flowrate_kg_per_s):
    return 10.1573390552631* mass_flowrate_kg_per_s**2 + 2.33678270779408 *
    ↪ mass_flowrate_kg_per_s + 1

def referenceDensity20C_kg_per_m3():
    # this is in kg per m3
    return 1061

def expt_pressure_loss_pascals(mass_rate_kg_per_s):
    delta_h = (m43_height_meters(mass_rate_kg_per_s) - m42_height_meters(
    ↪ mass_rate_kg_per_s))
    rho = referenceDensity20C_kg_per_m3()
    g = 9.81
    return delta_h * rho * g

# calling rust functions
def rustAdd4(value):
    return rust_functions_in_python.add_four_rust(value)

# frictionFactor in rust
def rustDarcy(ReynoldsNumber,roughnessRatio):
    return rust_functions_in_python.moody_rust(ReynoldsNumber,
    roughnessRatio)

# Digital Twin Isothermal Ciet functions
# implemented in rust

# these are functions straight from rust
# they get a pressure change given a set mass flowrate
# and temperature
def rust_get_heater_branch_pressure_change(
    mass_rate_kg_per_s,
    temperature_degrees_c):
    return rust_functions_in_python.
    ↪ get_heater_branch_isothermal_pressure_change_pascals_rust(

```

```

        mass_rate_kg_per_s,
        temperature_degrees_c)

def rust_get_ctah_branch_pressure_change(
    mass_rate_kg_per_s,
    temperature_degrees_c,
    pump_pressure_pascals):
    return rust_functions_in_python.
        ↪ get_ctah_branch_isothermal_pressure_change_pascals_rust(
            mass_rate_kg_per_s,
            temperature_degrees_c,
            pump_pressure_pascals)

def rust_get_dhx_branch_pressure_change(
    mass_rate_kg_per_s,
    temperature_degrees_c):
    return rust_functions_in_python.
        ↪ get_dhx_branch_isothermal_pressure_change_pascals_rust(
            mass_rate_kg_per_s,
            temperature_degrees_c)

def rust_get_heater_branch_hydrostatic_pressure(
    temperature_degrees_c):
    return rust_functions_in_python.
        ↪ get_heater_branch_isothermal_hydrostatic_pressure_pascals_rust(
            temperature_degrees_c)

# these are functions utilising the rust ci et functions
# but they will give mass flowrate given a pressure change
# using brent method from scipy
# which should be fast
def get_heater_branch_mass_flowrate(
    pressure_change_pascals,
    temperature_degrees_c):

    # basically im solving for the mass rate which
    # returns the correct pressure change
    def heater_pressure_chg_root(
        mass_rate_kg_per_s):
        return rust_get_heater_branch_pressure_change(
            mass_rate_kg_per_s,
            temperature_degrees_c) - pressure_change_pascals

```

```

mass_flowrate_kg_per_s_solution = scipy.optimize.brentq(
    heater_pressure_chg_root,
    -1.0,
    1.0)

return mass_flowrate_kg_per_s_solution

def get_ctah_branch_mass_flowrate(
    pressure_change_pascals,
    temperature_degrees_c,
    pump_pressure_pascals):

    # basically im solving for the mass rate which
    # returns the correct pressure change
    def ctah_pressure_chg_root(
        mass_rate_kg_per_s):
        return rust_get_ctah_branch_pressure_change(
            mass_rate_kg_per_s,
            temperature_degrees_c,
            pump_pressure_pascals) - pressure_change_pascals

    mass_flowrate_kg_per_s_solution = scipy.optimize.brentq(
        ctah_pressure_chg_root,
        -1.0,
        1.0)

    return mass_flowrate_kg_per_s_solution

def get_dhx_branch_mass_flowrate(
    pressure_change_pascals,
    temperature_degrees_c):

    # first let's check for reverse flow and return 0
    # flow if reverse, it's computationally cheaper
    hydrostatic_pressure = rust_get_heater_branch_hydrostatic_pressure(
        temperature_degrees_c)

    if (pressure_change_pascals > hydrostatic_pressure ):
        return 0.0

```



```

# basically im solving for the mass rate which
# returns the correct pressure change
def dhx_pressure_chg_root(
    mass_rate_kg_per_s):
    return rust_get_dhx_branch_pressure_change(
        mass_rate_kg_per_s,
        temperature_degrees_c) - pressure_change_pascals

mass_flowrate_kg_per_s_solution = scipy.optimize.brentq(
    dhx_pressure_chg_root,
    -1.0,
    1.0)

return mass_flowrate_kg_per_s_solution

# these methods solve for mass flowrate given a pump pressure
# and system temperature

def get_ciet_isothermal_mass_flowrate(
    pump_pressure_pascals,
    temperature_degrees_c):
    # the job of this function is to sum up the mass
    # flowrate of the branches in ciet
    # and solve for the value where the branch flowrates
    # sum up to zero
    # the convention is positive flowrate leaving the
    # top and negative flowrate entering the top

    def pressure_change_root(pressure_change_pascals):
        # both branches must be subject to the same
        # pressure change since they are in parallel
        ctah_branch_flowrate = get_ctah_branch_mass_flowrate(
            ↪ pressure_change_pascals,
            temperature_degrees_c,
            pump_pressure_pascals)

        heater_branch_mass_flowrate = get_heater_branch_mass_flowrate(
            pressure_change_pascals,
            temperature_degrees_c)

```

```

    dhx_branch_mass_flowrate = get_dhx_branch_mass_flowrate(
        pressure_change_pascals,
        temperature_degrees_c)

    dhx_and_heater_flowrate = numpy.add(dhx_branch_mass_flowrate,
        heater_branch_mass_flowrate)

    total_mass_flowrate = numpy.add(dhx_and_heater_flowrate,
        ctah_branch_flowrate)

    return total_mass_flowrate

# this scipy module will solve for the correct pressure change
# given the mass flowrate
# the intervals will be 50000 pascals plus or minus
# the hydrostatic pressure change of the heater

upper_bound = rust_get_heater_branch_hydrostatic_pressure(
    temperature_degrees_c) + 50000

lower_bound = rust_get_heater_branch_hydrostatic_pressure(
    temperature_degrees_c) - 50000

pressure_change_value = scipy.optimize.brentq(
    pressure_change_root,
    lower_bound,
    upper_bound)

# once we get the pressure change root value,
# we can get mass flowrate

ctah_branch_mass_flowrate = get_ctah_branch_mass_flowrate(
    pressure_change_value,
    temperature_degrees_c,
    pump_pressure_pascals)

return ctah_branch_mass_flowrate

```

```

# get ip address automatically
def getIPAddress():
    hostname = socket.gethostname()
    IPAddr = socket.gethostbyname(hostname)
    return IPAddr

async def main():
    _logger = logging.getLogger('asyncua')
    # setup our server
    server = Server()
    await server.init()
    server.set_endpoint('opc.tcp://'+getIPAddress()+':4840/freeopcua/server
        ↪ /')
    ## this IP address is for lab
    #server.set_endpoint('opc.tcp://192.168.10.177:4840/freeopcua/server/')

    # setup our own namespace, not really necessary but should as spec
    uri = 'http://examples.freeopcua.github.io'
    idx = await server.register_namespace(uri)

    # populating our address space
    # server.nodes, contains links to very common nodes like objects and
    ↪ root
    myobj = await server.nodes.objects.add_object(idx, 'MyObject')
    myvar = await myobj.add_variable(idx, 'MyVariable', 6.7)
    # Set MyVariable to be writable by clients
    await myvar.set_writable()
    await server.nodes.objects.add_method(
        ua.NodeId('ServerMethod', 2),
        ua.QualifiedName('ServerMethod', 2),
        func,
        [ua.VariantType.Int64],
        [ua.VariantType.Int64])

    ## here's where i add my reynolds number
    pipeObj = await server.nodes.objects.add_object(idx, 'pipeObj')

    ctah_pump_pressure = await pipeObj.add_variable(

```

```

        idx, 'ctah_pump_pressure_pascal', 16000.0)
await ctah_pump_pressure.set_writable()

ciet_temperature_degC = await pipeObj.add_variable(
    idx, 'ciet_temperature_degC',
    20.0)

ctah_flowrate = await pipeObj.add_variable(
    idx, 'ctah_mass_flowrate_kg_per_s', 0.0)

# i want to add a simulation time variable
simulation_time_ms = await pipeObj.add_variable(
    idx, 'simulation_time_ms',
    0.0)

roughnessRatio = 0.00015

_logger.info('Starting server!')

# server loop
async with server:
    while True:

        start = time.time()

        _logger.info('\n \n \n')
        _logger.info('to connect:')

        _logger.info('opc.tcp://'+getIPAddress()+':4840/freeopcua/server
            ↪ /')
        _logger.info('\n')

        # this is the main ciet digital twin calculation
        pump_pressure_pascals = await ctah_pump_pressure.get_value()
        fluid_temp_degC = await ciet_temperature_degC.get_value()
        mass_flowrate_kg_per_s = get_ciet_isothermal_mass_flowrate(
            pump_pressure_pascals,
            fluid_temp_degC)

```

```

        await ctah_flowrate.write_value(mass_flowrate_kg_per_s)

        _logger.info('pump_pressure_pascals: %.1f',
                     pump_pressure_pascals)
        _logger.info('ctah_mass_flowrate_kg_per_s: %.4f',
                     await ctah_flowrate.get_value())

        # this is the experimental ciet pressure loss at
        # 20C
        end = time.time()

        elapsed_loop_time_seconds = (end - start)
        await simulation_time_ms.write_value(
            elapsed_loop_time_seconds*1000)
        _logger.info('elapsed_loop_time_seconds (ms) %.1f',
                     await simulation_time_ms.get_value())

        wait_time = 1.0

        sleep_time = wait_time - elapsed_loop_time_seconds

        await asyncio.sleep(sleep_time)

if __name__ == '__main__':

    logging.basicConfig(level=logging.DEBUG)

    asyncio.run(main(), debug=True)

```

Appendix C

Supplemental Logs and Discussion for Main Text

C.1 Literature Review Supplemental Discussion

Justification for Type I, Type II and Type III Digital Twin

In the literature review, I provide some justification as to why I used a new “Type I”, “Type II” and “Type III” naming convention rather than use what was already available in literature. In this part of the appendix, I provide further justification as to why I use this particular naming convention.

Why I include Plant Simulators as a Digital Twin

One justification of using the proposed naming convention could come from how the word twin is used colloquially. For digital twins, the language of the word twin brings about the idea of making a copy or describing two individuals who look almost identical to each other regardless of whether they keep in contact for the rest of their lives. While we would also consider that twins do not need to look identical, we would normally distinguish such twins by adding the qualifier “fraternal” or the technical term “dizygotic” to it. [Lupton, 2021] When we use the word twin without any qualifier, the more common implied assumption is that the twins are identical. Also the twins do not need to stay in contact with each other throughout their lives to be considered twins.

I would suggest that likewise, digital twins should not need to have one-way or two-way data exchange with the physical asset they attempt to emulate to be considered a twin. I would simply say they are Type I digital twins. So, real-time system models used in simulators for operator training can be considered Type I digital twins. I would then go by the definition given by Boschert and Rosen [Boschert and Rosen, 2016]. In this definition, a digital twin is used to predict the behaviour of a real system in both acquisition phase (includes design) and operation phase [Boschert and Rosen, 2016]. Such intuition is also

used in the same way in literature; Several authors would agree they do not define digital twins such that digital twins are digital twins if and only if they share a two way data connection with the physical asset that they are meant to replicate [Chen, 2017; Rasheed, San, and Kvamsdal, 2020; Eckhart and Ekelhart, 2018].

Why I do not define a Digital Twin by when it is used in the Product Life Cycle

Finally, I would like to provide further justification for using such a naming convention as opposed to using other naming conventions. As mentioned earlier, the convention of narrowing down the definition of a digital twin such that a digital twin must have real-time one-way or two-way data exchange is more popular in literature than the convention that does not necessitate this data exchange. [Van der Valk et al., 2020; Kochunas and Huan, 2021]. However, this naming convention would be largely incompatible with other works in literature which do not conform to this definition. Therefore, I opt not to use this narrow definition but rather have digital twins categorised into Type I, Type II and Type III. This is not the only typing system used in literature, however.

One example is the use of the digital twin types α and β to denote when the digital twin is used in the life cycle of the technological process [Kholopov et al., 2019]. In this naming convention, an α digital twin is used to describe the technological process before it is implemented to manufacture a specific product while a β digital twin is used for monitoring the technological process after development is complete [Kholopov et al., 2019]. In this regard, the naming convention is tied more closely to the life cycle stage of the process or product rather than the end goal of the digital twin. For the end user of the digital twin, however, the specific use case of the digital twin may be of more immediate interest than the life cycle. At the end of the day, the user of the digital twin is seeking a specific benefit provided by the digital twin and considering the cost required to implement such a digital twin. In this regard, information about the life cycle of the digital twin may be of less immediate use to the user. In digital twin taxonomy then, we could develop a naming convention based solely upon the end use case of the digital twin. However, the use cases are so varied that it would be cumbersome for the user to get acquainted with all the terms. The middle ground is then to grade digital twins by their capabilities and relative cost of development. Capabilities and cost of development is tied closest to the degree of synchronisation required for digital twins. Therefore a typing system that is based on the digital twin's degree of synchronisation with the physical asset would make sense.

A summary of the types of digital twins and how they relate to other similar terms in literature is provided in Table 2.1. It should be noted that the α digital twin here is closest conceptually to the Type I digital twin since both are used in the development phase where debugging and implementation of the process is carried out. Whereas β digital twins are perhaps closely related to Type II digital twin since data about the technological process is sent to the operator in real-time [Kholopov et al., 2019]. The operator is then free to decide the actions to be performed thereafter.

Model Predictive Control vs Fault Tolerant Control

To readers familiar with process control and controller design, the use of digital twins to design fault tolerant controllers bears much resemblance to model based predictive control [Jones et al., 2020]. Therefore, there is a need to distinguish them to provide the reader with some degree of clarity.

Model Predictive Control (MPC) is the process of using a model to inform how controllers should behave so that the process can carry on in a stable, safe and profitable manner [Jones et al., 2020]. For example, internal model control (IMC) is a well established method of designing controllers for the petrochemical industry [Garcia, Prett, and Morari, 1989]. IMC takes advantage of a mathematical model of the process, such as a transfer function or state space model, and uses that to inform controller design. This method of using explicit process models to develop controllers is known as model predictive control (MPC). IMC is just one method in the MPC family [Garcia, Prett, and Morari, 1989]. And at a basic level, most IMC controllers are based on linear transfer function models of the process. In contrast to IMC, MPC allows us to use any form of mathematical model to help inform controller design. Since digital twins are a subset of mathematical process models, they can also be used in MPC controller design.

However, MPC methods are not necessarily fault tolerant. Basic Internal Model Control (IMC) based controllers for instance, do not necessarily account for system faults. Transfer functions used in IMC might reflect how the system behaves in the normal manner.

However, during system faults, the appropriate transfer function needed to describe the system might change. If this model is not updated to reflect the system fault, the IMC based controller may not be able to bring the system to a stable and safe state. Such a system is not fault tolerant. However, if the transfer function and IMC controller is updated in real-time to reflect system faults, we could perhaps consider such a control system fault tolerant.

In the same way, the use of digital twins in MPC methods may not necessarily guarantee fault tolerance. These Type III Digital Twins have to be synchronised in real-time to the physical system in order to be used for this use case.

Distinguishing Type II and Type III Digital Twins In the previous example of describing a β digital twin, which I denoted as a Type II digital twin, real-time data is sent from the plant to the digital twin so that the digital twin can use some internal model to estimate information about the process or plant that sensors are unable to measure. For convenience, I will use the term “beyond-sensor-data” to denote information about the state of the plant or process that cannot be directly measured by sensors and must instead be estimated with the use of models.

In a Type II digital twin setup, measured data is sent to the digital twin in real-time, and beyond-sensor-data is generated in real-time so that this information can be sent to the operator. The operator will then decide the next course of action as he or she sees fit. In this regard, the operator only periodically updates the state of the plant as he or she sees

fit because operators are human. It is reasonable for humans to have lapses in attention and therefore, they cannot update the state of the physical plant using data from the digital twin in real-time.

In contrast, a Type III digital twin setup would require real-time exchange between the digital twin and the physical asset (plant or process). This means that the digital twin sends beyond-sensor-data to plant controllers directly in real-time, thereby influencing the physical state of the plant in real-time. In this case, the Type III digital twin setup completely bypasses the operator when it needs to alter the state of the plant in real-time.

Distinguishing a Type I Digital Twin from a Real-Time Simulation Another possible point of confusion is how a Type I digital twin differs from a Real-Time Simulation. We consider that digital twins are defined in literature as digital representations of a physical asset which tracks the characteristics, properties, conditions and behaviours by means of models, information and data [Sleiti, Kapat, and Vesely, 2022]. While a Type I digital twin does not track the condition or state of a physical asset in real-time, it must at least be able to replicate the behaviour or characteristics of a plant in real-time. One key characteristic of a plant is that it can be observed and controlled by human operators in real-time. This capability, which could be thought of as a key characteristic or behaviour of the physical asset, should be properly replicated or copied in a digital twin. Of course, a digital twin must also be able to simulate the process of the plant in real-time. While a real time simulation could also perform real-time process simulation, I would distinguish digital twins from real time simulations in that a human user must be able to have a two way data exchange in order to interact with the digital twin in real time. This is because the physical asset has a capability for human users to interact with it in real time.

Digital Twin Synchronisation

Clarification on what output may mean in State Output Synchronisation

When I mean output I do not only mean desired process output, but also any sensor data which may depend on a process input. To illustrate this, take for example a system with a pump driving fluid through a water heater. The user may give an input, e.g a set point for water temperature exiting the heater. The term output as used in control systems terminology would be the actual temperature of the water exiting the heater. Hence, we would have installed a temperature sensor for the water heater outlet to measure the water heater outlet temperature. This is an output we desire to control. However, in this thesis, I would also consider intermediate temperatures such as the sensor measured surface temperature of the water heater to be an output since it depends on the set point. It may not be an output of interest, and could be termed as an intermediate process variable, but from the aspect of simulation and modelling, it is an output. This is because in the context of digital twins, we devise models that predict the surface temperature of the water heater, and that is in fact a model output or an output of some function or program. Of course, we

would more likely spend our efforts modelling an intermediate process variable such as the bulk temperature within the water heater tank. Regardless, we would also consider the bulk temperature within the water heater to be an important modelling output even though it is an intermediate process variable. Therefore when I refer to outputs in the context of digital twins, it will encompass these intermediate process variables as well.

Typical Fault Detection Setups in Literature

In a fault detection setup, a digital twin would emulate a physical asset operating under normal conditions (otherwise known as a healthy or fault free condition). When the behaviour of the physical system deviated too much from the digital twin, an alarm would be triggered alerting the user that a fault was present.

Now of course, the other reason that the physical asset can deviate from the type II digital twin in terms of behaviour is that the type II digital twin contains models with modelling errors. This can result in modelling errors causing a misdiagnosis of system faults. To prevent this from happening, we may need to periodically synchronise or calibrate the models within the digital twin with the physical system. Assuming this is done correctly, we can now use the deviations between the type II digital twin and the physical asset to alert users to faults which are present in the physical system.

In one example of such a case [Jain et al., 2019], photovoltaic cells were subject to a solar irradiance and cell temperature over a period of time. A digital twin of this experimental setup was constructed and has its inputs, cell temperature and solar irradiance, synced to that of the physical solar cell. This digital twin was assumed to be the fault free model of the photovoltaic cells.

The outputs of the photovoltaic cells (e.g. voltage and current) were subtracted from the estimated outputs of the digital twin to produce a residual vector. The elements of this residual vector were normalised using a scaling factor to ensure that the elements were dimensionless and were of the same order of magnitude [Jain et al., 2019]. Afterward, the L^∞ norm was taken of this vector and compared against a threshold denoted as Γ in the study [Jain et al., 2019]. Γ was determined by measuring the background noise of a fault free system, measurement uncertainties, discretisation error and then adding a margin of safety [Jain et al., 2019].

Now, fault detection is merely the first step in getting the user to the end goal of addressing the fault. By fault detection, I mean that the presence of the fault is detected. However, the type of fault plaguing the system still needs to be determined via fault identification. To begin fault identification, what was done in literature was that a series of faults were simulated in the digital twin [Jain et al., 2019; Dey, Perez, and Moura, 2017] prior to startup of the physical asset. Each fault would produce a specific behaviour or fault signature. These fault signatures would then be collated into a fault signature library [Jain et al., 2019]. If the behaviour of the system matches one of the signatures within the fault signature library, the nature or cause of the fault could then be identified. This was done in the case of the solar cell study as well [Jain et al., 2019].

Structural Analysis in Fault Detection

Of course, such a simple approach for fault detection is presented to help the reader understand why properly calibrated models are required for type II digital twins to detect faults. In literature, more sophisticated methods for fault detection are presented [Poresky, 2019]. One example described in Poresky's work is the use of structural analysis [Poresky, 2019] for fault detection. Structural analysis provides us with a systematic framework for identifying and isolating faults in complex systems with several modelling equations and several variables in the system. Structural analysis methods are extensively discussed in literature for fault detection [Izadi-Zamanabadi, 2002] and fault tolerant control [Izadi-Zamanabadi and Staroswiecki, 2000]. In fact, a fault diagnosis toolbox written for Matlab and Python has been written using this structural analysis method [Frisk, Krysander, and Jung, 2017]. This shows that structural analysis for fault detection is quite well established in literature.

In structural analysis, we also aim to diagnose faulty behaviour and determine the cause of faulty behaviour. Poresky uses the term fault detectability to describe our ability to identify faulty behaviour caused by a specific fault, and he uses the term fault isolability to describe if a detected fault can be distinguished from any other fault [Poresky, 2019].

For fault detection to occur, structural analysis dictates that we need an over-determined system. Let me illustrate with an example:

Suppose we are trying to conduct a food and beverage business. We are to receive crates of food from our supplier every month. These crates of food filled to the brim would weigh a set 50 kg if nothing goes wrong. However, if the crate weighs heavier than 50 kg, one could get suspicious of our suppliers. Of course, a truck driver delivering the crate would say it weighs 50 kg. One way then to determine how much the crate weighs is to believe what the truck driver says.

In this example, we can reduce this scenario to an equation:

$$m = 50 \text{ kg} \tag{C.1}$$

The mass of the crate, m is claimed by the driver to be 50 kg. We have one equation as described above, and one unknown. The equation simply describes what the truck driver claims the crate weighs. This system is just determined as the number of equations is equal to the number of unknowns. In such a system, there is no way to tell if the truck driver is lying. Of course, in real life, the truck driver could be ignorant or mistaken about the state of the crate, but for the sake of illustration, we assume that these other explanations do not apply.

We would need to cross-examine his claims with our own way of measuring the mass of the crate. Suppose we have our own weighing scale now, and it measures a mass measurement y . The variable y is of course known.

$$m = 50 \text{ kg} \quad (\text{C.2})$$

$$m = y \text{ kg} \quad (\text{C.3})$$

Supposing that y is known, and the uncertainty of y is negligible, we now have two ways of determining the mass of the crate. We have one unknown, m , and two equations to find the unknown. Such a system of equations is over-determined since the number of equations exceeds the number of unknowns. We can extend this logic to complex systems with many variables and equations.

If the number of equations (i.e ways of measuring variables) exceeds the number of variables, then we have an over-determined system and we are able to detect faults [Poresky, 2019]. However, identifying or isolating what fault is causing the errant behaviour is another matter altogether.

In our example, if $y \neq 50$, then we have a fault. Either the truck driver is lying or we have a faulty weighing scale. In this case, we have no way of telling what issue has arisen unless we assume the scale is accurate. In this case, the fault is detectable but not isolable. To isolate the fault, we need a way of determining if our weighing scale is working.

Suppose then we have an alternative way of checking if our scale is working properly using a reasonably large spring of known stiffness k_{spring} . We could attach a known test weight m_{test} of perhaps 55 kg to the spring and cause it to oscillate on some sort of low friction environment to measure the oscillation period T .

The additional equation describing this would be:

$$T = 2\pi \sqrt{\frac{k_{spring}}{m_{test}}} \quad (\text{C.4})$$

Supposing that our spring is well calibrated and experiment is conducted properly such that this equation applies, this becomes a way for us to measure the test weight m_{test} . We can then use our weighing scale to measure the test weight also. The measurement is denoted y_{test} . Our system of equations becomes:

$$m = 50 \text{ kg} \quad (\text{C.5})$$

$$m = y \text{ kg} \quad (\text{C.6})$$

$$m_{test} = y_{test} \quad (\text{C.7})$$

$$T = 2\pi \sqrt{\frac{k_{spring}}{m_{test}}} \quad (\text{C.8})$$

Supposing that the test mass measurement from the spring experiment matches y_{test} , then we have confidence that the problem is not with our equipment and can infer that the truck driver is lying. However, if the test mass measurement does not match y_{test} , we would likely know that our equipment is surely at fault, not necessarily the truck driver.

By using a known test mass, we have effectively found a way to check if our scales were working properly independent of the truck driver. If however, we did not use the test mass, and instead used the crate with our spring system, our system of equations becomes:

$$m = 50 \text{ kg} \quad (\text{C.9})$$

$$m = y \text{ kg} \quad (\text{C.10})$$

$$T = 2\pi \sqrt{\frac{k_{spring}}{m}} \quad (\text{C.11})$$

If the y matches the mass measurement of the spring system, then we would have confidence that our truck driver was lying. However, if y did not match the measurement of the crate's mass by our spring system, we would still have no way to tell if the measurement discrepancy was due to the truck driver was lying or a faulty scale, or if both cases were happening simultaneously. The only way to know for sure whether or not our scale is faulty is to calibrate our scale with a test mass regularly before using our weighing scale for checks. Therefore, we need that test mass to check our existing weighing scale.

This simple illustration helps us understand that for a fault to be isolable, the set of equations that helps us describe a specific fault must exclude all other model faults [Poresky, 2019]. For this to happen in this specific case, we wanted a system of equations which could check if our weighing scale was faulty independent of all other faults.

Therefore, we needed a test mass m_{test} to ensure that the system of equations describing the weighing scale calibration does not include the crate mass m .

Now, whether models are used for fault detection or fault isolation, we know we must have reasonably accurate and well calibrated models. Since those models are found in digital twins, we must ensure that the digital twin itself must be well calibrated. Secondly, for fault isolation, we must still accurately model how each fault impacts our system of equations. Only then can we properly distinguish one fault from another by devising alternative means of measurement. A more complete and formal explanation of this process can be found in literature [Poresky, 2019; Izadi-Zamanabadi, 2002].

In this master's thesis, I do not aim to explain structural analysis methodology for fault detection. However, I only want to elaborate enough to show that accurate modelling of the system is required to detect faults and accurate modelling of faults is required in order to isolate faults.

Synchronisation Requirements for Various Applications

Requirements for Fault Tolerant Control In such a setup, we could diagnose faults using Type II digital twins which are synchronised to the physical system in real-time. Information about these faults can be processed into a signal we could send to a controller to mitigate those faults without the need for immediate user intervention. This has been demonstrated in literature for simple manufacturing processes [Majdzik et al., 2022].

Sending such signals to the physical system in real-time from the digital twin by definition requires the digital twin to be a Type III digital twin. This is because there is a two-way real-time data exchange between the digital twin and the physical asset.

However, making control decisions based on a digital twin would put the plant at risk especially if those decisions were based on low accuracy or low fidelity data. Therefore, we would want our digital twin to provide data in as accurate and timely a manner as possible. This might sometimes mean that we would want accurate estimates about process information beyond what sensors can provide. Therefore, we might not only need to have any Type III digital twin, but a Type III digital twin that is able to synchronise itself to the full state of the plant in real-time. Using the terminology in section 2.4, we are performing full state synchronisation in real-time.

Prototyping Fault Tolerant Controllers In the operation of fault tolerant control, we might need a Type III digital twin with real-time full state synchronisation.

However, when we are developing and testing fault tolerant control designs, we might resort to using Type I digital twins. In such a setup, digital twins have to be used to generate fault signature libraries just as before [Saracian and Shirazi, 2022]. This is still important information we need for developing fault tolerant controllers.

Moreover, once we have controller prototypes ready for testing, we might use Type I digital twins to perform what-if analysis to analyse how these controllers behave in various scenarios. In literature, this approach was used to design controllers meant for the benchmark Tennessee Eastman (TE) chemical process [He et al., 2019].

We might not only perform prototyping of new controllers during the beginning of plant life. As the plant ages, we may want to update these controllers as well. As such, the use of Type I digital twins for prototyping fault tolerant controllers might extend well beyond the initial design phase of the plant.

Monitoring Process Variables of Physical Asset Digital twins can be used in monitoring of the process variables of a plant or physical asset in real-time. This goes beyond fault detection discussed in section 2.3. This is because fault detection only requires that a plant's behaviour deviates from the norm. In fault detection, we only need to compare the sensor readings of the physical asset against the estimated sensor readings of the digital twin in order to detect a fault. This was done in the photovoltaic cell study [Jain et al., 2019]. In contrast, the real-time monitoring of the process variables of a plant or physical asset involves using a digital twin to estimate data about the physical asset in areas where sensors are impractical to install. This means we are estimating "beyond sensor data".

In such a case, the end user may use a mathematical model of a physical asset in order to estimate data about the physical asset where said data is not reachable by sensors. For example, in the case of batteries, temperature data within the electrolytes of the battery may not be available. Nevertheless, in literature, they were estimated with the use of

lumped parameter models [Dey, Biron, et al., 2016] as well as partial differential equation (PDE) models [Dey, Perez, and Moura, 2017]. This allows the user to have an estimate of the core temperature of the battery or in the case of more sophisticated PDE models, temperature distribution within the battery [Dey, Perez, and Moura, 2017]. In that study, having internal temperature distribution estimates of the battery allowed for a more sophisticated version of fault detection to occur [Dey, Perez, and Moura, 2017]. However, it should be noted that this is not strictly required for fault detection in general. Nevertheless, having a digital twin with plant state monitoring capabilities could enhance fault detection algorithms provided the digital twin is kept synchronised with the physical asset. Additionally, the fault signature library or database should be kept up to date as well. Now of course, a digital twin meant to estimate all the sensor outputs for fault detection may likely already have all the data values necessary to monitor the entire state of the plant. This is especially true in the case of digital twins based on physical models or first principles. However, A designer might opt to use a data driven model because the first principles model might be too slow for real-time calculations and become a bottleneck in digital twin development [Rasheed, San, and Kvamsdal, 2020]. As a result, we might have the digital twin were based on a purely data driven model. If the designer had not included the other sensor unreachable process variables in the data driven model, then these models would not likely have the predictive power present in first principle type models. In such a case, the fault detection function can be performed without knowing the full internal state of the plant.

Therefore, we can show that monitoring the internal state of a plant, including all its process variables, is distinct from fault detection processes. You could also perform monitoring of the plant without any fault detection algorithms in place. These two functions are essential parts of online plant health monitoring, but are distinct from each other.

Requirements for Process Variable Monitoring of Physical Asset If we want to know or estimate the internal state of the physical asset (beyond-sensor-data) in real time, then input, output and model synchronisation has to be done in real-time as defined in 2.4. This by definition necessitates a Type II digital twin at least.

Predictive Maintenance Now digital twins have been shown to have utility in fault detection and in developing fault tolerant controllers. In fact, the operation of fault tolerant controllers usually necessitates a Type III digital twin with real-time full state synchronisation capabilities. These digital twins should also have capability to provide the user with information beyond what can be measured directly with sensors (i.e beyond-sensor-data).

Such a digital twin would take considerable effort to develop. However, an additional reason to develop such a digital twin is the potential to use such a digital twin in predictive maintenance [Rasheed, San, and Kvamsdal, 2020; Fotland, Haskins, and Rølvåg, 2020].

Without digital twins, maintenance is usually done periodically to prevent any kind of stress related failures. However, the frequency of maintenance can often be higher than what is actually required, and performing unnecessary maintenance can accelerate the aging of components. While this may be the case, periodic maintenance is done so that the physical asset can be operated in a safe manner. However, if we want to reduce the frequency of maintenance in a safe manner, we would need to understand how the physical asset undergoes wear and tear using some simulation model. When the estimated wear and tear exceeds some threshold, we stop the operation of the physical asset and perform maintenance. The overall effect of the latter approach is a higher degree of operation time and increased component life.

An example of this case is in cranes in offshore vessels [Fotland, Haskins, and Rølvåg, 2020]. Offshore cranes will experience stress and fatigue during lifting operations. To detect if the crane is experiencing unsafe levels of stress, we could install sensors within the crane or in a physical twin of the crane. However, similar to the case of batteries, sensors are limited in their capabilities to measure stress along the entire structure of the crane. Moreover, having physical twins of the cranes would be quite costly to construct and operate. The alternative then is to construct a digital twin of the crane and predict the accumulated stresses in each part of the crane so that the operators can better predict if or when parts of the crane would fail. This digital twin can then be used as a basis for risk analysis when planning for operations. [Fotland, Haskins, and Rølvåg, 2020]

Requirements for Predictive Maintenance In these examples, we take advantage of the digital twin's ability to have an educated guess about the state of the physical asset everywhere within the said physical asset, even for state parameters that cannot be measured directly with sensors.

This means that we would like a digital twin to have real-time full state synchronisation capabilities as described in section 2.4. This is because we may want to know the information about the physical assets beyond what sensors can tell us. Furthermore, we may want to keep a history of the state of the system. This is because the state of the system presently might depend of the history of the system state. For example, the metal fatigue within some region of the offshore crane will depend on the magnitude of stress experienced by the crane in the past as well as the duration of stress experienced. This examples illustrates how fatigue is dependent on the history of load cycles and how material stress and strain relationships show hysteresis or memory dependence [Erber, Guralnick, and Michels, 1993]. As such it is important for us to have a record of the past states of the system over time in order to predict its present state. Only when one has sufficiently accurate knowledge about the state of the physical system can one execute predictive maintenance safely.

By virtue of these requirements, this digital twin will at least have to be a Type II digital twin since the digital twin receives data from the physical asset in real-time. We might not necessarily need a Type III digital twin depending on the use case. For example, when

users wish to check if a crane or support has experienced a critical level of low cycle fatigue, the user may need only check the digital twin periodically (perhaps every week or every month) rather than in real-time (every 0.3 seconds).

C.2 Methods for Constructing Digital Twins

Algorithms for Parallel Flow Paths

Returning Zero Flow if Forward Pressure Gradient Detected

With some degree of optimisation, such as removing the code that calculated the empirical manometer values in Python, as well as some other code used during debugging and sandboxing, the forward flow calculation times were reduced to about 85 ms.

However, reverse flow peak calculation times were somewhere in the region of 125 ms still due to the artificially high K values implemented. This was due to the fact that the brent methods used to iterate out the pressure loss included both forward and reverse flow.

The typical bracket in the brent equation was -1.0 kg/s and 1.0 kg/s of mass flow rate.

Thus, we would have had some iterations where the high K values were calculated which slowed the iterations down.

Thus, for simple check valve behaviour, it is best not to use the high K value method because it will slow down convergence speed. One can simply return zero flowrate in future when forward flow pressure gradient is detected within the DHX branch.

Effect of Pressure on Simulated Time It should be noted that at different given pressures, the simulation time would also change. For example, and a 16 kPa loop pressure drop was applied, the simulation time was consistently at around 85ms. However, when loop pressure drop was changed to -16 kPa, the simulation time was around 108 ms.

Usually simulation times at pressures close to zero are higher, for example -0.1 kPa pressures had simulation times around 125 ms. Whereas 0.1 kPa pressure had simulation times around 105 ms. This is probably due to different rates of convergence given different pressure values.

Computation Speed when High K Behaviour Removed

Now, with the high K behaviour removed from FM-20, the typical simulation time at 16 kPa loop pressure drop was about 75 ms with maximum time of 83 ms. This is not too different from before. When -16 kPa loop pressure drop was supplied, in reverse flow, the simulation time was around 100 ms, with peaks at around 105 ms.

At -0.1 kPa pressure, simulation time was around 115 ms, which is faster than before, but not sufficient to get it below 100 ms. Whereas at 0.1 kPa pressure, simulation time was around 105 ms, which is not too different from before where we had the high K value check valve.

These results show that High K behaviour for check valve would somewhat increase the time required for the solvers to converge. Nevertheless, it also shows that serial calculation is insufficient for such a low end computer.

Trial on Gaming Laptop

On the gaming laptop, with the high K behaviour removed from FM-20, and DHX branch added, peak computation times were at around 63 ms for 16 kPa, 72 ms for -16 kPa, 72 ms for 0.1 kPa and about 78 ms for -0.1 kPa.

Doing so allowed the simulation times to fall below 100 ms which was the the desired time limit. This was just to demonstrate that The digital twin was still operable on a desktop or laptop. Nevertheless, to make it work on a lower end laptop such as the Ideapad 3 with Ryzen 3500U, extra optimisation will be needed.

Methods for Real-Time State Synchronisation of Digital Twins

Now, we have explored some important use cases of digital twins. These digital twins might need to undergo partial or full state synchronisation depending on the use case. By full state synchronisation I mean that the model of the digital twin is synchronised to that of the physical asset, and that the digital twin's model inputs and outputs are also synchronised to the sensor data from the physical asset. For partial state synchronisation, I mean that at least the digital twin's models are synchronised or calibrated to the physical asset. However, synchronisation of the digital twin to all sensor data from the physical asset is not necessary for partial state synchronisation. Also, the frequency of state synchronisation might depend on the use case.

Regardless, the fact is that state synchronisation is an important aspect of digital twins, specifically Type II and Type III digital twins. Now, for this thesis, we will not provide our digital twin with Type II or Type III capabilities. We will be constructing a Type I digital twin. However, the option to convert this Type I digital twin into a Type II or Type III digital twin remains to be explored in future work. Doing so would likely require us to explore state synchronisation methods.

Therefore, we only want to briefly review such methods as stated in literature. We do so briefly because this work will not demonstrate state synchronisation capabilities, but we do so anyway because it may influence how we design the Type I digital twin.

Some methods shown to work in literature are the use of Kalman Filters [Akbarian, Fitzgerald, and Kihl, 2020]. This method seeks to take the measured inputs and outputs of the physical system and use them to generate the simulated outputs of the digital twin.

More sophisticated methods of synchronisation using machine learning have also been demonstrated in a Siemens digital twin of a smart factory [Rasheed, San, and Kvamsdal, 2020] and studies of a nuclear power plant steam generator [H. Song, M. Song, and X. Liu, 2022].

One might also use an objective function based method which varies digital twin inputs in such a way to minimise the differences between digital twin simulated outputs and physical plant measured outputs [Zipper and Diedrich, 2019]. One could mix and match algorithms to ensure that results are satisfactory. One of the algorithms used in literature for this purpose is Dijkstra's shortest path algorithm [Zipper and Diedrich, 2019].

Another way is to use a simple Proportional, Integral and Derivative (PID) controller that takes the measured physical system output as a desired set point [Akbarian, Fitzgerald, and Kihl, 2020]. It then measures the error between the digital twin outputs and physical system measured outputs and attempts to minimise them by modifying the inputs into the digital twin.

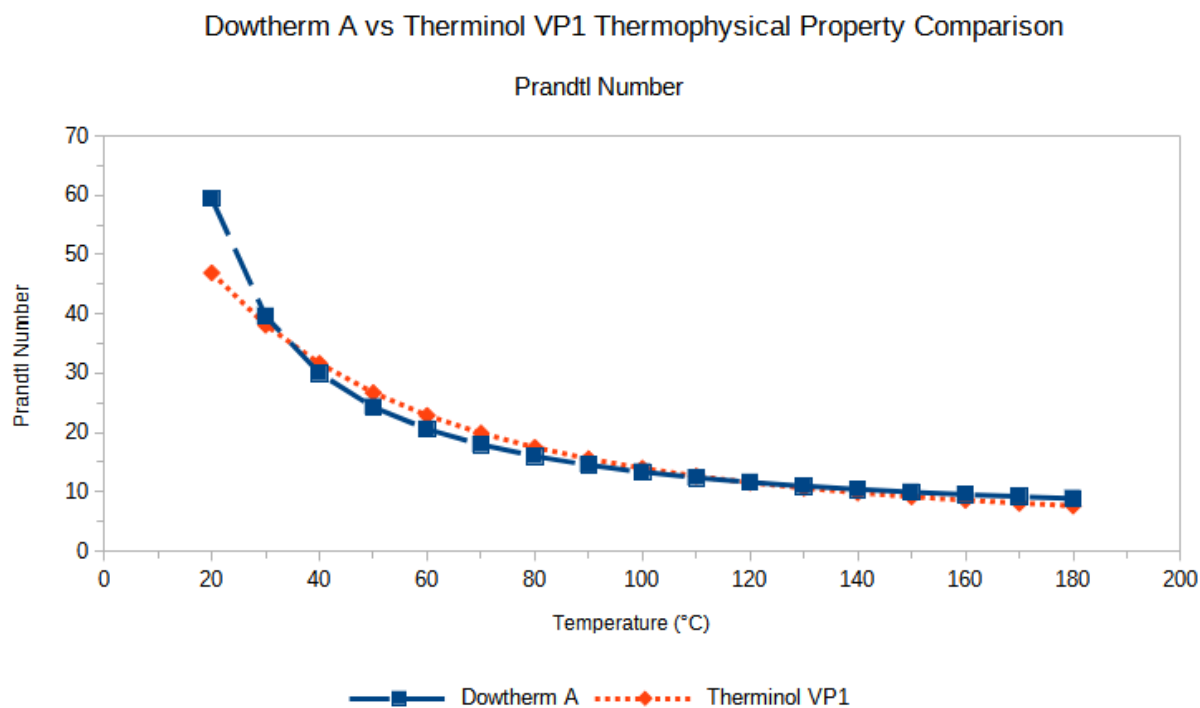
Each strategy has its merits and use case, there is no one size fit all strategy. For example, if we were to use data driven machine learning models to provide digital twin outputs, we may very well synchronise our digital twin to the physical asset. However, without physics based models, we may wonder if the predictions made about the system state are accurate if the digital twin is suddenly subject to conditions outside the data range. Of course, if the designers of the digital twin did a good job of training the neural network to anticipate all possible scenarios, this may not happen. Nevertheless, this would require much more effort than using a first principles model in order to generate digital twin data. Therefore, we can see that physics based models still retain their value in an era of data driven models.

Now, we can see that there are a myriad of synchronisation methods available for use in digital twins. However, the choosing of the specific synchronisation method would greatly depend on the demands of synchronisation. For example, whether periodic synchronisation or real-time synchronisation required. In periodic synchronisation where we may not care about transient values, a Kalman Filter may be appropriate [Akbarian, Fitzgerald, and Kihl, 2020], whereas when transient values are important, a PID controller method of synchronisation may be more appropriate [Akbarian, Fitzgerald, and Kihl, 2020].

It is therefore important for the user to be keenly aware of the specific use case of the digital twin before selecting a synchronisation strategy. This process may not be covered in this thesis, but is reserved for future work.

Open Source Thermodynamic Libraries

Thermophysical Properties Dowtherm A vs Therminol VP1



Temp Degrees C	Dowtherm A	Therminol VP 1	Error Percentage
20	59.4	46.9	21
30	39.6	38.1	3.7
40	30	31.6	5.6
50	24.3	26.7	9.9
60	20.6	22.9	11.3
70	17.9	19.9	10.8
80	16	17.5	9.3
90	14.5	15.5	7.2
100	13.3	14	4.7
110	12.4	12.6	2.1
120	11.6	11.5	0.5
130	11	10.6	3
140	10.4	9.8	5.5
150	9.9	9.2	7.8
160	9.5	8.6	10
170	9.2	8.1	12
180	8.9	7.7	13.9

Table C.1: Prandtl Number Comparison for Therminol VP 1

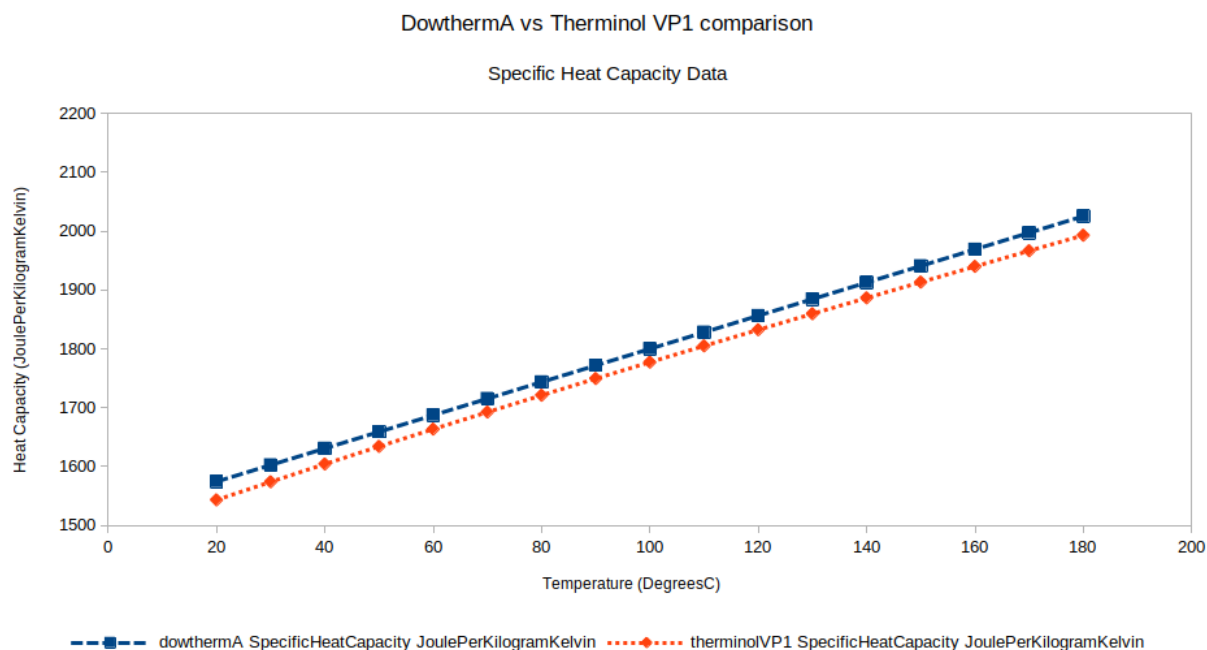
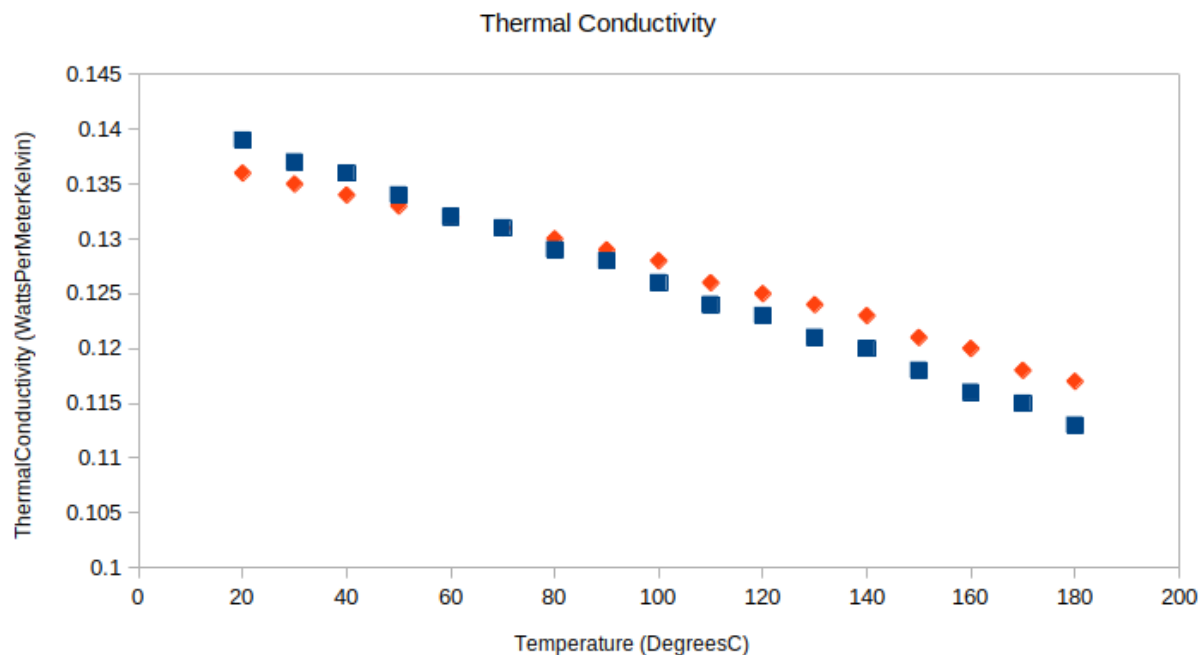


Table C.2: Specific Heat Capacity Comparison ($J/(kg\ K)$) for Therminol VP 1

Dowtherm A vs TherminolVP1 Thermophysical Properties



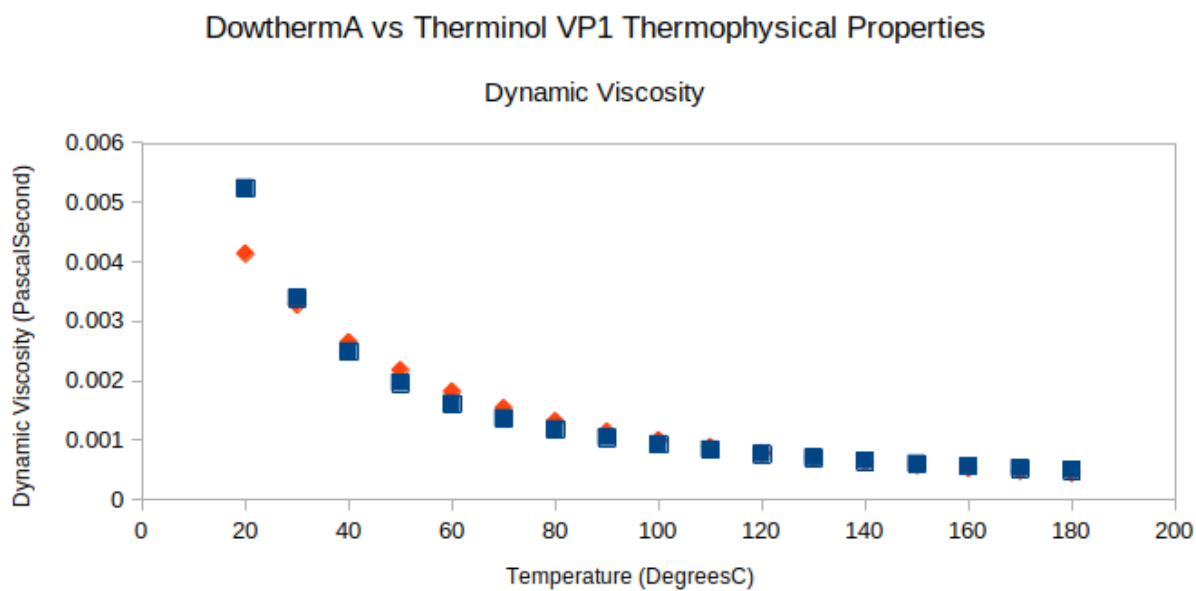
■ dowthermA ThermalConductivity WattPerMeterKelvin ◆ therminolVP1 ThermalConductivity WattPerMeterKelvin

Temp Degrees C	Dowtherm A	Therminol VP 1	Error Percentage
20	0.139	0.136	1.807
30	0.137	0.135	1.358
40	0.136	0.134	0.924
50	0.134	0.133	0.506
60	0.132	0.132	0.103
70	0.131	0.131	0.282
80	0.129	0.13	0.65
90	0.128	0.129	0.999
100	0.126	0.128	1.33
110	0.124	0.126	1.642
120	0.123	0.125	1.933
130	0.121	0.124	2.203
140	0.12	0.123	2.452
150	0.118	0.121	2.678
160	0.116	0.12	2.88
170	0.115	0.118	3.057
180	0.113	0.117	3.209

Table C.3: Thermal Conductivity Comparison ($W/(m\ K)$) for Therminol VP 1

For viscosity, the formula used to round the data (take the data to be in cell A1 for a libreoffice calc document) to three significant figures is:

```
=ROUND(A1,3+CEILING(ABS(LOG10(A1))))
```

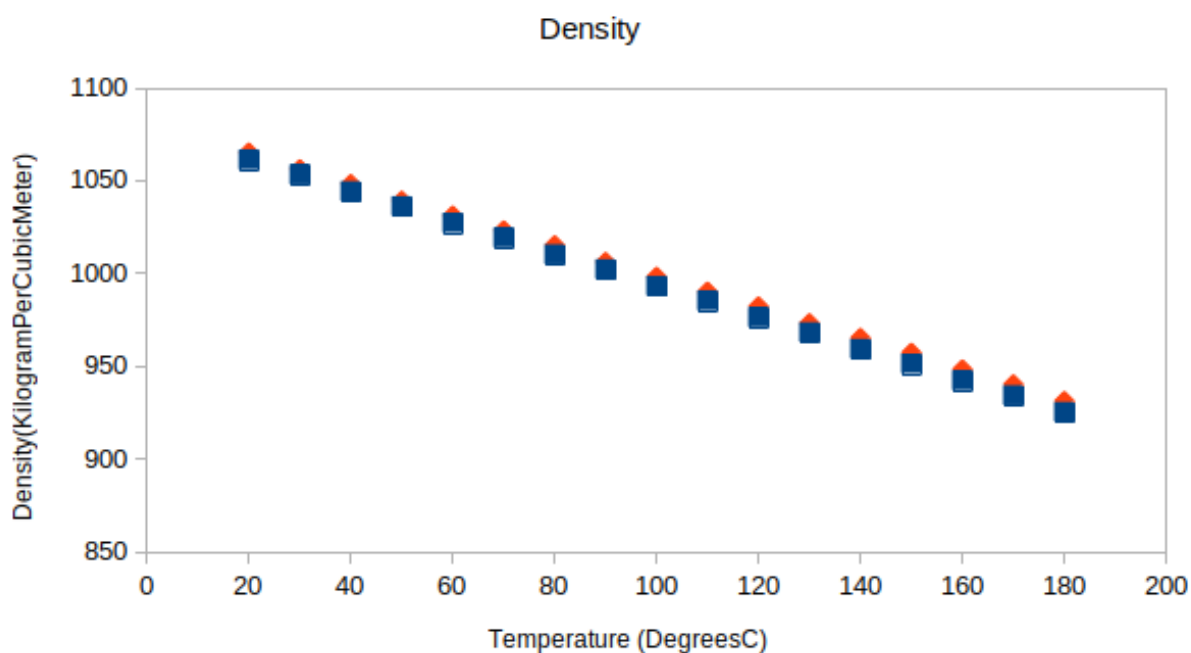


■ dowthermA DynamicViscosity PascalSecond ◆ therminolVP1 DynamicViscosity PascalSecond

Temp Degrees C	Dowtherm A	Therminol VP 1	Error Percentage
20	0.00524	0.00414	20.8937
30	0.00339	0.00328	3.3113
40	0.002492	0.002649	6.3082
50	0.001962	0.002178	11.0355
60	0.001613	0.001819	12.7639
70	0.001368	0.001541	12.6658
80	0.001185	0.001321	11.4738
90	0.001045	0.001146	9.6487
100	0.000933	0.001003	7.4824
110	0.000843	0.000886	5.1596
120	0.000767	0.000789	2.797
130	0.000704	0.000708	0.466
140	0.000651	0.000639	1.787
150	0.000604	0.00058	3.9389
160	0.000564	0.00053	5.975
170	0.000528	0.000487	7.8898
180	0.000497	0.000449	9.6828

Table C.4: Dynamic Viscosity Comparison ($Pa \cdot s$) for Therminol VP 1

DowthermA vs Therminol VP1 Thermophysical Properties



■ dowthermA Density KilogramPerCubicMeter ♦ therminolVP1 Density KilogramPerCubicMeter

Temp Degrees C	Dowtherm A	Therminol VP 1	Error Percentage
20	1061	1065	0.4
30	1053	1056	0.4
40	1044	1048	0.4
50	1036	1039	0.4
60	1027	1031	0.4
70	1019	1023	0.4
80	1010	1015	0.5
90	1002	1006	0.5
100	993	998	0.5
110	985	990	0.5
120	976	982	0.6
130	968	973	0.6
140	959	965	0.6
150	951	957	0.6
160	942	948	0.6
170	934	940	0.7
180	925	931	0.6

Table C.5: Density Comparison (kg/m^3) for Therminol VP 1

C.3 Methodology Supplemental Discussion

finite volume and finite element methods

Continuum methods such as finite element methods and finite volume methods are used in literature. For example, finite element model (FEM) is used by SAM [Zou, R. Hu, and Charpentier, 2019], which solves the Navier Stokes Equations [Perry and Green, 2015]:

$$\begin{aligned}\nabla \vec{u} &= 0 \\ \rho \frac{D\vec{u}}{Dt} &= -\nabla P + \mu \nabla^2 \vec{u} + \rho g\end{aligned}$$

Nevertheless, FEM methods or even finite volume methods (FVM) may require partial differential equation (PDE) solvers to solve properly. This means it likely would be time intensive, thus this method was not explored at all. Therefore, nodalised methods are preferred.

fanning and darcy friction factors

While it is common knowledge for engineers, I have decided to write the conversion between fanning and darcy friction factors for the convenience of the user. In literature, both fanning and darcy friction factor can be represented by f and it can just be ambiguously called the “friction factor”. In this work, unless otherwise stated, the friction factor is almost always referring to the darcy friction factor.

The conversion is listed below:

$$f_{darcy} = 4f_{fanning} \quad (C.12)$$

Which means for the pressure loss correlations, the equation should look like this [Perry and Green, 2015]:

$$\Delta P_{loss\ i} = \frac{1}{2} \rho u^2 (4f_{fanning} \frac{L}{D} + K) \quad (C.13)$$

On the Newton Raphson Method

In this section of the appendix, we discuss how one might occur the set of discretised nonlinear equations that we can, in theory, solve using the multivariate Newton Raphson method.

How to Obtain F

Suppose first that we have a pipe described by a control volume. This control volume has the equation describing enthalpy as follows:

$$\frac{\partial H}{\partial t} = \sum_i H_{in} - \sum_i H_{out} + \Delta Q \quad (C.14)$$

Supposing that we discretise this equation using an implicit Euler method, and supposing that we demonstrate this for a typical pipe, represented by a control volume:

$$\begin{aligned} & m_{pipe} \frac{c_{p,pipe}(T_{pipe}^{t+1})T_{pipe}^{t+1} - c_{p,pipe}(T_{pipe}^t)T_{pipe}^t}{\Delta t_{timestep}} \\ &= \dot{m}^{t+1} c_{p,in}(T_{in}^{t+1})T_{in}^{t+1} - \dot{m}^{t+1} c_{p,out}(T_{out}^{t+1})T_{out}^{t+1} \\ & \quad - (h_{wall}(T_{pipe}^{t+1}, T_{wall}^{t+1}, Re)A_{wall})(T_{pipe}^{t+1} - T_{wall}^{t+1}) \end{aligned} \quad (C.15)$$

Here, m_{pipe}^t , $c_{p,pipe}^t$, T_{pipe}^t are the mass of fluid in the pipe, heat capacity of the pipe and bulk temperature of the pipe at time t . T_{wall}^t is the wall temperature of the pipe at time t , and $\dot{m}^t$ is the mass flowrate through the pipe at time t . The $t+1$ superscript indicates that these values evaluated at the next timestep. This forms just one of the energy balance equations within system. We can, in general move all the terms to the left hand side:

$$\begin{aligned} & m_{pipe} \frac{c_{p,pipe}(T_{pipe}^{t+1})T_{pipe}^{t+1} - c_{p,pipe}(T_{pipe}^t)T_{pipe}^t}{\Delta t_{timestep}} \\ & \quad - \dot{m}^{t+1} c_{p,in}(T_{in}^{t+1})T_{in}^{t+1} + \dot{m}^{t+1} c_{p,out}(T_{out}^{t+1})T_{out}^{t+1} \\ & \quad + (h_{wall}(T_{pipe}^{t+1}, T_{wall}^{t+1}, Re)A_{wall})(T_{pipe}^{t+1} - T_{wall}^{t+1}) = 0 \end{aligned} \quad (C.16)$$

We might call this equation f_1 and f_1 is a function of all the unknown temperatures, flowrates, Reynold's numbers and so on. The set of all unknown variations, both dependent and independent, is called $\vec{x}$. We would now repeat this process over all pipes and control volumes to get a set of equations. Additionally, we would also add in correlations for mass balance as well. In the simplest of cases, we might just look at a single loop of pipes. Additionally, we can add set of equations for the wall temperatures describing thermal inertia within each pipe and convection to surrounding air. Moreover, we need to consider the fluid mechanics equations as well and add them to our set of equations:

$$\Delta P_{loss\ i} = \frac{1}{2} \frac{\dot{m}^2}{\rho A_{xs}^2} (f_{darcy} \frac{L}{D} + K) \quad (C.17)$$

$$\Delta P_{change} = - \sum_i^n \Delta P_{loss\ i} + \sum_i^n \Delta P_{hydrostatic\ i} + \sum_i^n \Delta P_{source\ i} \quad (C.18)$$

For a single loop of pipes, we can use Kirchoff's voltage law for circuits and adapt it for pipes. For a single series of pipes, the ΔP_{change} can be set to zero. And this set of fluid mechanics equations can be used across all the pipes. For each equation, we can once more

shift all the terms in the equation to the left hand side and equate the right hand side to zero. Each equation can be labelled f_i and they are in general dependent on $\vec{x}$. Suppose now we have n equations, $f_1, f_2, \dots, f_n$. This set of equations can be described by a matrix, called $\vec{F}(\vec{x})$.

How to obtain J

The Jacobian is obtained as follows

$$\vec{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \dots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \quad (\text{C.19})$$

Here, I did not specify x_1, x_2 or any variable in $\vec{x}$. However, one should know that they can represent any of the unknowns in the set of equations $\vec{F}$ such as the mass flowrates or any of the control volume temperatures.

A brief on the Solution Procedure

Now that we, conceptually speaking, have the Jacobian and the set of equations $\vec{F}(\vec{x})$, we then solve for $\Delta\vec{x}$ using:

$$\vec{F}(\vec{x}_k) = -\vec{J}(\vec{x}_k) * (\Delta\vec{x}) \quad (\text{C.20})$$

Where $\Delta\vec{x}$ represents the vector of difference between two successive iterative guesses of $\vec{x}$, $\vec{x}_k$ and $\vec{x}_{k+1}$ which are the guesses of $\vec{x}$ at the k^{th} and $(k+1)^{th}$ iteration. This should be done by any matrix solver algorithm which could, for example, be a Lower Upper (LU) decomposition solver. If the norm of $\vec{x}$ is too large, we would then update $\vec{x}_k^t$ as $\vec{x}_{k+1}^t$ and repeat the iteration process by finding the Jacobian again, and evaluating the jacobian and $\vec{F}$ at $\vec{x}_k$. This should be done until the norm of $\Delta\vec{x}$ is within a user set tolerance.

Problems with Multivariate Newton Raphson Method

In the discretised equation C.15, we can see that if we were to use an implicit Euler equation, calculating the temperature distribution for such a network of pipes results in a nonlinear system of equations being formed. The most significant term of these is the heat transfer coefficient at the wall. The heat transfer coefficient is itself dependent on the Reynold's number and the Prandtl number correlations which are in turn dependent on T_{wall} and T_{pipe} . Additionally, the local Re is dependent upon the flow solution, which is in turn dependent on the temperature distribution in a tightly coupled case. Writing code to solve such a system of equations can prove tedious and computationally expensive because it requires a nonlinear solver such as the newton raphson solver to obtain a solution. This

may make the Newton Raphson method prohibitively expensive for digital twin applications. Moreover, the newton raphson solver has regions of numerical instability when solving systems of nonlinear equations [Kochunas and Huan, 2021]. We would like to avoid such a situation.

Onto Operator Splitting!

A faster alternative is to use operator splitting similar to how we decouple solving energy and mass balances at each time step. This means that the energy balances are not solved simultaneously by iterating them to convergence. We could solve the fluid and solid control volume equations separately using the same operator splitting method. [MacNamara and Strang, 2016] This is mainly to help the simulation or digital twin meet the simulation time requirements.

A second way to speed things up is to use an explicit Euler method instead of an implicit one so that one can solve the control volume equations separately rather than simultaneously. This means that we evaluate all fluid properties, enthalpies, heat fluxes etc. at the previous timestep.

We could then solve the mass balance equations first, and then each energy equation would only have one unknown: which is the temperature of the control volume at the next timestep. However, for operator splitting methods and explicit Euler solutions, we may encounter stability issues in future. This is because PISO algorithms will have stability issues when the Courant Number is more than 1. So long as the courant number is less than 1, numerical stability should generally not be an issue. Note that for heat transfer, the Courant number criterion for stability is even stricter due to the way heat flux changes with temperature gradients. Extensive discussion is reserved for future work.

With this approach, we could compute a new flow and heat transfer solution independent of each other. There may be some errors caused by not solving these simultaneously, but should errors arise, we could then explore methods to reduce those errors. [MacNamara and Strang, 2016] Also, we may want to explore methods to ensure that the solutions are numerically stable.

Such discussion is outside the scope of this thesis. For now, we shall just assume that it is possible to split the solution procedure to speed things up. And based on that assumption, formulate a method for fluid mechanics equations to be solved within the time constraint assuming temperature profile from the last time steps are used.

Therefore, for a first iteration of a digital twin, we will attempt to solve just for the fluid mechanics equations first. Heat transfer equations are outside the scope of this thesis and should be discussed in future work.

A Chronological Account of Trial and Error for Parallel Flow Branches

For readers interested in making a similar digital twin, I have outlined my thought process here for reference. Hopefully it prevents would be digital twin writers from making mistakes.

Possibility 1: condensing parallel branches into one single equivalent component

To solve for flow in parallel branches, one could make an argument for the fact that a set of parallel flow paths could be condensed into one equivalent component.

That means that the heater branch, heater branch and DRACS branch would be represented by one equivalent component with an effective fdk term. We could then map out the pressure losses given mass flowrate through the component prior to simulation via some empirical correlation. Therefore, during simulation, only the empirical correlation is used to return the appropriate pressure loss values given a fixed mass flow rate.

What this means practically is to find out the correlation of the fdk term over the set of parallel branches:

$$\Delta P_{loss \ parallel} = \frac{1}{2} \frac{\dot{m}^2}{\rho A_{xs}^2} (f_{darcy} \frac{L}{D} + K) \quad (C.21)$$

And during simulation, we would calculate Re for the parallel branch and extract the fdk term out to calculate the friction factor losses.

However, going by this approach, one would have to find the effective viscosity and density to use within these branches. Deriving such a method would not be trivial especially when the temperature profiles within these branches would be vastly dissimilar. Furthermore, we would also need to consider natural convection within each branch. These issues reduce the attractiveness of this method as a possible solution.

Moreover, if one were to think of such a method, one would have to validate it against a method which is proven to be accurate. The only gains from embarking on this endeavour would perhaps be a decrease in computation time.

I did embark on this method early on in trying to set up the digital twin, only to find that derivations were incredibly laborious and error prone. Time was better spent trying another solution method.

Possibility 2: Brute Force Iteration

Regardless of the solution method I would eventually adopt, I would have to validate the results with a more rigorous method. This is where brute force iteration comes in. Of course, if brute force iteration happens to be fast enough, I would not explore other methods later on.

To solve for the fluid flow within the branches, we would first need to think about how to solve for a mass flowrate over a series of pipes given a pressure loss term. After we have

this solution procedure set, we can then solve for pressure losses over parallel branches given a fixed mass flowrate. The third step is to solve for mass flowrate over the entire loop given a fixed loop pressure drop.

Let's first consider solving for mass flowrate given pressure loss over a series of pipes. Now due to the fact that friction factor is explicit in Reynolds number (Re), we have to implicitly solve for Re given a pressure loss term. This can be done using any iterative solver, although bisection or brent's method is preferred due to its better convergence as compared to newton raphson or secant methods. [P. Li and H, 2022]

For each series of pipes in each branch we could adapt:

$$\Delta P_{pump} = \sum_i^n \Delta P_{loss\ i} \quad (C.22)$$

We would then apply the Darcy Weisbach Equation to each component:

$$\Delta P_{loss\ i} = \frac{1}{2} \frac{\dot{m}^2}{\rho A_{xs}^2} (f_{darcy} \frac{L}{D} + K) \quad (C.23)$$

And then iteratively solve for mass flowrate given a fixed total pressure loss value.

Step two is to solve for pressure losses over parallel flowpaths given a fixed mass flow rate. Now suppose we have a three series of pipes in parallel as in CIET, we can take advantage of the fact that pressure change across each branch must be equal. Therefore we could use this to form an equation.

$$\Delta P_{change} = - \sum_i^n \Delta P_{loss\ i} + \Delta P_{hydrostatic} + \Delta P_{source} \quad (C.24)$$

We note that the total change of pressure between the start and end of the parallel branches is equal to the contributions in pressure losses, hydrostatic pressure and any pressure sources eg. pumps within that branch.

For the simple case where there is only one branch and the start and end point are the same,

$$\Delta P_{change} = 0\ Pa ; \Delta P_{hydrostatic} = 0\ Pa \quad (C.25)$$

And we would reduce back to:

$$\Delta P_{pump} = \sum_i^n \Delta P_{loss\ i} \quad (C.26)$$

Here, $\Delta P_{source} = \Delta P_{pump}$. To solve for the mass flow rate of each branch, we would have to do so iteratively due to the fdk term being implicit in Re.

$$\Delta P_{change} = - \sum_i^n \frac{1}{2} \rho u^2 (f_{darcy}(Re) \frac{L}{D} + K) + \Delta P_{hydrostatic} + \Delta P_{source} \quad (C.27)$$

Of course, it is more convenient to write this in terms of mass flowrate:

$$\Delta P_{change} = - \sum_i^n \frac{1}{2} \frac{\dot{m}^2}{A^2 \rho} (f_{darcy}(Re) \frac{L}{D} + K) + \Delta P_{hydrostatic} + \Delta P_{source} \quad (C.28)$$

Here, A is cross sectional area and $\dot{m}$ the mass flowrate through the series of pipes. If we were to set a fixed pressure change for each parallel branch, then we could perform iterations to find the mass flowrate for each branch. Summing them together, we would get the total mass flowrate the parallel collection of fluid components.

Issues with Brute Force Nested Iteration Procedure Suppose now that this parallel collection of pipes were part of a larger series of fluid components. The iteration procedure would be that given a set pressure loss, we would iterate our way to have a fixed total flowrate along the entire system hopefully within the time limit.

This iteration procedure is quite suitable if we wanted to incorporate state synchronisation capabilities into our digital twin in future. A possible setup could be that the user puts in a set frequency or pressure into CIET's pump. This user input is then translated into a loop pressure drop (which I would like to call "pump pressure") input for the digital twin at the same time. If the digital twin is able to obtain estimates of flowrate data using equation C.28 in real-time, then we are able to use this digital twin for fault detection purposes. This digital twin would have real-time state input synchronisation with CIET. To enable this capability, we would need to iteratively solve for mass flowrate values in every pipe branch in CIET. To do so, we would have to keep guessing the system mass flowrate until the desired pressure loss is reached. This is the first outermost iteration. To obtain the pressure losses for each mass flowrate value, we would then have to set a constant mass flowrate over each fluid component, including the parallel collection of pipes. For simple fluid components, we do not really have to iterate.

For parallel collections of pipes however, iteration is quite necessary. The exception is where form losses of the system greatly exceed the darcy friction factor term. In such a case:

$$\Delta P_{change} = - \sum_i^n \frac{1}{2} \frac{\dot{m}^2}{A^2 \rho} (K) + \Delta P_{hydrostatic} + \Delta P_{source} \quad (C.29)$$

Now, if there is little density variation, we may proceed to factorise the other terms in the summation term out:

$$\Delta P_{change} - \Delta P_{hydrostatic} - \Delta P_{source} = - \frac{1}{2} \frac{\dot{m}^2}{\rho} \sum_i^n \left(\frac{K}{A^2} \right) \quad (C.30)$$

Rearranging and simplifying:

$$\Delta P_{hydrostatic} + \Delta P_{source} - \Delta P_{change} = \frac{1}{2} \frac{\dot{m}^2}{\rho} \sum_i^n \left(\frac{K}{A^2} \right) \quad (C.31)$$

$$\dot{m}^2 = 2\rho \frac{\Delta P_{hydrostatic} + \Delta P_{source} - \Delta P_{change}}{\sum_i^n (\frac{K}{A^2})} \quad (C.32)$$

In such a case, mass flowrate in each series of pipes found without iteration provided we know the pressure change.

If the cross sectional area varies little across the series of pipes, we can find a simplified expression for mass flowrate over each branch:

$$\dot{m} = \sqrt{2\rho(\Delta P_{hydrostatic} + \Delta P_{source} - \Delta P_{change})} \frac{A}{\sqrt{\sum_i^n (K)}} \quad (C.33)$$

For parallel flow paths without pressure sources and equivalent hydrostatic pressure, and with nearly constant flow area, flow across parallel paths is proportional to $\frac{A}{\sqrt{K_{total}}}$. It is a useful case where we can save ourselves from iterating in order to obtain mass flowrate across each flow path.

However, for CIET, flow is often in the laminar region especially when the loop operates in natural circulation mode [N. Zweibaum et al., 2015] or perhaps in loss of forced cooling (LOFC) scenarios where the pump ceases to work. If we wished to pick an algorithm which worked in all cases, then iteration would be the way to go.

Now given a set mass flowrate, we would then iteratively guess the same pressure change across each branch until the sum of mass flowrates across each branch is equal to the set mass flowrate. This is the second layer of iterations.

However, obtaining mass flowrates from each series of pipes within the is itself an iterative process. This is the third layer of iterations. We can perform some simple estimated calculations to estimate how much computation time is required.

Suppose that a simple estimation to obtain pressure loss from mass flowrate takes 1 μs . Each branch has about 10 components. Thus, without parallelisation, we could estimate around 10 μs for each branch to obtain pressure loss for given mass flowrate.

Suppose now that each iteration algorithm requires about 10 iterations to converge upon the solution for mass flowrate given a pressure loss or pressure change. That means to calculate a given mass flowrate across each branch we would require about 100 μs per iteration. This is the first layer of iteration.

The second layer would be to iterate out the pressure loss term given a total mass flowrate. Suppose again that to obtain total mass flowrate given a pressure loss term we require 100 μs per branch and 300 μs for three branches. Supposing that 10 iterations are required, the calculations would take at least 3 ms.

The third layer of iteration would be to iterate a total mass flowrate term given a total pressure loss over the entire system.

This calculation could take again 10 iterations. This would put us at 30 ms per calculation if we were optimistic. This of course assumes we need 10 iterations to achieve convergence and that we don't have numerical stability issues. If let's say we needed 20 iterations, then we would need 8 times as long to calculate.

This would be 240 ms, which is unacceptable because each calculation timestep is aimed at 100 ms which is approximately how often CIET yields data, every 95 to 100 ms.

Again, we also assumed that each pipe calculation takes $1\mu s$, which is rather optimistic. If let's say it were to take $10\mu s$, then our calculation time would be 2.4 s. This is an order of magnitude slower than what is required.

It is clear now that a brute force nested iteration procedure would begin to be problematic as we would need to iteratively solve for mass flowrates given pressure loss terms over each series of pipes. We would then have a case of nested iterations which could prolong computation times to a point where the method becomes unsuitable for digital twins.

Furthermore, it would make real-time state synchronisation of the digital twin nearly impossible.

Nevertheless, brute force iteration still serves as to be useful for validation of other heuristic methods should they be used. However, we still need a code which simulates fluid flow within the specified time limit.

Therefore my first attempt would be to use compiled languages I was familiar with, eg. C#. I also used this language because DWSIM was written in the .Net ecosystem, and it would have the thermodynamic libraries available for me to compute heat transfer for thermol VP-1 and other fluids without writing the libraries from scratch.

Possibility 3: Attempt to Adapt SpiceSharp to Fluid Flow

Foreseeing this issue, alternative methods were sought after for solving pipe network problems within time. One possible route explored was the adaptation of circuit solvers for pipe networks. Circuit voltage would be analogous to pressure losses, whereas current would be analogous to mass or volumetric flowrates.

The specific software attempted was spice sharp, which was written in C#. [Boulanger, Golebiowski, and Sodalom, 2017] This software was designed for circuits and networks with complex parallel branches. To account for nonlinear components such as diodes, a Newton Raphson solver is used.

It was found, however, that while spice sharp had multidimensional Newton Raphson solvers to iteratively solve for non linear resistors which do not obey Ohm's law (of which a pipe is analogous to a non ohmic resistor), the solver often failed to converge to the correct values, and more often than not, encountered stability issues due to these nonlinear components such as pipes and valves.

The only times spiceSharp's Newton Raphson solvers worked was in simple fluid circuits where one pipe was in each parallel branch. This was compared against a nested iteration solver during unit tests.

With such meager results, I decided to move on from spiceSharp. Failing which, I then decided to attempt writing solvers for networks of series connections of pipes using C# code.

Timed Brute Force Nested Iteration Calculations in C# I decided to continue experimenting in C# because the DWSIM [Tangsriwong et al., 2020] thermodynamic libraries were written in Visual Basic .Net, which is part of the .Net ecosystem, of which C# code belongs. Not only that, this was convenient because I also wrote brute force nested iteration solvers into my custom spiceSharp resistors (pipes) because they were needed to verify the solver accuracy for the Newton Raphson solvers. These were in turn verified against fixed values of the darcy friction factor and Re correlations using Moody's chart [Perry and Green, 2015].

For these series of tests, I then created a C# library to simulate brute force iteration for branches of series pipes in parallel described above. This library was based off the spiceSharp library which I adapted to fluid flow. They do not use any of the solvers however; only the custom structures and objects used to store data such as IEntity and IEntityCollection were used. I just added class methods to these existing classes to enable brute force nested iteration calculation. As the experiments went on, I kept adding capabilities and libraries such as coolprop [Bell et al., 2014]. Tests for the code were run using XUnit in .Net Core C# code which by default would run test cases in parallel. Even for a simple pipe setup of three pipes in parallel, the nested iteration loops often took 47s to more than a minute to solve depending on the pressure loss I gave to the pipe system. This is 3 levels of nested iteration to obtain a solution.

It should be noted however, that the 47s does not truly reflect the calculation speed. When I used the C# stopwatch classes to measure time taken specifically for the iterations and excluded time to initialise the classes, I found that the calculation took about 175 ms. This was way long for my time requirement of 100 ms.

Whereas when I was just solving for three pipes in parallel given a fixed mass flowrate using two levels of nested iteration, the calculation times often took 33 ms. This was much better and proved that taking nested iterations out of the equation would truly help.

For these tests, I used:

AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx (8) @ 2.100GHz

While the code was not optimised in any way, it showed that for a simple system of three pipes in parallel, performance was quite unsatisfactory. In CIET, I would have to implement a system with perhaps 30 to 60 components, and two or three parallel branches. This meant that the required times for calculation would only increase.

Possibility 4: Parallel Branch Approximation

To help solve the issue of long calculation times, one idea that came to mind was that it was best to solve the outer CTAH loop as if it were a fourth parallel branch. Such an approximation would allow us to solve for the fluid mechanics equations within the stipulated timeframe.

Essentially we are solving for the total mass flowrate of each branch combined and setting it to zero:

$$\sum_i^n \dot{m}_i = 0 \quad (\text{C.34})$$

Each branch will have the mass flowrate equal to that of $\dot{m}$ within equation C.28, this is the mass the mass flowrate of the series of pipes within said branch.

We see that with one less layer of nested iterations, the speed increase is approximately five to six times as shown using a simple C# demonstration. Therefore, while there are perhaps some inaccuracies caused by this approximation, where perhaps I would ignore some of the frictional losses caused by some of the piping systems, the benefit of performance gains far outweighed the cost for CIET.

Therefore, moving forward, this would then be the desired approximation with which to simulate CIET.

Data Race and Thread Safety Issues with C# and its Underlying Libraries

Even with the parallel branch approximation, it was apparent that C# libraries were not quite suitable due to speed issues. A faster code was needed to carry out the iterations in real time. Furthermore, I found another issue with C# code and the underlying libraries: CoolProp thermodynamic libraries as well as Engineering Units for unit safe calculations were not thread safe. This would make debugging arduous if I ever wanted to move to parallelisation using multiple threads. Not only might I require such capabilities in future to increase computation speed, I may also wish to carry out the state synchronisation for the digital twin in parallel with the calculations in future. Therefore, I wanted to write the digital twin in a language which enabled these capabilities with few issues as possible.

Unfortunately, C# was not such a language as I had data race issues. I found this issue out as I ran unit tests for the pipe calculation code using the XUnit test framework

[James Newkirk, 2022] as described earlier when I was testing spiceSharp code. By default, XUnit runs tests in different test classes in parallel. I found that as I wrote several test classes, I ran into race conditions. Sometimes, wrong unit exceptions would be thrown, and at other times, when two threads try to access the CoolProp thermodynamic libraries written in C, there would be byte errors. Some other times, there were no errors at all. In other words, if I wished to introduce multithreading into the code to speed up simulation speed, I foresee that race conditions would be common as the C# compiler compiles code which is thread unsafe.

Race conditions (of which data races are possible cause) are one of the hardest bugs to solve seeing that they are not consistently repeatable but still can cause the program to crash. To help solve the issue of concurrency unsafe code, I began to wonder if a language with thread safety built into its compiler would be a better choice. Therefore I began to consider Rust. [Klabnik and Nichols, 2022]

Issues with C# OPC-UA Server

However, Rust was a new language for me and would cost some time to learn. I thought that the most time efficient means of learning would be to first try setting up an OPC-UA server with the pipe code in C# even if the calculations do not meet the desired time of <100 ms per loop.

Unfortunately, I found out later that when I tried integrating the flow solvers into a C# OPC-UA server, I found that the C# OPC-UA code was impractical to work with. I therefore decided eventually to use a Python OPC-UA server. While Rust also has an OPC-UA server implementation which is relatively more user friendly compared to C# , Python is no doubt more user friendly than Rust. In terms of development speed, Python will beat Rust. If performance bottlenecks are found to be due to Python, OPC-UA server code can then be translated from Python to Rust once a working model was built.

Conclusion: C# may not be suitable for this Digital Twin With speed issues, thread safety issues and difficulty with writing OPC-UA server and client in C#, I decided to abandon C# for writing the digital twin altogether.

Transition into Rust

Due to thread safety and speed issues, I began to think about transitioning into Rust. Nevertheless, I was concerned that Python's lack of computational speed would be detrimental for progress towards real time digital twin. To speed up computations in Python, I needed to link it to libraries written in compiled code; there were a few options for me, I could either construct a library in C#, and link it to Python [Lostmsu, Vmuriart, and Filmor, 2006] or use another code such as Rust.

A third option would be to use Python's just in time (JIT) compiler such as Pypy to speed up the code. PyPy is used to translate Python code to cPython. This results in faster run times and this is able to make Python comparable in speed to C#.

However in terms of writing thread safe code, these do not come near to the level that Rust offers. Rust's compilers will not compile the code if it is written with the possibility of data races. It also comes close to C and C++ in terms of computing speed and performance. Therefore, it seems Rust is a superior language with which to write a high performance iteration and calculation code.

Nevertheless, we still wish to quantify the relative speeds of Python, C#, C++, C and Rust to check if Rust is fast enough a language suitable to write our digital twin in.

A verbose account of OPC-UA Server Selection

Experience with C# Implementation of OPC-UA Server

When starting the project, C# seemed to be the right balance of speed and code maturity since it had many useful libraries such as EngineeringUnits.

In addition, C# code has a well documented OPC-UA library hosted on github by the OPC foundation available as a nuget package. [Regen, Barnstedt, and Moldovean, 2016]

<https://github.com/OPCFoundation/UA-.NETStandard>

However, creating a server using such a library is no trivial task; the only practical way to construct a server (without purchasing additional licenses) is using OPC Foundations model compiler. [Armstrong, 2015; Manditereza, 2021]

<https://github.com/OPCFoundation/UA-ModelCompiler>

The user first had to write out their OPC-UA model in an xml model design file. The user would then compile the UA-ModelCompiler code which would create an executable to generate C# code from the xml file.

The generated C# code then had to be used to create custom sub classes necessary for creating the server. And only then the server could be created.

Such a method was found to be extremely tedious even for a simple 10 component system. For CIET with more than 50 components, this was not going to be practical. If one were to design CIET's digital twin with heat transfer capabilities, this would mean creating more than 100 node objects using this method.

This was not practical and the C# OPC-UA server was abandoned. While there are higher level C# OPC-UA server and client software development kits (SDKs) which greatly speed up the development process, they are often proprietary software. Some prices quoted online went as high as 1000-3000 euros. [Traeger, 2022]

<https://opcua.traeger.de/en/#prices>

Choosing Between Python and Rust

In comparison, both Python and Rust had significantly lower barriers to entry compared to C# code whether in terms of cost as these codes are released under open source licenses.

And comparing Python and Rust OPC-UA opens source implementations to the C# counterparts, both Python and Rust code seemed less complex as to program a server, Python and Rust required fewer less interaction with the low level OPC-UA code and third party compilers like the UA Model Compiler.

The Python implementation of OPC-UA server and client code is found here [Oroulet et al., 2022]:

<https://github.com/FreeOpcUa/opcua-asyncio>

Whereas the Rust OPC-UA server and client code is found here [Locka99, 2022]:

<https://github.com/locka99/opcua>

While Rust may be faster than Python even when Python uses PyPy JIT compiler, the ease of setting up a minimal Python OPC-UA server compared to Rust was the deciding factor in choosing to continue writing OPC-UA servers in Rust for this prototype.

I also conjecture that the bottleneck for calculation speed at each timestep would be the nested iterations required to solve for the fluid flow in the CIET system rather than the Server code itself. This is of course excluding the synchronisation tasks that might be required of the digital twin in future.

By then, when state synchronisation is desired for the digital twin, we may find ourselves bottlenecked when we wish to estimate the various temperatures within CIET using heat transfer models. For a handful of components, this may not be an issue.

However, CIET has about 30 or more components if we include the DRACS loop. Some of these components have several sensors attached to them. Furthermore, we may want to keep track of the state of each component. For a typical pipe, we may have an entrance temperature, bulk fluid temperature, inner surface temperature, pipe temperature and insulation temperature to keep track of. We may also be keeping track of the enthalpy of these different parts of the pipe. This means we are already keep track of 10 variables per pipe. Furthermore, we may want to keep track of fluid density and viscosity for the sake of mass flowrate calculations. This means we have at least twelve variables to keep track of per pipe. If we were to multiply that by 30 components, it means we may eventually have to synchronise 360 variables at every timestep.

This list of variables is not exhaustive. Furthermore, we may want to, at some point in future, use this information for fault detection in real-time. Otherwise, we may wish to use this data for a fault tolerant controller. This adds on to the task load required of the digital twin in each second.

In reality, depending on the fidelity of the model, we may require more variables to be kept in sync. Keeping each variable in sync requires some amount of computational time.

However, synchronisation is out of scope for this thesis and for development of a Type I digital twin of CIET. So for now, we will leave that out. The bottleneck for this first iteration of an isothermal-flow digital twin is likely the calculation speed of the fluid equations. This is because of the nested iterations required to solve the equations.

To help with this bottleneck, we could take a page from Pyomo's book and use compiled code to speed up Python's slower code. However, instead of using C, C++ or fortran, I would opt to use Rust because of its high degree of memory safety without requiring garbage collection and due to lower cognitive complexity. [Ardito et al., 2021]

Hence, a possible setup would be that the most time intensive computations could be calculated in Rust for obtaining flow solutions via fluid mechanics equations, whereas the relatively lighter computation load of running the OPC-UA server could be done in Python without too much of an issue in terms of time.

For this thesis, a Python server with a calculation backend written in Rust was developed. Iterative calculations could be based on the peroxide library in written in Rust [Kim, 2022]:

<https://github.com/Axect/Peroxide>

Alternatively, for any iterative calculations within Python, I used Python's scipy library with the brent method to implicitly calculate mass flowrate over each branch given a pressure loss term. This ensures that I have a high degree of speed and stability when iterating for a mass flowrate solution. A similar process is performed for both layers of iteration. [Virtanen et al., 2020]

Scipy has a root finding function based on brent's method, and has a backend based on compiled code (C, C++ or fortran) similar to numpy [Virtanen et al., 2020], this ensures that we are not bottlenecked by Python's slow speed.

Brent's method, can be as fast as the secant method, but falls back to bisection during its calculations. Thus, it provides the stability of the bisection method but could be as fast as the secant method. [Stage, 2013]

While other iteration methods exist, not all of them are implemented in Scipy or Peroxide libraries. The emphasis for a first attempt is numerical stability first and speed second. If extra speed is required, modifications can be made in future iterations.

The most important excerpts of the server code are as follows:

We first have some functions which calculate branch mass flowrates using Rust code:

```
def rust_get_heater_branch_pressure_change(
    mass_rate_kg_per_s,
    temperature_degrees_c):
    return rust_functions_in_python.
        ↪ get_heater_branch_isothermal_pressure_change_pascals_rust(
            mass_rate_kg_per_s,
            temperature_degrees_c)
def rust_get_ctah_branch_pressure_change(
    mass_rate_kg_per_s,
    temperature_degrees_c,
    pump_pressure_pascals):
    return rust_functions_in_python.
        ↪ get_ctah_branch_isothermal_pressure_change_pascals_rust(
            mass_rate_kg_per_s,
            temperature_degrees_c,
            pump_pressure_pascals)
def rust_get_heater_branch_hydrostatic_pressure(
    temperature_degrees_c):
    return rust_functions_in_python.
        ↪ get_heater_branch_isothermal_hydrostatic_pressure_pascals_rust(
            temperature_degrees_c)
```


For each branch, we must get the mass flowrate given a pressure drop. I opted to use Scipy libraries as compared to Rust's Peroxide libraries because of convenience. Every function in Rust that I need to import over to Python requires me to program an interface using PyO3. This is extra code and work which I preferred to avoid in favour of expediency. This caused me to choose scipy instead of Rust's peroxide's libraries to help solve for the first level of nested iterations.

```
def get_heater_branch_mass_flowrate(
    pressure_change_pascals,
    temperature_degrees_c):

    # basically im solving for the mass rate which
    # returns the correct pressure change
    def heater_pressure_chg_root(
        mass_rate_kg_per_s):
        return rust_get_heater_branch_pressure_change(
            mass_rate_kg_per_s,
            temperature_degrees_c) - pressure_change_pascals

    mass_flowrate_kg_per_s_solution = scipy.optimize.brentq(
        heater_pressure_chg_root,
        -1.0,
        1.0)

    return mass_flowrate_kg_per_s_solution
```

The above code is meant to calculate mass flowrate for the heater branch; similar calculations are performed for the DHX and CTAH branch as well.

In this code, the Brent method is used. The Brent method in the Scipy libraries require upper and lower bounds for their solutions similar to bisection because it falls back to bisection in case the secant method part of the Brent's method fails. For this case, the bounds i gave are flowrates of -1 kg/s and 1 kg/s. This is because CIET's experiments tend to have typical flowrates of 0.18 kg/s. I just gave an arbitrary safety bound of ± 1 kg/s because 1 is a convenient number to give. However, this amounts to a safety factor of just over five times CIET's typical flowrates. I use these same bracketing bounds for the DHX and CTAH branch as well. This has proven to find the roots with minimal convergence error.

Once these calculations are complete, we can then move on to calculate the mass flowrate inside each branch given a fixed loop pressure drop. Similar to the SAM version of simulation [Zou, R. Hu, and Charpentier, 2019], any pump curve will be up to the end user to program in, and it is outside the scope of this thesis.

The following code shows an example of an early prototype which calculates the flow rate for flow only considering the heater branch and CTAH branch:

```

def get_ciet_isothermal_mass_flowrate(
    pump_pressure_pascals,
    temperature_degrees_c):
    # the job of this function is to sum up the mass
    # flowrate of the branches in ciet
    # and solve for the value where the branch flowrates
    # sum up to zero
    # the convention is positive flowrate leaving the
    # top and negative flowrate entering the top

    def pressure_change_root(pressure_change_pascals):
        # both branches must be subject to the same
        # pressure change since they are in parallel
        return get_ctah_branch_mass_flowrate(pressure_change_pascals,
            temperature_degrees_c,
            pump_pressure_pascals) + get_heater_branch_mass_flowrate(
                pressure_change_pascals,
                temperature_degrees_c)

    # this scipy module will solve for the correct pressure change
    # given the mass flowrate
    # the intervals will be 50000 pascals plus or minus
    # the hydrostatic pressure change of the heater

    upper_bound = rust_get_heater_branch_hydrostatic_pressure(
        temperature_degrees_c) + 50000

    lower_bound = rust_get_heater_branch_hydrostatic_pressure(
        temperature_degrees_c) - 50000

    pressure_change_value = scipy.optimize.brentq(
        pressure_change_root,
        lower_bound,
        upper_bound)

    # once we get the pressure change root value,
    # we can get mass flowrate

    ctah_branch_mass_flowrate = get_ctah_branch_mass_flowrate(

```

```
        pressure_change_value,  
        temperature_degrees_c,  
        pump_pressure_pascals)  
  
    return ctah_branch_mass_flowrate
```

The Brent method here tries to guess the pressure change over each branch. To give bounds for Brent's method, the hydrostatic pressure ± 50000 Pa is used. This is because the loop pressure drop at 0.18 kg/s is about 16000 Pa. I just arbitrarily gave it a safety factor of 50000 Pa as it was a convenient round number. This is about three times the typical loop pressure drop we expect at 0.18 kg/s.

Now that we have discussed how Brent's method was used, we can move on to the server code.

In the server code, we first start by defining the endpoint, (IP Address), nodes and objects of the server.

After which, in the main server loop, the mass flowrate is calculated at each timestep:

```
# this is the main ciet digital twin calculation
pump_pressure_pascals = await ctah_pump_pressure.get_value()
fluid_temp_degC = await ciet_temperature_degC.get_value()
mass_flowrate_kg_per_s = get_ciet_isothermal_mass_flowrate(
    pump_pressure_pascals,
    fluid_temp_degC)

await ctah_flowrate.write_value(mass_flowrate_kg_per_s)

_logger.info('pump_pressure_pascals: %.1f',
    pump_pressure_pascals)
_logger.info('ctah_mass_flowrate_kg_per_s: %.4f',
    await ctah_flowrate.get_value())

# this is the experimental ciet pressure loss at
# 20C

pressure_loss_expt = expt_pressure_loss_pascals(
    mass_flowrate_kg_per_s)
await ciet_pressure_loss_manometer_data.write_value(
    pressure_loss_expt)
_logger.info('pressure_loss_expt (Pa): %.1f',
    await ciet_pressure_loss_manometer_data.get_value())

end = time.time()

elapsed_loop_time_seconds = (end - start)
_logger.info('elapsed_loop_time_seconds (ms) %.1f',
    elapsed_loop_time_seconds * 1000)

wait_time = 1
```

```

sleep_time = wait_time - elapsed_loop_time_seconds

await asyncio.sleep(sleep_time)

```

For the digital twin, we want to make sure that flowrate data is returned at regular intervals according to the desired timestep eg. every 0.1s when the timestep is 0.1s. However, we know that calculation times will fluctuate depending on the CPU. To address this, we shall subtract the elapsed time for the digital twin calculation from the wait time. In the above case it is 1 second because I wanted print and log outputs to be readable during debugging. However, in real operations, it can very well be adjusted to 0.1s. This is so that regardless of calculation time, the calculations are performed to update server data every 0.1s.

C.4 Residual Plot Discussion for DHX and CTAH Branch

We are not entirely sure of what is causing the deviations in figure 4.2. One possibility is that This may be indicative of a missing component. One may suspect that this is due to the approximations of branch 5 and branch 17 in figure 3.10. However, this should not be the case because if one considers blocking out the heater branch in figure 3.9 and figure 3.10, the diagrams look quite similar. The approximation I made was to combine 5a and 5b into a component called branch 5, and the components 17a and 17b into one branch. Therefore, this approximation should not be the cause of these deviations. Nevertheless, we can show that these deviations are not too significant using error propagation analysis to estimate the effects of various uncertainties. Hence, when residual plots are discussed, we also want to discuss error propagation.

We first explore some basics on error propagation here, and then we will conduct some analysis on how some typical errors such as flowmeter error and pipe tolerance errors can affect results for loop pressure drop. We also use numerical methods to obtain loop pressure drop errors rather than analytical methods to obtain the error bars. This section also explains why. I was also unsure if the 10% ζ correlation modelling error provided matched the data. Hence I also added a section displaying that I used graphreader to see if the residuals were within 10% of the true value.

Error Propagation Variance and Covariance Methodology

Error propagation is of great importance in nuclear engineering design, it has been talked about extensively in nuclear thermal hydraulics texts [Todreas, Kazimi, and Massoud, 2021] and in generally literature [Meyer, 2007]. From literature, one common standard for uncertainty quantification and propagation is the Guide to Expression of Uncertainty in Measurement (GUM) [BIPM et al., 2008].

Error Propagation for Independent Variables

For independent variables, error propagation can be described as using a differential equation [Todreas, Kazimi, and Massoud, 2021; BIPM et al., 2008; Meyer, 2007]:

$$u_c^2(M) = \sum_{i=1}^N \left(\frac{\partial M}{\partial x_i} \right)^2 u^2(x_i) \quad (\text{C.35})$$

The term $u_c(M)$ stands for combined uncertainty of a measurand M , and $u(x_i)$ stands for the uncertainty of a quantity x_i used in turn to calculate M . This is useful when we look at the error due to manometer readings. Usually we subtract one manometer reading h_2 from the another manometer reading h_1 to find the head loss Δh . The readings are quite independent of each other.

$$\Delta h = h_1 - h_2 \quad (\text{C.36})$$

Note that we can readily replace uncertainty with standard deviation σ as it is a common measure of uncertainty.

Hence the uncertainty can be described as:

$$\sigma_c^2(\Delta h) = \sigma^2(h_1) \left(\frac{\partial \Delta h}{\partial h_1} \right)^2 + \sigma^2(h_2) \left(\frac{\partial \Delta h}{\partial h_2} \right)^2 \quad (\text{C.37})$$

Due to the simple linear relation, the square of partial derivatives, also known as sensitivity coefficients [BIPM et al., 2008] evaluate to 1. Thus leaving us with the expression

$$\sigma_c(\Delta h) = \sqrt{\sigma^2(h_1) + \sigma^2(h_2)} \quad (\text{C.38})$$

Uncertainty Statistics for Measured Values

Normally, readings such as manometer readings are not read or sampled 30 or more times in order to secure a large sample size so that we can obtain sample means to apply the central limit theorem to. The variance $\sigma^2(x_i)$ or uncertainty $\sigma(x_i)$ is evaluated from scientific judgement in what is known as a “Type B evaluation of standard uncertainty” [BIPM et al., 2008].

In some cases, it is only possible to estimate the upper and lower bounds of a reading, just like in a manometer reading where we have a instrument measurement error $\epsilon_{measurement}$ of $\pm 0.001m$, which is the smallest marking on a manometer line. In such a case, we can expect a reading, for example $1.000m \pm 0.001m$ to lie anywhere from $0.999m$ to $1.001m$. Having little clue on what the actual distribution is, we can only assume a uniform distribution within that interval, and a zero probability of measuring values outside this interval [BIPM et al., 2008]. For such a rectangular distribution, the variance can be calculated as:

$$\sigma^2(x_i) = \frac{(a_+ - a_-)^2}{12} \quad (C.39)$$

Given that the bounds are determined by this measurement error, we can write the variance as:

$$\sigma^2(x_i) = \frac{\epsilon_{measurement}^2}{3} \quad (C.40)$$

Variance of Modelling Errors

To validate our digital twin, we want to check if the modelling errors exceed some set tolerance. This tolerance should in turn be determined by measurement errors.

The modelling error for validation experiments is defined as the difference between the measured loop pressure drop and the modelled loop pressure drop using the correlations used in the digital twin:

$$\varepsilon_{modelling} = \Delta P_{loop \text{ measured}} - \Delta P_{loop \text{ modelled}} \quad (C.41)$$

$$\Delta P_{loop \text{ modelled}} = \sum_{i=1}^N \frac{1}{2} \frac{\dot{m}^2}{\rho A_{xs}^2} (f_{darcy \ i} \frac{L_i}{D_i} + K_i) \quad (C.42)$$

The modelling error “should” have a mean value of zero or at least be minimised. In real life, some of these modelling errors may arise out of measurement error. The variances in modelling error can be using the Taylor series approximation can be shown to be:

$$(\delta \varepsilon_{modelling})^2 = (\delta \Delta P_{loop \text{ measured}})^2 + \Delta P_{loop \text{ modelled}}^2 \quad (C.43)$$

The variance will depend on the manometer data uncertainty, which is easy to measure, but also the mass flowrate uncertainty and other uncertainties as well. We consider some of them in the following section.

Flowmeter and Manometer Error

In CIET, the flowmeter ¹ has errors of around $\pm 2\%$ with flowrates in the range of operation, namely around $\pm 0.18 \text{ kg/s}$ [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]. We could take it to mean 2% of 0.18 kg/s or 2% of whatever flowrate value is being read. I choose the approach where I measure 2% of the current mass flowrate because I do not want to overestimate measurement error when we use it as a benchmark to compare our simulated results with. Manometer reading errors are assumed at $\pm 0.001m$. This would then cause errors within loop pressure drop estimations:

$$\Delta P_{\text{loop pressure drop}} = (h_1 - h_2)\rho g \quad (\text{C.44})$$

Analytical Way of Estimating Flowmeter Error Contributions to Loop Pressure Drop We can estimate the contribution of manometer measurement error using:

$$(\delta \Delta P_{\text{loop measured}})^2 = \rho^2 g^2 (\delta h_1^2 + \delta h_2^2) \quad (\text{C.45})$$

$$(\delta \Delta P_{\text{loop measured}})^2 = \rho^2 g^2 (0.001^2 + 0.001^2) m^2 \quad (\text{C.46})$$

$$(\delta \Delta P_{\text{loop measured}}) = (1061 \text{ kg/m}^3)(9.81 \text{ m/s}^2) \sqrt{0.001^2 + 0.001^2} m^2 \approx 14.7 \text{ Pa} \quad (\text{C.47})$$

The second error we want to consider is the 2% measurement error for flowrate. This measurement error will affect the modelled loop pressure drop.

$$\Delta P_{\text{loop modelled}} = \sum_{i=1}^N \frac{1}{2} \frac{\dot{m}^2}{\rho A_{xs}^2} (f_{\text{darcy } i} \frac{L_i}{D_i} + K_i) \quad (\text{C.48})$$

Now, mass flowrate errors not only going to affect the dynamic pressure, but also the pressure loss coefficient. To obtain the mass measurement error,

$$(\delta \Delta P_{\text{loop modelled}})_{\text{mass measurement}}^2 = (\delta \dot{m})^2 \left(\frac{\partial \Delta P_{\text{loop modelled}}}{\partial \dot{m}} \right)^2 \quad (\text{C.49})$$

We can estimate the partial derivative as:

$$\frac{\partial}{\partial \dot{m}_i} (\Delta P_{\text{loop modelled}}) = \sum_i \left[\frac{1}{2} \frac{2\dot{m}_i}{\rho A_{xs}^2} (f_i \frac{L_i}{D_i} + K_i) + \frac{1}{2} \frac{\dot{m}_i^2}{\rho A_{xs}^2} \frac{\partial Re_i}{\partial \dot{m}_i} \frac{\partial}{\partial Re_i} (f_i \frac{L_i}{D_i} + K_i) \right] \quad (\text{C.50})$$

We note that

$$\frac{\partial Re_i}{\partial \dot{m}_i} = \frac{A_{xs} i}{\mu_i D_{H i}} \quad (\text{C.51})$$

¹FC340 with a DN25 sensor [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]

If we were to multiply this by the flowrate error δm_i and take a 2% error [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]:

$$\delta m_i = 0.02 m_i \quad (\text{C.52})$$

$$\begin{aligned} & (\delta \Delta P_{\text{loop modelled}})_{\text{mass measurement}} \\ &= 0.02 \sum_i \left[\frac{1}{2} \frac{2m_i^2}{\rho A_{xs\ i}^2} (f_i \frac{L_i}{D_i} + K_i) + \frac{1}{2} \frac{m_i^2}{\rho A_{xs\ i}^2} Re_i \frac{\partial}{\partial Re_i} (f_i \frac{L_i}{D_i} + K_i) \right] \end{aligned} \quad (\text{C.53})$$

If we were to put it in terms of pressure drop for each component ΔP_i and modelled loop pressure drop $\Delta P_{\text{loop modelled}}$:

$$\begin{aligned} & (\delta \Delta P_{\text{loop modelled}})_{\text{mass measurement}} \\ &= 0.02 \Delta P_{\text{loop modelled}} + 0.02 \sum_i \left[\Delta P_i + \frac{1}{2} \frac{m_i^2}{\rho A_{xs\ i}^2} Re_i \frac{\partial (f_i \frac{L_i}{D_i} + K_i)}{\partial Re_i} \right] \end{aligned} \quad (\text{C.54})$$

For the Reynold's number derivatives, we note that there are two bounding cases. Firstly that we have laminar flow, secondly that we have turbulent flow. For the components such as the Coriolis flowmeters, CTAH and static mixers, we have:

$$(f_i \frac{L_i}{D_i} + K_i)_{\text{coriolis}} = 18 + \frac{93000}{Re^{1.35}} \quad (\text{C.55})$$

$$(f_i \frac{L_i}{D_i} + K_i)_{\text{static mixer}} = 21 + \frac{4000}{Re} \quad (\text{C.56})$$

$$(f_i \frac{L_i}{D_i} + K_i)_{\text{CTAH}} = 400 + \frac{52000}{Re} \quad (\text{C.57})$$

For laminar flow, let us calculate the derivative assuming $K_i = 0$:

$$\frac{\partial (f_i \frac{L_i}{D_i} + K_i)}{\partial Re_i} = -\frac{64}{Re_i^2} \frac{L_i}{D_i} = -\frac{1}{Re_i} (f_i \frac{L_i}{D_i} + K_i) \quad (\text{C.58})$$

In the bounding case that we have all components as laminar:

$$\begin{aligned} & (\delta \Delta P_{\text{loop modelled}})_{\text{mass measurement}} = 0.02 \Delta P_{\text{loop modelled}} \\ & + 0.02 \sum_i \left[\Delta P_i + \frac{1}{2} \frac{m_i^2}{\rho A_{xs\ i}^2} Re_i \left(-\frac{1}{Re_i} (f_i \frac{L_i}{D_i} + K_i) \right) \right] \end{aligned}$$

The right hand side term evaluates to zero:

$$(\delta\Delta P_{loop \text{ modelled}})_{mass \text{ measurement}} = 0.02\Delta P_{loop \text{ modelled}} + 0.02 \sum_i [\Delta P_i - \Delta P_i]$$

Therefore, the error contribution is 2% of the total loop pressure drop.

Whereas in the fully turbulent case, where the form loss coefficient can be considered to be constant,

$$\frac{\partial(f_i \frac{L_i}{D_i} + K_i)}{\partial Re_i} = 0 \quad (C.59)$$

$$(\delta\Delta P_{loop \text{ modelled}})_{mass \text{ measurement}} = 0.02\Delta P_{loop \text{ modelled}} + 0.02 \sum_i [\Delta P_i]$$

Therefore the error contribution is 4% of the total loop pressure drop.

For components such as the CTAH which have a laminar like term where there is a constant term in the pressure loss coefficient and a laminar like term, this is considered somewhere between the two bounding cases. At lower Re in creeping flow, it behaves like a laminar component while in higher Re, the component behaves like a purely turbulent flow component. We can infer the error contribution due to manometers is somewhere between 2% and 4%.

For the Coriolis flowmeter:

$$\frac{\partial(f_i \frac{L_i}{D_i} + K_i)}{\partial Re_i} = -1.35 \frac{93000}{Re_i^{2.35}} = -\frac{1.35}{Re_i} \frac{93000}{Re_i^{1.35}} \quad (C.60)$$

At lower Re, where the K value of 18 can be neglected,

$$\frac{\partial(f_i \frac{L_i}{D_i} + K_i)}{\partial Re_i} = -1.35 \frac{93000}{Re_i^{2.35}} = -\frac{1.35}{Re_i} (f_i \frac{L_i}{D_i} + K_i) \quad (C.61)$$

for this component,

$$(\delta\Delta P_i)_{mass \text{ measurement}} = 0.02 \left[\frac{1}{2} \frac{2m_i^2}{\rho A_{xs \ i}^2} (f_i \frac{L_i}{D_i} + K_i) + \frac{1}{2} \frac{m_i^2}{\rho A_{xs \ i}^2} Re_i \left(-\frac{1.35}{Re_i} (f_i \frac{L_i}{D_i} + K_i) \right) \right]$$

We can factorise out the component pressure drop and get:

$$(\delta\Delta P_i)_{mass \text{ measurement}} = 0.02(0.65) \left[\frac{1}{2} \frac{m_i^2}{\rho A_{xs \ i}^2} (f_i \frac{L_i}{D_i} + K_i) \right] \quad (C.62)$$

The error for this component at low flow is about 1.3%. Hence, we get an error of 1.3% at low flow to 4% at higher flows.

We can of course, calculate the error percentage rigorously. However, it seems that where laminar flow is concerned, 2% error estimates are reasonable. If we were estimating error bounds for measurement to compare it to modelling errors obtained from the digital twin, it should be conservative to take the 2% error. 1.3% error is the most conservative but almost an unnecessary one to make given that the Coriolis flowmeter is only one or two of the components of the in each flow loop.

If we were to calculate this rigorously, we can calculate the following for each component:

$$(\delta\Delta P_i)_{mass\ measurement} = 0.04\Delta P_i + \frac{1}{2} \frac{m_i^2}{\rho A_{xs\ i}^2} Re_i \frac{\partial(f_i \frac{L_i}{D_i} + K_i)}{\partial Re_i} \quad (C.63)$$

Once the mass flowrate and pressure losses (not pressure change, otherwise we have to include hydrostatic pressure) in each of the branches in parallel is found, we can then code a function to return the pressure error due to mass measurement to the environment. We then need to take the square root of the sum of these pressure errors to find the total pressure error.

Numerical Approach to Measuring Error Propagation of Flowmeter Reading Error on Loop Pressure Drop

The analytical approach may work well for error propagation for pipes in series. For pipe branches in parallel however, this method breaks down altogether because now there is more than one branch available for flow to go through. To further complicate things, the mass flowrates used to calculate pressure drop error propagations in parallel branches are not independent of each other. Hence, we may want to develop an error propagation method that can be (1) agnostic to parallel branches of flow and (2) calculated quickly without the use of iteration.

For this, we consider only the CTAH branch, where the loop pressure drop equals the CTAH pump pressure at operating point regardless of whether the DHX and Heater branches are both valved open or not. The error propagation term due on loop pressure drop due to mass flowrate reading error ($\delta(\Delta P_{loop\ pressure\ drop})$), where $\delta\dot{m}_{CTAH} \approx 2\%$ [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]) can be estimated as follows:

$$\delta(\Delta P_{loop\ pressure\ drop})_{flowmeter\ error}^2 = \left(\frac{\partial \Delta P_{loop\ pressure\ drop}}{\partial \dot{m}_{CTAH}} \right)^2 \delta\dot{m}_{CTAH}^2 \quad (C.64)$$

Given that we know the measurement error of the Coriolis flowmeter FM-40 $\delta\dot{m}_{CTAH}$, which is 2% of the prevailing flowrate through FM-40 (The CTAH Branch). We then need a way of finding the partial derivative.

There are some algorithms we could use. For example, perturb the loop pressure drop by a set small set amount and obtain the mass flowrate through the CTAH. This is problematic however, for calculation time since we are using two more nested iterations to find the partial derivative $\frac{\partial \Delta P_{loop\ pressure\ drop}}{\partial \dot{m}_{CTAH}}$.

Given that we already know the mass flowrate and existing CTAH pump pressure at operating point, we can use this information to obtain the partial derivative $\frac{\partial \Delta P_{loop \text{ pressure drop}}}{\partial \dot{m}_{CTAH}}$. We can first find the branch pressure change, which is the same for each branch, namely the CTAH, DHX and Heater branches. This can be found using mass flowrate and ctah pump pressures at operating point. Using the branch pressure change, we can obtain mass flowrates through the CTAH branch by summing up mass flowrates through the heater and DHX branches. Once we have the CTAH branch mass flowrate, we can then calculate the branch pressure drop across the CTAH branch. We can subtract the branch pressure drop and hydrostatic pressure from the branch pressure change to obtain the CTAH pump pressure at operating point.

Hence, using only the branch pressure change, we are already able to parametrically obtain a set of CTAH branch mass flowrates $\dot{m}_{CTAH}$ and loop pressure drop $\Delta P_{loop \text{ pressure drop}}$ without nested iteration. We can perturb the branch pressure change by a small amount $\pm$ a small pressure change. This enables us to obtain two sets of $\dot{m}_{CTAH}$ and $\Delta P_{loop \text{ pressure drop}}$ with which we can use to estimate the partial derivative $\frac{\partial \Delta P_{loop \text{ pressure drop}}}{\partial \dot{m}_{CTAH}}$ using the central difference approximation.

Correlation Modelling Error

In validation experiments, the pressure loss coefficient correlations were shown to have up to 10% error at which is clearly visible at lower Reynold's numbers [N. Zweibaum et al., 2015]. This applies to the heater, TCHX, CTAH, static mixers and Coriolis flowmeters [N. Zweibaum et al., 2015]. These errors can also propagate to errors in the individual pressure drops.

$$(\delta \Delta P_i)_{correlation \text{ error}}^2 = \left(\delta \left(f_i \frac{L_i}{D_i} + K_i \right) \right)^2 \left(\frac{\partial \Delta P_i}{\partial \left(f_i \frac{L_i}{D_i} + K_i \right)} \right)^2 \quad (C.65)$$

I have used graphreader to manually click and obtain the information used in the plots in figure C.1 from Nicolas Zweibaum's PhD Thesis [N. Zweibaum et al., 2015]. Here, we only consider the TCHX, CTAH, static mixers and Coriolis flowmeters. CIET's heater is ignored because the first version of the heater matches quite well with experimental data.

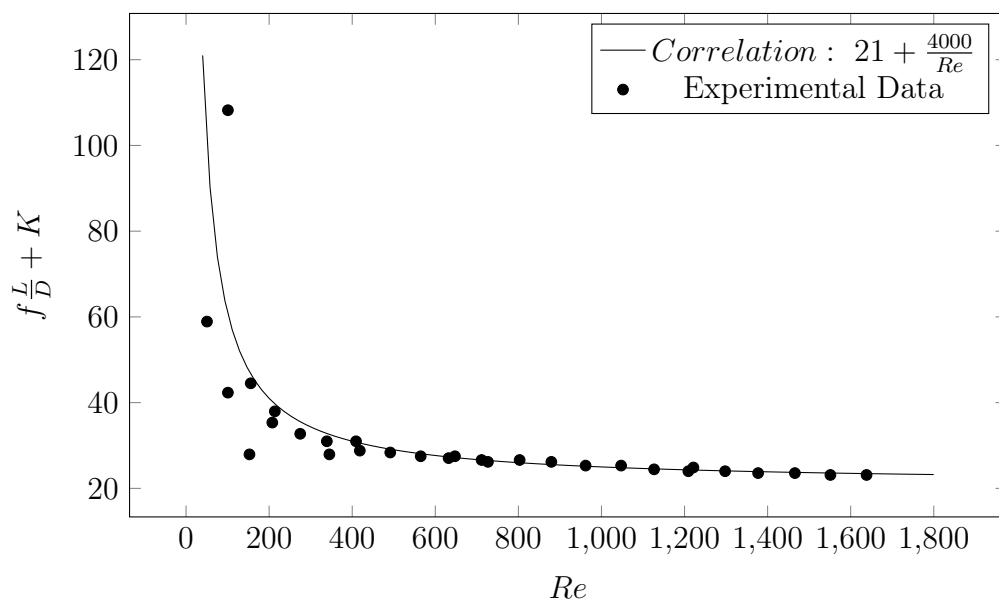


Figure C.1: MX10 Correlation and Experimental Data Plots

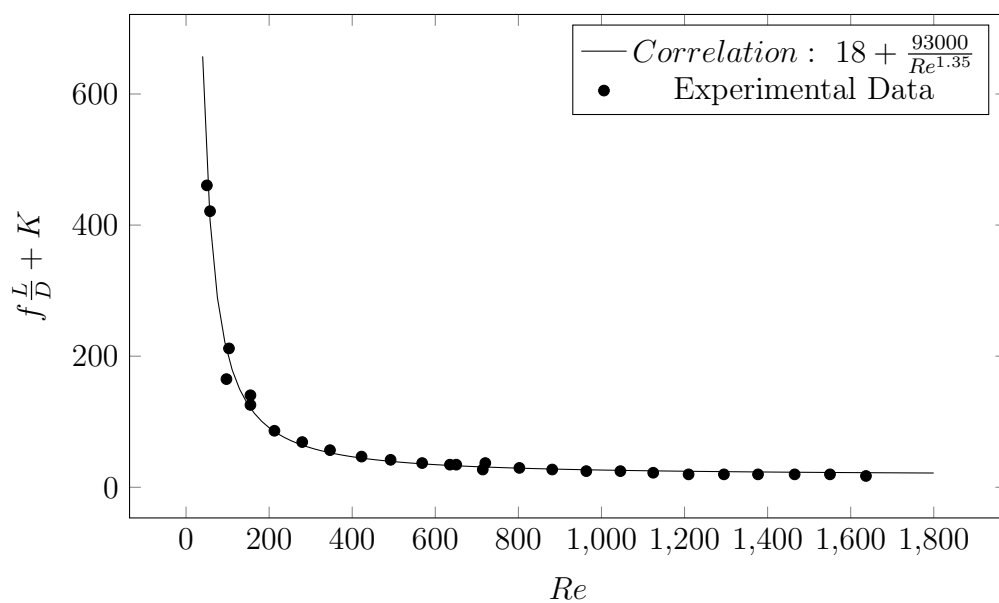


Figure C.2: FM40 Correlation and Experimental Data Plots

The data used to plot the graphs is on the following table:

Re	K+fD/L(MX10)	Re	K+fD/L(FM40)
50.38	58.909	50.15	460.591
100.76	42.327	57.75	421.182
100.76	108.218	97.26	165.025
152.67	27.927	103.34	211.823
155.73	44.509	155.02	125.616
207.63	35.345	155.02	140.394
213.74	37.964	212.77	86.207
274.81	32.727	279.64	68.966
338.93	30.982	346.5	56.65
345.04	27.927	422.49	46.798
409.16	30.982	492.4	41.872
418.32	28.8	568.39	36.946
491.6	28.364	635.26	34.483
564.89	27.491	650.46	34.483
632.06	27.055	714.29	27.094
647.33	27.491	720.36	36.946
711.45	26.618	802.43	29.557
726.72	26.182	881.46	27.094
803.05	26.618	963.53	24.631
879.39	26.182	1045.59	24.631
961.83	25.309	1124.62	22.167
1047.33	25.309	1209.73	19.704
1126.72	24.436	1294.83	19.704
1209.16	24	1376.9	19.704
1221.37	24.873	1465.05	19.704
1297.71	24	1550.15	19.704
1377.1	23.564	1636.78	17.241
1465.65	23.564		
1551.15	23.127		
1638.17	23.127		

Table C.6: Raw Data used to Plot Experimental Data Left MX-10, Right FM-40 using Graphreader, Significant Figures not Adjusted

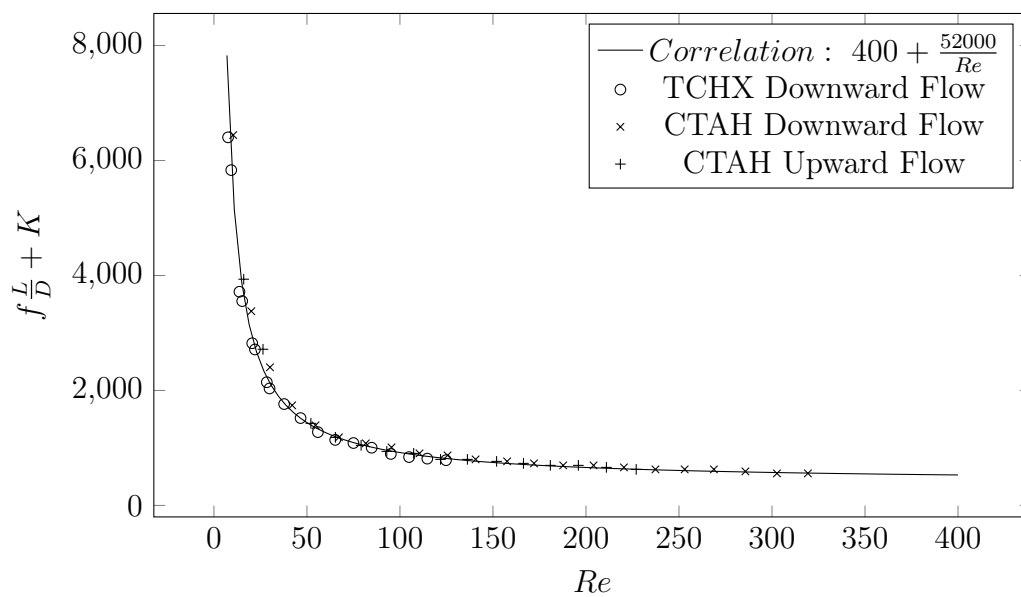


Figure C.3: CTAH and TCHX Correlation and Experimental Data Plots

Re	K+fD/L			Re	K+fD/L(TCHX)
10.361	6442.786			7.598	6403.101
20.094	3378.109			9.314	5833.333
30.141	2402.985			13.725	3717.054
42.072	1741.294			15.196	3554.264
54.631	1393.035			20.588	2821.705
67.19	1184.08			22.059	2713.178
81.633	1079.602			28.431	2143.411
95.447	1009.95			29.902	2034.884
110.518	905.473			37.745	1763.566
125.589	870.647			46.569	1519.38
140.659	800.995			55.882	1275.194
157.614	766.169			65.196	1139.535
172.057	731.343			75	1085.271
187.755	696.517			84.804	1003.876
204.082	696.517			95.098	895.349
220.408	661.692			104.902	841.085
237.363	626.866			114.706	813.953
253.061	626.866			124.755	786.822
268.76	626.866				
285.714	592.04				
302.669	557.214				
319.309	557.214				

Re	K+fD/L
16.013	3935.323
26.374	2716.418
52.119	1427.861
65.306	1184.08
79.121	1044.776
92.936	940.299
107.378	905.473
121.821	800.995
136.264	800.995
151.962	766.169
166.405	731.343
180.848	696.517
195.918	696.517
210.989	661.692
227.002	626.866

Table C.7: Raw Data used to Plot Experimental Data Left: CTAH Downward Flow, Middle: CTAH Upward Flow, Right: TCHX Downward Flow using Graphreader, Significant Figures not Adjusted

Now, to verify whether a 10% error bar is suitable or an overestimation especially at higher Re, we want to use some of this data from graphreader and plot residuals to see the percentage residuals at all ranges of Re.

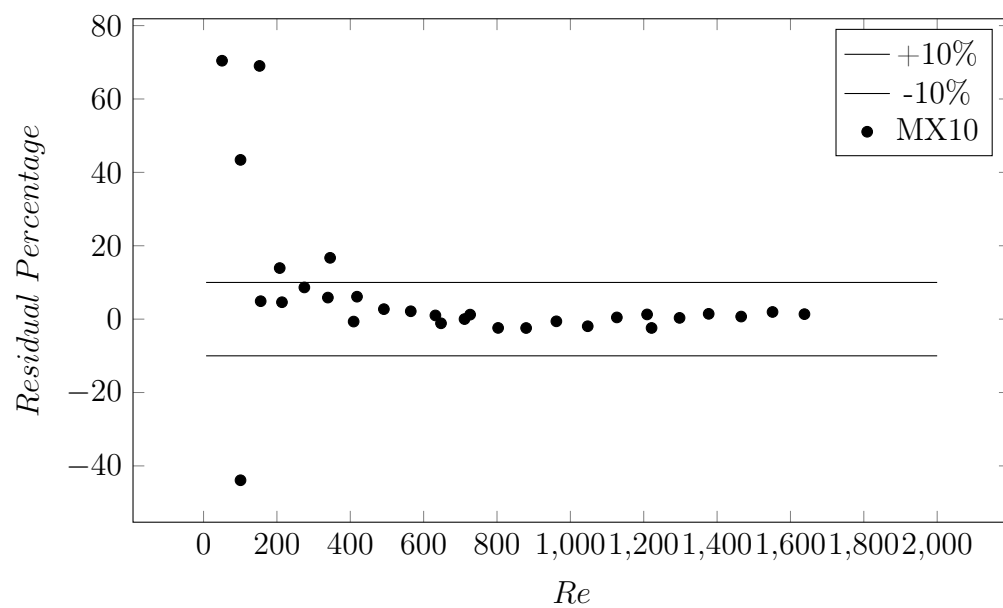


Figure C.4: MX10 Estimated Residual Plots

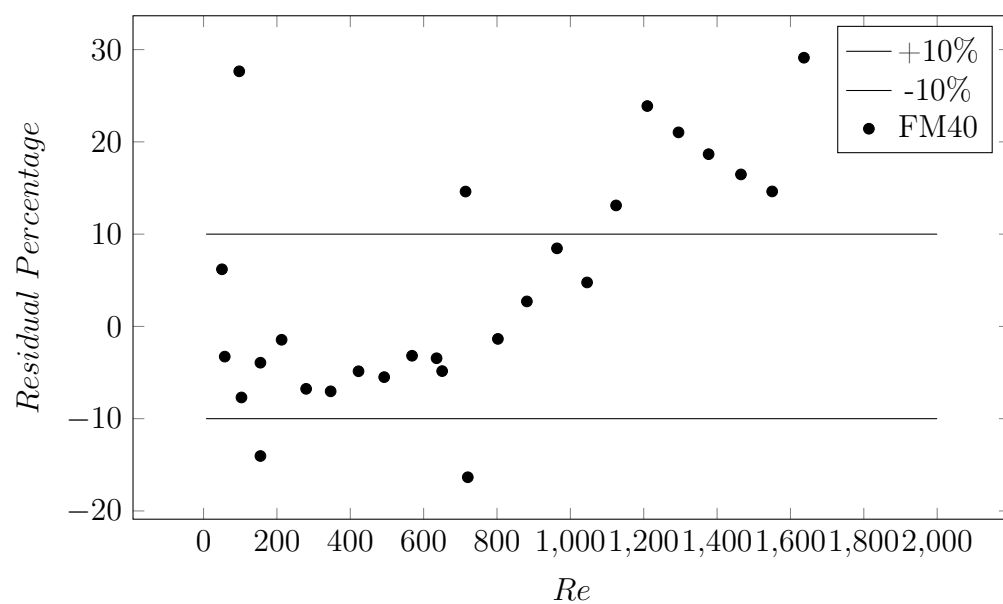


Figure C.5: FM40 Estimated Residual Plots

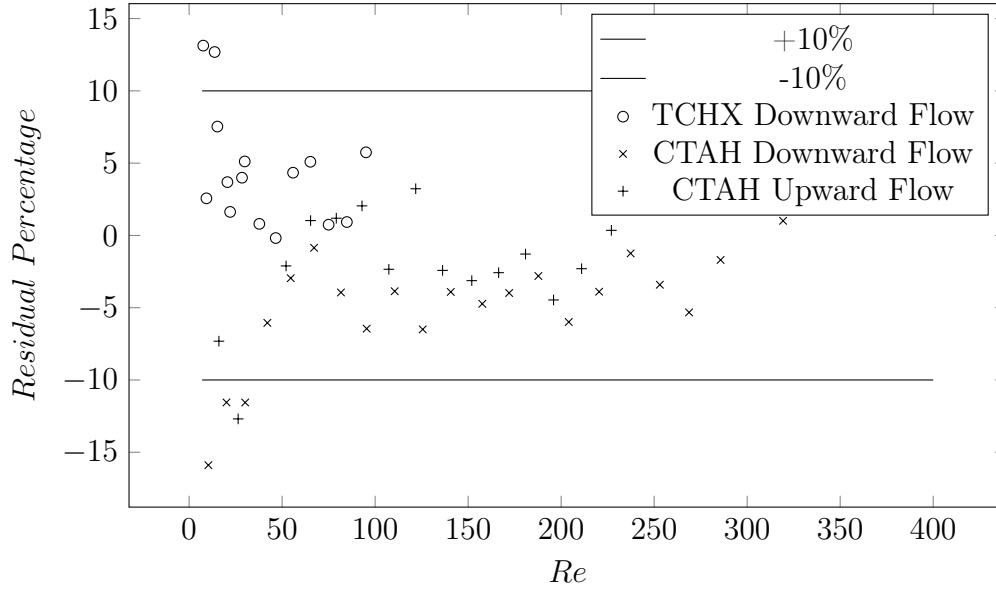


Figure C.6: CTAH and TCHX Estimated Residual Plots

It seems from the CTAH and TCHX plots, a 10% residual percentage may overestimate errors in higher Re ranges. It seems that the errors there are closer to 5% rather than 10%. However, the residuals at lower Re such as $Re < 30$ are about 10 to 15%. Whereas for FM-40, a 10% residual estimates the average error of the plots. However, at higher Re , the percentage error could be as high as 20% or 30%. For MX-10, a 10% residual measures the average percentage error as well. It overestimates error at high Re ($Re > 500$) and underestimates error at low Re , ($Re < 200$). If one wishes to estimate these errors accurately, it may be better to fit a function or some statistical distribution to them. In this analysis, there are caveats though. The pixellation errors occurring when I used graphreader to read this data may have a great degree of impact on the residuals. This was not investigated. However, this does show for the most part, that a 10% bound would seem reasonable error bound to use except for some outliers. But we cannot be sure that these outliers are due to pixellation errors or some other factor. Given these considerations, I've opted to just use the 10% ζ correlation error as given.

Other Errors that may Contribute to Modelling Errors

One error that was particularly worrying was that schedule 10 pipes have a $\pm 12.5\%$ tolerance. I initially thought that this would amount to 12.5% or greater error in the loop pressure drop. In this part of the Appendix, I do a quick calculation to show that the additional error caused by this thickness tolerance would cause at most a $\pm 2\%$ error in the diameter of the pipe. Despite calculating this, I opted not to consider this error as it would only widen the error bars. This still led to the same conclusion.

Let us consider the effect of diameter tolerances as CIET is manufactured using Schedule 10 (0.28-cm-wall-thickness) 1 in nominal diameter 304L stainless steel piping [Bickel, Nicholas Zweibaum, and Per F Peterson, 2014]. The hydraulic diameter is modelled at about 2.79 cm on most of these pipes [N. Zweibaum et al., 2015]. These will have tolerances which will introduce extra uncertainties into the calculated values of $\Delta P_{loop\ modelled}$. As of now, I don't know the exact tolerances of the pipes' inner diameter here. However, a quick web search shows the thickness tolerance can be as high as $\pm 12.5\%$ of the thickness which is 0.28 cm for this case. The outside diameter tolerances are 0.015 in over the nominal diameter and 0.031 in under the nominal diameter.

We can approximate inner diameter errors as:

$$\sigma^2 = \frac{(0.28cm * 0.125)^2}{12} + \frac{((0.031 + 0.015) * 2.54 \frac{cm}{inch})^2}{12} = 0.001239cm^2 \quad (C.66)$$

The error bounds σ based on these is about ± 0.0352 cm or about $\pm 1.386\%$. If we include the factor of $\sqrt{3}$ into this formula, where we assume a uniform error distribution we get an upper and lower bound of $\pm 2.4\%$.

The flowmeter error and manometer error are, fortunately independent of each other. The manometer error is essentially constant while the flowmeter error can be assumed to be a function of mass flowrate.

However things are not so simple yet, as the modelling error is considered to be a linear function of $\delta P_{loop\ measured}$ but a nonlinear function of $m_{i\ measured}$. However, we can ignore this because the conservative estimate is to have a lower variance of modelling error.

C.5 Computation Speed in CTAH, Heater and DHX Branch Digital Twin

In the Results Section, I wrote about the computation speed in the CTAH, Heater and DHX branch digital twin which takes about 120-125 ms to calculate. A slightly more detailed discussion is presented here for the user's convenience, and it pertains mainly to the dependence of the computation speed on how the check valve was simulated in code. Peak computation times exceeded 120 ms for the high K check valve setting for both forward and reverse flow. By forward flow, I mean that fluid flows downwards in the CTAH branch. Whereas with the method of returning zero flowrate if pressure gradients favoured upward flow through the DHX, the peak computation times for reverse flow was 125 ms. However, time savings were indeed observed for forward flow with peak calculation times around 85 ms. These calculation times were mainly due to the time it took for the solver to converge.

This is likely because the large K values slowed down the convergence of the brent solvers due to them being stiff. Thus, the solvers could have fell back to bisection. Doing so however, would slow the solver down to unacceptable times.

Appendix D

Useful References for CIET Schematics

Some of the data such as pump curves for CIET and DHX loop isothermal pressure drop data are slightly harder to find as they are not found in PhD Dissertations or Journals. However, they are available in white papers and IRP reports of for the FHR. Some of the links are pasted here for reference.

CIET Specific Instrumentation, P&ID, and pump curve data:

http://fhr.nuc.berkeley.edu/wp-content/uploads/2017/05/UCBTH-15-006_CIET-IC-White-Paper.pdf
http://fhr.nuc.berkeley.edu/wp-content/uploads/2015/04/14-009_CIET-IRP-Final-Report.pdf

Reverse fluid flow data through the DRACS loop (page 74 of 103):

http://fhr.nuc.berkeley.edu/wp-content/uploads/2015/04/14-009_CIET-IRP-Final-Report.pdf